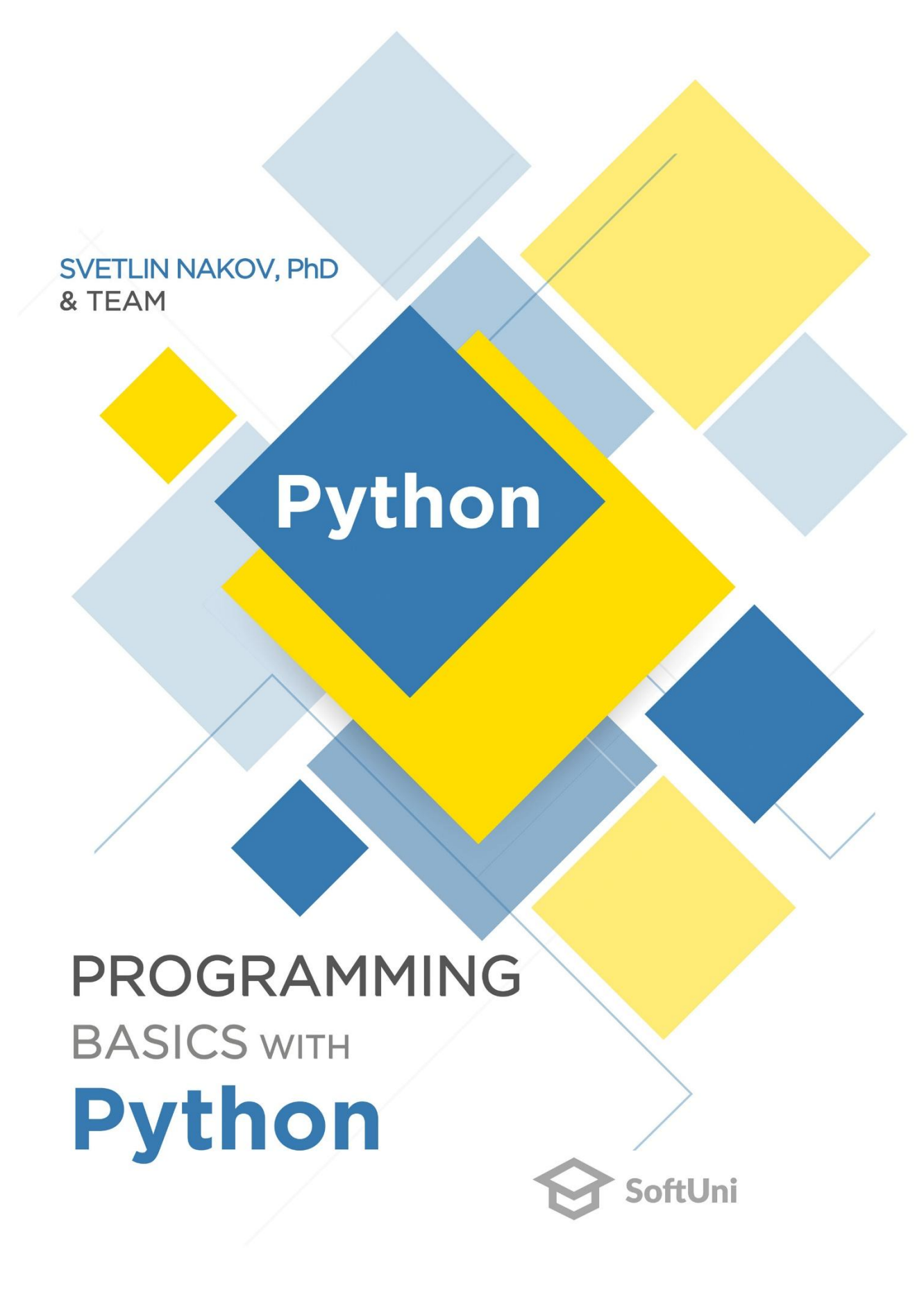


SVETLIN NAKOV, PhD
& TEAM



Python

PROGRAMMING BASICS WITH **Python**



SoftUni

Contents

Preface	13
Chapter 1. First Steps in Programming.....	27
Chapter 2.1. Simple Calculations	59
Chapter 2.2. Simple Calculations – Exam Problems	85
Chapter 3.1. Simple Conditions.....	101
Chapter 3.2. Simple Conditions – Exam Problems.....	117
Chapter 4.1. More Complex Conditions	129
Chapter 4.2. More Complex Conditions – Exam Problems	147
Chapter 5.1. Loops	165
Chapter 5.2. Loops – Exam Problems	183
Chapter 6.1. Nested Loops.....	199
Chapter 6.2. Nested Loops – Exam Problems.....	219
Chapter 7.1. Complex Loops	231
Chapter 7.2. Complex Loops – Exam Problems	263
Chapter 8.1. Practical Exam Preparation – Part I.....	275
Chapter 8.2. Practical Exam Preparation – Part II	299
Chapter 9.1. Problems for Champions – Part I.....	315
Chapter 9.2. Problems for Champions – Part II.....	329
Chapter 10. Functions.....	343
Chapter 11. Tricks and Hacks.....	369
Conclusion	381

Free Book "Programming Basics with Python"

The free book "Programming Basics with Python" introduces the readers to writing **programming code** at a beginner level (variables and data, conditional statements, loops, and functions) using the Python programming language.

Book Editions

This book is available in several versions in different programming languages:

- [Programming Basics with C# \(English\)](#)
- [Programming Basics with Java \(English\)](#)
- [Programming Basics with JavaScript \(English\)](#)
- [Programming Basics with Python \(English\)](#)
- [Programming Basics with C# \(Bulgarian\)](#)
- [Programming Basics with Java \(Bulgarian\)](#)
- [Programming Basics with JavaScript \(Bulgarian\)](#)
- [Programming Basics with Python \(Bulgarian\)](#)
- [Programming Basics with C++ \(Bulgarian\)](#)

Book Details

Brief information about this edition:

- Title: **Programming Basics with Python**
- Authors: **Svetlin Nakov & Team**
 - Main authors: Boncho Vulkov, Hristo Minkov, Iliya Iliev, Martin Tsarev, Miglen Evlogiev, Milena Angelova, Mirela Damyanova, Nikolay Kostov, Petar Ivanov, Petya Gospodinova, Svetlin Nakov, Tanya Evtimova, Tanya Staneva, Teodor Kurtev, Ventsislav Petrov, Vladimir Damyanovski, Yordan Darakchiev
 - Other contributors: Aleksander Krastev, Aleksander Lazarov, Aleksander Peev, Aleksandra Stoycheva, Angel Dimitriev, Ariet Motzeva, Daniel Tsvetkov, Denis Milanov, Dimitar Dzhapunov, Dimitar Tatarski, Dimo Dimov, Diyan Tonchev, Dzhaner Hasan, Elena Rogleva, Hristiyan Hristov, Hristo Hristov, Iskra Nikolova, Ivelin Kirilov, Jordan Liubenov, Julieta Atanasova, Kalin Primov, Kaloyan Nenchev, Kristiyan Pamidov, Luboslav Lubenov, Martin Zhelev, Mihail Georgiev, Natali Dimitrova, Nikola Slavchev, Nikolay Bankin, Nikolay Dimov, Pavlin Petkov, Petar Ivanov, Preslav Mihaylov, Rositsa Nenova, Ruslan Filipov, Simeon Stavrev, Stefka Vasileva, Svetlin Nakov, Teodor Kurtev, Tonyo Zhelev, Tsvetan Iliev, Vasko Viktorov, Venelin Bekyarov, Ventsislav Petrov, Yanitsa Vuleva, Yulian Linev, Zahariya Pehlivanova, Zhivko Nedyalkov
- ISBN: **978-619-00-1403-4** (9786190014034)
- Edition: **Faber Publishing, Sofia, Dec 2021** (expected)
- License: **CC-BY-SA**
- Source code: <https://github.com/SoftUni/Programming-Basics-Book-Python-EN>

Programming Basics with Python

Svetlin Nakov and Team

Aleksander Krastev	Jordan Liubenov	Preslav Mihaylov
Aleksander Lazarov	Julietta Atanasova	Rositsa Nenova
Aleksander Peev	Kalin Primov	Ruslan Filipov
Aleksandra Stoycheva	Kaloyan Nenchev	Simeon Stavrev
Angel Dimitriev	Kristiyan Pamidov	Svetlin Nakov
Ariet Motzeva	Luboslav Lubenov	Teodor Kurtev
Boncho Vulkov	Martin Tsarev	Tsvetan Iliev
Daniel Tsvetkov	Martin Zhelev	Tanya Evtimova
Denis Milanov	Miglen Evlogiev	Tanya Staneva
Dimitar Dzhapunov	Milena Angelova	Tonyo Zhelev
Dimitar Tatarski	Mirela Damyanova	Vasko Viktorov
Dimo Dimov	Mihail Georgiev	Ventsislav Petrov
Diyan Tonchev	Natali Dimitrova	Ventsislav Petrov
Dzhaner Hasan	Nikola Slavchev	Vladimir Damyanovski
Elena Rogleva	Nikolay Bankin	Yanitsa Vuleva
Hristiyan Hristov	Nikolay Dimov	Yordan Darakchiev
Hristo Hristov	Nikolay Kostov	Yulian Linev
Iliya Iliev	Pavlin Petkov	Zahariya Pehlivanova
Iskra Nikolova	Petar Ivanov	Zhivko Nedyalkov
Ivelin Kirilov	Petya Gospodinova	

ISBN: 978-619-00-1403-4

<https://python-book.softuni.org/>

Sofia, 2021

“Programming Basics with Python” Book and Video Lessons

© Svetlin Nakov and Team, 2021

First Edition, October 2021

This book and video lessons are distributed **freely** under the [CC-BY-SA](#) open source license, which defines the following **rights** and **obligations**:

- **Sharing** – you can copy and distribute the book and videos freely in any format or media.
- **Adaptation** – you can copy, remix and modify portions of this book and video lessons and produce new content based on them.
- **Attribution** – when you use portions of this book and video lessons you should attribute the original source, along with link to it and this license, but not in any way that suggests the licensor endorses you or your use.
- **Share Alike** – if you remix, transform, or build upon this book and video lessons, you must distribute your contributions under the same license as the original.

All trademarks used in this book are the property of their respective owners.

Publisher: Faber Publishing, Sofia

ISBN: 978-619-00-1403-4

Official Web Site: <https://python-book.softuni.org/>

Official Facebook Page: <https://fb.com/IntroProgrammingBooks>

Cover: Marina Shideroff – <https://behance.net/marinashideroff>

Source Code: <https://github.com/SoftUni/Programming-Basics-Book-Python-EN>

Table of Contents

Contents.....	3
Free Book "Programming Basics with Python"	4
Book Editions	4
Book Details	4
Table of Contents.....	7
Preface	13
Who is This Book Intended for?	13
Why Did We Choose the Python Language?.....	13
The Book in Other Programming Languages: Java, JavaScript, C#, C++	14
Programming is Learned by a Lot of Writing, Not Reading!	14
The Software University (SoftUni).....	15
How to Become a Software Developer?.....	18
The History Behind This Book	23
Chapter 1. First Steps in Programming.....	27
What Does 'To Program' Mean?.....	27
Computer Programs	27
Python Interpreter: Installation and Use	30
Starting The Python Interpreter.....	31
Computer Programs	32
How Can We Write a Console Program?	33
Development Environment (IDE).....	34
Problem: Creating a Console Program 'Hello Python'.....	38
Typical Mistakes in Python Programs	41
What Have We Learned from This Chapter?.....	41
First Steps in Programming – Problems	42
Console, Graphical and Web Applications	46
Graphical Application: 'Numbers Summator'	47
Empty Python Project.....	47
Web Application: 'Numbers Summator'	51
Chapter 2.1. Simple Calculations	59
Calculations in Programming	59

Data Types and Variables.....59

Printing Results on The Screen59

Reading User Input in Integer Type60

Reading a Sequence of Numbers.....61

Reading Floating-Point Numbers.....62

Reading a Text from The Console62

Concatenating Text and Numbers.....63

Arithmetic Operations64

Concatenating Text and Numbers.....66

Printing Formatted Text in Python67

Numerical Expressions.....69

Rounding of Numbers.....71

What Have We Learned from This Chapter?73

Problems: Simple Calculations.....74

Graphical Applications with Numerical Expressions.....77

Chapter 2.2. Simple Calculations – Exam Problems.....84

 Reading Numbers from The Console.....85

 Printing Text Using Placeholders85

 Arithmetic Operators86

 Concatenation.....86

 Exam Problems.....87

Chapter 3.1. Simple Conditions101

 Number Comparison..... 101

 Simple If Comparisons 102

 If-Else Conditional Constructs..... 102

 About Blocks of Code..... 103

 The Lifetime of a Variable 105

 Conditional Chaining..... 105

 Problems: Simple Conditions 106

 Debugging - Simple Operations With a Debugger 110

 Problems: Simple Conditions..... 112

Chapter 3.2. Simple Conditions – Exam Problems.....	117
Exam Problems.....	117
Problem: Transport Price.....	117
Problem: Pipes in Pool	119
Problem: Sleepy Tom Cat.....	122
Problem: Harvest	124
Problem: Firm	126
Chapter 4.1. More Complex Conditions	129
Nested Conditions.....	129
More Complex Conditions	131
Logical "AND"	132
Logical "OR"	134
Logical Negation	135
The Parenthesis () Operator	136
More Complex Conditions - Problems	136
What Have We Learned from This Chapter?.....	141
Problems: More Complex Conditions.....	142
Chapter 4.2. More Complex Conditions – Exam Problems	147
Nested Conditions.....	147
If-Elif-Else Conditions	147
Exam Problems.....	148
Chapter 5.1. Loops	164
Repeating Blocks of Code (For Loop)	165
What Have We Learned from This Chapter?.....	172
Problems: Loops.....	173
Lab: Graphics and Web Applications	176
Chapter 5.2. Loops – Exam Problems	183
Problem: Histogram.....	183
Problem: Smart Lily	187
Problem: Back to The Past.....	190
Problem: Hospital	192
Problem: Division.....	194
Problem: Logistics.....	196

Chapter 6.1. Nested Loops	199
Nested Loops	200
Drawing More Complex Figures.....	205
What Have We Learned from This Chapter?	211
Lab: Drawing Figures in a Web Environment.....	212
Chapter 6.2. Nested Loops – Exam Problems.....	218
Problem: Draw Fort.....	219
Problem: Butterfly	221
Problem: Stop.....	223
Problem: Arrow.....	225
Problem: Axe	227
Chapter 7.1. Complex Loops.....	230
Loops with a Step	231
While Loop.....	233
Greatest Common Divisor (GCD).....	235
The Euclidean Algorithm	235
While True + break loop.....	236
Infinite Loops and the break Operator.....	239
Nested Loops and the break Operator	241
Dealing with Exceptions: try-except	243
Problems with Loops.....	244
What Have We Learned from This Chapter?	248
Problem: Web Applications with More Complex Loops.....	249
Chapter 7.2. Complex Loops – Exam Problems	262
Problem: Stupid Password Generator.....	263
Problem: Magic Numbers.....	265
Problem: Stop Number	268
Problem: Special Numbers	270
Problem: Digits.....	271
Chapter 8.1. Practical Exam Preparation – Part I.....	275
The “Programming Basics” Practical Exam	275

The Online Evaluation System (Judge)	275
Problems with Simple Calculations.....	275
Problems with Simple Conditions.....	279
Problems with Complex Conditions	282
Problems with Simple Loops.....	286
Problems for Drawing Figures on The Console	290
Problems with Nested Loops	294
Chapter 8.2. Practical Exam Preparation – Part II.....	299
Problem: Distance	299
Problem: Change Tiles	302
Problem: Flowers	304
Problem: Grades	307
Problem: Christmas Hat.....	309
Problem: Letters Combinations.....	312
Chapter 9.1. Problems for Champions – Part I.....	315
More Complex Problems on The Studied Material	315
Problem: Crossing Sequences	315
Problem: Magic Dates.....	319
Problem: Five Special Letters	323
Chapter 9.2. Problems for Champions – Part II.....	328
More Complex Tasks on The Studied Material	329
Problem: Passion Days	329
Problem: X Expression	333
Problem: Bulls and Cows.....	338
Chapter 10. Functions.....	343
What is a Function?	343
Functions with Parameters (Complex Functions)	347
Returned Value by a Function.....	351
Variations of Functions.....	354
Problem: Greater of Two Values	355
Nested Functions (Local Functions)	356
Classic Approaches in The Use of Functions	357

What Have We Learned from This Chapter?	359
Problems.....	360
Chapter 11. Tricks and Hacks	369
Code Formatting.....	369
Naming Code Elements	371
Shortcuts in PyCharm	372
Code Snippets	373
Code Debugging Techniques	376
A guide to tricks	378
What Have We Learned from This Chapter?	380
Conclusion.....	381
This Book is Only the First Step!	381
What to Do After Reading This Book?	382
Online Communities for Beginners in Programming.....	383
Good Luck to All of You!	384

Preface

The book introduces readers to writing **programming code** at a beginner's level (basic coding skills), working the **development environment** (IDE), using **variables** and data, **operators** and **expressions**, working with the **console** (reading input data and printing output), usage of **conditional statements** (**if**, **if-else**, **if-elif-else**), **loops** (**for**, **while**) and working with **functions** (declaring and calling functions, passing parameters, and returning values). The book uses **Python** programming language and **PyCharm** development environment. The covered training material provides basic preparation for a deeper study of programming.



This book only gives you **the first steps to programming**. It covers very basic skills that you must develop for years, to reach a high enough level and start working as a programmer.

The book is also used as an unofficial **textbook** for school-level programming courses in the **high schools**, studying professions like "Programmer", "Application Programmer" and "System Programmer".

Who is This Book Intended for?

This book is suitable for **complete beginners in programming**, who want to find out what programming is and learn the main constructions for writing programming code that is used in software development. The book provides a **solid basis** of practical skills that are used for further study in programming and software development.

Why Did We Choose the Python Language?

For this book, we chose **Python**, because it is a **modern** language for high-level programming and at the same time it is easy to learn and suitable for **beginners**. As a use **Python** is **widespread**, with a well-developed ecosystem, numerous libraries, and technological frameworks, and therefore gives many **prospects** for development. **Python** combines the paradigms of procedural, functional, and object-oriented programming in a modern way with easy-to-use syntax. In the book, we'll use the **Python language** and the **PyCharm** development environment, which is available for free from JetBrains.

As we will explain more about later, the programming language you start with does not make a significant difference in the process of learning to code, but you still need to choose one. For this book, we will focus specifically on Java. You can find the principles and concepts explained here illustrated with other programming languages like C#, JavaScript, and Python [here](#).

The Book in Other Programming Languages: Java, JavaScript, C#, C++

This programming book for complete beginners is available in several programming languages (or is in the process of being adapted for them):

- [Programming Basics with Java \(English\)](#)
- [Programming Basics with JavaScript \(English\)](#)
- [Programming Basics with C# \(English\)](#)
- [Programming Basics with Python \(English\)](#)
- [Programming Basics with JavaScript \(Bulgarian\)](#)
- [Programming Basics with Python \(Bulgarian\)](#)
- [Programming Basics with C# \(Bulgarian\)](#)
- [Programming Basics with Java \(Bulgarian\)](#)
- [Programming Basics with C++ \(Bulgarian\)](#)

If you prefer a different language, choose from the list above.

Programming is Learned by a Lot of Writing, Not Reading!

If anyone thinks they're going to read a book and learn to program without writing a code and solve tasks hard, they're delusional. Programming is learned with **many, many practices**, with writing code every day and solving hundreds, even thousands of tasks, seriously and with perseverance, for years.

You need to **solve a lot of problems**, to make mistakes, to fix them, search for solutions and information on the Internet, to try, to experiment, to find better solutions, get used to the code, syntax, programming language, development environment, error search and debugging of non-working code, task reasoning, algorithmic thinking, breaking down problems in steps and implementing every step, gaining experience and improving your skills every day, because learning to write code is just **the first step towards the profession of a software engineer**. You have a lot to learn!

We advise the reader, as a minimum, **to try all the examples from the book**, to interact with them, to change and test them. Even more important than the examples **are the exercises** because they develop the practical skills of a developer.

Solve all the tasks in the book, because programming is learned by practice! The tasks after each topic are carefully selected to cover in-depth the included learning material. The purpose of solving all tasks from all the topics covered is to give **complete writing skills to a program code** at the beginners' level (as is the purpose of this book).



Solve all the exercises in the book. Otherwise, you won't learn anything! Programming is learned by writing a lot of code and solving thousands of problems!

The Software University (SoftUni)

[The Software University \(SoftUni\)](#) is the largest training center for software engineers in South-Eastern Europe. Tens of thousands of students pass through the university every year. SoftUni was founded in 2014 as a continuation of the hard work of [Dr. Svetlin Nakov](#) in training **skillful software engineering professionals** by a practical, contemporary and high-quality education that combines fundamental knowledge with modern software technologies and a lot of practice.

SoftUni: High-Quality Practical Tech Education

The Software University provides **quality education, profession, job and diploma** for programmers, software engineers and IT professionals. SoftUni builds an extremely successful and strong **connection between education and industry** by collaborating with hundreds of software companies, provides job and internships of its students, creates quality professionals for the software industry, and directly responds to the needs of employers via the training process.

The SoftUni Interactive Classroom

SoftUni's self-paced programs teach software developers worldwide in the innovative **Interactive Classroom** that combines **video lessons, coding sessions, examples, and exercises** in a single platform on the Web.

Using the **SoftUni Interactive Platform**, you **write, execute and test code directly into the browser** and your exercise solutions are automatically evaluated by the integrated **Judge system**. If you have difficulties with an assignment, you can **get assistance** in multiple ways: with automated hints and guidelines or through SoftUni's **help center**.

This is how the **SoftUni Interactive Classroom** looks like:

Statements

- The **actions** that a program takes, are expressed as **statements**
- Common **statements** (actions / commands) include:
 - Declaring a variable**
`int counter;`
 - Assigning a value**
`counter = 1;`
 - Declaring + initializing**
`int counter = 1;`
 - Printing a value**
`System.out.println(counter);`
 - Modifying a value**
`counter++;`
`sum = a + b;`

Calculate Speed

```
1 import java.util.Scanner;
2
3 public class Program {
4     public static void main(String[] args) {
5         // Write code here
6     }
7 }
```

Description

Create a program that calculates the speed of a moving vehicle given the distance and time travelled.

- Read 2 floating-point numbers: distance and time
- Calculate the speed at which the vehicle needs to be traveling in order to get from point A to point B for the specified time.
- Print out the result
- Use the formula: $\text{speed} = \text{distance} / \text{time}$

Example

Input	Output
15	7.5
2	

Video: SoftUni and SoftUni Judge

Watch a video lesson about SoftUni and the SoftUni Judge system here:

<https://youtube.com/watch?v=IwLET8SDBE4>.

The Automated Judge System

The SoftUni Judge system (<https://judge.softuni.org>) is an automated Internet system for checking the solutions of programming exercises via series of tests. The submission and verification happen in **real time**: you submit the solution and within seconds you get an

answer whether it is correct. You earn points for each task depending on your answer. For a completely right one, you get the maximum points for the problem. With a partially correct solution, you receive a fraction of the points. And with a completely wrong answer, you get 0 points. This how the **SoftUni Judge** looks like:



02. Expression

```

1 using System;
2
3 namespace Expression
4 {
5     class Program
6     {
7         static void Main (string[] args)
8         {
9             Console.WriteLine((3522 + 52353) * 23 - (2336*501 + 23432 - 6743) * 3);
10        }
11    }
12 }

```

Allowed working time: 0.100 sec.
 Allowed memory: 16.00 MB
 Size limit: 16.00 KB
 Checker: Numbers Checker ?

C# code Submit

All problems from the current book are available for testing in **SoftUni Judge** and we strongly recommend testing them after you solve them to be sure you don't miss anything and that your solution works correctly according to the task requirements.

Keep in mind some **specifics about SoftUni Judge**:

- For each task the **Judge system keeps the best score you had**. Therefore, if you upload a solution with wrong code or lower score compared to the previous one, the system won't take away your points.
- The output of your program is **compared** by the system to a strictly expected result. Every **unnecessary symbol, missing comma or space** may lead to 0 points on a particular test. The **output** that the Judge system expects is **described in the requirements of every task** and **nothing else should be added**.
- **Example:** If the output requires to print a number (ex. **25**), do not display any descriptive messages such as **The result is: 25**, just print as it is required, i.e. only the number.

The SoftUni Judge system is **available any time** via its website: <https://judge.softuni.org>.

- Using the SoftUni system is **free** and it's not bound with the participation in SoftUni's courses.

We are convinced that after sending a few tasks **you will like getting instant feedback** for your solutions and the Judge system will become your favorite assistant in your programming practice.

How to Become a Software Developer?

Dear readers, probably many of you have the ambition to become programmers and develop software for a living, or work in the IT area. That's why we have prepared for you a **short guide on "How to Become a Software Developer"**, so we can help you take the first steps towards this desired profession.

You can become a programmer (working in a software company) after **at least 1-2 years of intensive training and coding every day**, solving thousands of programming tasks, development of several more serious practical projects and gaining a lot of experience with code writing and software development. You can't become a programmer for a month or two! The profession of software engineer requires **a lot of knowledge**, covered with **a lot of practice**.

Video: Become a Software Engineer – 4 Essential Skills

Watch a **video lesson** about the 4 essential developer skills, needed to become a developer: <https://youtu.be/Ds5PD3UW57k>.

The 4 Essential Skills of the Software Developers

There are **4 main skill groups** where all programmers must have. Most of these skills are resistant in time and are not influenced by the development in specific technologies (that are changing constantly). These are the skills that **any good programmer** has and to which every beginner must strive:

- Coding (20%)
- Algorithmic thinking (30%)
- Computer science and software engineering concepts (25%)
- Languages and software technologies (25%)

Skill #1 – Coding (20%)

Writing code forms around 20% of the minimum knowledge and skills of a programmer, needed for starting a job in a software company. The skill of coding includes the following components:

- Working with variables, conditional statements, loops
- Using functions, methods, classes and objects
- Data manipulation: arrays, lists, hash tables, strings

The skill of coding **can be acquired in a few months** of hard learning and solving practical problems by writing code every day. This book covers only the first point of coding: **working**

with variables, conditional statements and loops. The rest remains to be learned in follow-up trainings, courses and books.

The book (and the courses based on it) gives only the beginning of one long and serious training on the way to professional programming. If you don't learn perfectly the material from this book, you can't become a programmer. You are going to miss fundamentals and it will be harder in the future. For this reason, **give enough time to programming basics**: solve many problems and write code every day for months until you learn to **solve every problem from the book very easily**. Then go ahead.

We specifically note that **the programming language does not matter** for the ability to code. You can code or not. If you can code with C#, you will easily learn to code with Java, C++ or other languages. That's why **the coding skills** are being studied quite seriously at the in all [SoftUni software engineering programs](#), and each programming book for beginners starts with them, including this one.

Skill #2 – Algorithmic Thinking (30%)

The algorithmic (logical, engineering, mathematical, abstract) thinking forms around 30% of the minimum skills of a programmer needed to start the profession. **Algorithmic thinking** is the ability to break a particular problem into a logical sequence (algorithm), to find a solution for every separate step and then assemble the steps into a working solution. That is the most important skill of any programmer.

How to **build algorithmic thinking**?

- Algorithmic thinking is developed by solving **multiple programming (1000+) problems**, as diverse as possible. That is the recipe: solving thousands of practical problems, building algorithms, and implementing the algorithms, along with debugging the issues that come up in the process.
- Sciences like physics, mathematics and similar ones helps a lot, but they are not mandatory! People with **engineering and technical skills** usually learn very easily to think logically, because they already **have problem solving skills**, even if it is not algorithmic.
- The ability of **solving programming problems** (for which algorithmic thinking is needed) is extremely important for a programmer. Many companies test particularly this skill during their job interviews.

The current book develops the **beginner level of algorithmic thinking**, but it's not enough to make you a good programmer. To become good at this profession you must add **logical thinking and problem-solving skills**, beyond the range of this book. For example, working with **data structures** (arrays, lists, matrices, hash-tables, binary trees) and basic **algorithms** (searching, sorting, searching in tree structures, recursion, etc.).

Algorithmic thinking skill can be developed while studying in the [Software Developer programs at SoftUni](#), as well as in specialized algorithmic courses data structures and algorithms.

As you may have guessed, the choice of **programming language does not matter** for the development of algorithmic thinking. It is a skill unrelated to Programming. Because of their

well-developed logical thinking, there is the misconception that all programmers are smart people and having a high IQ is a requirement for entering into the profession.

Skill #3 – Computer Science and Software Engineering Concepts (25%)

Fundamental knowledge and skills for programming, software development, software engineering and computer science comprise around 25% of the developer's minimum skills to start a job. Here are the more important of these skills and knowledge:

- **Basic mathematical concepts** related to programming – coordinate systems, vectors and matrices, discrete and non-discrete mathematical functions, state automata and state machines, combinatorics and statistics concepts, algorithm complexity, mathematical modeling, and others.
- **Programming skills** – code writing, working with data, using conditional statements and loops, working with arrays, lists and associative arrays, strings and text processing, working with streams and files, using programming interfaces (APIs), working with IDE, debugger, developer tools, etc.
- **Data structures and algorithms** – lists, trees, hash-tables, graphs, search, sorting, recursion, binary search trees, etc.
- **Object-oriented programming (OOP)** – working with classes, objects, inheritance, polymorphism, abstraction, interfaces, data encapsulation, exceptions management, design patterns.
- **Functional programming (FP)** – working with lambda functions, higher order functions, functions that return a function as a result, closure, etc.
- **Databases** – relational and non-relational databases, database modeling (tables and links between them), SQL query language, object-relational mapping (ORM), transactions and transaction management.
- **Network programming** – network protocols, network communication, TCP/IP, concepts, tools and technologies from computer networks.
- **Client-server interaction**, peer to peer communication, back-end technologies, front-end technologies, MVC architectures.
- **Technologies for server-side development (back-end)** – Web server architecture, HTTP protocol, MVC architecture, REST architecture, web development frameworks, templating engines.
- **Front-end technologies (client-side development)** – HTML, CSS, JS, HTTP, DOM, AJAX, communication with back-end, calling REST API, front-end frameworks, basic design and UX (user experience) concepts.
- **Mobile technologies** – mobile apps, Android and iOS development, mobile user interface (UI), calling server logic.

- **Embedded systems** – microcontrollers, digital and analog input and output control, sensor access, peripheral management.
- **Operating systems (OS)** – working with operating systems (Linux, Windows, etc.), installation, configuration and basic system administration, process management, memory, file system, users, multitasking, virtualization and containers.
- **Parallel and asynchronous programming** – thread management, asynchronous tasks, promises, common resources, and access synchronization.
- **Software engineering** – source control systems, development management, task planning and management, software development methodologies, software requirements and prototypes, software design, software architectures, software documentation.
- **Software testing** – unit testing, test-driven development, QA engineering, error reporting and error tracking, automation testing, build processes and continuous integration.

We need to once again mention that **the programming language does not matter** for acquiring all these skills. You will accumulate them slowly, over years. Some fundamental knowledge can be learned theoretically, but to get an in-depth understanding of it you need a lot of practice.

Fundamental knowledge and skills for programming, software development, software engineering, and computer science are taught during the [Software Developer Program at SoftUni](#), as well as a number of additional training courses. Working with a variety of software libraries, APIs, frameworks and software technologies and their interaction gradually builds this knowledge and skills, so do not expect that you will understand them from a single course, book or project.

Having **basic knowledge in the areas listed above** is enough to start a job as a programmer. Your further improvement will come as a result of the technology and development tools you will use in your day-to-day work.

Skill #4 – Programming Languages and Software Technologies (25%)

Programming languages and software development technologies form around 25% of a programmer's skills. They have the largest learning content, but they change very fast over time. If we look at the **job offers** in the software industry, they usually mention words like the ones below, but in the job offers they secretly mention **the three main skills**: coding, algorithmic thinking and knowing the fundamentals of computer science and software engineering.

For those clearly technological skills **the programming language does matter**.

- **Note**: only for these 25% of the profession the programming language does matter!
- **For the rest 75% of the skills the programming language doesn't matter**, and these skills are resistant in time and transferable between different languages and technologies.

Here are some commonly used software development stacks which software companies are looking for (as of Jan 2019):

- **C#** + OOP + FP + classes from .NET + SQL Server databases + Entity Framework (EF) + ASP.NET MVC + HTTP + HTML + CSS + JS + DOM + jQuery + cloud + containers.
- **JavaScript (JS)** + OOP + FP + databases + MongoDB or MySQL + HTTP + web programming + HTML + CSS + JS + DOM + jQuery + Node.js + Express + Angular or React + cloud + containers.
- **Python** + OOP + FP + databases + MongoDB or MySQL + HTTP + web development + HTML + CSS + JS + DOM + jQuery + Django or Flask + cloud + containers.
- **Java** + Java API classes + OOP + FP + databases + MySQL + HTTP + web programming + HTML + CSS + JS + DOM + jQuery + JSP / Servlets + Spring MVC or Java EE / JSF + cloud + containers.
- **PHP** + OOP + databases + MySQL + HTTP + web development + HTML + CSS + JS + DOM + jQuery + Laravel or Symfony or other MVC framework for PHP + cloud + containers.
- **C++** + OOP + STL + Boost + native development + databases + HTTP + other languages and technologies.
- **Swift** + OOP + MacOS + iOS + Cocoa + Cocoa Touch + XCode + HTTP + REST + other languages and technologies
- **Go** + OOP + Linux + gRPC + cloud + containers + other languages and technologies.

If the words above look scary and incomprehensible, then you are at the very beginning of your career and you need **many years of learning** until you reach the profession of a software engineer. Do not worry, every programmer goes through one or several technology stacks and needs to study **a set of interconnected technologies**, but it is imperative for you is know how to write **programming logic** (to code) and **think algorithmically** (to solve programming problems). Becoming a good software engineer is impossible without those skills!

The Programming Language Doesn't Matter!

As it already became clear, **the difference between programming languages** and more specifically between the skills of developers in different languages and technologies forms around **10-20% of the skills**.

- All programmers have around **80-90% of the same skills** that do not depend on the programming language! These are the skills to program and to design and develop software, that are very similar in different programming languages and development technologies.
- The more languages and technologies you know, the faster you will learn new ones, and the less you will feel the difference between them.

Let us state once again that the **choice of programming language (mostly) does not matter**, you just need to learn to program. This process starts with **coding** (by reading this book or enrolling in the [SoftUni Software Developer program](#)), continues with mastering more complex **programming concepts** (like data structures, algorithms, OOP, and FP), and includes

using **fundamental knowledge and skills for software development, software engineering, and computer science**.

Only when you start working with a specific technology into a software project you will need **a specific programming language**, knowledge about specific programming libraries (APIs), frameworks and software technologies (front-end UI technologies, back-end technologies, ORM technologies, etc.). Keep calm, you will learn them, all programmers are learning them, but first you need to learn the foundation: **to program and do it well**.

This book uses the C# language, but it is not required and can be replaced with Java, JavaScript, Python, PHP, C++, Ruby, Swift, Go, Kotlin, or any other language. To be a **software developer**, you need to learn **coding** (20%), learn **algorithmic thinking**, and **solve problems** (30%), to have **fundamental knowledge of programming and computer science** (25%) and to master a **specific programming language and the technologies around it** (25%). Be patient, for a year or two all this can be mastered on a good starting level, if you are serious.

The History Behind This Book

The main engine and project manager for the creation of the current **free programming book for beginners** with open source is [Svetlin Nakov, PhD](#).

In 2014, the SoftUni initiative was launched. In the beginning, it had a wider reach and included more theory, but in 2016 Svetlin Nakov, PhD completely revised them, renovated, simplified, and **directed them to be more practical**. This is how the **learning content core of this book was created**.

SoftUni's free programming start-up trainings are probably the largest ever conducted in Bulgaria. On the principle of free software and free knowledge, Svetlin Nakov led a **team of volunteers** and started this open-source project, initially to create a book based on programming with the C# language and later with other programming languages.

Authors Team

This book is developed by a broad author's team of **volunteers** who dedicated their time to give away the systematized knowledge and guide you at the start of programming. Below is a list of the main book authors (in alphabetical order):

Boncho Vulkov, Hristo Minkov, Iliya Iliev, Martin Tsarev, Miglen Evlogiev, Milena Angelova, Mirela Damyanova, Nikolay Kostov, Petar Ivanov, Petya Gospodinova, Svetlin Nakov, Tanya Evtimova, Tanya Staneva, Teodor Kurtev, Ventsislav Petrov, Vladimir Damyanovski, Yordan Darakchiev

A large team of authors has a significant contribution to this book. Below is a list of the additional book contributors (editors, translators, others):

Aleksander Krastev, Aleksander Lazarov, Aleksander Peev, Aleksandra Stoycheva, Angel Dimitriev, Ariet Motzeva, Daniel Tsvetkov, Denis Milanov, Dimitar Dzhapunov, Dimitar Tatarski, Dimo Dimov, Diyan Tonchev, Dzhaner Hasan, Elena Rogleva, Hristiyan Hristov, Hristo Hristov, Iskra Nikolova, Ivelin Kirilov, Jordan Liubenov, Julieta Atanasova, Kalin Primov, Kaloyan Nenchev, Kristiyan Pamidov, Luboslav Lubenov, Martin Zhelev, Mihail Georgiev, Natali Dimitrova, Nikola Slavchev, Nikolay Bankin, Nikolay Dimov, Pavlin Petkov,

Petar Ivanov, Rositsa Nenova, Ruslan Filipov, Simeon Stavrev, Stefka Vasileva, Svetlin Nakov, Teodor Kurtev, Tonyo Zhelev, Tsvetan Iliev, Vasko Viktorov, Venelin Bekyarov, Ventsislav Petrov, Yanitsa Vuleva, Yulian Linev, Zahariya Pehlivanova, Zhivko Nedyalkov.

Book cover design: Marina Shiderova.

The book is written in the period June-October 2018.

Official Website of The Book

The current book on **basics of Python programming for beginners** is available for free use on the Internet from: <https://python-book.softuni.org>.

This is **the official website of the book** and there will be uploaded its latest version.

License and Distribution

The book is distributed **freely** in electronic format under an open license [CC-BY-NC-SA](#). The book is published and distributed **on paper** by SoftUni, and you can buy a hard copy from online bookstores like Amazon.

The **source code** of the book can be found in its official GitHub repo: <https://github.com/SoftUni/Programming-Basics-Book-Python-EN>.

International Standard Book Number (ISBN): **978-619-00-1403-4**.

The Book That Helps Teachers

If you are a **teacher of programming**, informatics, or information technology or want **to teach programming**, this book gives you more than a well-structured learning material with many examples and tasks. **Free of charge** with the book you receive **quality educational content** for teaching in school, in the **Bulgarian language**, by the school requirements:

- **Educational presentations** (PowerPoint slides) for each learning topic tailored to 45-minute hours in schools – free of charge.
- Well-designed **tasks** for class and homework, with detailed conditions and sample entrance and exit – free of charge.
- **An automated task and homework verification system** (Online Judge System) to be used by students, also free of charge.
- **Video lessons** with methodological instructions from the free course for programming teachers, which is held regularly by the SoftUni Foundation.

Bug Reports

If you find **bugs**, inaccuracies, or defects in the book, you can report them in the official tracker of the project:

<https://github.com/SoftUni/Programming-Basics-Book-Python-EN/issues>

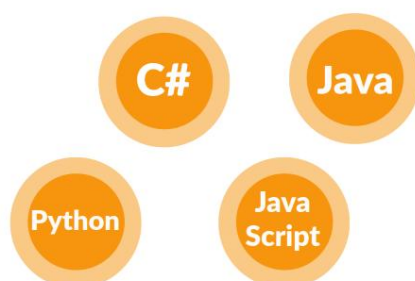
We can't promise that we will edit everything you send us, but we are willing to constantly improve the quality of this book so that the reported errors and all reasonable suggestions will be addressed.

Enjoy Your Reading!

And be sure **to write code** in large quantities, **try the examples** from each topic and, above all, **solve the tasks from the exercises**. You won't learn to program just by reading, so make sure you put time into solving exercises!

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 1. First Steps in Programming

In this chapter, we are going to find out **what programming** is at its core. We are going to become familiar with the idea of **programming languages** and explore the environments for software development (**Integrated Development Environment** – IDEs); we'll see how to work with them, in particular with **PyCharm**. We will write and execute our **first program** written in the programming language Python and then we're going to practice with a few tasks: we will create a console-based program, a graphical application, and a Web application. We will learn how to use **SoftUni's Judge System** to check the correctness of our solutions to the problems in this book. Finally, we'll look at some typical mistakes made during code writing and we're going to learn how to avoid them.

What Does 'To Program' Mean?

To program means to give instructions to the computer, for example, *play a certain sound*, *print something on the screen*, or *multiply two numbers*. When there's a series of commands, we have a computer program (script). The computer program's text is called **program code** (source code or just code).

Computer Programs

Computer programs represent a **series of commands** written in a particular **programming language**, like Python, C#, Java, JavaScript, Ruby, PHP, C, C++, Go, etc. To write commands, we need to be familiar with the **syntax and semantics of the language** we'll be using, in our case – **Python**. In this book, we're going to cover programming in general, as well as Python's particular syntax and semantics. We will examine each step of writing code, starting with the simplest and eventually reaching more complex programming constructs.

Algorithms

Computer programs usually execute some sort of algorithm. **Algorithms** are a series of steps used to complete a task and achieve an expected result – something like a recipe. For example, if we need to fry some eggs, we follow a certain recipe (algorithm): we heat some oil in a pan, then we break the eggs, we wait until they're cooked, and finally, move the pan away from the heat. Similarly, in programming, **computer programs execute algorithms** – a series of commands needed to complete a certain task. If we want to print a series of numbers in ascending order, for example, we need an algorithm. It will go through all the numbers, find the smallest one and print it, then go through the rest of them and do the same until there are no more numbers left. To create programs, write program code (commands), execute it and other operations that have to do with making programming more convenient, we need a **development environment** (IDE) like **PyCharm**.

Programming Languages, Compilers, Interpreters and Development Environments

Programming languages are artificial languages (syntax for expression), designed to **issue commands** which we expect the computer to read, process and execute. With the help of

programming languages, we write a series of commands (**programs**) that **tell the computer what to do**. Execution of computer programs can be achieved by using either a **compiler** or an **interpreter**.

The **compiler** translates code from a programming language into **machine code** and for each construct (command) in the code, it chooses an appropriate, predefined fragment of machine code while simultaneously checking the program's text for **errors**. Together, all the compiled fragments represent the original program translated in machine code, exactly how the computer's microprocessor is expecting it. After it's been compiled, the program can be directly executed by the microprocessor in cooperation with the operating system. Compiled programming languages **compile the program** before executing it and find syntactic errors (incorrect commands) during compile time. Languages like C++, C#, Java, Swift and Go work with compilers.

Some programming languages don't use a compiler but are **directly interpreted** by specialized software called an interpreter. The **interpreter** is a **program that executes programs**, written in some programming language. It executes the program's commands in sequence and understands not only single commands and series of commands but also other language constructs (checks, iterations, functions, etc.). PHP, Python and JavaScript are languages that work with an interpreter and are executed without the need to be compiled. Due to lack of preliminary compilation, interpreted languages' errors are found during run time, after the program has already started working.

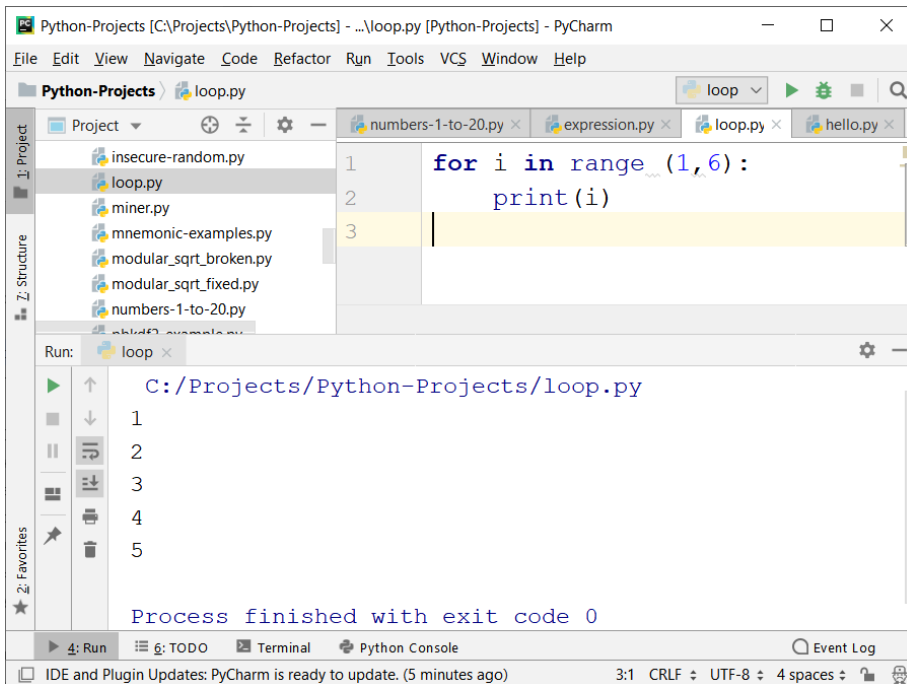
Overall, compiled languages work slower than their interpreted counterparts. The difference in execution speed is sometimes insignificant, but in other cases massive. In interpreted programming languages we can change code quicker and re-execute (to remove a mistake, for example), while compiled languages are slower after a code change has been introduced, due to the need for compilation.

The **development environment** (Integrated Development Environment – IDE) combines traditional instruments for software development. In the development environment we write code, compile and execute programs. Development environments incorporate a text editor for writing code, a **programming language, a compiler or interpreter**, an **execution environment** where our program is executed, a **debugger** that traces our program and looks for errors, **instruments for design and user interface** as well as other instruments and add-ons.

Development environments are convenient because they incorporate everything necessary to develop a program, so there's no need to leave the environment. If we don't use a development environment, we'll have to write code in a text editor, compile it with a command from the console, execute it with another command from the console and write additional commands, when needed, which would cost a lot of time. This is why most programmers use IDEs for everyday work.

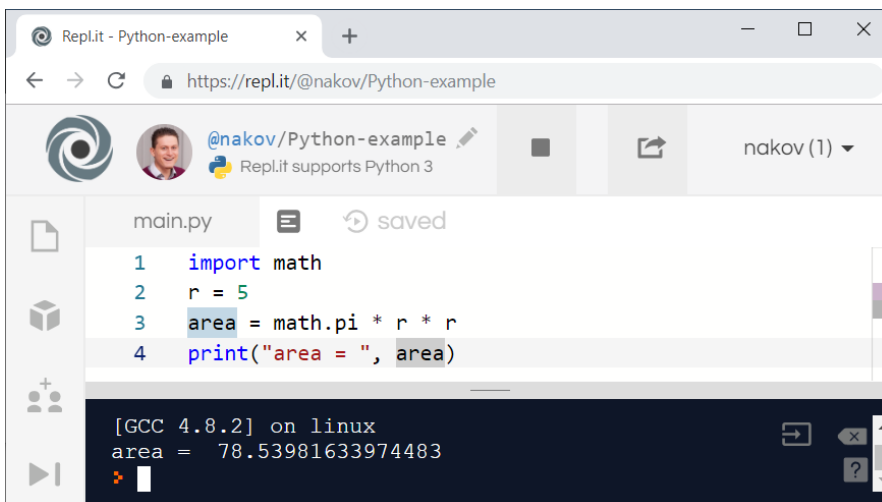
Programming with **Python** is usually done in the development environment **PyCharm** which is developed and distributed by JetBrains. You can download it from:

<https://www.jetbrains.com/pycharm>. Here's what the PyCharm environment looks like in action:



Alternatives to PyCharm for writing code in Python are **Visual Studio Code** (<https://code.visualstudio.com>), **Atom** (<https://atom.io>), **Eclipse for Python** (<https://www.eclipse.org>) together with the add-on **PyDev** (<https://pydev.org>) and others. In this book, we'll use the development environment PyCharm.

Alternatives to desktop development environments (IDE) are online environments for Python programming such as **Repl.it** and **PythonAnywhere**. They're great for testing short examples, when we cannot install an IDE locally, or when we want to easily share our code with colleagues. Here's an example of how to execute Python code in Repl.it:



Low-Level, High-Level Languages and Runtime Environments

A program is, in essence, a **set of instructions** that ask the computer to carry out certain tasks. They are entered by the programmer and **executed unconditionally by the machine**.

There are different types of **programming languages**. **Lowest-level** languages can be used to write the very **instructions commanding the processor** - **Assembler** is one such language. Higher-level languages can be utilized to create an operating system, drivers for managing hardware (video card drivers, for example), web browsers, compilers, engines for game graphics (game engines) and other system components and programs. Even higher-level languages like **JavaScript**, **C#** and **Python** are used to create application software, like programs for reading mail or chatting.

Low-level languages command hardware directly and require a lot of effort and a vast number of commands, to carry out a certain task. **Higher-level languages** require less code to achieve the same result, but have no direct access to hardware. They are used to develop application software like web and mobile applications.

Most of the software we use daily like music players, video players, GPS programs, etc. is written on high-level **languages for application programming** like Python, JavaScript, C#, Java, C++, PHP, etc.

Python is an interpreted language which means we write commands and they're executed immediately after running the program. As a result of that, we will realize we've made a mistake in our code only after we've started our program and reached the incorrect command. In cases like these, **IDEs** like PyCharm are extremely helpful as they check our code while we're writing it and alarm us of potential issues. After we've written our code and want to test it, we can save it in a file with extension **.py**.

Python Interpreter: Installation and Use

To program in Python, we first need to **install a Python interpreter**. It executes commands and runs programs, written in Python, and is essential if we want to use Python on our computer.

Installing The Python Interpreter

Installing Python is quite simple. Just go to <https://python.org/downloads> and download the latest version for your platform.



On **Windows**, the installation is a straightforward process of clicking **[Next]**, **[Next]** and **[Finish]**. On **Linux**, you need to use a package installer like `sudo apt-get install python3`.



Use **Python version 3.x or newer**. Python 2 is outdated technology and although it's accessible by default on many systems, it's problematic and many of the examples in this book won't work on it.

Starting The Python Interpreter

After the Python interpreter has been installed, we can **get it started** and have a tinker at it. If your environment is Windows, use the **[Start]** menu and find the application you just installed – 'Python 3.7 (64-bit)', for instance. On a Linux / MacOS environment, write the command `python3` in the console. Here's what the Python interpreter should look like on Windows:

```
Python 3.7 (64-bit)
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:57:15) [MSC v.1915 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> _
```

Working with The Python Interpreter

Python is an **interpreter** and it can be used in **command mode**: we write a **command**, it's executed and we receive the **result**. The simplest command we can use is to make Python calculate a numeric expression – `2 + 3`, for example:

```
Python 3.7 (64-bit)
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:57:15) [MSC v.1915 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+3
5
>>> _
```

As we can see, the answer is correct: **5**. Let's try a few more commands: store the value **5** in a variable named **a** and print the result of `2 * a`:

```
Python 3.7 (64-bit)
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:57:15) [MSC v.1915 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+3
5
>>> a = 5
>>> 2 * a
10
>>> _
```

Play around with the Python interpreter yourself. Try calculating `1234567890 234567890 34567890`. Did it work? Try using an **incorrect command** (write your name, for example). Try calculating `2 ** 20` and guessing what that might be. Have a go at printing the text *"Hello Python, how are you?"* using the command `print("some text")`. Is it working?

Computer Programs

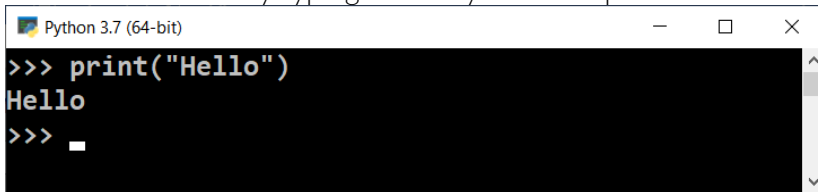
As we already mentioned, a program is a **series of commands**, in other words, it describes a series of calculations, checks, iterations and numerous other operations, which aim at producing a result. The program is written in text format and the text is called **source code**. It's saved in a file with extension `.py` (`main.py`, for instance), and then it can be executed through your browser or the **console** with the help of **Python Shell**. In **script and interpreted languages**, like JavaScript, Python and PHP, the source code is executed step by step by an interpreter.

Problem: A Program That Prints 'Hello'

Let's start with a very basic example of a short **Python program**. It will print the word *"Hello"* like so:

```
print("Hello")
```

We can execute it by typing in the Python interpreter:

A screenshot of a Python 3.7 (64-bit) interpreter window. The window has a title bar with the text "Python 3.7 (64-bit)" and standard window controls (minimize, maximize, close). The main area is a black console with white text. It shows the prompt ">>>" followed by the command "print('Hello')", the output "Hello", and another prompt ">>>" followed by a cursor. There is a vertical scrollbar on the right side of the console area.

Problem: A Program That Plays the Music Tone 'A'

Our next program will consist of a single **Python command** which plays the music tone 'A' (432 hertz) for half a second (500 milliseconds):

```
import winsound  
  
winsound.Beep(432, 500)
```

In a Windows environment we will **hear a sound**. Make sure your speakers aren't muted. On Linux and MacOS, the example won't work.

Problem: A Program That Plays a Series of Music Tones:

We can make our last program a bit more complex by creating a loop that passes a series of notes with increasing height to the command:

```
import winsound

for i in range(200, 4000, 200):
    winsound.Beep(i, 300)
```

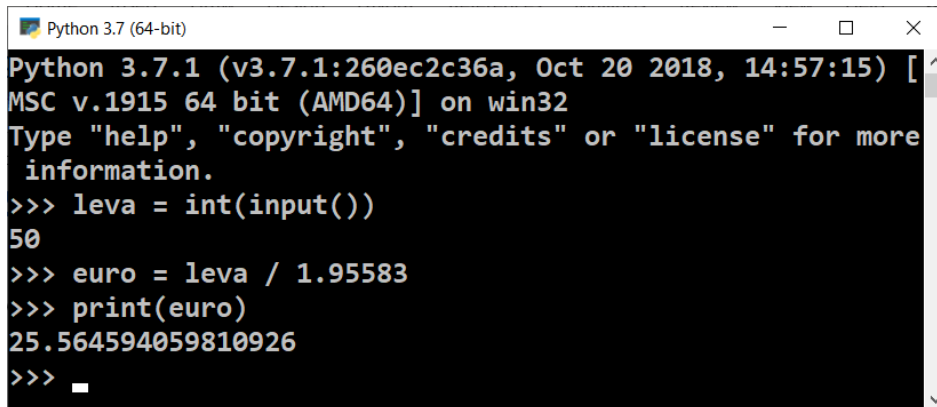
In the example above we make the computer play very briefly (300 milliseconds) each subsequent tone with height 200, 400, 600, etc. hertz until reaching 4000 hertz. The result is something similar to a melody. We will learn how iterations (loops) in programming work in [chapter "Loops"](#), but for now, let's assume that they just repeat a command many times.

Problem: Program That Converts Levs (BGN - Bulgaria's currency) into Euros

Let's have a look at another simple program that reads a number of leva (BGN) from the user, converts them into euro (divides them by the euro exchange rate) and prints the result. This is a program of **three consecutive commands**. Enter and execute them in sequence:

```
leva = int(input())
euro = leva / 1.95583
print(euro)
```

Here's what the **result** of the above series of commands might look like:



```
Python 3.7 (64-bit)
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:57:15) [
MSC v.1915 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> leva = int(input())
50
>>> euro = leva / 1.95583
>>> print(euro)
25.564594059810926
>>> _
```

We looked at **three examples of computer programs**: single command, series of commands in a loop and a sequence of three commands. Let's now move on to the more exciting bit: writing our programs in **Python** and executing them.

How Can We Write a Console Program?

Let's go through the necessary steps to **create and execute a computer program** that uses a text console (window for entering and printing text) to read and write its data. Such programs are called **console programs**. But first of all, we need to **install and prepare the development environment** where we will write and execute the Python programs from this book and the exercises that go along with it.

Development Environment (IDE)

As previously mentioned, we need a **development environment** (IDE) to program. The IDE is a program editor, where we write program code and execute it, discover our mistakes, correct them and start the program again.

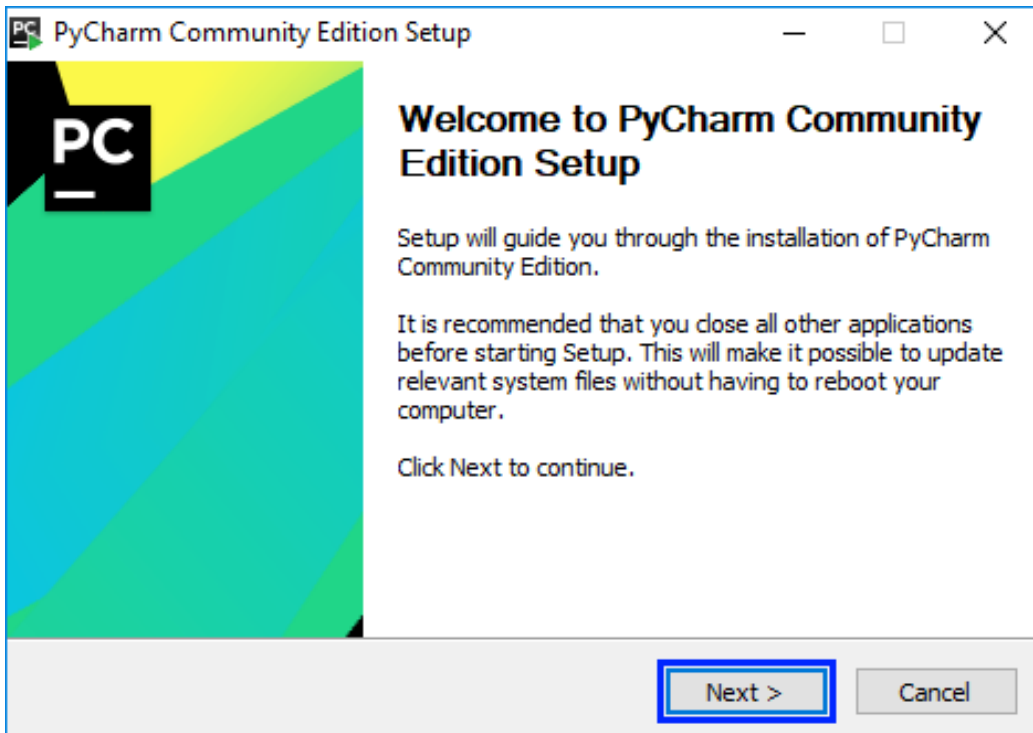
- For programming in Python, we use the **PyCharm** environment, suitable Windows, Linux and MacOS operating systems.
- If we are programming in Java we might use **IntelliJ IDEA**, **Eclipse** and **NetBeans**.
- If we are programming in C#, we might use **Visual Studio**.

Installing PyCharm Community

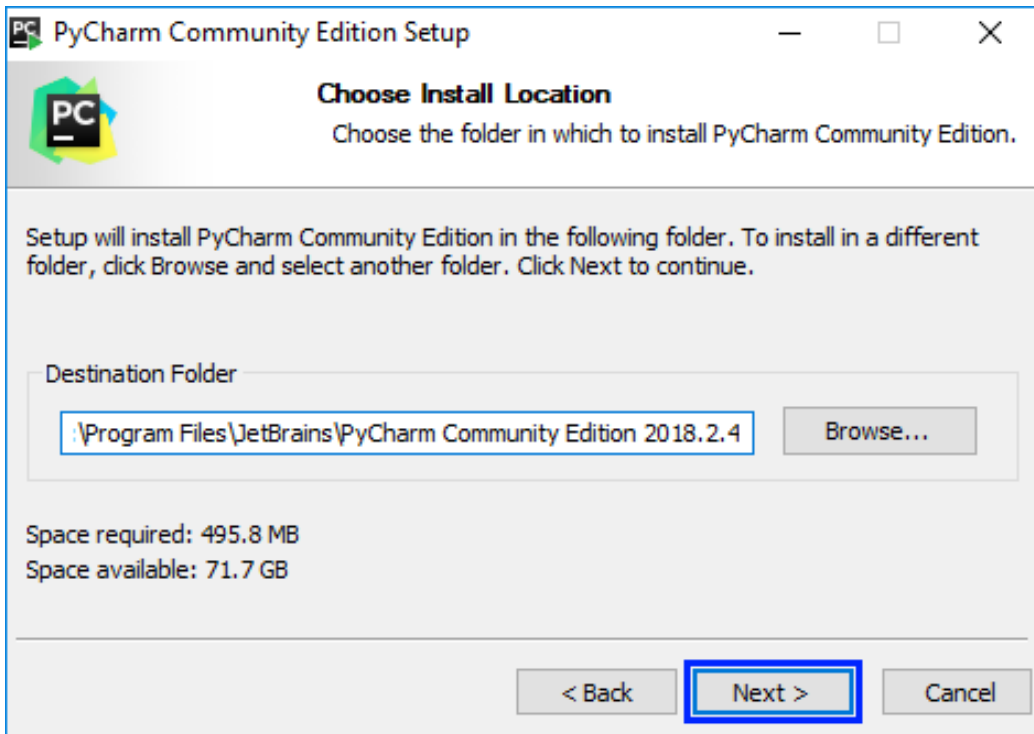
Let's start by installing the **PyCharm Community** integrated environment (in our example we'll be using version 2018, current as at October 2018). The Community version of PyCharm is freely distributed by JetBrains and can be downloaded from:

<https://www.jetbrains.com/pycharm/download>.

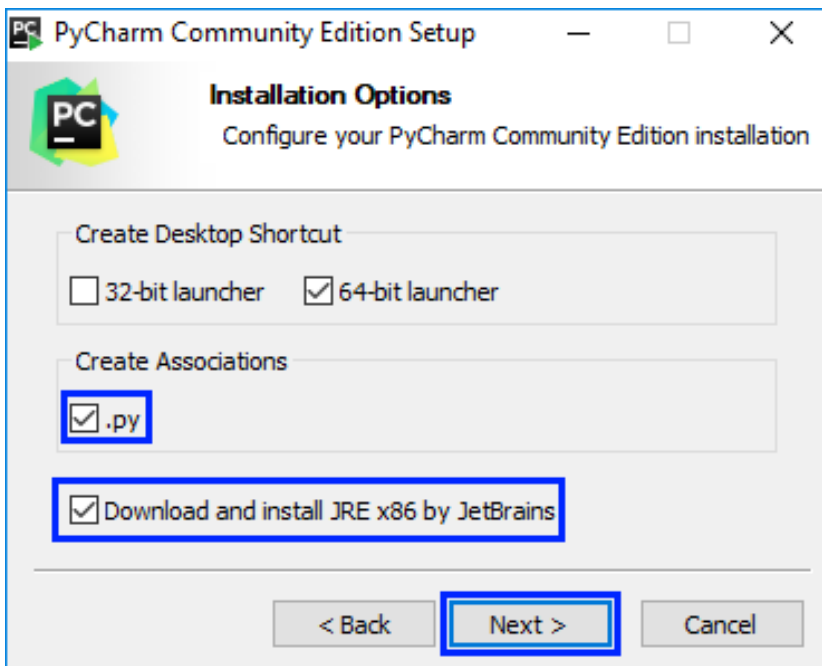
The installation is typical for Windows - simply use the **[Next]** and **[Finish]** buttons. There's a Linux version available as well. In the following paragraphs, we've described in detail **every step of the PyCharm Community 2018 installation process**. After we download the installation file and run it, we're greeted with the following screen:



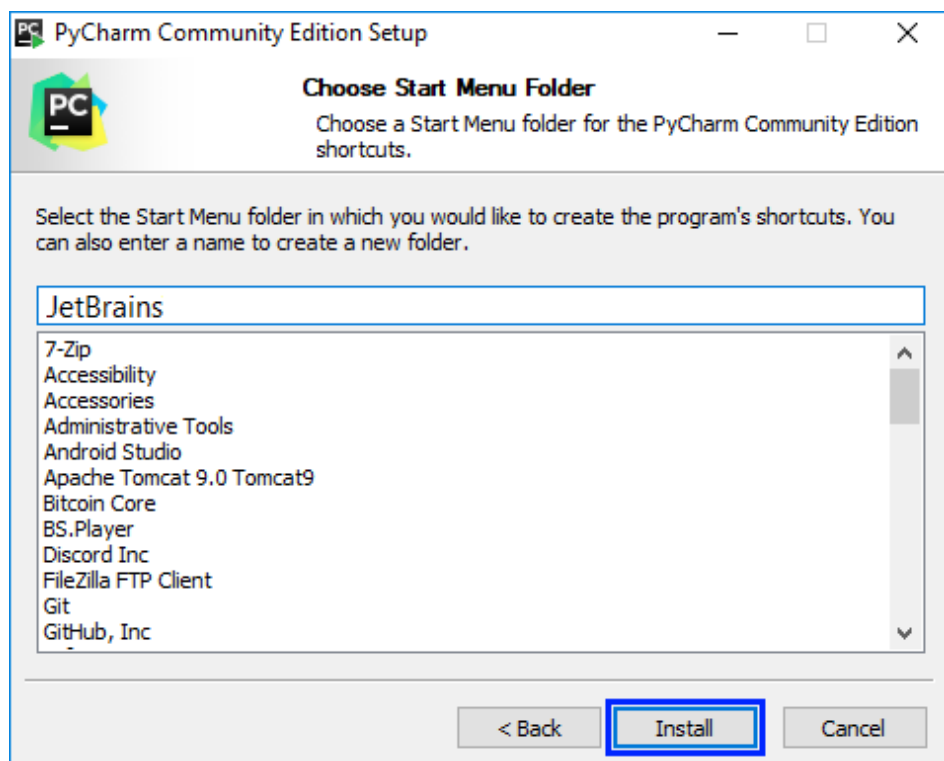
We hit **[Next]**, which takes us to the screen you see below:



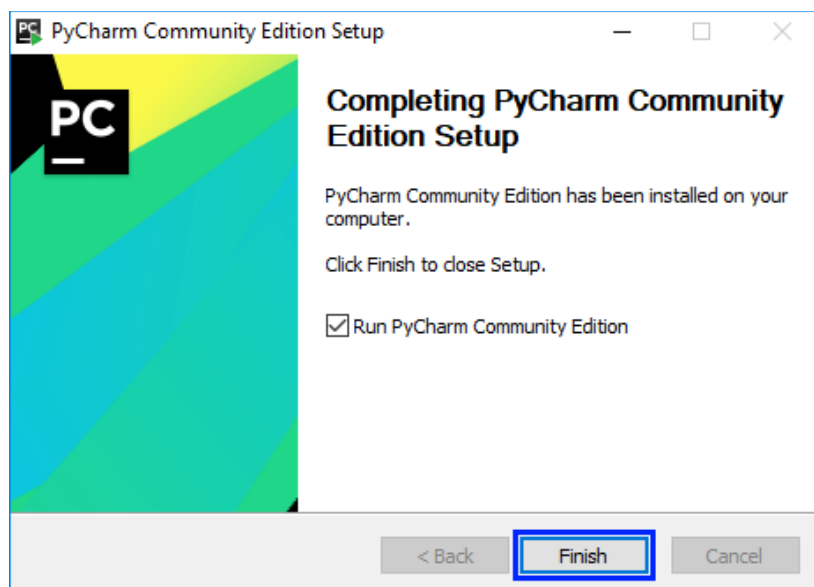
A window with PyCharm's installation panel is loaded:



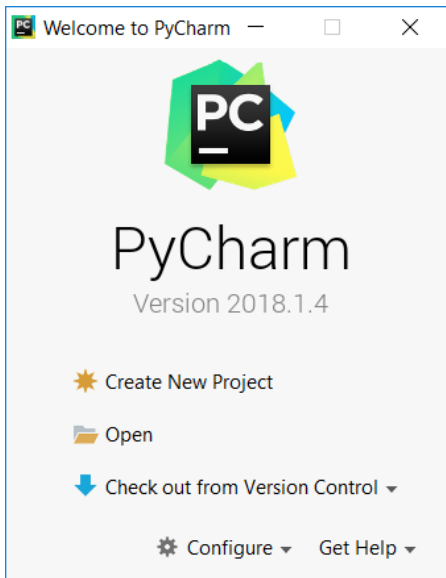
We tick the box which allows us to choose a shortcut depending on our operating system [.py] and [Download and install JRE x86 by JetBrains], and then click [Next]:



Then we hit the [Install] button.



After PyCharm installs, we'll see an information screen, where we can tick a box that says [Run PyCharm] which is going to start our IDE. After PyCharm has started, a screen similar to the one below will be displayed:



That's it. We're ready to start working with PyCharm. **Older Versions of PyCharm**

We could also use older versions of PyCharm (version 2016 or 2012, for example) but it's **not recommended** since they don't have the newest capabilities for development and not all the examples in this book will run on them.

Online Development Environments

There are also **alternative environments to develop online**, directly in your web browser. They aren't very easy to use but if you have no other option, you can start your education with them and install PyCharm later. Here are some links:

- Tutorials Point allows writing and executing code online: https://www.tutorialspoint.com/online_python_ide.php.
- Another useful online development environment for writing and executing Python code is Repl.it: <https://repl.it/languages/python3>.

Projects in PyCharm

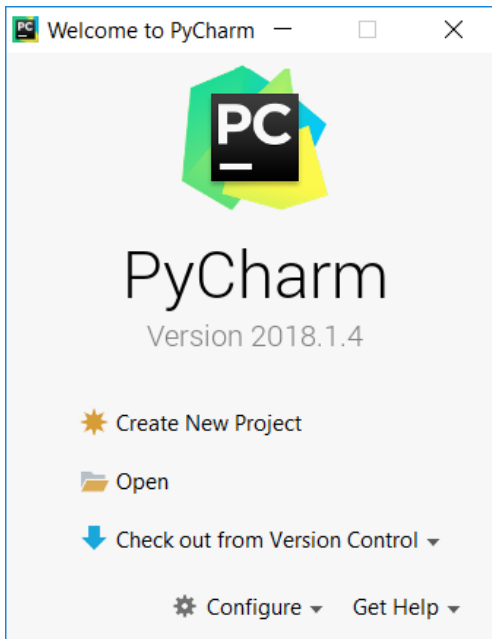
Before we start working with PyCharm, let's get familiar with the term **PyCharm Project** - it represents the project we're currently working on.

In the beginning that will be a console program which we'll learn how to write with the help of this book. After some more thorough studying and with time and practice, these projects will turn into applications, web apps and other projects. The PyCharm project uses **logic to group the numerous files** that make up a certain application or component.

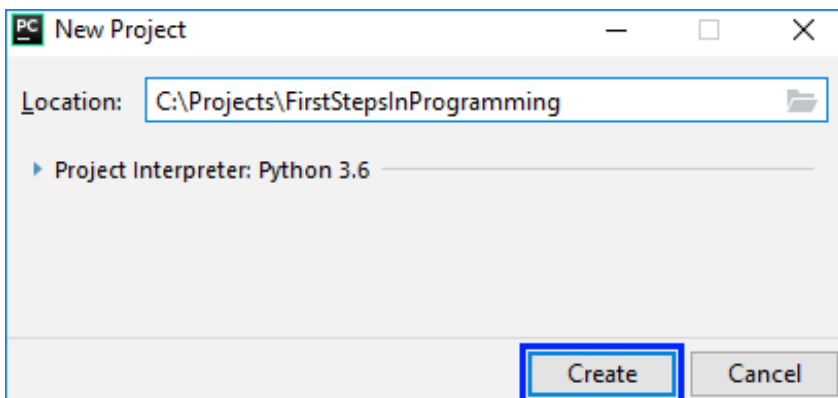
A single **Python project** consists of several **Python source files (.py)**, configuration files and other resources. Every Python source file can have one or more **classes** and **functions**. **Classes** contain **functions**(actions), which are comprised of a **series of commands**. It may seem complicated, but in larger projects, this structure is extremely convenient and allows for the good organization of our work files.

Problem: Creating a Console Program 'Hello Python'

Let's get back to our console program. Now that we have PyCharm installed, we can start it. Then we will create a new project: [Create New Project].



We will give our project a meaningful name, such as `FirstStepsInProgramming`:



PyCharm will create an **empty project** for us and we can add Python files to it. We'll add a new Python file - [File] or right-click on our project -> [New] -> [Python File] and give it a meaningful name like `HelloPython.py`.

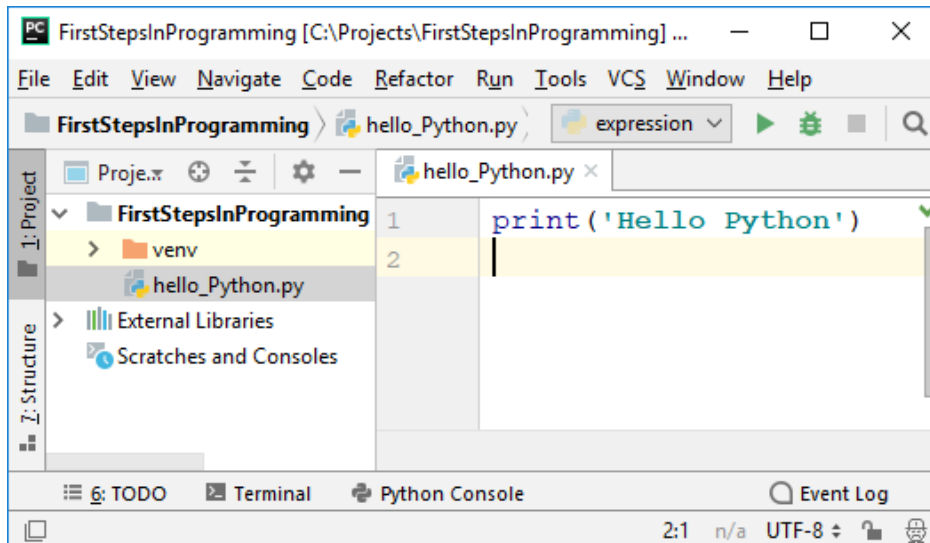
We can adjust the version and location of the Python interpreter, which we're going to be using. We must have a Python interpreter, otherwise, we won't be able to execute the programs we've written. It's possible to have more than one version of Python on the same computer but for ease of use, in the beginning, **just install the latest version and use that**. If you haven't already installed Python, please do so before you continue.

Writing Program Code

Writing Python code requires no additional preparation – creating a file with the `.py` extension is all we need. So let's directly proceed with writing our first line of code. We will write the following command:

```
print('Hello Python')
```

Here's what our program might look like in PyCharm:

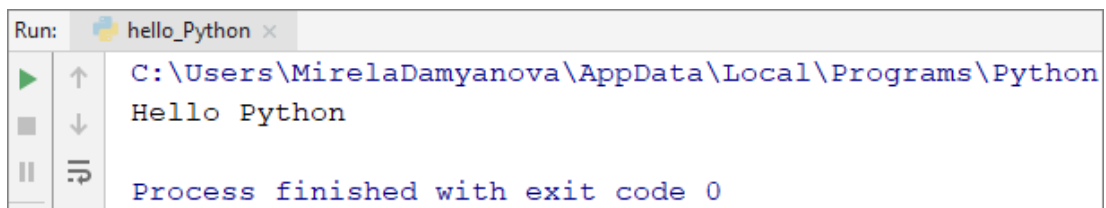


The `print('Hello Python')` command allows us to print the text message **Hello Python** on the console. The message needs to be surrounded with quotes, signifying text.

This command is quite typical for programming: we call the function `print(...)` and pass the text literal `'Hello Python'` as its parameter.

Starting The Program

To start the program, we need to press **[Ctrl + Shift + F10]** or right-click – **[Run]**. If there are no errors, the program will execute. The result will be displayed on the console:



The output from the program is the following text message:

```
Hello Python
```

The `'Process finished with exit code 0'` message is displayed at the end of the PyCharm console after the program has been executed, to signify that the program has finished with no errors.

Testing in The Judge System

Testing the solutions to the problems in this book is completely automated via the **Judge System**'s website: <https://judge.softuni.org>. The solutions are judged immediately by the system. Each solution goes through a series of tests that are hidden; every test that passes grants the user certain points.

The program we just wrote can be tested here: <https://judge.softuni.org/Contests/Practice/Index/1046#0>. To do that, simply paste the entire source code in the black field and choose **Python code** as is shown below:

01. Hello Python

```
1 print('Hello Python')
2
```

Allowed working time: 0.100 sec.
Allowed memory: 16.00 MB
Size limit: 16.00 KB
Checker: Trim ?

Python code

Submit

Then we need to send our solution by clicking the **[Submit]** button. The system returns a result in a few seconds which is displayed in the table of submitted solutions. If necessary, we can hit the 'Refresh' button at the upper right-hand corner of the table of submitted solutions:

Submissions			
⏮ ⏪ 1 ⏩ ⏭			Refresh
Points	Time and memory used	Submission date	
✓ 100 / 100	Memory: 8.20 MB Time: 0.056 s	20:49:18 30.09.2018	Details
✗ 0 / 100	Memory: 8.20 MB Time: 0.071 s	20:48:40 30.09.2018	Details
⏮ ⏪ 1 ⏩ ⏭			

- In the table, the Judge System will display one of the following **possible results**:
- A number of points (between 0 and 100), when the submitted code is compiled successfully (there are no syntactic errors) and can be tested.
 - If our solution is **completely correct**, all tests are marked in green and we receive **100 points**.

- If our solution is **incorrect**, some tests are marked in red and we receive less than 100 or 0 points.
- If there are syntactic errors in our program, we receive a **compile-time error message**.

Test Your Tone-Playing Programs

Now that you've learned how to **execute programs**, you can test the sample note-playing programs you have above. Have fun with them, try out different things. Try changing them and playing with them. Swap the `print('Hello Python')` command with `windsound.Beep(432, 500)` and add `import windsound` at the top, then start your program. Check whether your computer sound is on and turned up. If you're using an online development environment, you won't hear anything, because the program isn't executed on your computer, but somewhere else. The `windsound` Module may not work with some operating systems like Linux and MacOS.

Typical Mistakes in Python Programs

One of the usual mistakes beginners make is mixing up **capital and lowercase letters**. However, they matter when we call commands and can impede proper functioning. Here's an example of such a mistake:

```
Print('Hello Python')
```

In the example above, `print` is written incorrectly and the capital letter needs to be changed to lowercase.

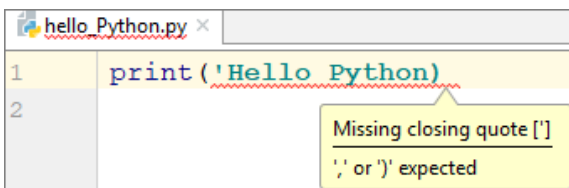


In Python we don't use a **semicolon (;)** at the end of each command. Commands are separated by a **new line or by indentation**.

A missing **quotation mark** or **closing bracket** can also cause issues – the program will either **function improperly** or **not execute at all**. Such mistakes are hard to notice, especially with larger amounts of code. Here's an example:

```
print('Hello Python)
```

This program will return an **error** and the code will be underlined to draw the programmer's attention to the missed closing bracket.



What Have We Learned from This Chapter?

First of all, we learned **what programming is** – issuing commands written in a **programming language** that the machine can understand and carry out. We also found out what a

computer program is – a series of commands that aim to achieve a certain result. We gained some basic knowledge of the Python programming language and learned how to create simple console programs using PyCharm. Then we examined Python's program code structure. We looked at printing with the function `print(...)` and starting the program with [Ctrl + Shift + F10]. And last but not least, we now know how to test our code in SoftUni's Judge System.

Top work! Let's now tackle the exercises. You do remember that learning how to program involves a lot of code writing and problem-solving, right? Let's do just that and put what we've learned into practice.

First Steps in Programming – Problems

Welcome to the exercises. We are now going to write a couple of console applications, which will help us take a few more steps into programming. After that we will show you how to program something more complex – programs with a graphical and web user interface.

Problem: Expression

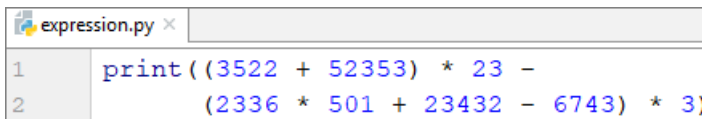
Write a console-based Python program that calculates and prints the value of the following numerical expression:

- $(3522 + 52353) * 23 - (2336 * 501 + 23432 - 6743) * 3$

Note: you are not allowed to previously calculate the value (for example with Windows Calculator).

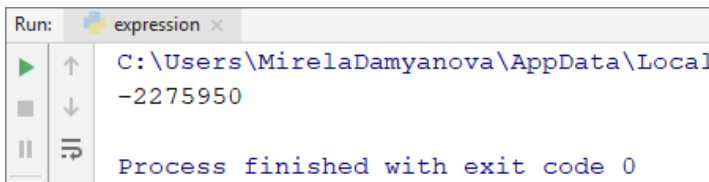
Hints and Guidelines

Create a new Python file and name it **Expression**. Then we need to write the code which will calculate the numeric expression above and print its value on the console. Pass the expression to the `print(...)` command by writing it within its brackets:



```
expression.py x
1 print((3522 + 52353) * 23 -
2       (2336 * 501 + 23432 - 6743) * 3)
```

Start the program with [Ctrl + Shift + F10] and check whether the result matches the one from the picture:



```
Run: expression x
C:\Users\MirelaDamyanova\AppData\Local
-2275950
Process finished with exit code 0
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1046#1>.

02. Expression

```
1 print((3522 + 52353) * 23 - (2336 * 501 + 23432 - 6743) * 3)
2
```

Allowed working time: 0.100 sec.
 Allowed memory: 16.00 MB
 Size limit: 16.00 KB
 Checker: Numbers Checker ?

Python code Submit

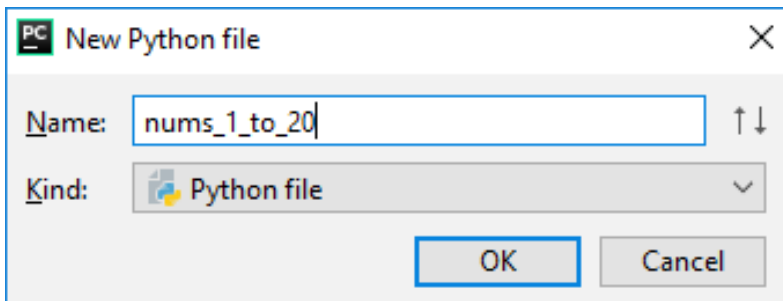
Submissions		
⏮ ⏪ 1 ⏩ ⏭		
Points	Time and memory used	Submission date
✓ 100 / 100	Memory: 8.20 MB Time: 0.056 s	21:06:39 30.09.2018
		Details

Problem: Numbers from 1 to 20

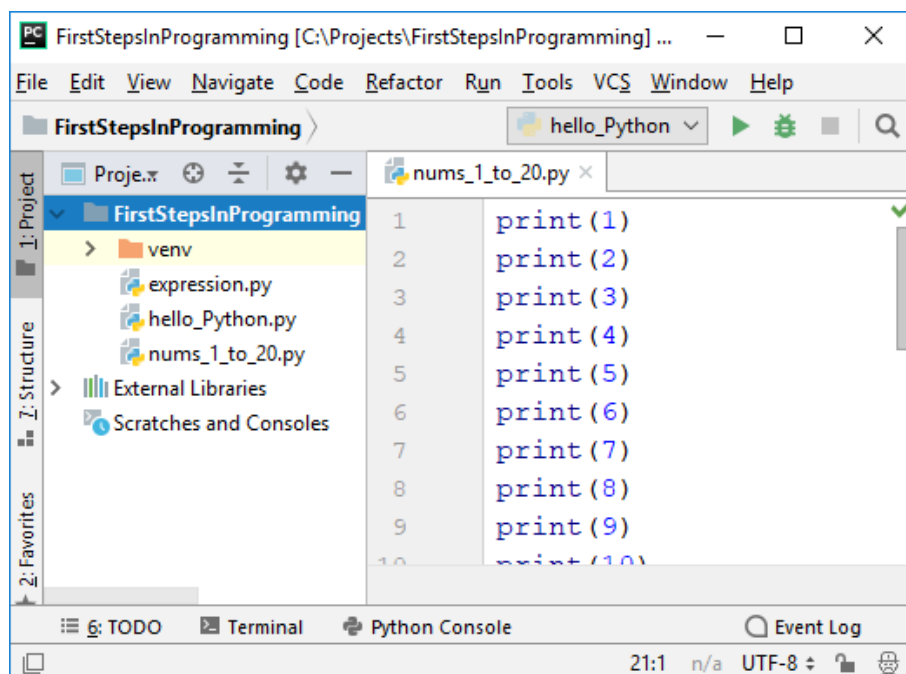
Write a Python console program that prints the numbers from 1 to 20 on separate lines on the console.

Hints and Guidelines

Create a Python file and name it `nums_1_to_20.py`:



Inside the file write 20 `print()` commands, each on a separate line, to print the numbers from 1 to 20 one after another. Some of you may be wondering if there is a more clever way. Don't worry, there is, but we will mention it later on.



Let's now **start the program** and check whether the result is what we're expecting:

```
1
2
...
20
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1046#2>. Now try and **think of a more clever way** we can write this program, to avoid repeating the same command so many times. Look up information for "[for loop Python](#)" on the Internet.

Problem: Triangle of 55 Stars

Write a Python console program that prints a **triangle of 55 stars** on 10 lines:

```
*
**
***
****
*****
*****
*****
*****
*****
*****
```


Hints and Guidelines

Create a new Python file and name it `triangle_of_55_stars.py`. Inside we need to write code that prints our triangle, using 10 `print()` commands, like this:

```
print('*')
print('**')
...
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1046#3>.

Try and **improve your solution** to avoid repeating the same command so many times. Can this be achieved with a **for**-loop? Did you manage to invent a better solution (with a for-loop, for example) to the previous problem? The current problem can be solved with a similar, but a little more complex approach (a loop within another loop). It's completely fine if you can't figure it out, you'll remember this problem when we learn about loops in a few chapters.

Problem: Rectangle Area

Write a Python program that receives two numbers **a** and **b**, then calculates and prints the area of a rectangle with sides **a** and **b**.

Sample Input and Output

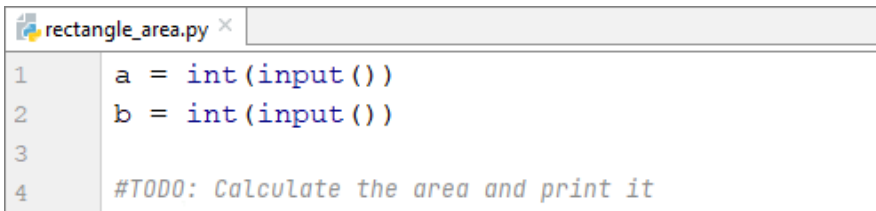
a	b	area
2	7	14

a	b	area
12	5	60

a	b	area
7	8	56

Hints and Guidelines

Create a new Python file. We'll use the following commands to read our numbers:



```
rectangle_area.py ×
1 a = int(input())
2 b = int(input())
3
4 #TODO: Calculate the area and print it
```

What's left is to finish the program so it can calculate the rectangle's area and print it. Pass the product of **a** and **b** to the already familiar `print(...)` command. In programming, we multiply using the `*` operator.

Test Your Solution

Test your solution with a few examples. You should receive a result identical to the one you see below (we enter 2 and 7 as input and the program prints 14 – their product, as output):

```
2
7
```

14

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1046#4>.

Problem: Square of Stars

Write a Python console program that reads an integer positive number N from the console and prints a square of N stars on the console, like in the examples below.

Sample Input and Output

Input	Output	Input	Output	Input	Output
3	*** * * ***	4	**** * * * * ****	5	***** * * * * * * *****

Hints and Guidelines

Create a new Python file. To read the number N ($2 \leq N \leq 100$), we use the following code:

```
square_of_stars.py x
1  n = int(input())
2
3  #TODO: Print the rectangle
```

Finish the program so that it prints a square made up of stars. You may need to use **for**-loops.

Caution: this task is more difficult than the rest and is presented to you at this point purposefully. It's marked with a star, to provoke you **to look for information on the Internet**. This is one of the most important skills you have to develop while you're learning to program. This is what you'll be doing every day if you work as a developer, so don't be scared, try it out. If you're having difficulties, you can also ask for help in the **SoftUni's Reddit Community**: <https://www.reddit.com/r/softuni/>.

Testing in The Judge System

Test your solution here:<https://judge.softuni.org/Contests/Practice/Index/1046#5>.

Console, Graphical and Web Applications

With **console applications**, as you've already noticed, **all operations** for reading input and printing output are done on the console. The input data is **entered on the console** and then

it is read by the application; the console is also used to **print the output data** after or during the runtime of the program.

While a console application **uses the text console**, web applications use a **web-based user interface**. To execute them, two things are needed – a **web server** and a **web browser**. The **browser** plays the main role in the **visualization of data and interaction with the user**. Web applications are much more pleasant for the user, they look better, and a mouse and touch screen can be used (for tablets and smartphones). Programming stands under all of that, of course. This is why **we need to learn how to program** and we have already made our first tiny steps towards achieving that.

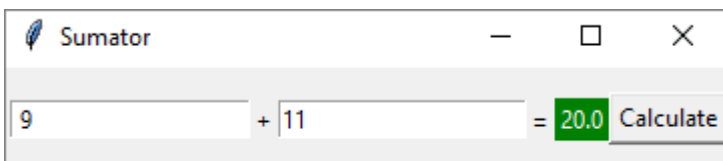
Graphical (GUI) applications have a **visual user interface**, directly on your computer or mobile device, without a web browser. Graphical applications (also known as desktop applications) contain one or more graphical windows where certain controllers are located (text fields, buttons, pictures, tables and others), which **allow dialog** with the user to be more intuitive. Similar to them are mobile applications on your smartphone or tablet: we use forms, text fields, buttons and other controls and we manage them with program code. This is why we learn to write code now: **code is everywhere in software development**.

Problem: Graphical and Web Applications

We will now build a **basic web application** and after that, a simple **graphical application**, to have a look at what we will be able to create once we progress with programming and software development. We are not going to examine the techniques and constructions used in much detail. Rather, we are just going to have a peek at the arrangement and functionality of what we've created. After we progress with our knowledge, we will be able to develop bigger and more complex software applications and systems. We hope that the examples given below will boost your interest, rather than make you give up.

Graphical Application: 'Numbers Summator'

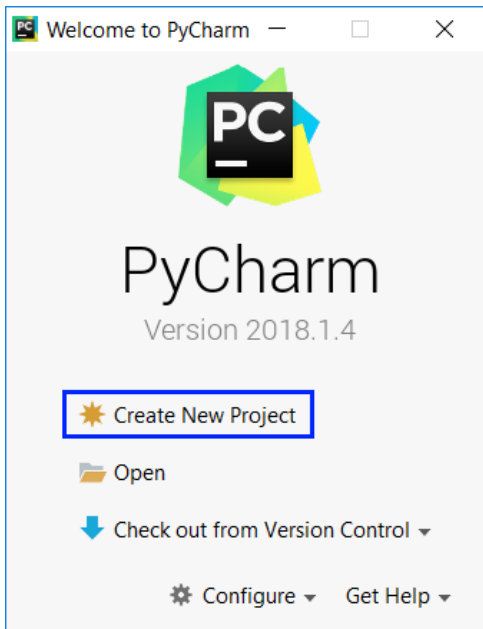
Write a **graphical (GUI) application** that **calculates the sum of two numbers**. After the numbers are entered in the first text fields, and the button **[Calculate]** is clicked, calculate their sum and display the result in the third text field colored in green:



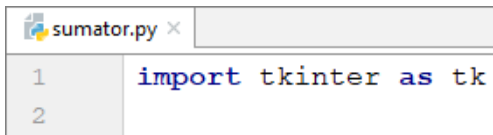
We'll use the standard Python graphical library **TKinter** for our application (<https://wiki.python.org/moin/Tkinter>). It enables us to create **graphical (GUI) applications** using the **Python** programming language.

Empty Python Project

Create a **new Python project** named **"Summator-GUI"** in PyCharm:



Add a new Python file named **sumator.py** to the project. Then add **tkinter** as well:



Creating The Basis for The GUI Project

Next, we need to write the code for our graphical application, starting with the basis:

```
import tkinter as tk
```

```
class Application(tk.Frame):  
    def __init__(self, master=None):  
        super().__init__(master)  
  
        self.pack()
```

```
# create the application  
app = Application()  
app.master.title("Sumator")  
app.master.minsize(width=100, height=50)
```

```
# start the program  
app.mainloop()
```

We'll need the following components (controls):

- **Entry** – will enter the numbers that need to be calculated.
- **Label** – will serve to statically display text and the result of the calculation.
- **Button** – will calculate the sum of our numbers.

```
def __init__(self, master=None):
    super().__init__(master)

    self.pack()
    self.create_widgets()
```

Let's initialize and visualize our controls (widgets):

```
def create_widgets(self):
    # create widgets
    self.firstNumberEntry = tk.Entry()
    self.plusSign = tk.Label(text="+")
    self.secondNumberEntry = tk.Entry()

    self.equalSign = tk.Label(text="=")
    self.resultLabel = tk.Label(text="Result...", bg="green", fg="white")
    self.calculateButton = tk.Button(text="Calculate")

    # place widgets
    self.firstNumberEntry.pack(side="left")
    self.plusSign.pack(side="left")
    self.secondNumberEntry.pack(side="left")
    self.equalSign.pack(side="left")
    self.resultLabel.pack(side="left")
    self.calculateButton.pack(side="left")
```

And try to start our application with [Ctrl + Shift + F10] or right-click + [Run]. It should start but **not function correctly**, because we haven't specified what happens when the button is clicked:



Application Logic

Now we're going to write the code responsible for summing our numbers and displaying the result in the third field. For that to work, we add the **calculate** command upon the button's initialization:

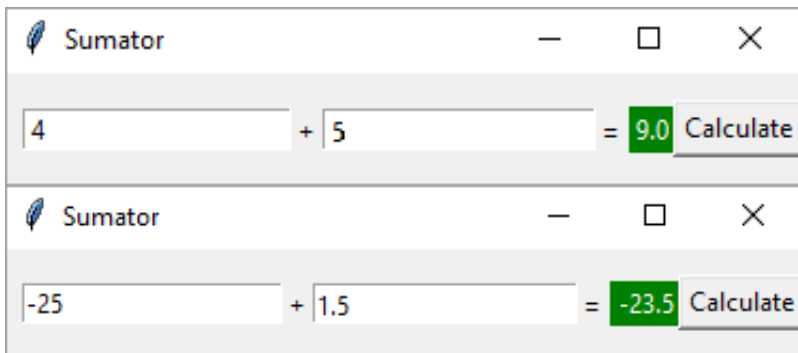
```
tk.Button(text="Calculate", command=self.calculate)
```

Then we write the **calculate** function's code:

```
def calculate(self):
    first_value = float(self.firstNumberEntry.get())
    second_value = float(self.secondNumberEntry.get())
    result = first_value + second_value
    self.resultLabel.config(text=str(result), bg="green", fg="white")
```

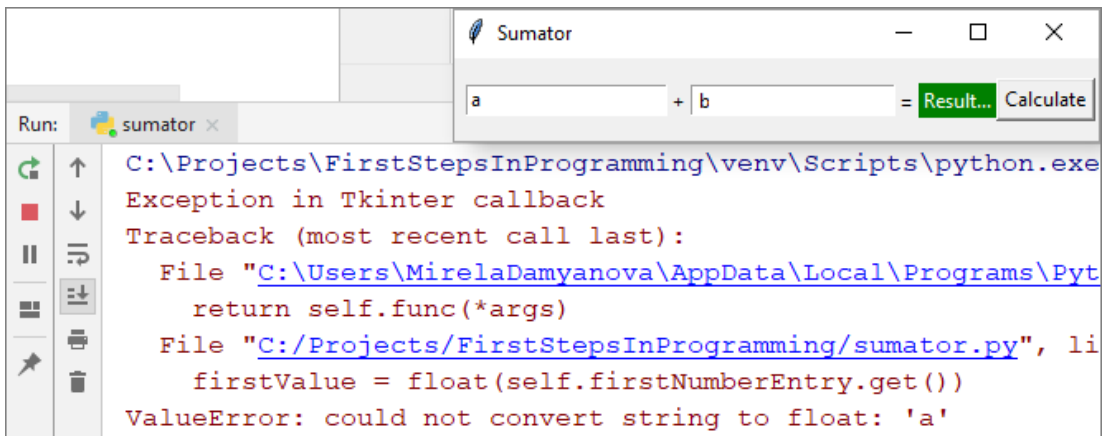
This code takes the first number from the **firstNumberEntry** field and stores it in the **first_value** variable. Then it stores the second number from the **secondNumberEntry** field in the **second_value** variable. It sums **first_value** and **second_value** and stores the result in the **result** variable. Finally, it displays the text representation of the **result** variable in the **resultLabel** field.

Let's start the program again using [Ctrl + Shift + F10] or right-click + [Run] and check whether it's working correctly. Try and calculate $4 + 5$, then $-2.5 + 1.5$:



Managing Invalid Numbers

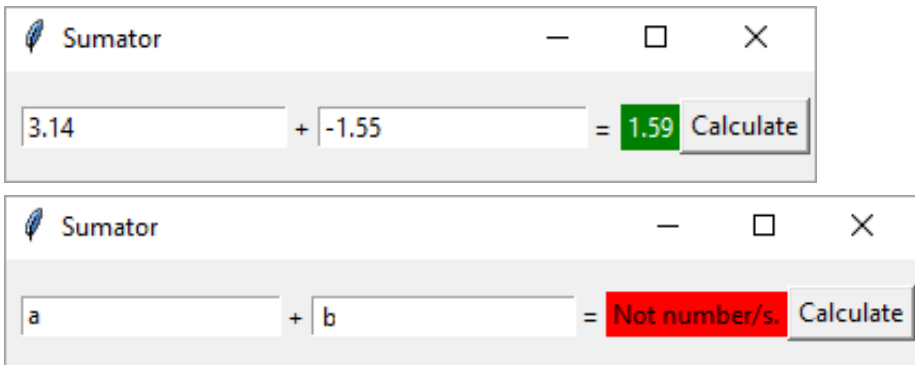
Entering invalid numbers like 'a' and 'b' seems to create issues:



This is due to the conversion of the information entered in our text field into a number. If the value in the field isn't a number, our program returns an error. We can fix the code to resolve this issue:

```
def calculate(self):
    try:
        first_value = float(self.firstNumberEntry.get())
        second_value = float(self.secondNumberEntry.get())
        result = first_value + second_value
        self.resultLabel.config(text=str(result), bg="green", fg="white")
    except ValueError:
        self.resultLabel.config(text="Not number/s.", bg="red", fg="black")
```

The code you see above **intercepts number-handling errors** (catches exceptions) and in case of an error, **returns a Not number/s** value in the result field. If we start our program once more and enter an improper value, the **result is Not number/s** and our program continues working as intended:

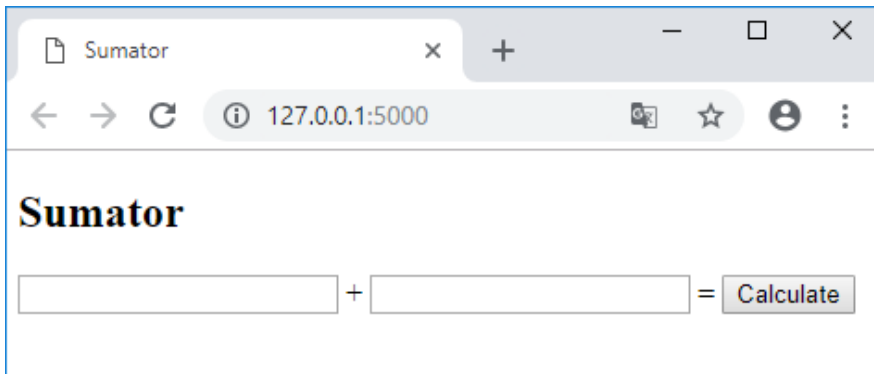


Does that seem complicated? If it does, that's completely normal. We are at the very start of our programming journey. The example above requires much more knowledge and skills, which we are going to develop using this book and later on as well. Just allow yourself to have some fun with desktop programming. Or bravely move forward to the next example or the next chapter in the book. There will come a time when this becomes easy for you, but you have to put in some effort and be persistent. Learning programming is a slow process involving lots and lots of practice.

Web Application: 'Numbers Summator'

Now we are going to create something even more complex, but also more interesting: a Web application that **calculates the sum of two numbers**. By entering two numbers in the first two text fields and clicking the [Calculate] button, **their sum is calculated** and the result is displayed in the third text field.

Keep in mind that we are creating a **Web-based application**. This is an application that is accessible through a web browser, just like your favorite email or news website. The web application is going to have a server-side (back-end), which is written in the Python language with **Flask** (<https://flask.palletsprojects.com>) technology, and a client-side (front-end), which is written in the **HTML** language (this is a language for visualization of information in a web browser). The Web application is expected to resemble the following:

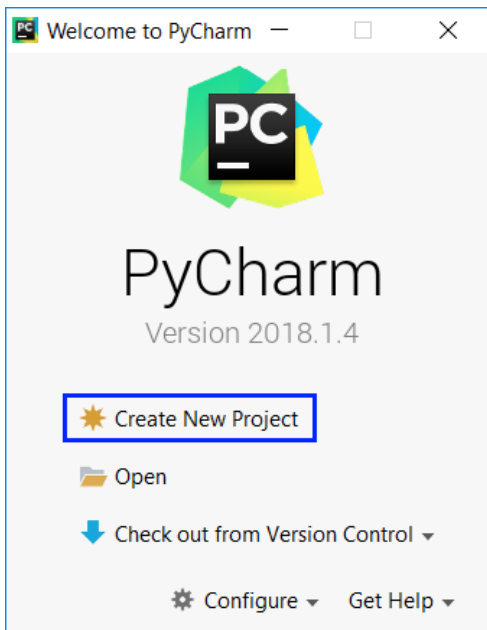


As opposed to console applications that read and write data in the form of text on the console, Web applications have a **Web-based user interface**. Web applications are **loaded from some Internet address** (URL) through a standard web browser. Users write input data on a page, visualized by the web browser, the data is processed on a web server and the results are shown again on a page in the web browser. For our web application, we are going to use **Flask**, a light Python library, which allows creating **web applications with the programming language Python**.

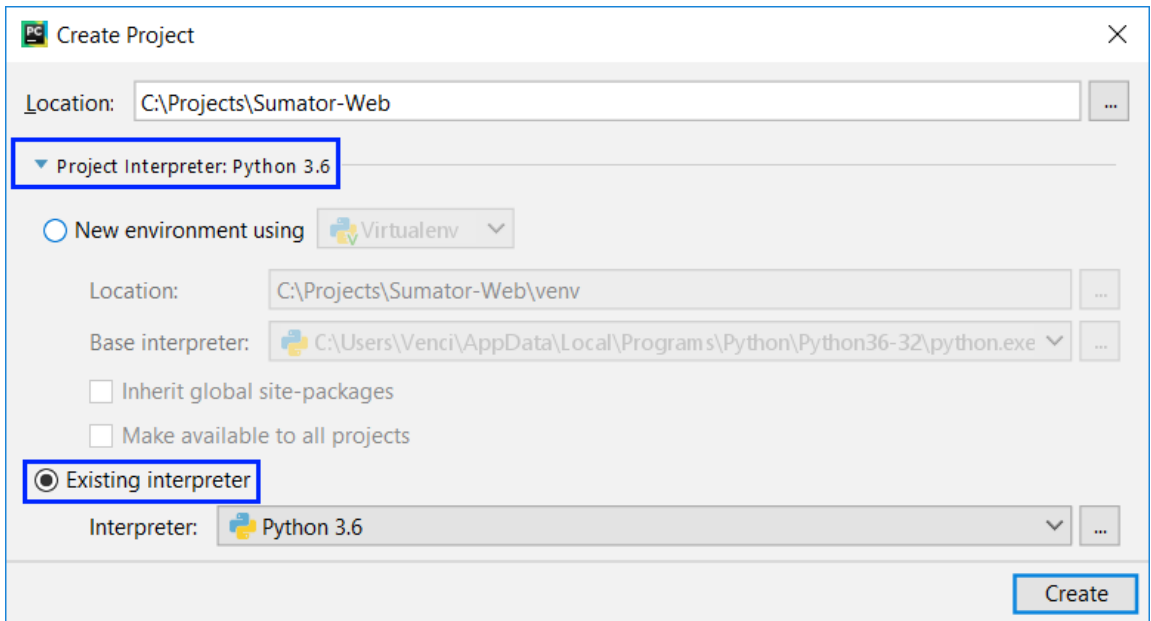
Next are the steps for the implementation of the "Numbers Summator" web application.

Empty Python Solution

We'll create an **empty Python project** in PyCharm to organize our application's code:

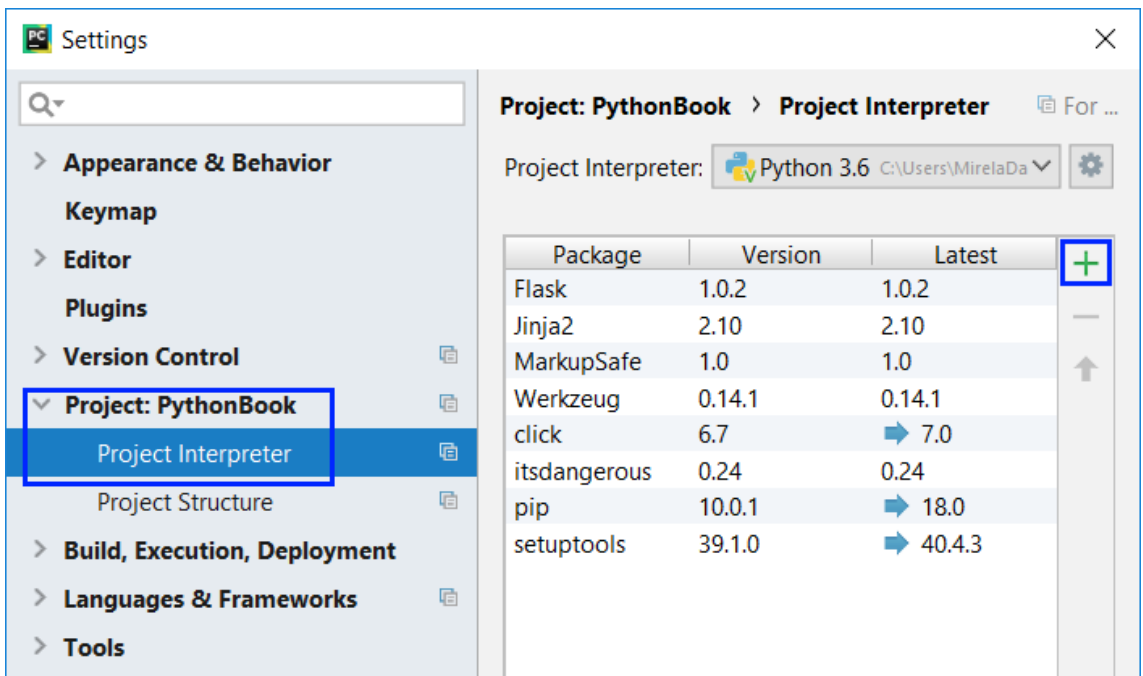


Then we need to give the project a meaningful name like **"Summator-Web"**. We'll also set the Python interpreter to this one by default:

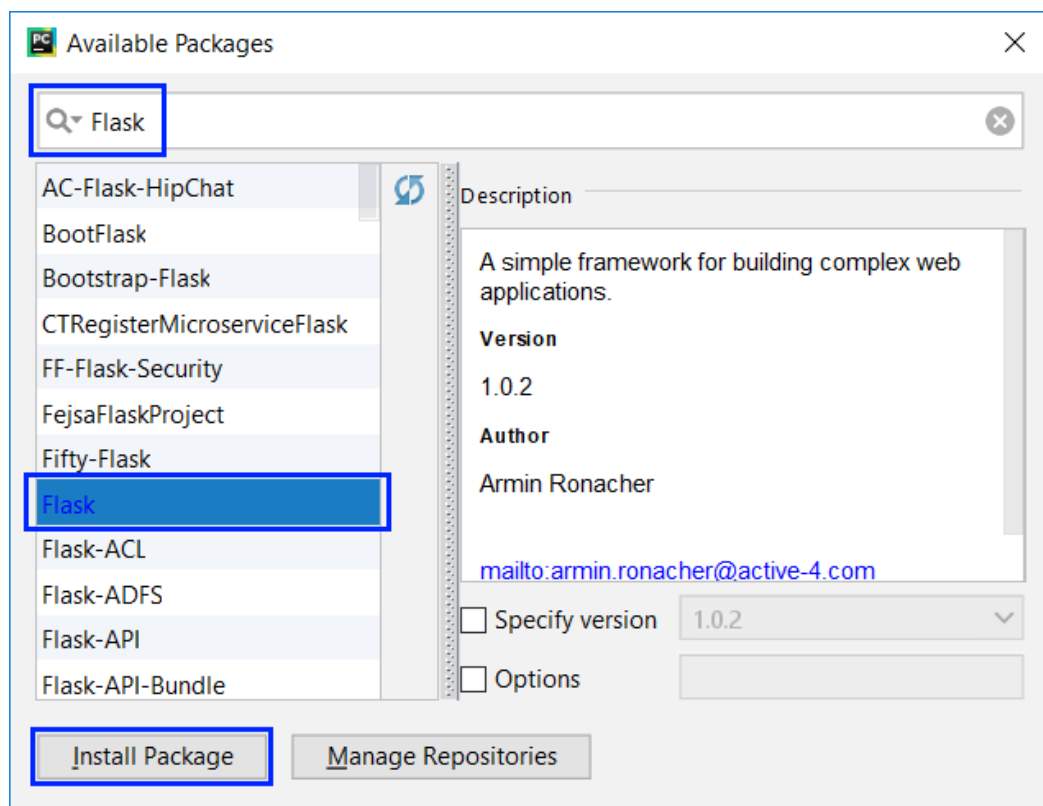


Installing Flask

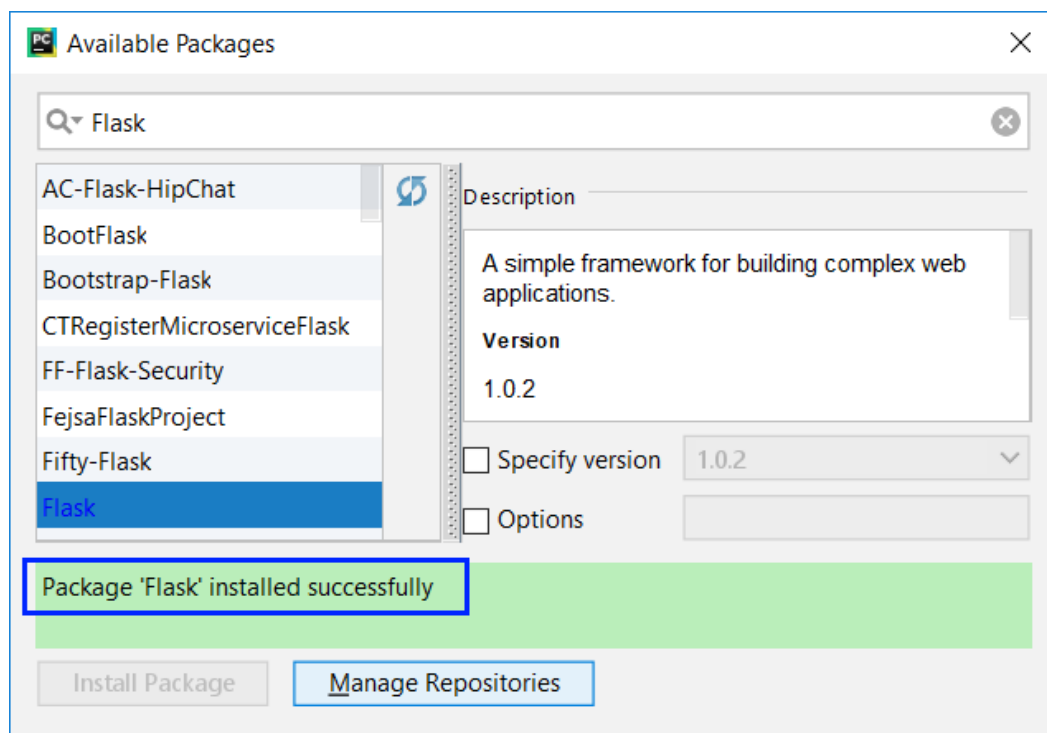
Before we start writing code, we need to install Flask. Let's go to PyCharm's settings [File] -> [Settings] then go to [Project: Sumator-Web] -> [Project Interpreter]. There we need to click the **+** button:



When you find Flask in the window that appears, click [Install package]:

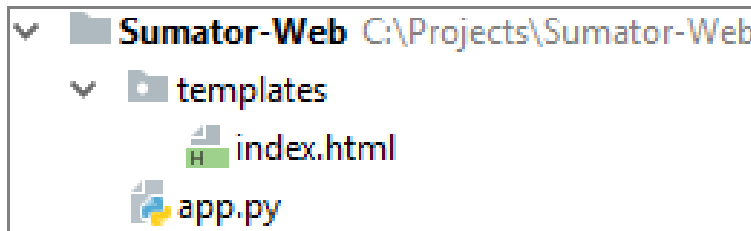


If all goes well, you'll receive the following message on the same screen:



Creating The Project Structure

Let's create the structure of our project. To do that, we're going to create a Python file named **app.py** where we will keep our program code. Then we need a folder which we'll call **templates** and an HTML file named **index.html** which will help visualize our content in the browser.



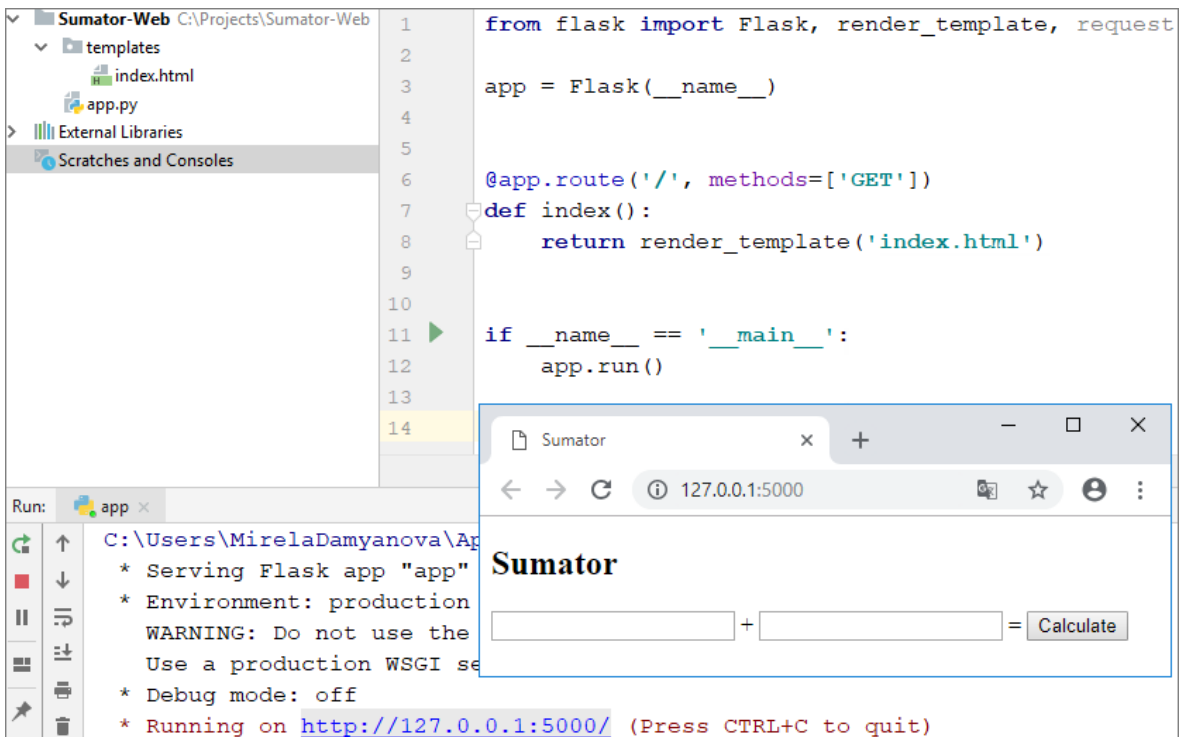
Writing The Application's Logic

Time to create our application's controls. The aim is to add fields where our numbers will be entered, a button for summing them and a place to display the result. We can achieve this by writing the following code in **templates/index.html**:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sumator</title>
</head>
<body>
  <h2>Sumator</h2>
  <form method="POST" action="/">
    <input type="number" name="firstNumber" />
    <span>+</span>
    <input type="number" name="secondNumber" />
    <span>=</span>
    <span>{{result}}</span>
    <input type="submit" value="Calculate" />
  </form>
</body>
</html>
```

This code creates a web form **<form>** with a field and a button **[Calculate]** which will send the data from the form to our server. Our Python **app.py** file will be responsible for data manipulation. Notice the curly braces – they are used to switch between the **HTML** and **Python** languages and come from the **Jinja2** syntax used for creating dynamic web pages.

After we've written the following code in **app.py**, we can test if our program is working by running it:



Next, we need to write the code which will handle our request:

```
from flask import Flask, render_template, request
```

```
app = Flask(__name__)
```

```
@app.route('/', methods=['GET', 'POST'])
```

```
def index():
```

```
    if request.method == 'GET':
```

```
        return render_template('index.html')
```

```
    elif request.method == 'POST':
```

```
        first_value = float(request.form.get('firstNumber'))
```

```
        second_value = float(request.form.get('secondNumber'))
```

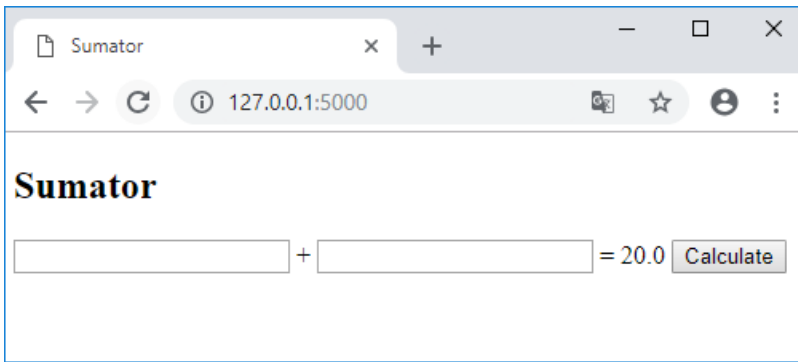
```
        result = eval(str(first_value + second_value))
```

```
        return render_template('index.html', result=result)
```

```
if __name__ == '__main__':
```

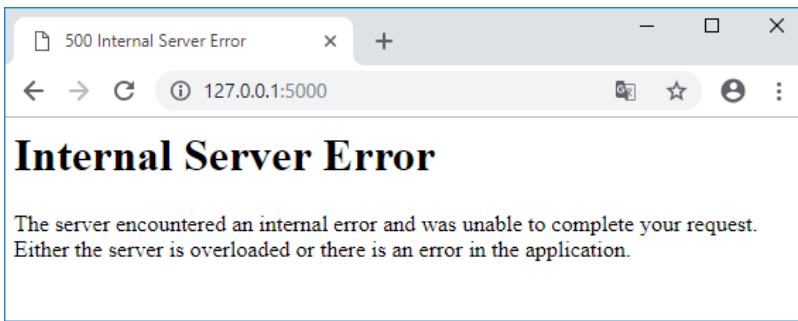
```
    app.run()
```

If we click the calculation button now, our program will be able to sum the two digits from the form. Let's run the program again and test whether it's working correctly. Upon entering the numbers **9** and **11**, we'll receive the correct result:



Managing Invalid Numbers

If we try entering **invalid data**, we'll receive an error:

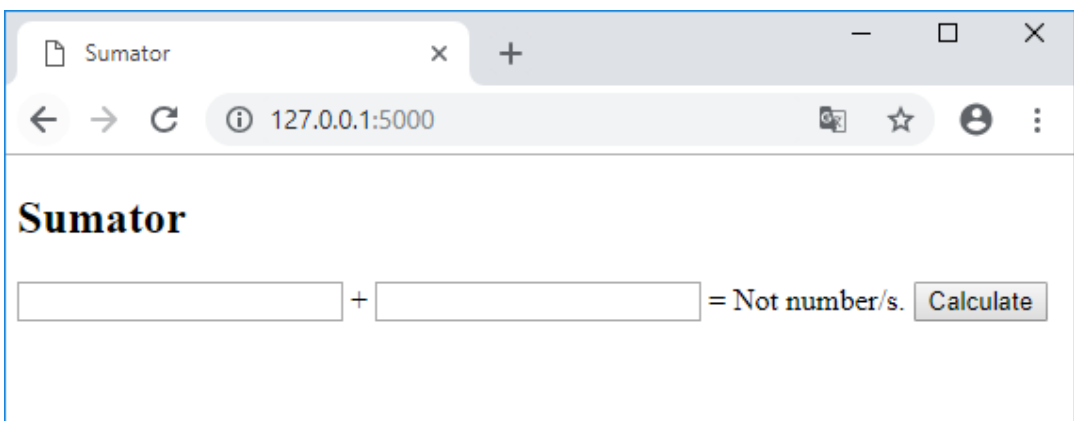


To handle the error, we need to write the following code:

```
@app.errorhandler(500)
def page_not_found(e):
    return render_template('index.html', result="Not number/s.")
```

This is not the best solution for error handling, but it will work.

If we run the program one more time and test it by entering **invalid data**, we'll receive the **Not number/s.** message.



Does all this seem scary? **There's no need to be afraid.** We have a lot more to learn, to reach the level of knowledge and skills required to write web-based applications with ease like in the example above, as well as much bigger and more complex applications. If it all makes

little sense, just keep going without worrying. In time, you will remember with a smile how incomprehensible and exciting your first collision with web programming was.

The purpose of both examples (graphical desktop application and web application) is not to teach you, but to allow you to dive a little deeper into programming, to **fuel your interest** in software development, and to inspire you to study hard. **You have a lot more to learn**, but it is interesting, isn't it?

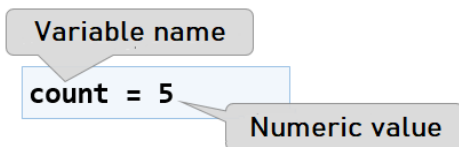
Chapter 2.1. Simple Calculations

In this chapter, we are going to become familiar with the following concepts and programming techniques:

- How to work with **data types and variables**, which are necessary when processing numbers and strings.
- How to **print** a result on the console.
- How to **read** user input.
- How to do simple **arithmetic operations**: addition, subtraction, multiplication, division, string concatenation.
- How to **round** numbers.

Calculations in Programming

We know that computers are machines that process data. All **data** is held in the computer memory (RAM) as **variables**. Variables are named areas of the computer memory which hold data of a certain type, for example, a number or text. Each **variable** in Python has a **name** and **value**. Here is how to define a variable, assigning a value to it at the time of declaration:



After processing, data is still held in variables (i.e. somewhere in the computer memory, allocated by our program).

Data Types and Variables

In programming, each variable stores a certain **value** of a particular **type**. For example, data types can be: **number**, **letter**, **text** (string), **date**, **color**, **image**, **list** and others. Here are some examples of data types and their values:

- **int** - integer: 1, 2, 3, 4, 5, ...
- **float** - floating-point number: 0.5, 3.14, -1.5, ...
- **str** - text (string) of symbols: 'a', 'Hello', 'Hi', 'Beer', ...
- **datetime** - date: 21-12-2017, 25/07/1995, ...

In **Python**, the data type is defined by the value that is assigned and is not explicitly specified when declaring variables (as opposed to C#, Java and C++).

Printing Results on The Screen

To print text, numbers, or another result on the screen, we need to call the built-in function **print()**. It allows us to print the value of a variable as well as to directly print a string or a number:

```
print(9) # prints a number

print('Hello!') # prints text

msg = 'Hello, Python!'
print(msg) # prints value of a variable
```

Reading User Input in Integer Type

To read a user input as an integer, it is necessary to **declare a variable** and use the built-in functions `input (...)` to read a text line from the console and `int (...)` to convert a text value to numerical:

```
num = int(input())
```

If it weren't for this conversion, for the program **every number** will be just **text** with which **we wouldn't be able to do** arithmetic operations. When using `input(...)` we can include a prompt to the user to tell them what we want them to enter, for example:

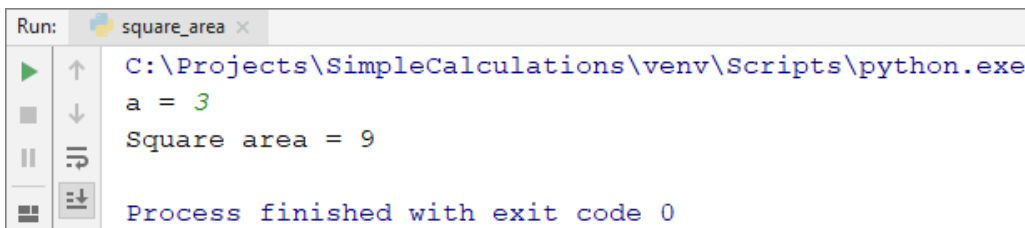
```
size = int(input('Enter the size = '))
```

Problem: Square Area

Let's take the following program as an example. It reads an integer, multiplies it by itself (i.e. **squares it**) and prints the result of the multiplication. This is how we can calculate the area of a square by using a given length of its side **a**:

```
a = int(input('a = '))
area = a * a
print('Square area = ', area)
```

Here is how the program would work for a square with a side equal to 3:

A screenshot of a terminal window titled "Run: square_area x". The terminal shows the execution of a Python script. The first line is the command prompt "C:\Projects\SimpleCalculations\venv\Scripts\python.exe". The next line shows the user input "a = 3" in green. The following line shows the program output "Square area = 9". The final line shows the program completion message "Process finished with exit code 0". On the left side of the terminal, there are standard IDE icons for running, stepping through, and other debugging actions.

```
Run: square_area x
C:\Projects\SimpleCalculations\venv\Scripts\python.exe
a = 3
Square area = 9
Process finished with exit code 0
```

Try to type in an invalid number, for example **"hello"**. You will receive an **error** message during the execution of the program (exception). This is normal. We will find out later how we can catch such errors and make the user re-enter a number.

How Does the Example Work?

The first line `a = int(input('a = '))` prints a message which tells the user to input the side of the square `a`, then it reads a text (string) and converts it into an integer (this is called parsing) utilizing the function `int(...)`. The result is assigned to a variable with the name `a`.

The next command `area = a * a` assign to a new variable `area` the result of the multiplication of `a` by `a`.

The last line `print('Square area = ', area)` prints the given text and next to it, the calculated area of the square, held in the variable `area` is concatenated.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#0>.

Reading a Sequence of Numbers

When we want to read **several numbers from the console**, if they are given **each on a new line**, we read them sequentially like this:

```
num1 = input()
num2 = input()
num3 = input()
print(num1, num2, num3)
```

If we type in the following input:

```
10
20
30
```

we will receive the following result:

```
10 20 30
```

When we want to read **several numbers from the console** and they are given **together on the same line**, separated by an interval, we can use the following construction:

```
num1, num2, num3 = map(int, input().split())
print(num1 + num2 + num3)
```

If we type in the following input:

```
100 200 300
```

we will receive the following result:

```
600
```

How does the code mentioned above work? By using `.split(...)` we separate the elements from the given text line by the interval separator. If we type in the input from above, we will receive 3 elements: `'100'`, `'200'`, and `'300'`. After that, utilizing the function `map(int, elements)` we convert the sequence of elements from text to numbers.

Reading Floating-Point Numbers

To read a user input as a **floating-point number**, we need to **declare a variable**, this time using the function `float(...)` which converts the given text value to a fractional number:

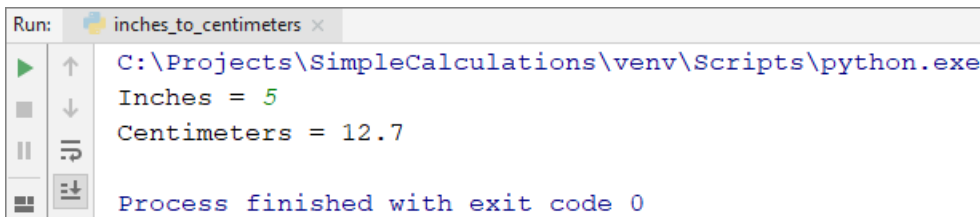
```
num = float(input())
```

Problem: Inches to Centimeters

Let's write a program that reads a fractional number in inches and converts it to centimeters:

```
inches = float(input('Inches = '))
centimeters = inches * 2.54
print('Centimeters = ', centimeters)
```

Let's start the program and make sure that when we input a value in inches, we receive the correct result in centimeters:

A screenshot of a Python IDE terminal window. The title bar shows 'Run: inches_to_centimeters x'. The terminal output shows the command prompt 'C:\Projects\SimpleCalculations\venv\Scripts\python.exe', followed by the input 'Inches = 5' and the output 'Centimeters = 12.7'. At the bottom, it says 'Process finished with exit code 0'. On the left side of the terminal, there are icons for running, stepping through, and other debugging actions.

```
Run: inches_to_centimeters x
C:\Projects\SimpleCalculations\venv\Scripts\python.exe
Inches = 5
Centimeters = 12.7
Process finished with exit code 0
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#1>.

Reading a Text from The Console

To read a **text** (string) from the console, again, we have to **declare a new variable** and use the standard command for reading a text from the console:

```
str = input()
```

Problem: Greeting by Name

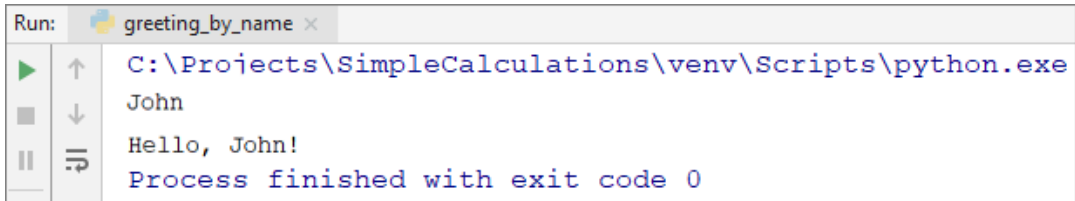
Let's write a program that asks the user for their name and salutes them with the text `"Hello, {name}!"`.

```
name = input()
```

```
print('Hello, ', end='')
print(name, end='!')
```

By default, the built-in function `print(...)` prints the result and continues to the next line. This is because `print(...)` uses the parameter `end`, which by default has a value `\n` (new line). To stay on the same line, we can change the value of `end=''`.

Here is the result if we call the function with the name "John":



```
Run: greeting_by_name x
C:\Projects\SimpleCalculations\venv\Scripts\python.exe
John
Hello, John!
Process finished with exit code 0
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#2>.

Concatenating Text and Numbers

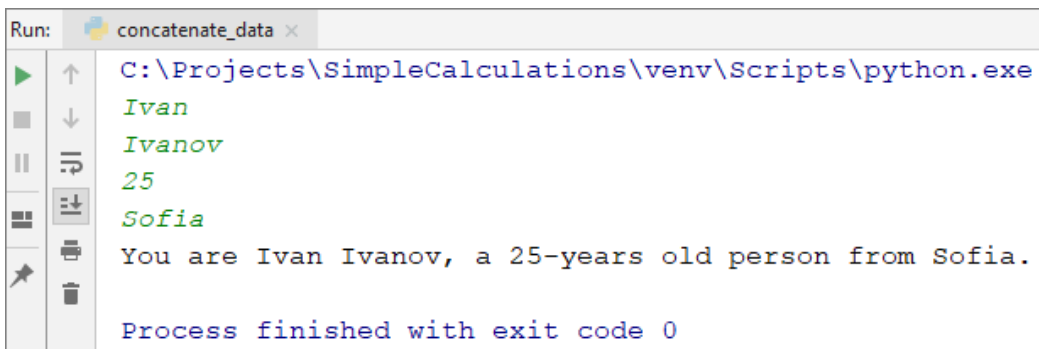
When printing text, numbers and other data, we can concatenate them by using templates `{0}`, `{1}`, `{2}`, etc. In programming, these templates are called placeholders.

In Python we use the built-in method `.format(...)`, through which we list the variables we want to put in place of the templates (placeholders). Example: we type in **first name** and **last name** of a student as well as their **age** and **hometown** and we print text in the format "You are `{name}` `{family name}`, a `{age}`-year-old person from `{hometown}`". The following is a solution with text templates:

```
first_name = input()
last_name = input()
age = int(input())
town = input()

print('You are {0} {1}, a {2}-year-old person from {3}.'
      .format(first_name, last_name, age, town))
```

Here is the result that we will get after the execution of this example:



```
Run: concatenate_data x
C:\Projects\SimpleCalculations\venv\Scripts\python.exe
Ivan
Ivanov
25
Sofia
You are Ivan Ivanov, a 25-years old person from Sofia.
Process finished with exit code 0
```

Note how each variable must be passed in the **order, in which we want it to print**. Essentially, the template (placeholder) accepts all types of variables and this creates great convenience when printing.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#3>.

More on Printing Formatted Text

It is possible for one number of a **template** to be used several times and it is not required for the templates of the function **.format(...)** to be numbered consecutively. Here is an example:

```
print('{1} + {1} = {0}'.format(1+1, 1))
```

The result is:

```
1 + 1 = 2
```

Arithmetic Operations

Let's examine the basic arithmetic operations in programming.

Summing Numbers: Operator +

We can sum numbers by using the **+** operator:

```
a = 5
b = 7
sum = a + b # the result is 12
```

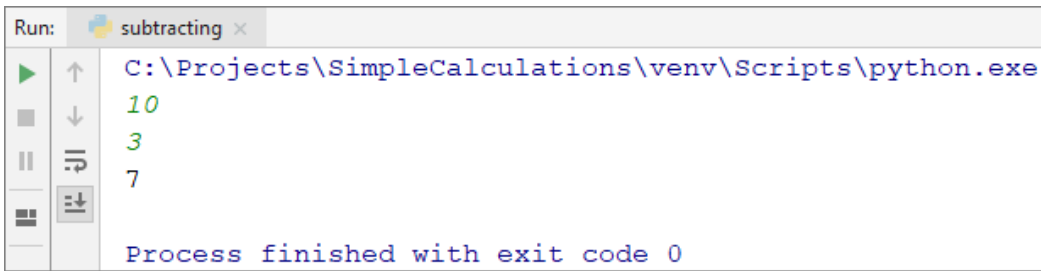
Subtracting Numbers: Operator -

Subtracting numbers is done with the **-** operator:

```
a = int(input())
b = int(input())

result = a - b
print(result)
```

Here is the result of the execution of this program (with numbers 10 and 3):



The screenshot shows a terminal window titled "Run: subtracting x". The command prompt is "C:\Projects\SimpleCalculations\venv\Scripts\python.exe". The input is "10", "3", and "7" on separate lines. The output is "Process finished with exit code 0".

Multiplying Numbers: Operator *

For multiplication of numbers, we use the `*` operator:

```
a = 5
b = 7
product = a * b # 35
```

Dividing Numbers: Operator /

Dividing numbers is done with the `/` operator.

Note: Dividing by 0 causes an **error** during execution (runtime exception) - `ZeroDivisionError`.

Here are a few examples with the division operator:

```
print(10 / 2.5) # Result: 4
print(10 / 4)   # Result: 2.5
print(10 / 6)   # Result: 1.6666666666666667

print(5 / 0)    # Result: ZeroDivisionError: division by zero
print(-5 / 0)   # Result: ZeroDivisionError: division by zero
print(0 / 0)    # Result: ZeroDivisionError: division by zero
print(2.5 / 0)  # Result: ZeroDivisionError: float division by zero
```

Raising to a Power: Operator **

To raise to power in Python the operator `**` is used:

```
print(2 ** 10)    # 1024
print(2 ** 20)    # 1048576
print(2 ** 30)    # 1073741824
print(2 ** 64)    # 18446744073709551616
print(2 ** 128)   # 340282366920938463463374607431768211456
print(2.5 ** 1.5) # 3.952847075210474
```

As seen in the example, Python works efficiently with **big integer values**, without special requirements. Here is a bigger example:

```
print(2 ** 1024)
```

The result is the following big integer:

```
179769313486231590772930519078902473361797697894230657273430081157732
675805500963132708477322407536021120113879871393357658789768814416622
492847430639474124377767893424865485276302219601246094119453082952085
005768838150682342462881473913110540827237163350510684586298239947245
938479716304835356329624224137216
```

Concatenating Text and Numbers

Besides for summing numbers, the operator `+` is also used for joining pieces of text (concatenation of two strings one after another). In programming, joining two pieces of text is called "concatenation". Here is how we can concatenate a text with a number using the `+` operator:

```
first_name = 'Maria'
last_name = 'Ivanova'
age = 19

str = first_name + ' ' + last_name + ' @ ' + str(age)
print(str) # Maria Ivanova @ 19
```

In Python, we cannot directly concatenate a number to a given text, so we first parse the number to text utilizing the `str (...)` function.

Here is another example:

```
a = 1.5
b = 2.5

sum = 'The sum is: ' + str(a) + str(b)
print(sum) # The sum is 1.52.5
```

Did you notice something strange? Maybe you expected the numbers `a` and `b` to be summed? The concatenation works from left to right and the result above is correct. If we want to sum the numbers, we have to use **brackets**, to change the order of execution of the operations:

```
a = 1.5
b = 2.5

sum = 'The sum is: ' + str(int(a + b))
print(sum) # The sum is: 4
```

Repeating Text: Operator *

The operator `*` can be used to repeat a given text a number of times:

```
text = 'hi'
print(3 * text)  # hihhi
```

This feature of the operator for multiplication can lead to the following **wrong result** if we don't keep in mind our value type:

```
count = input()  # Enter 20
print(5 * count) # 2020202020, not 100
```

If we want to multiply a number read from the console by 5, we first have to convert it to an integer using the function `int()`:

```
count = int(input()) # Enter 20
print(5 * count)     # 100
```

Printing Formatted Text in Python

In Python, there are **several ways** to print **formatted text**, i.e. text, combined with numbers, values of variables, and expressions. We have already encountered some of them, but let's look at the issue in more detail.

Printing with Commas

When printing using `print(...)` we can list multiple values with commas:

```
width = 5
height = 7
print('width =', width, '; height =', height, '; area =', width *
      height)
```

The result is the following:

```
width = 5 ; height = 7 ; area = 35
```

As you can see from the example, in such printing every two values of the listed are separated by an **interval**.

Concatenating Text with The Operator +

We already know how to concatenate text and numbers with the operator `+`. Using `+`, the example above can be written in the following way:

```
width = 5
height = 7
```

```
print('width = ' + str(width) + ' ; height = ' + str(height) + ' ;  
area = ' + str(width * height))
```

The result is the same as in the last example:

```
width = 5 ; height = 7 ; area = 35
```

Because in Python text cannot be directly concatenated with numbers, they first have to be parsed to text using the function `str(num)`.

Formatting Strings %d, %s, %f

Printing with templates can be done with **formatting strings** (also used in C and Java). Here is an example:

```
width = 5  
height = 7  
text = "area"  
print('width = %d ; height = %d ; %s = %d' % (width, height, text,  
width * height))
```

In the example above we use the operator `%`, which substitutes values in a **text template**, given as a sequence of elements in brackets. The following main **placeholders** are used (formatting strings): `%d` signifies an integer, `%f` signifies a floating-point number, `%s` signifies text. The outcome of the code above is the same as in the last examples:

```
width = 5 ; height = 7 ; area = 35
```

Using a wrong formatting string could result in an error.

When formatting floating-point numbers, we can round them to a specified number of digits after the decimal point, for example with the formatting string `%.2f` we can print a floating-point number rounded to 2 digits after the decimal point:

```
print('price = %.2f \nVAT = %.3f' % (1.60, 1.60 * 0.2))
```

The result from the code above is the following:

```
price = 1.60  
VAT = 0.320
```

In the example above we use the special symbol `\n`, which means „go to the next line“. Analogically, the special symbol `\b` returns the cursor with one position backward (deletes the last printed symbol), respectively the following code:

```
print('ab\bcb')
```


prints the following result:

```
ac
```

Formatting Using `.format(...)`

We already looked at how we can format text and numbers using `.format(...)` and numbered templates `{0}`, `{1}`, `{2}` etc. Actually, when enumeration is not necessary, we can use just `{}`, to create a **placeholder**. Here is an example of how we can use formatting with `text.format(...)` to print the result from the last example:

```
width = 5
height = 7
print('width = {} ; height = {} ; area = {}'.format(width, height,
width * height))
```

The result is the same again:

```
width = 5 ; height = 7 ; area = 35
```

Formatting Using F-String

Maybe the easieast, the most intuitive and the shortest way to **format text and numbers** in Python is to use the **f-string syntax**:

```
width = 5
height = 7
print(f'width = {width} ; height = {height} ; area = {width *
height}')
```

Using the **F-String formatting** is very easy: we put a prefix `f` in front of the string and insert values of variables and expressions in curly braces `{expression}` in random positions. In the example above we have three expressions: `{width}`, `{height}` and `{width * height}` which are calculated and are substituted by their text value. We can use text expressions, integer expressions, floating-point expressions, and all other kinds of expressions.

Formatting with **F-String** in Python is **the recommended method** for printing formatted text because it uses the shortest syntax and the code is easily read and understood.

Numerical Expressions

In programming, we can calculate **numerical expressions**, for example:

```
expr = (3 + 5) * (4 - 2) # 16
```

The standard rule for priorities of arithmetic operations is applied: **multiplication and division are always done before addition and subtraction**. In the case of an **expression in brackets**, it is **calculated first**, but we already know all of that from school math.

Problem: Concatenate Data

Write a program, that reads a first name, last name, age and city from the console and prints a message of the following kind: **You are <firstName> <lastName>, a <age>-year-old person from <town>.**

Hints and Guidelines

We add to the existing PyCharm project one more Python file named "concatenate_data". We **write the code** which reads the input from the console:

```
first_name = input()
last_name = input()
age = int(input())
town = input()
```

Next, the solution should be tested locally using [Ctrl + Shift + F10] or with right-click - [Run] and enter sample input.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#3>.

Problem: Trapeziod Area

Let's write a program that inputs the lengths of the two bases of a trapezoid and its height (one floating-point number per line) and calculates the area of the trapezoid by the standard math formula:

```
b1 = float(input())
b2 = float(input())
h = float(input())

area = (b1 + b2) * h / 2
print('Trapezoid area = ' + str(area))
```

Because we want the program to work for both integer and floating-point numbers, we use **float(...)**. If we start the program and we input for the sides respectively **3**, **4** and **5**, we will get the following result:

```

Run: trapeziod_area x
C:\Projects\SimpleCalculations\venv\Scripts\python.exe
3
4
5
Trapezoid area = 17.5
Process finished with exit code 0

```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#4>.

Rounding of Numbers

Sometimes when we work with floating-point numbers, we need to convert the numbers to the same type of format. This conversion is called **rounding**. Python offers several methods for rounding of numbers:

- **math.ceil(...)** - rounding up, to the next (bigger) integer number:

```
up = math.ceil(23.45) # up = 24
```

- **math.floor(...)** - rounding down, to the previous (smaller) integer number:

```
down = math.floor(45.67) # down = 45
```

- **round(...)** - rounding is done following the **basic rule for rounding** - if the decimal part is less than 5, the rounding is down and vice versa, if it is greater than 5 - up:

```
round(5.439) # 5
round(5.539) # 6
```

- **round(..., [number of symbols after the decimal point])** - rounding to the closest number with a specified number of symbols after the decimal point:

```
round(123.456, 2) # 123.46
round(123, 2) # 123.0
round(123.456, 0) # 123.0
round(123.512, 0) # 124.0
```

Problem: Circle Area and Perimeter

Let's write a program that, when given the **radius r** of a circle, calculates the **area** and the **perimeter** of the circle.

Formulas:

- $\text{Area} = \pi * r * r$

- $\text{Perimeter} = 2 * \pi * r$
- $\pi \approx 3.14159265358979323846\dots$

```
import math

r = float(input('Enter circle radius => r = '))

area = math.pi * r * r
perimeter = 2 * math.pi * r

print('Area = ', area)
print('Perimeter = ', perimeter)
```

Let's try the program with radius $r = 10$:

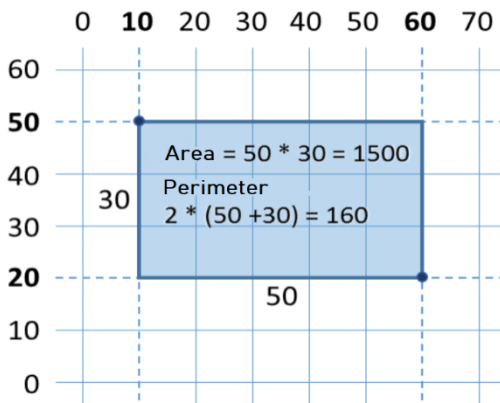
```
Run: C:\Projects\SimpleCalculations\venv\Scripts\python.exe
Enter circle radius => r = 10
Area = 314.1592653589793
Perimeter = 62.83185307179586
Process finished with exit code 0
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#5>.

Problem: 2D Rectangle Area

A rectangle is given with coordinates of its two opposite corners. You have to calculate its area and perimeter:



For this exercise, we have to take into consideration that by subtracting the smaller **x** from the bigger **x** we can get the long side of the rectangle. Analogically, by subtracting the smaller **y** from the bigger **y**, we can get the height of the rectangle. Finally, we have to multiply them. Here is an example implementation of the described logic:

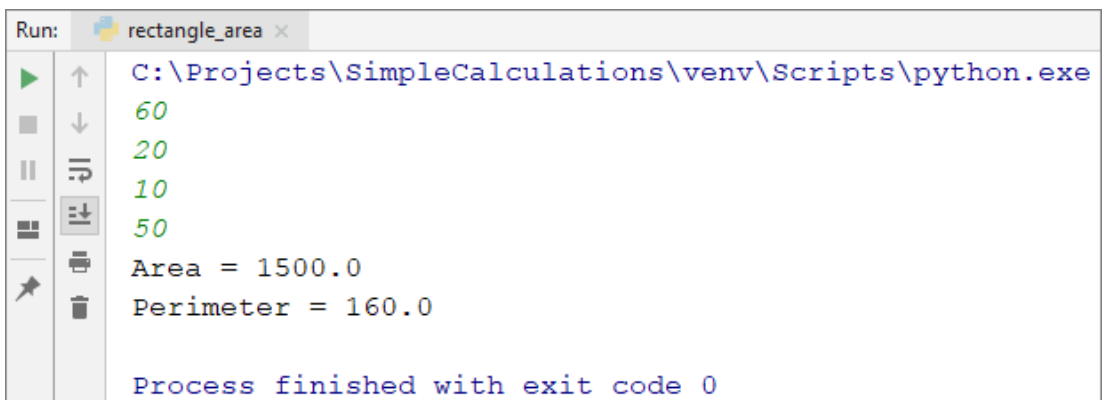
```
x1 = float(input())
y1 = float(input())
x2 = float(input())
y2 = float(input())

width = max(x1, x2) - min(x1, x2)
height = max(y1, y2) - min(y1, y2)

area = width * height
perimeter = 2 * (width + height)

print('Area = ', area)
print('Perimeter = ', perimeter)
```

The function **max(a, b)** is used to find the bigger value between **a** and **b** and analogically **min(a, b)** - to find the smaller value between the two. When starting the program with the values from the coordinate system, the following result is given:



The screenshot shows a terminal window titled "Run: rectangle_area x". The command prompt is "C:\Projects\SimpleCalculations\venv\Scripts\python.exe". The program takes four inputs: 60, 20, 10, and 50. It then outputs "Area = 1500.0" and "Perimeter = 160.0". At the bottom, it says "Process finished with exit code 0".

```
Run: rectangle_area x
C:\Projects\SimpleCalculations\venv\Scripts\python.exe
60
20
10
50
Area = 1500.0
Perimeter = 160.0
Process finished with exit code 0
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#6>.

What Have We Learned from This Chapter?

Let's summarize what we have learned in this chapter:

- Reading text: **str = input()**.
- Reading an integer: **num = int(input())**.
- Reading a floating-point number: **num = float(input())**.
- Calculations with numbers and using the suitable arithmetic operators **[+, -, *, /, ()]**:
sum = 5 + 3.

- Printing a text by placeholders on the console: `print('{0} + {1} = {2}'.format(3, 5, 3 + 5))`.

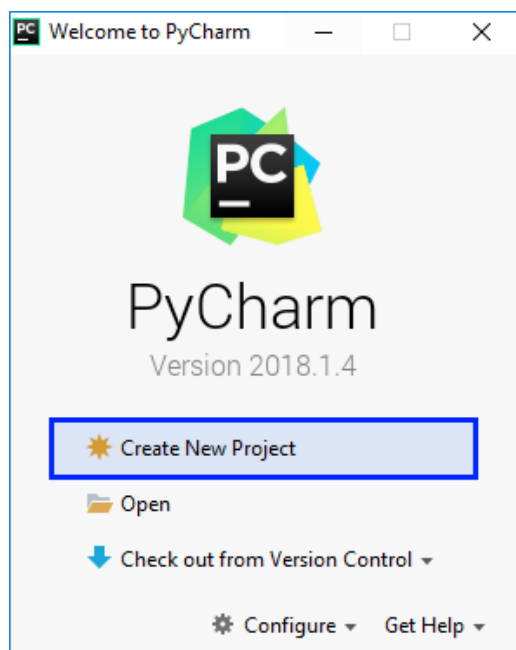
Problems: Simple Calculations

Let's strengthen the knowledge gained throughout this chapter with a few more exercises.

Creating a New Project in PyCharm

We have to create a new project in PyCharm (from [Create New Project] or [File] -> [New Project]) to better organize our problems for exercise. The idea of this **project** is for it to contain **one Python file for each problem** of the exercises:

- We start PyCharm.
- We create a new project: [Create New Project] (or [File] -> [New Project]).



Problem: Triangle Area

Write a program that reads from the console **aside** and a **height of a triangle** and calculates its area. Use the **formula** for triangle area: `area = a * h / 2`. Round the result to **2 digits after the decimal point** using `round(area, 2)`.

Sample Input and Output

Input	Output	Input	Output	Input	Output
20 30	Triangle area = 300	15 35	Triangle area = 262.5	7.75 8.45	Triangle area = 32.74

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#7>.

Problem: Celsius to Fahrenheit

Write a program that reads **degrees on the Celsius scale** (°C) and converts them to **degrees on the Fahrenheit scale** (°F). Look on the Internet for a proper [formula](#) to do the calculations. Round the result to 2 digits after the decimal point.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
25	77	0	32	-5.5	22.1	32.3	90.14

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#8>.

Problem: Radians to Degrees

Write a program, that reads **an angle in radians** (**rad**) and converts it to **degrees** (**deg**). Look for a proper formula on the Internet. The number π in Python programs is available through **math.pi** but before that, we have to refer to **the math library** using **import math**. Round the result to the nearest integer number using the method **round()**.

Sample Input and Output

Input	Output	Input	Output
3.1416	180	0.7854	45
6.2832	360	0.5236	30

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#9>.

Problem: USD to BGN

Write a program for the **conversion of US dollars (USD) to Bulgarian leva (BGN)**. Round the result to 2 digits after the decimal point. Use a fixed rate between a **dollar (USD)** and a **lev (BGN)**: 1 USD = 1.79549 BGN.

Sample Input and Output

Input	Output	Input	Output	Input	Output
20	35.91 BGN	100	179.55 BGN	12.5	22.44 BGN

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#10>.

Problem: * Currency Converter

Write a program for **conversion of money from one currency to another**. It has to support the following currencies: BGN, USD, EUR, GBP. Use the following fixed currency rates:

Exchange Rate	USD	EUR	GBP
1 BGN	1.79549	1.95583	2.53405

The **input** is the **sum for conversion**, **input currency**, and **output currency**. The **output** is one number – the converted value of the above currency rates rounded **2 digits** after the decimal point.

Sample Input and Output

Input	Output
20 USD BGN	35.91 BGN
100 BGN EUR	51.13 EUR

Input	Output
12.35 EUR GBP	9.53 GBP
150.35 USD EUR	138.02 EUR

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#11>.

Problem: * 1000 Days After Birth

Write a program that enters a **birth date** in the format **dd-MM-yyyy** and calculates the date on which **1000 days** are turned since this birth date and prints it in the same format.

Sample Input and Output

Input	Output
25-02-1995	20-11-1997
07-11-2003	02-08-2006

Input	Output
14-06-1980	10-03-1983
01-01-2012	26-09-2014

Input	Output
30-12-2002	24-09-2005

Hints and Guidelines

- Look for information about the data type `datetime` in Python and in particular look at the methods `strptime(str, format)`, `timedelta(days=n)`. With their help, you can solve the problem without the need to calculate days, months, and leap years.
- Don't print anything additional on the console except for the wanted date!

Testing in The Judge System

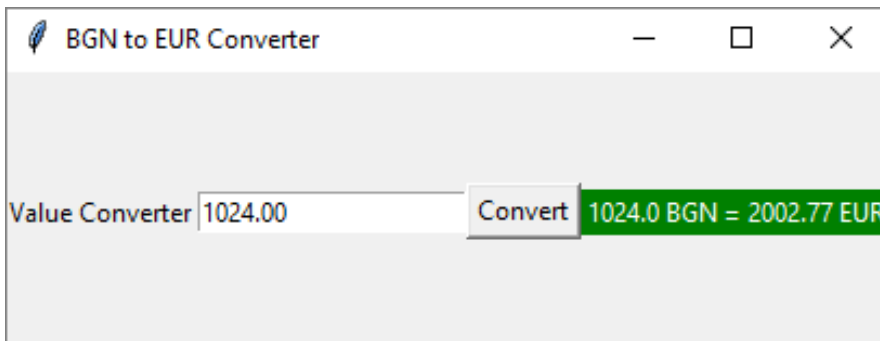
Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1047#12>.

Graphical Applications with Numerical Expressions

To exercise working with variables and calculations with operators and numerical expressions, we will make something interesting: we will develop a **desktop application** with a graphical user interface. We will use calculations with floating-point numbers in them.

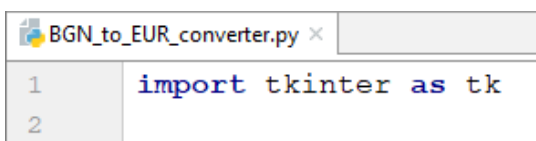
Graphical Application: Converter from BGN to EUR

We need to create a **graphical application** (GUI application), that calculates the value in **Euro** (EUR) of monetary amount given in **Bulgarian leva** (BGN). We use a fixed rate BGN / EUR: 1.95583.



Note: This exercise goes beyond the material learned in this book and aims not to teach you how to program GUI applications, but to strengthen your interest in software development. Let's get to work.

We add to the existing PyCharm project one more Python file. We name it "BGN_to_EUR_converter". To create a Graphical Application using Python we will use the standard library `tkinter`.



First, we create a Graphical **Application**, which represents a rectangle **Frame** for components. At this stage, everything will be done without explanations, as if it is **magic**:

```
import tkinter as tk

class Application(tk.Frame):
    def __init__(self, master=None):
        super().__init__(master)

        self.pack()

# create the application
app = Application()
app.master.title("BGN to EUR Converter")

# start the program
app.mainloop()
```

Now we can add the components of our application to the so-called **function** which we will call with the function **init**, or the so-called **constructor**:

```
def __init__(self, master=None):
    super().__init__(master)

    self.pack()
    self.create_widgets()
```

We order the following UI components:

- **Label** - will serve for a static display of text
- **Entry** - will input the value for conversion
- **Button** - will convert the given value
- One more **Label** will show the result of the conversion.

Our components are found in the function **create_widgets()**. We add text for visualization on the first **Label**, named **label1**. **numberEntry** is **Entry** where the conversion value will be input. **convertButton** will **catch** an event and execute **command** (in our task it will call the function **convert()** which we will write shortly). **output** is our **Label** for displaying a result after we have input a value and we have clicked a button:

```

class Application(tk.Frame):
    def __init__(self, master=None):
        super().__init__(master)

        self.pack()
        self.create_widgets()

    def create_widgets(self):
        # create widgets
        self.label = tk.Label(text="Value Converter")
        self.numberEntry = tk.Entry()
        self.convertButton = tk.Button(text="Convert", command=self.convert)
        self.output = tk.Label()

```

After we have initialized our components, we have to visualize them. This is easily done using the built-in method from **tkinter** - **pack()**:

```

# place widgets
self.label.pack(side="left")
self.numberEntry.pack(side="left")
self.convertButton.pack(side="left")
self.output.pack(side="left")

```

It is left to write the **code** (program logic) for conversion from leva to euro. We will do this by using the function **convert()**:

```

def convert(self):
    entry = self.numberEntry.get()
    value = float(entry)
    result = round(value * 1.95583, 2)

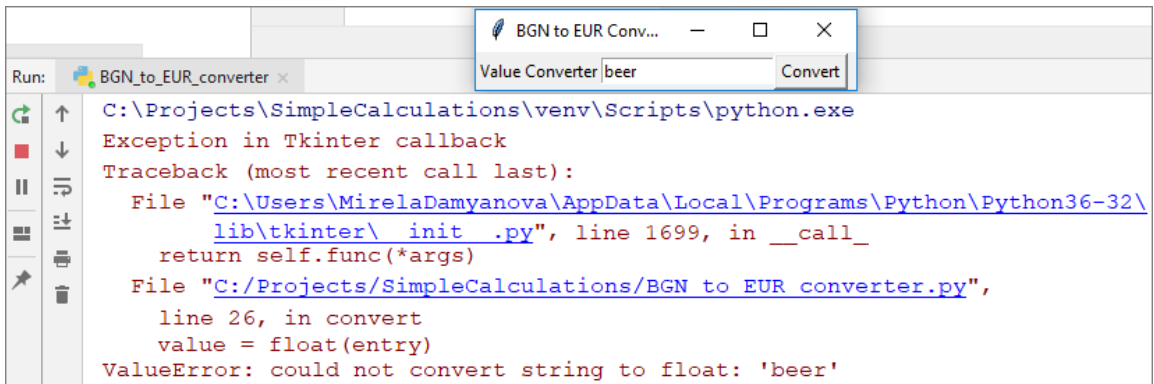
    self.output.config(
        text=str(value) + ' BGN = ' + str(result) + ' EUR',
        bg="green", fg="white")

```

On the last line, we input the result of our **Label output** and we set the color of the background (**bg**) and the color of the text (**fg**).

We run the application with [Ctrl + Shift + F10] or with right-click + [Run], and we test if it works correctly.

We will have a problem with this code. What will happen if we input something different than a number?

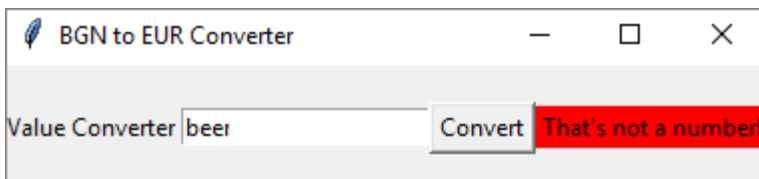


We can **catch** this error and prompt a user-friendly message in our application. To do this, let's change the code of our program logic:

```
def convert(self):
    entry = self.numberEntry.get()

    try:
        value = float(entry)
        result = round(value * 1.95583, 2)
        self.output.config(
            text=str(value) + ' BGN = ' + str(result) + ' EUR',
            bg="green", fg="white")
    except ValueError:
        self.output.config(
            text="That's not a number!",
            bg="red", fg="black")
```

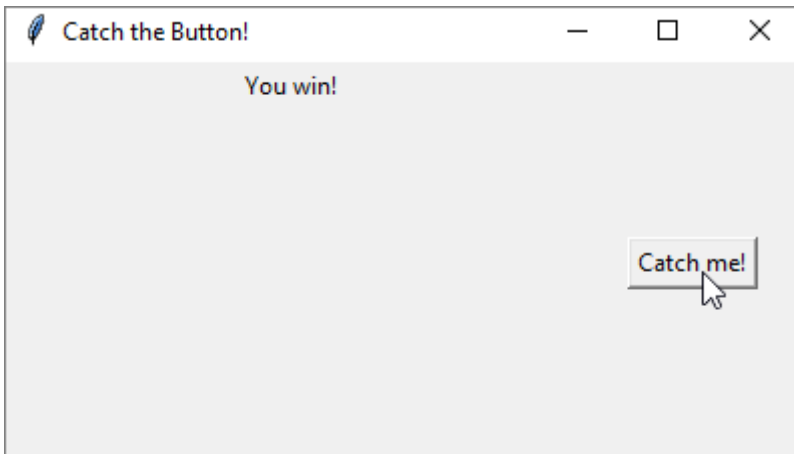
This way we catch the error in a **try block** and when we input something different than a number we will receive a message **That's not a number!**.



Finally, we **run the application** with [Ctrl + Shift + F10] or with right-click + [Run] and we test if it works correctly.

Graphical Application: *** Catch the Button!

Create a fun graphical application **"catch the button"**. Upon moving the mouse cursor onto the button, it moves to a random position. This way it creates the impression that **"the button runs from the mouse and it is hard to catch"**. When the button gets **"caught"**, a congratulations message is shown.



We will begin with the same code from our last exercise. Let's change the name of our application to **Catch the Button!** and this time we will set the window size:

```
import tkinter as tk

class Application(tk.Frame):
    def __init__(self, master=None):
        super().__init__(master)

        self.pack()

# create the application
app = Application()
app.master.minsize(width=400, height=200)

# start the program
app.mainloop()
```

We will have the following components:

- **Button** - the button which we have to **catch**.
- **Label** - the congratulations message.

We create a button containing the text "Catch me!" and **command** - the function **on_click()**, which we will define later. We visualize the components by using the method **pack()**. Let both components be **on the top (top)**:

```

self.pack()
self.create_widgets()

def create_widgets(self):
    # create widgets
    self.button = Button(self, text="Click me")
    self.button.pack()

    # place widgets
    self.button.place(x=10, y=10)

```

In our examples, we will use the so-called **binding** operation. This represents **catching** of a change and executing a specified function. Using this code we instruct our program to execute the function **on_enter()** when the mouse cursor is over the button and to execute the function **on_leave()** when the mouse cursor leaves the button:

```

# bind functions to events
# mouse is over -> move button
# leave button -> clear text
self.button.bind("<Enter>", self.on_enter)
self.button.bind("<Leave>", self.on_leave)

```

To move the button to a random position we use **random**:

```
import random
```

Let's implement the three functions which represent the program logic in our application:

- **on_enter(self, event)** - we select random **x** and **y** coordinates which will change every time we move our cursor over the button; we also change the location of the button - in our example it will be **static to the right**, but you can create your logic and change the direction of the button together with the random numbers selected for coordinates.
- **on_leave(self, event)** - when the cursor of the mouse is not over the button, our congratulation should not appear; we configure our **label** to **not have text**.
- **on_click()** - when we click the button, our **label** will now display text - **You win!**

```

def on_enter(self, event):
    x = random.randrange(0, 100)
    y = random.randrange(0, 100)
    self.button.pack(side="right", padx=x, pady=y)

def on_leave(self, event):
    self.button.config(text="")

def on_click(self):
    self.button.config(text="You win!")

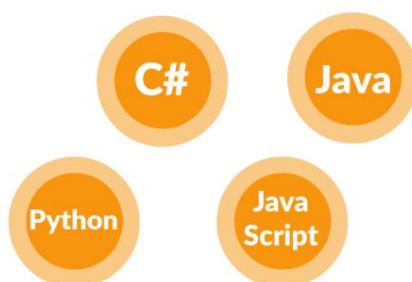
```

Not everything may run smoothly from the first try. These exercises above go **beyond the material learned** and their purpose is to strengthen your interest and make you search the web and **look for solutions** to the emerging problems.

If you are having problems with the tasks above, feel free to ask questions in the **SoftUni's Reddit Community**: <https://www.reddit.com/r/softuni/>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 2.2. Simple Calculations – Exam Problems

In the previous chapter, we explained how to work with the system console – how to **read numbers** from it and how to **print the output**. We went through the main arithmetical operations and briefly mentioned data types. Now, we are going to practice what we have learned by solving a few **more complex exam problems**.

Reading Numbers from The Console

Before going to the tasks, we are going to revise the most important aspects of what we have studied in the previous chapter. We will start by reading numbers from the console.

Reading an Integer

We need to create a variable to store the integer (for example, **num**) and use a standard command for reading input from the console (the function **input(...)**), combined with the function **int(...)** which converts a string to an integer:

```
num = int(input())
```

Reading a Floating-Point Number

We read a floating-point number, the same way we read an integer one, but this time we use the function **float(...)**:

```
num = float(input())
```

Printing Text Using Placeholders

A **placeholder** is an expression that is replaced with a particular value while printing an output. The function **print(...)** supports printing a string based on a placeholder, there are a few ways to do that, as we reviewed in the previous chapter:

- The first argument that we pass to the function is the formatted string, followed by the symbol **%** and the number of arguments, equal to the number of placeholders, placed in brackets:

```
print("You are %s %s, a %d-years old person from %s." % ("Ivan",  
"Ivanov", "16", "Burgas"))
```

- The second way is the so-called "f-strings", where before the formatted string we put the symbol **f** and the arguments are in the string between curled brackets **{}**:

```
first_name = "Ivan"
```

```
last_name = "Ivanov"  
age = 21  
town = "Burgas"  
print(f"You are {first_name} {last_name}, a {age}-years old person  
from {town}.")
```

There are other ways to format a string and the more curious of you can check them in the following article, as well as find the difference between them: <https://cito.github.io/blog/f-strings/>

Arithmetic Operators

Let's revise the main arithmetic operators for simple calculations.

Operator +

```
result = 3 + 5 # the result is 8
```

Operator -

```
result = 3 - 5 # the result is -2
```

Operator *

```
result = 3 * 5 # the result is 15
```

Operator / and //

```
result = 5 / 2 # the result is 2.5 (fractional division)  
result2 = 7 // 3 # the result is 2 (integer division)
```

Concatenation

By using the operator `+` between string variables (or between a string and a number), concatenation is being performed (combining strings):

```
firstName = "Ivan";  
lastName = "Ivanov";  
age = 19;  
str = firstName + " " + lastName + " is " + age + " years old";  
# Ivan Ivanov is 19 years old
```

Exam Problems

Now, after having revised how to make simple calculations and how to read and print numbers from the console, let's go to the tasks. We will solve a few **problems from a SoftUni practical exam**.

Problem: Training Lab

A **training lab** has a rectangular size **l** x **w** meters, without columns on the inside. The hall is divided into two parts- left and right, with a hallway approximately in the middle. In both parts, there are **rows with desks**. In the back of the hall, there is a big **entrance door**. In the front, there is a **podium** for the lecturer. A single **working place** takes up **70 x 120 cm** (a table with size 70 x 40 cm + space for a chair and passing through with size 70 x 80 cm). **The hallway width is at least 100 cm**. It is calculated that due to the **entrance door** (which has 160 cm opening) **exactly one working space is lost**, and due to the **podium** (which has a size of 160 x 120 cm) **exactly two working spaces are lost**. Write a program that reads the size of the training lab as input parameters and calculates the **number of working places in it** (look at the figure).

Input Data

Two numbers are read from the console, one per line: **l** (length in meters) and **w** (width in meters).

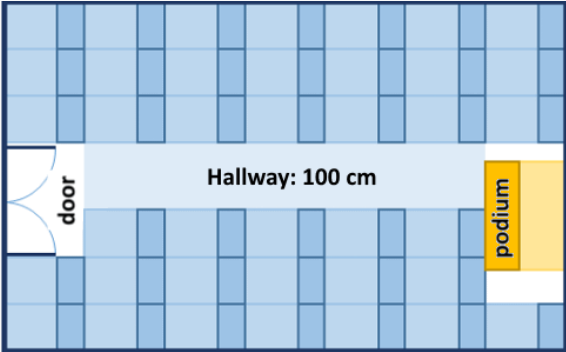
Constraints: $3 \leq w \leq l \leq 100$.

Output Data

Print an integer in the console: **the number of working places** in the training lab.

Sample Input and Output

Input	Output	Figure
15 8.9	129	

Input	Output	Figure
8.4 5.2	39	

Clarification of The Examples

In the first example, the hall length is 1500 cm. **12 rows** can be situated in it ($12 \cdot 120 \text{ cm} = 1440 + 60 \text{ cm difference}$). The hall width is 890 cm. 100 cm of them are for the hallway in the middle. The rest 790 cm can be situated by **11 desks per row** ($11 \cdot 70 \text{ cm} = 770 \text{ cm} + 20 \text{ cm difference}$). **Number of places** = $12 \cdot 11 - 3 = 132 - 3 = 129$ (we have 12 rows with 11 working places = 132 minus 3 places for podium and entrance door).

In the second example, the hall length is 840 cm. **7 rows** can be situated in it ($7 \cdot 120 \text{ cm} = 840$, no difference). The hall width is 520 cm. 100 cm from them is for the hallway in the middle. The rest 420 cm can be situated by **6 desks per row** ($6 \cdot 70 \text{ cm} = 420 \text{ cm}$, no difference). **Number of places** = $7 \cdot 6 - 3 = 42 - 3 = 39$ (we have 7 rows with 6 working places = 42 minus 3 places for podium and entrance door).

Hints and Guidelines

Try to solve the problem on your own first. If you do not succeed, go through the hints.

Idea for Solution

As with any programming task, **it is important to build an idea for its solution**, before having started to write code. Let's carefully go through the problem requirements. We have to write a program that calculates the number of working places in a training lab, where the number depends on the hall length and height. We notice that the provided input data will be **in meters** and the information about how much space the working places and hallway take, will be **in centimeters**. To do the calculations, we will use the same measuring units, no matter whether we choose to convert length and height into centimeters or the other data in meters. The first option is used for the presented solution.

Next, we have to calculate **how many columns and how many rows** with desks will fit. We can calculate the columns by **subtracting the width by the necessary space for the hallway (100 cm) and dividing the difference by 70 cm** (the length of a workplace). We find the rows by **dividing the length by 120 cm**. Both operations can result in **a real number** with a whole and a fractional part, but we have to **store only the whole part in a variable**. In the end, we

multiply the number of rows by the number of columns and divide it by 3 (the lost places for entrance door and podium). This is how we calculate the needed value.

Choosing Data Types

From the example, we see that a real number with whole and fractional parts can be given as an input, therefore, it is not appropriate to choose data type **int**. This is why we use **float**. Choosing a data type for the next variables depends on the method we choose to solve the problem. As with any programming task, this one has **more than one way to be solved**.

Solution

It is time to go to the solution. We can divide it into three smaller tasks:

- Reading input from the console.
- Doing the calculations.
- Printing the output on the console.

The first thing we have to do is read the input from the console. With **input(...)** we read the values from the console and with the function **float(...)** string is converted into **float**.

```
length = float(input()) * 100
width = float(input()) * 100
```

Let's move to the calculations. The special part here is that after having divided the numbers, we have to store only the whole part of the result in a variable.



Search in Google! Whenever we have an idea how to solve a particular problem, but we do not know how to write it in Python or we are dealing with one that many other people have had before us, the easiest way to solve it is by looking for information on the Internet.

In this case, we can try with the following search: "*Python gets whole number part of float*". One possible way is to use the method **math.trunc(...)** and don't forget to refer to the **math** library. The code down below is blurred on purpose and it should be completed by the reader:

```
import math
length = float(input()) * 100
width = float(input()) * 100
seats = math.trunc(length * width / 3)
```

With **print(...)** we print the result in the console:

```
print(seats)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1048#0>.

Problem: Vegetable Market

A gardener is selling his harvest on the vegetable market. He is selling **vegetables** for **N BGN per kilogram** and **fruits** for **M BGN per kilogram**. Write a program that **calculates the earnings of the harvest in Euro**. (Assume that **1 EUR** is equal to **1.94 BGN**).

Input Data

Four numbers are read from the console, one per line:

- First line: Vegetable price per kilogram – a floating-point number.
- Second line: Fruit price per kilogram – a floating-point number.
- Third line: Total kilograms of vegetables – an integer.
- Fourth line: Total kilograms of fruits – an integer.

Constraints: all numbers will be within the range from 0.00 to 1000.00.

Output Data

Print on the console **one floating-point number**: the earnings of all fruits and vegetables in EUR.

Sample Input and Output

Input	Output
0.194 19.4 10 10	101

Clarification for the first example:

- Vegetables cost: 0.194 BGN. * 10 kg. = **1.94 BGN**.
- Fruits cost: 19.4 BGN. * 10 kg. = **194 BGN**.
- Total: **195.94 BGN. = 101 EUR**.

Input	Output
1.5 2.5 10 10	20.6185567010309

Hints and Guidelines

First, we will give a few ideas, followed by particular hints for solving the problem and the essential part of the code.

Idea for Solution

Let's first go through the problem requirements. In this case, we have to calculate **the total income** from the harvest. It equals **the sum of the earnings from the fruits and vegetables** which we can calculate by multiplying **the price per kilogram by the quantity**. The input is given in BGN and the output should be in euros. It is assumed that 1 EUR equals 1.94 BGN, therefore, to get the wanted **output value**, we have to divide the sum by 1.94.

Choosing Data Types

After we have a clear idea of how to solve the task, we can continue with choosing appropriate data types. Let's go through the **input**: we have **two integers** for total kilograms of vegetables and fruits, therefore, the variables we declare to store their values will be of **int** type. The prices of the fruits and vegetables are said to be **two floating-point numbers** and therefore, the variables will be of **float** type.

We can also declare two variables to store the income from the fruits and vegetables separately. The **output** should be a **floating-point number**, so the result should be **float** type.

Solution

It is time to get to the solution. We can divide it into three smaller tasks:

- Reading input from the console.
- Doing the calculations.
- Printing the output on the console.

To read the input, we declare variables, which we have to name carefully so that they can give us a hint about the values they store. With **input(...)**, we read values from the console and with the functions **int(...)** and **float(...)**, we convert the particular string value into int and double:

```
vegetables_price = float(input())
fruits_price = float(input())
vegetables_quantity = int(input())
fruits_quantity = int(input())
```

We do the necessary calculations:

```
vegetables_total_price = vegetables_price * vegetables_quantity
fruits_total_price = fruits_price * fruits_quantity
```

The task does not specify a special output format, therefore, we just have to calculate the requested value and print it on the console. As in mathematics and so in programming, the division has a priority over addition. However, in this task, first, we have to **calculate the sum** of the two input values and then **divide by 1.94**. To give priority to addition, we can use brackets. With **print(...)** we print the output on the console:

```
print((vegetables_total_price + fruits_total_price) / 1.94)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1048#1>.

Problem: Change Tiles

On the ground in front of an apartment building **tiles need to be placed**. The ground has a **square shape with a side of N meters**. The tiles are **"W" meters wide** and **"L" meters long**. There is one bench on the ground with a **width of "M" meters** and a **length of "O" meters**. There is no need to place tiles under it. Each tile is replaced for **0.2 minutes**.

Write a program that **reads from the console the size of the ground, the tiles and the bench**, and calculates **how many tiles are needed** to cover the ground and what is the total **time for placing all of the tiles**.

Example: ground with size 20 m has an **area of 400 m2**. A bench that is 1 m wide and 2 m long, has an area of **2 m2**. One **tile is 5 m wide and 4 m long** and has an **area of 20 m2**. The **space that needs to be covered is $400 - 2 = 398$ m2**. $398 / 20 = 19.90$ tiles are necessary. The **time needed is $19.90 * 0.2 = 3.98$ minutes**.

Input Data

The input data comes as **5 numbers**, which are read from the console:

- **N – length of a side of the ground** within the range of [1 ... 100].
- **W – width per tile** within the range of [0.1 ... 10.00].
- **L – length per tile** within the range of [0.1 ... 10.00].
- **M – width of the bench** within the range of [0 ... 10].
- **O – length of the bench** within the range of [0 ... 10].

Output Data

Print on the console **two numbers**:

- **number of tiles** needed for the repair
- **total time for placing them**

Each number should be on a new line and rounded to the second decimal place.

Sample Input and Output

Input	Output
20	
5	19.9
4	3.98
1	
2	

Explanation of the example:

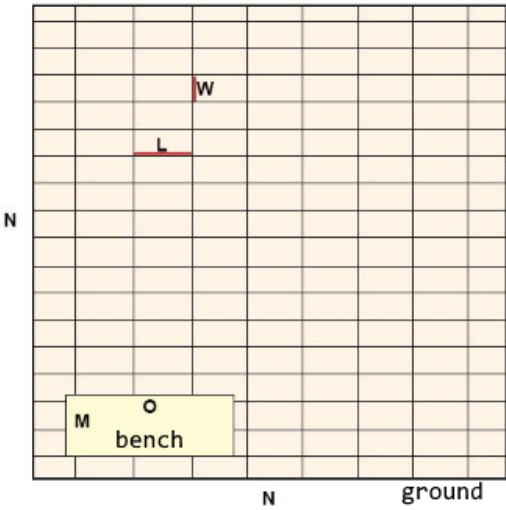
- **Total area = $20 * 20 = 400$.**

- Area of the bench = $1 * 2 = 2$.
- Area for covering = $400 - 2 = 398$.
- Area of tiles = $5 * 4 = 20$.
- Needed tiles = $398 \setminus 20 = 19.9$.
- Needed time = $19.9 * 0.2 = 3.98$.

Input	Output
40	
0.8	
0.6	3302.083333333333
3	660.4166666666667
5	

Hints and Guidelines

Let's make a draft to clarify the task requirements. It can look the following way:



Idea for Solution

It is required to calculate the **number of tiles** that have to be placed, as well as the **time for placing them**. To find that number, we have to calculate **the area that needs to be covered** and **divide it by the area per tile**. By the requirements of the problem, the ground is square, therefore, we find the total area by multiplying its side by its value **$N * N$** . After that, we calculate **the area that the bench takes up** by multiplying its two sides as well **$M * O$** . After subtracting the area of the bench from the area of the whole ground, we obtain the area that needs to be repaired.

We calculate the area of a single tile by **multiplying its two sides with one another $W * L$** . As we already stated, now we have to **divide the area for covering by the area of a single tile**. This way, we find the number of necessary tiles. We multiply it by **0.2** (the time needed for changing a tile). Now, we have the wanted output.

Choosing Data Types

The length of the side of the ground, the width and the length of the bench will be given as **integers**, therefore, to store their values, we can use the system function **int(...)**. We will be given floating-point numbers for the width and the length of the tiles and this is why we will use **float(...)**.

Solution

As in the previous tasks, we can divide the solution into three smaller tasks:

- Reading the input.
- Doing the calculations.
- Printing the output on the console.

The first thing we have to do is go through **the input** of the task. It is important to pay attention to the sequence they are given in. With **input(...)** we read values from the console and with **int(...)** and **float(...)**, we convert the particular string value into **int** or **float**:

```
# Ground length
n = int(input())
# Tile width
w = float(input())
# Tile length
l = float(input())
# Bench width
m = int(input())
# Bench length
o = int(input())
```

After we have initialized the variables and have stored the corresponding values in them, we move to the **calculations**. The code below is blurred on purpose, so the reader can think by himself over it:

```
calculating_area = n * l
bench_area = m * o
area_with_tiles = calculating_area - bench_area
tiles_count = n * l
```

We calculate the values that we have to print on the console. The number of necessary tiles is obtained by **dividing the area** that needs to be covered **by the area of a tile**:

```
tiles_count = area_with_tiles / tile_area
time = tiles_count * 0.2
```

In the task is specified that the number of the output should be rounded **to the second decimal place**. That is why we cannot just print the value with **print(...)**. We will use the method **round(...)**, which is rounding to the closest integer number with exactly n-numbers after the decimal place. For example, the number 1.35 will be rounded to 1, and 1.65 - to 2:

```
print(round(tiles_count, 2))
print(round(time, 2))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1048#2>.

Problem: Money

Some time ago, **Pesho** bought **Bitcoins**. Now, he is going on vacation in Europe and he needs **euros**. Apart from **Bitcoins**, he has **Chinese yuans** as well. Pesho wants to **exchange his money for euros** for the tour. Write a program that **calculates** how much euro he can buy, depending on the following exchange rates:

- 1 Bitcoin = 1168 BGN.
- 1 Chinese yuan (CNY) = 0.15 USD.
- 1 USD = 1.76 BGN.
- 1 EUR = 1.95 BGN.

The exchange office has a **commission fee** of 0% to 5% from the final sum in euro.

Input Data

Three numbers are read from the console:

- On the first line – **number of Bitcoins**. Integer within the range of [0 ... 20].
- On the second line – **number of Chinese yuans**. Floating point number within the range of [0.00 ... 50 000.00].
- On the third line – **commission fee**. Floating point number within the range of [0.00 ... 5.00].

Output Data

Print one number on the console – **the result of the exchange of currencies**. The output should be rounded **to the second decimal place**.

Sample Input and Output

Input	Output
1	569.67
5	
5	

Explanation:

- 1 Bitcoin = 1168 BGN
- 5 Chinese yuan (CNY) = 0.75 USD
- 0.75 USD = 1.32 BGN
- **1168 + 1.32 = 1169.32 BGN = 599.651282051282 EUR**
- **Commission fee: 5% of 599.651282051282 = 29.9825641025641**
- **Result: 599.651282051282 - 29.9825641025641 = 569.668717948718 = 569.67 EUR**

Input	Output	Input	Output
20 5678 2.4	12442.24	7 50200.12 3	10659.47

Hints and Guidelines

Let's first think of the way we can solve the task, before having started to write code.

Idea for Solution

We see that the **number of bitcoins** and **the number of Chinese yuans** will be given in the input. The **output** should be in euros. The exchange rates that we have to work with are specified in the task. We notice that we can only exchange the sum in BGN to EUR, therefore, we **first have to calculate the whole sum** that Pesho has in BGN, and then **calculate the output**.

As we have information for the exchange rate of Bitcoins to BGN, we can directly exchange them. On the other hand, to get the value of **Chinese yuans in BGN**, first, we have to **exchange them in USD**, and then the **USD to BGN**. Finally, we will **sum the two values** and calculate how much euro that is.

Only the final step is left: **calculating the commission fee** and subtracting the new sum from the total one. We will obtain a **floating-point number** for the commission fee, which will be a particular **percent of the total sum**. Let's divide it by 100, to calculate its **percentage value**. Then we will multiply it by the sum in euro and divide the result from the same sum. Print the final sum on the console.

Choosing Data Types

Bitcoins are given as an **integer**, therefore, we can declare a variable using the **int(...)** function. For **Chinese yuan** and **commission fee**, we obtain a **floating-point number**, therefore, we are going to use **float(...)**.

Solution

After we have built an idea on how to solve the task and we have chosen the data structures that we are going to use, it is time to get to **writing the code**. As in the previous tasks, we can divide the solution into three smaller tasks:

- Reading input from the console.
- Doing the calculations.
- Printing the output on the console.

We declare the variables that we are going to use and again we have to choose **meaningful names**, which are going to give us hints about the values they store. We initialize their values: we create variables, where we will store the string arguments passed to the function and convert them to int or double:

```
bitcoins = int(input())
yuans = float(input())
commission = float(input()) / 100
```

Then we do the necessary calculations:

```
bitcoins_to leva = bitcoins * 1168
yuans_to_dollars = yuans * 0.15
dollars_to_leva = yuans_to_dollars * 1.76
```

```
Sum = bitcoins_to_leva + dollars_to_leva
euro = Sum / 1.95
```

Finally, we **calculate the commission fee** and **subtract it from the sum in euro**. Let's pay attention to the way we could write this: **euro -= commission * euro** is the short way to write **euro = euro - (commission * euro)**. In this case, we use a **combined assignment operator (-=)** that **subtracts the value of the operand to the right from the one to the left**. The operator for multiplication (*) has a **higher priority** than the combined operator, this is why the expression **commission * euro** is performed first and then its value is divided.

Finally, we have to print the result in the console. We should notice that we have to format the output **to the second decimal place**. The difference between this and the previous task is that here, even if the number is an integer, **we have to print it to the second decimal place** (for example **5.00**). One way to do so is to use the function **print(...)** and to format a string using a placeholder **%.2f**. By using it, we can convert a number into a string, saving the specified numbers after the decimal point:

```
euro -= euro * commission
print("%.2f" % euro)
```

Let's pay attention to something that applies to all other problems of this type: written like that, the solution of the task is pretty detailed. As the task itself is not too complex, in theory, we could write one big expression, where right after having taken the input, we calculate the output. For example, such an expression would look like this:

```
euro = ((bitcoins * 1168) + (yuans * 0.15 * 1.76)) / 1.95  
- (commission * ((bitcoins * 1168) + (yuans * 0.15 * 1.76)) / 1.95)
```

This code would print a correct result, **but it is hard to read**. It won't be easy to find out how it works and whether it contains any mistakes, as well as finding such and correcting them. **Instead of one complex expression**, it is **better to write a few simpler ones** and store their values in variables with appropriate names. This way, the code is cleaner and easily maintainable.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1048#3>.

Problem: Daily Earnings

Ivan is a programmer in an American company, and he **works** at home **approximately N days per month** by earning **approximately M USD per day**. At the end of the year, Ivan **gets a bonus**, which **equals 2.5 of his monthly salary**. In addition, **25% of his annual salary goes for taxes**. Write a program that **calculates what is the amount of Ivan's net average earnings in BGN per day**, as he spends them in Bulgaria. It is accepted that one year has exactly 365 days. The exchange rate of US USD to BGN will **be passed to a function**.

Input Data

Three numbers are read from the console.

- On the first line – **workdays per month**. An integer within the range of [5 ... 30].
- On the second line – **daily earnings**. A floating-point number within the range of [10.00 ... 2000.00].
- On the third line – **exchange rate of USD to BGN** /1 dollar = X BGN/. A floating-point number within the range of [0.99 ... 1.99].

Output Data

Print one number on the console – **the average daily earnings in BGN**. The result will be rounded up to the second digit after the decimal point.

Sample Input and Output

Input	Output
21 75.00 1.59	74.61

Explanation:

- One monthly salary = $21 * 75 = 1575$ USD.
- Annual income = $1575 * 12 + 1575 * 2.5 = 22837.5$ USD.

- Taxes = 25% of 22837.5 = 5709.375 USD.
- Net annual income in USD = 17128.125 USD = 27233.71875 BGN.
- Average earnings per day = 27233.71875 / 365 = 74.61 BGN.

Input	Output	Input	Output
15	80.24	22	196.63
105		199.99	
1.71		1.50	

Hints and Guidelines

Firstly, we have to analyze the task and think of a way to solve it. Then, we will choose data types and, finally, we will write the code.

Idea for Solution

Let's first calculate **how much the monthly salary** of Ivan is. We do that by **multiplying the working days per month by his daily earnings**. We **multiply the result** by 12, to calculate his salary for 12 months, and then, we multiply it **by 2.5**, so that we can calculate the bonus. After having summed up the two values, we calculate **his annual income**. Then, we will reduce **the annual income by 25% taxes**. We can do that by multiplying the total income by **0.25** and subtracting the result out of it. Depending on the exchange rate, **we exchange the USD to BGN** and after that, we divide the result by 365.

Choosing Data Types

The **working days** per month are given as **an integer**, therefore, we can convert the string into an int with the function **int(...)**. For both **the earned money** and **the exchange rate of USD to EUR**, we will obtain a **floating-point number**, therefore, we will use the function **float(...)**.

Solution

Again: after we have an idea of how to solve the problem and we have considered the data types that we are going to use, we can start **writing the program**. As in the previous tasks, we can divide the solution into three smaller tasks:

- Reading the input.
- Doing the calculations.
- Printing the output on the console.

Then we **declare the variables** that we are going to use by trying to choose **meaningful names**. We create a variable to store the arguments passed to the function, by converting the string to integer or floating number by using **int(...)/float(...)**:

```
working_days = int(input())
monthly_pay = float(input())
exchange_rate = float(input())
```

After that we do the calculations:

```
monthly_salary = working_days * profit_per_day
annual_earnings = monthly_salary * 12 + monthly_salary * 2.5
taxes = annual_earnings * 0.25
net_annual_earnings = (annual_earnings - taxes)
salary_in_leva = net_annual_earnings * currency_rate
```

We could write an expression that calculates the annual income without brackets as well. As multiplication is an operation that has a higher priority over addition, it will be performed first. Despite that, **writing brackets is recommended when using more operators**, as this way the code is **easily readable** and chances of making a mistake are smaller.

Finally, we have to print the result on the console. We notice **that the number has to be rounded up to the second digit after the decimal point**. To do that, we can use the same placeholder, just like the previous task:

```
print("%.2f" % (salary_in_leva / 365))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1048#4>.

Chapter 3.1. Simple Conditions

In the current chapter, we're going to be taking a look at the **conditional constructs in the Python programming language**. By implementing these constructs, our program can produce a different output based on a given specific input. We will explain the syntax of the conditional operators (**if**, **if-elif** and **else**) by implementing appropriate examples and also, we are going to take a look at the range in which a variable lives (its **scope**). Finally, we will go over different **debugging** techniques, to follow the programming steps through which our program goes during its run.

Number Comparison

In programming, we can compare values through the use of the following **operators**:

- Operator **<** (less than)
- Operator **>** (greater than)
- Operator **<=** (less than or equals)
- Operator **>=** (greater than or equals)
- Operator **==** (equals)
- Operator **!=** (not equal; different than)

The result from a comparison is the so-called Boolean value – **True** or **False**, depending on the evaluated result being either true or false.

Problems of Number Comparisons

```
a = 5
b = 10
print(a > b)           # False
print(a == b)          # False
print(a < 10)           # True
print(b < 10)           # False
print(a != b)           # True
print(b >= 10)          # True
print(b == a * 2)       # True
```

Note that when printing **true** and **false** values in **Python**, they are capitalized, **True** and **False**, respectively.

Comparison Operators

In Python, we can use the following operators to compare data:

Operator	Notation	Applicable for
Equals	==	numbers, strings, dates

Not equal	!=	numbers, dates, other comparable data types
Greater than	>	
Greater than or equal	>=	
Less than	<	
Less than or equal	<=	

Here is an example:

```
result = "hello" == "Hello"
print(result)  # False
```

Simple If Comparisons

In programming, we often **check particular conditions** and perform various actions depending on the result of the comparison. This is done through **if** clauses, which have the following structure:

```
if condition:
    # body of the conditional construct
```

Problem: Excellent Result

We take the grade as an input in the console and check if it is excellent (≥ 5.50).

```
grade = float(input())
if grade >= 5.5:
    print("Excellent!")
```

Test the example code locally. Try entering different grades, for example, 4.75, 5.49, 5.50 and 6.00. For grades **less than 5.50** the program will not give any output, however for grades of **5.50 or greater**, the output will be "Excellent!".

Testing in The Judge System

You can test the solution example here:
<https://judge.softuni.org/Contests/Practice/Index/1049#0>.

If-Else Conditional Constructs

The **if** conditional can also have an **else** option to provide a specific action to be performed in case the Boolean expression (which is specified at the beginning **if Boolean expression**) returns a negative result (**False**). Written in this way, the **conditional statement** is called **if-else** and its behavior is as follows: if the result of the condition is

positive (**True**) - a set of instructions is executed. By contrast, when the result is **negative** (**False**) - a different set is executed. The format of this structure is as shown:

```
if condition:
    # Condition body to be executed if a condition is true
else:
    # else structure body to be executed if a condition is false
```

Problem: Excellent or Not

Similarly, to the example above, we input a grade and check if it is excellent, but this time we should **output a result in both cases**:

```
grade = float(input())
if grade >= 5.5:
    print("Excellent!")
else:
    print("Not excellent.")
```

Testing in Judge System

You can test your solution at the following link:

<https://judge.softuni.org/Contests/Practice/Index/1049#1>.

About Blocks of Code

By pressing the **tab** key, we create a block of code through which a group of commands can be executed. When we have code in **if**, **elif**, **else** (and other structures) and we want to perform a series of operations, we put them in a block after the condition.



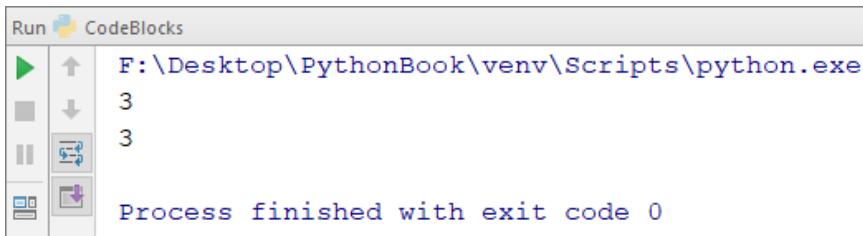
It is a good practice **to use a tab (or four spaces)** since this makes the code more readable, neater and cleaner. In this way, we avoid errors during code execution.

Here is an example of bad indentation:

```
firstNum = 4
secondNum = 1

if firstNum > secondNum:
    result = firstNum - secondNum
    print(result)
elif secondNum > firstNum:
    result = secondNum - firstNum
print(result)
```

The above code will either give an error because it is incorrectly formatted, or its execution will display the wrong result on the console:

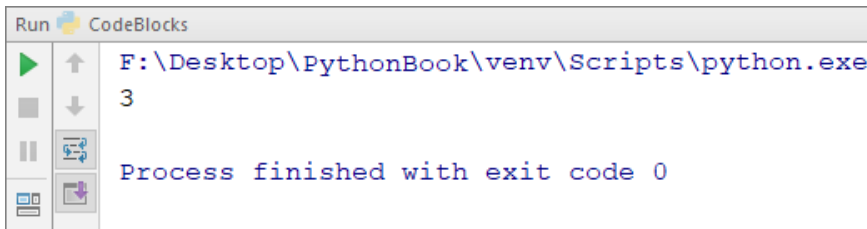


```
Run CodeBlocks
F:\Desktop\PythonBook\venv\Scripts\python.exe
3
3
Process finished with exit code 0
```

With correct indentation:

```
firstNum = 4
secondNum = 1
result = 0
if firstNum > secondNum:
    result = firstNum - secondNum
elif secondNum > firstNum:
    result = secondNum - firstNum
print(result)
```

The following output will be printed on the console:



```
Run CodeBlocks
F:\Desktop\PythonBook\venv\Scripts\python.exe
3
Process finished with exit code 0
```

Problem: Even or Odd

Write a program that checks whether a given integer number is **even** or **odd**.

The problem can be solved with a single **if-else** structure and the operator **%**, which returns the **division remainder** of two numbers:

```
number = int(input())
if number % 2 == 0:
    print("even")
else:
    print("odd")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#2>.

Problem: Greater Number

Write a program that reads two integers and outputs the number of higher value between the two.

Our first task is to **read** the two numbers. After which through the use of a simple **if-else** structure, in combination with the **greater than operator (>)**, to perform the comparison. We have deliberately blurred parts of the code so that the reader can implement the learned so far.

```
print("Enter two integers: ")
firstNumber = int(input())
secondNumber = int(input())

if firstNumber > secondNumber:
    print("Bigger number is: ", firstNumber)
else:
    print("Bigger number is: ", secondNumber)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#3>.

The Lifetime of a Variable

Every variable has a scope in which it exists, called **variable scope**. This scope specifies where the variable can be used and accessed. In Python, **variables could be used anywhere if they are initialized at least once**.

In the example below, on the last line, we try to print the variable **my_name**, which is defined in the **else** structure. We will get an **error**, because in this case the body of the **else** clause, in which we initialize the variable, is not executed. However, there is no problem printing the variable **can_drive** because the program entered the body of the **if** clause and initialized the variable. As you can see the variables **can_drive** and **my_name** are colored yellow. This is a warning from **PyCharm**, that we may get an error. Therefore, we should be careful where we initialize the variables.

```
my_age = 20
if my_age >= 18:
    can_drive = True
else:
    my_name = 'Ivan'

print(my_age)
print(can_drive)
print(my_name)
```

Conditional Chaining

Sometimes we have to do a series of checks, before deciding what actions our program will execute. In such cases we can apply the structure **if-elif ... else** in series. For this purpose, we employ the following structure:

```
if condition:
    # condition body;
elif condition2:
    # condition body;
elif condition3:
    # condition body;
...
else:
    # else structure body
```

Problem: Number 0...9 to Text

Print the digits one through nine in English on the console (the numbers are read from the console). We can take the digit and through a **series of conditions** print the corresponding English word on the console:

```
number = int(input())

if number == 1:
    print("one")
elif num == 2:
    print("two")
elif ...:
    ...
elif num == 9:
    print("nine")
else:
    print("number too big")
```

The program logic of the above example **sequentially compares** the input number with the digits from 1 to 9 with **each consecutive comparison being performed only in case the previous result is not true**. Eventually, if none of the **if** conditionals are satisfied, the last **else** clause is executed.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#4>.

Problems: Simple Conditions

To practice the implementation of the conditional constructs **if** and **if-elif** we will take a look at a few practical problems.

Problem: Bonus Score

We are given an **integer** – several points. Additional **bonus points** are awarded as per the rules described below. Write a program that calculates the **bonus points** for the given number and outputs the **total points** including the bonus.

- If the number is **up to 100** inclusive, the bonus points are 5.
- If the number is **larger than 100**, the bonus points are **20%** of the number.
- If the number is **larger than 1000**, the bonus points are **10%** of the number.
- Additional points are awarded as below (added separately from the described above):
 - For **even** numbers -> + 1 p.
 - For numbers, **ending with 5** -> + 2 p.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
20	6 26	175	37 212	2703	270.3 2973.3	15875	1589.5 17464.5

Hints and Guidelines

We can calculate the base and additional bonus score with a series of **if-elif-else** statements. For the **main bonus points we have 3 cases** (the input is less than or equal to 100, it is between 100 and 1000, and finally it is greater than 1000), for the **additional bonus score - further 2 cases** (whether the number is even or odd):

```
points = int(input())
bonus = 0

if points > 1000:
    bonus += points * 0.1
elif #TODO: finish the logic
elif #TODO: finish the logic
if #TODO: finish the logic
if #TODO: finish the logic

print(bonus)
sum = bonus + points
print(sum)
```

Here's what the solution to the problem might look like:

Run CodeBlocks

```
F:\Desktop\PythonBook\venv\Scripts\python.exe
F:\Desktop\PythonBook\PythonBook.py
Enter score: 20
Bonus score: 6
Total score: 26

Process finished with exit code 0
```

Please note that for this problem the Judge system is set up to ignore any non-number outputs, so we may print explanations along with the number output.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#5>.

Problem: Sum Seconds

Three athletes finish with some **number of seconds** (between **1** and **50**). Write a program that reads the times of the contestants and calculates their **combined time** in "minutes:seconds" format. Seconds are to be printed with a **leading zero** (2 -> "02", 7 -> "07", 35 -> "35").

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
35 45 44	2:04	22 7 34	1:03	50 50 49	2:29	14 12 10	0:36

Hints and Guidelines

Firstly, we sum the three numbers, to obtain the total seconds. As we know that **1 minute = 60 seconds**, we should calculate the minutes and seconds in the range 0 to 59:

- If the result is between 0 and 59, we print 0 minutes + calculated seconds.
- If the result is between 60 and 119, we print 1 minute + calculated seconds minus 60.
- If the result is between 120 and 179, we print 2 minutes + calculated seconds minus 120.
- If the seconds are less than 10, we print the number with a leading zero.


```

firstCompetitor = int(input())
# TODO: Read second and third competitors' time
seconds =
    firstCompetitor + secondCompetitor + thirdCompetitor
minutes = 0

if seconds > 59:
    minutes += 1
    seconds = seconds - 60
if seconds > 59:
    minutes += 1
    seconds = seconds - 60
if seconds < 10:
    print(f'{minutes}:0{seconds}')
else:
    print(f'{minutes}:{seconds}')

```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#6>.

Problem: Metric Converter

Write a program, that **converts distance** between the following 8 units of measure: **m, mm, cm, mi, in, km, ft, yd**. You may use the conversion table below:

Input measure	Output measure
1 meter (m)	1000 millimeters (mm)
1 meter (m)	100 centimeters (cm)
1 meter (m)	0.000621371192 miles (mi)
1 meter (m)	39.3700787 inches (in)
1 meter (m)	0.001 kilometers (km)
1 meter (m)	3.2808399 feet (ft)
1 meter (m)	1.0936133 yards (yd)

The input will consist of three parameters:

- First line: A number.
- Second: Input unit of measure.
- Third: Output unit of measure (for the result).

Sample Input and Output

Input	Output	Input	Output	Input	Output
12 km ft	39370.0788	150 mi in	9503999.99393599	450 yd km	0.41147999937455

Hints and Guidelines

We take the input data and to the units of measure, we can add the function `lower()`, which will convert all letters to lower case. As we can see from the conversion table above, we have data for **converting only between meters and any other measuring unit**. To make the conversion, firstly we must calculate the input measurement in meters. To this effect, we need to create a set of conditionals to determine the input measuring unit and then the output.

```
size = float(input())
sourceMetric = input().lower()
destMetric = input().lower()
if sourceMetric == "km":
    size = size / 0.001
# Check the other metrics: mm, cm, ft, yd, ...
if destMetric == "ft":
    size = size * 3.2808399
# Check the other metrics: mm, cm, ft, yd, ...
print(f'{size} {destMetric}')
```

Testing in The Judge System

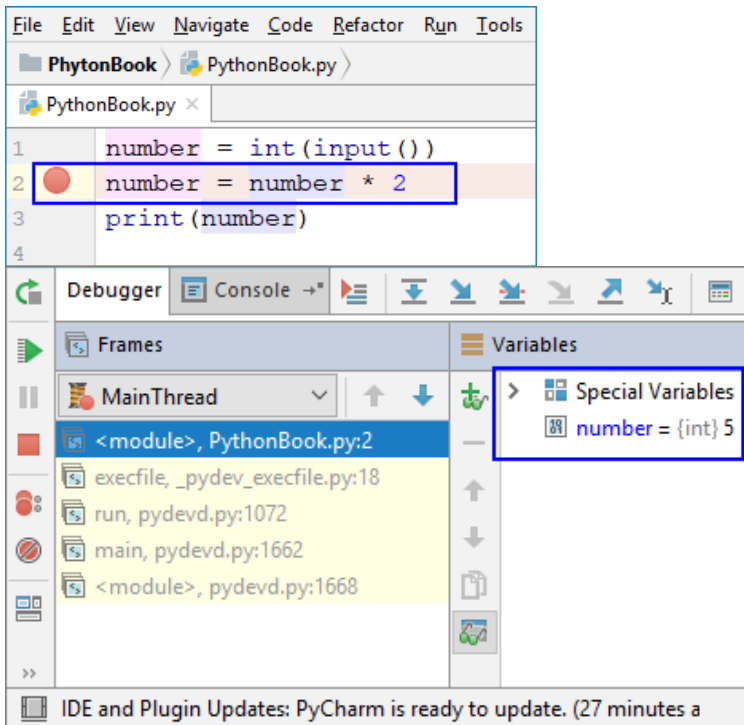
Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#7>.

Debugging - Simple Operations With a Debugger

So far, we have written quite a lot of code and oftentimes there have been mistakes, haven't there? Now we can show you a tool to make finding mistakes easier.

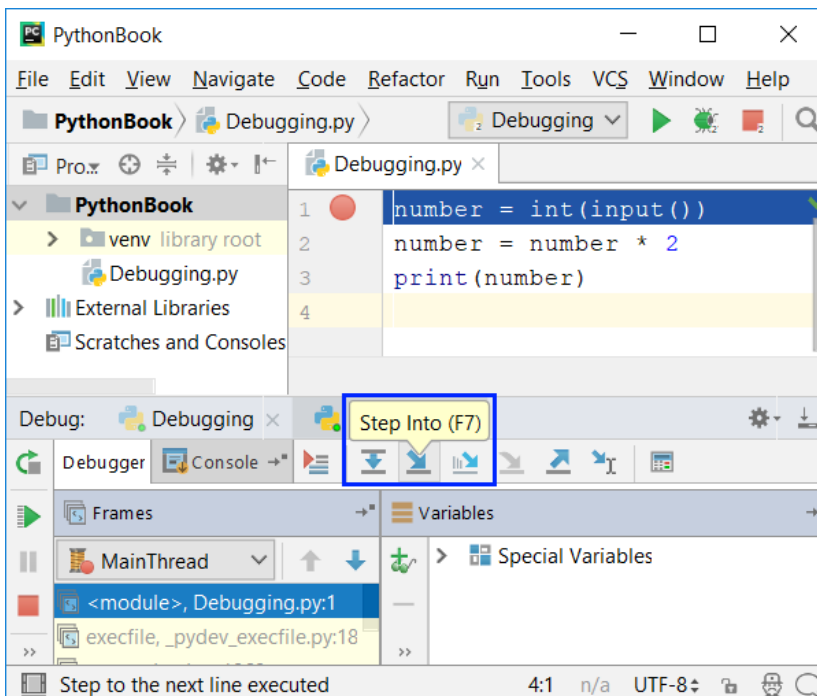
What is "Debugging"?

Debugging is the process of "attaching" to a program's execution, which allows us to follow closely the execution of our program. We can follow **line by line** the events in our program, what its evaluation route is, what the intermediate values of the declared variables at each step of the execution are, among other useful information and thus allowing us to locate errors - the so-called **bugs**:



Debugging in PyCharm

By pressing [Shift + F9], we start the program in **Debug mode**. We move on to the next line of execution with [F7]:



With [Ctrl + F8] we create **breakpoints**, which we can reach directly using [Shift + F9] (when starting the program in **Debug mode**).

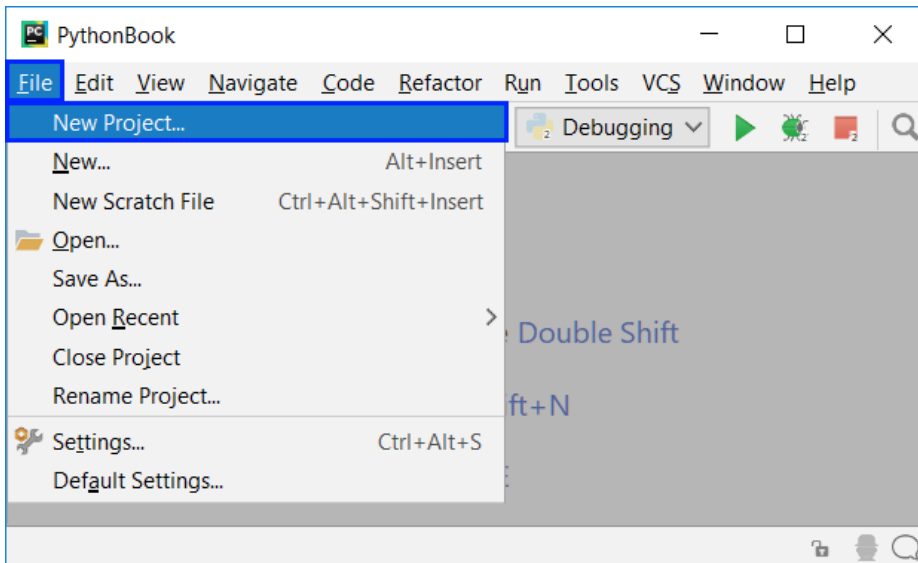
Problems: Simple Conditions

Now let's practice the lessons learned in this chapter with a few practical exercises.

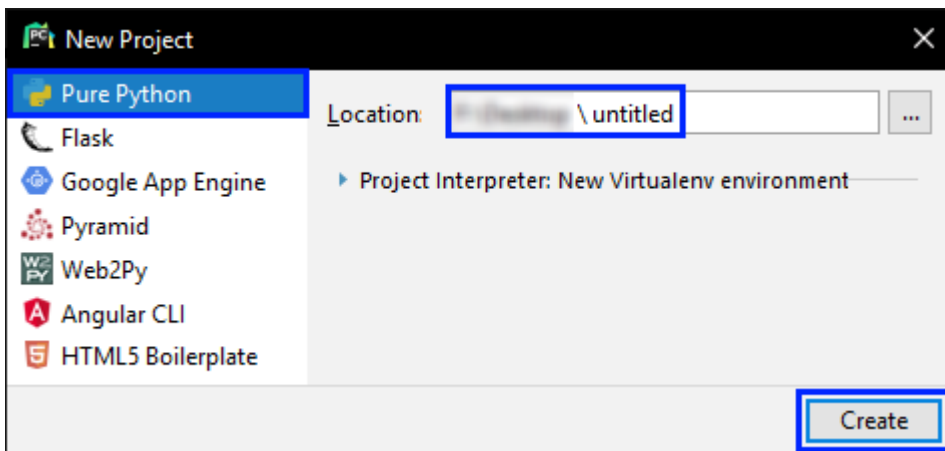
Empty PyCharm Solution (Project)

We create empty solution in PyCharm so we can organize the solutions to the tasks from the exercises – each task will be in a separate file and all tasks will be in the same Project.

We start PyCharm and create a new **Project**: [File] -> [New Project].



Select **Pure Python** from the field on the left and set the project directory, putting the name of your project in place of the **untitled** one:



Now we have an empty project (no files in it).

Problem: Password Guess

Write a program that **reads input data from the console** - a password (one line of random text) and checks if the input **matches** the phrase "s3cr3t!P@ssw0rd". If it matches, print "Welcome", otherwise print "Wrong password!".

Sample Input and Output

Input	Output	Input	Output	Input	Output
qwerty	Wrong password!	s3cr3t!P@ssw0rd	Welcome	s3cr3t!p@ss	Wrong password!

Hints and Guidelines

Use an **if-else** statement.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#8>.

Problem: Number 100...200

Write a program that **reads input data from the console** - an integer and checks if it is **below 100**, **between 100 and 200** or **over 200**. Print the appropriate messages as per the examples below.

Sample Input and Output

Input	Output	Input	Output	Input	Output
95	Less than 100	120	Between 100 and 200	210	Greater than 200

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#9>.

Problem: Equal Words

Write a program that **reads input data from the console** - two words and checks if they are the same. A comparison should be case-insensitive and the output should be either "yes" or "no".

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
Hello Hello	yes	SoftUni softuni	yes	Soft Uni	no	beer vodka	no

Hints and Guidelines

Before the comparison, both words should be in lower case, so that case (uppercase / lowercase) does not influence the result `word = word.lower()`.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#10>.

Problem: Speed Info

Write a program, that reads input data from the console - speed (decimal number) and prints speed information. For speed up to 10 (inclusive), print "slow". For speed over 10 up to 50, print "average". For speed over 50 and up to 150, print "fast". For speed over 150 and up to 1000, print "ultra fast". For any higher speed, print "extremely fast".

Sample Input and Output

Input	Output	Input	Output	Input	Output
8	slow	126	fast	3500	extremely fast
49.5	average	160	ultra fast		

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#11>.

Problem: Area of Figures

Write a program that reads input data from the console - the measures of a geometric shape and calculates its surface area. There are four types of shapes: square, rectangle, circle and triangle.

The first line of input is the type of shape (square, rectangle, circle, triangle):

- If the shape is a square, the next argument will be one number - the length of its side.
- If the shape is a rectangle, the next argument will be two numbers - the lengths of its sides.
- If the shape is a circle, the next argument will be one number - the radius of the circle.

- If the shape is a **triangle**, the next argument will be two numbers - its base and the corresponding altitude.

The result should be rounded up to the **third decimal point**.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
square 5	25	rectangle 7 2.5	17.5	circle 6	113.097	triangle 4.5 20	45

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#12>.

Problem: Time + 15 Minutes

Write a program that reads **two integers - hours and minutes** based on a 24-hour day and calculates what time it will be **15 minutes later**. The result should be printed in the following format **hh:mm**. Hours should always be between 0 and 23, while minutes should always be between 0 and 59. Hours should be written with one or two digits as needed, while the minutes should always be written with two digits - add a **leading zero**, as needed.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
1 46	2:01	0 01	0:16	23 59	0:14	11 08	11:23

Hints and Guidelines

Add 15 minutes and check using a set of conditions. If minutes are over 59 **increase the hours** by 1 and **decrease the minutes** by 60. You may handle the case when hours are over 23 similarly. Take care when printing the minutes to add a **leading zero** where appropriate.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#13>.

Problem: 3 Equal Numbers

Write a program that reads **3 numbers** and prints whether they are the same ("yes" / "no").

Sample Input and Output

Input	Output	Input	Output	Input	Output
5	yes	5	no	1	no
5		4		2	
5		5		3	

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#14>.

Problem: Number 0...100 to Text

Write a program that converts numbers in the range of [0 ... 100] in text.

Sample Input and Output

Input	Output	Input	Output	Input	Output
25	twenty five	42	forty two	6	six

Hints and Guidelines

Firstly, we should check for **single-digit numbers** and if this is the case, print the corresponding word. Then we can check if the number is a **double-digit number**. These can be printed in two parts: left part (**double-digit** = number / 10) and right part (**single-digit** = number % 10). If the number has three digits, then it must be 100 and this can be handled as a special case.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1049#15>.

Chapter 3.2. Simple Conditions – Exam Problems

In the previous chapter, we went through **the simple conditional statements** in Python, which we can use to execute different actions depending on a given condition. We mentioned what is the **scope** of a variable and how to track the execution of our program step by step (the so-called **debugging**). In this chapter, we'll be working with **simple conditions** by going through some Exam Problems. To do this, let's first revise their construction:

```
if bool expression:
    # condition body
else:
    # else-construction body
```

if conditions consist of:

- **if** clause
- bool expression - a variable of bool type (**bool**) or bool logical expression (an expression that results in **true/false**)
- condition body - contains a random block of source code
- **else** clause and its block of source code (**optional**)

Exam Problems

After having revised how to write simple conditions, let's solve a few Exam Problems to practice the **if-else** construction.

Problem: Transport Price

A student has to travel **n kilometers**. He can choose between **three types of transportation**:

- **Taxi**. Starting fee: **0.70 BGN**. Day rate: **0.79 BGN/km**. Night rate: **0.90 BGN/km**.
- **Bus**. Day / Night rate: **0.09 BGN/km**. Can be used for distances of a minimum of **20 km**.
- **Train**. Day / Night rate: **0.06 BGN/km**. Can be used for distances of a minimum of **100 km**.

Write a program that reads the number of **kilometers n** and **period of the day** (day or night) and calculates **the price for the cheapest transport**.

Input Data

Two lines are read from the console:

- The first line (arguments) contains a number **n** – number of kilometers – an integer in the range of **[1 ... 5000]**.

- The second line contains the word "day" or "night" – traveling during the day or the night.

Output Data

Print to the console **the lowest price** for the given number of kilometers.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
5 day	4.65	7 night	7	25 day	2.25	180 night	10.8

Hints and Guidelines

We will read the input data and, depending on the distance, we will choose the cheapest transport. To do that, we will write a few conditional statements.

Processing The Input Data

In the task, we are given **information about the input and output data**. Therefore, in the first **two lines** of the solution, we will declare and initialize the two **variables** that are going to store **the values of the input data**. The **first line** contains **an integer** and that is why the declared variable will be converted with the function **int(...)**. The **second line** contains **a word**, therefore, the variable will be of **string** type:

```
distance = int(input())
day_or_night = input()
```

Before starting with the conditional statements, we need to **declare** one more **variable** that stores the value of **the transport price**:

```
price = 0.00
```

Checking The Conditions and Calculating

After having **declared and initialized** the input data and the variable that stores the value of the price, we have to decide which **conditions** of the task have to be **checked first**.

The task specifies that the rates of two of the vehicles **do not depend** on whether it is **day** or **night**, but the rate of one of the transports (taxi) **depends**. This is why the **first condition** will be whether it is **day or night** so that it is clear which rate the taxi will be **using**. To do that, we **declare one more variable** that stores **the value of the taxi rate**:

```
taxi_rate = 0.00
```

To calculate **the taxi rate**, we will use a conditional statement of type **if-else**:

```
if day_or_night == 'day':
    taxi_rate = 0.79
else:
    taxi_rate = 0.90
```

After having done that, now we can start calculating the transport price itself. The constraints in the task refer to the distance that the student wants to travel. This is why we will use an **if-elif-else** statement that will help us find the price of the transport, depending on the given kilometers:

```
if distance < 20:
    price = 0.70 + distance * taxi_rate
elif distance < 100:
    price = distance * 0.09
else:
    price = distance * 0.06
```

First, we check whether the kilometers are **less than 20**, as the task specifies that the student can only use a **taxi** for **less than 20** kilometers. If the condition is **true** (returns **true**), the variable that is created to store the value of the transport (**price**), will store the corresponding value. This value equals the **starting fee** that we will **sum** with its **rate**, **multiplied** by the **distance** that the student has to travel.

If the condition of the variable is **not true** (returns **false**), the next step of our program is to check whether the kilometers are **less than 100**. We do that because the task specifies that in this range, a **bus** can be used as well. The **price** per kilometer of a bus is **cheaper** than a taxi one. Therefore, if the result of the condition is **true**, we store a **value**, equal to the result of the **multiplication** of the **rate** of the bus by the **distance** to the variable for the transportation **price** in the **elif** statement body.

If this condition **does not return true** as a result, we have to store a **value**, equal to the **result** of the **multiplication** of the **distance** by the train **rate** to the price variable in the **else** body. This is done because the train is the **cheapest** transport for the given distance.

Printing The Output Data

After we have checked the distance conditions and we have calculated the price of the cheapest transport, we have to **print** it. The task **does not** specify how to format the result, therefore, we just print the variable:

```
print(price)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1050#0>.

Problem: Pipes in Pool

A pool with volume **V** fills up via **two pipes**. Each pipe has a certain **flow rate** (the liters of water, flowing through a pipe for an hour). A worker starts the pipes simultaneously and

goes out for **N hours**. Write a program that finds the state of the pool **the moment the worker comes back**.

Input Data

Four lines are read from the console:

- The first line contains a number **V – the volume of the pool in liters** – an integer in the range of [1 ... 10000].
- The second line contains a number **P1 – the flow rate of the first pipe per hour** – an integer in the range of [1 ... 5000].
- The third line contains a number **P2 – the flow rate of the second pipe per hour** – an integer in the range of [1 ... 5000].
- The fourth line contains a number **H – the hours that the worker is absent** – a floating-point number in the range of [1.0 ... 24.00].

Output Data

Print to the console **one of the two possible states**:

- To what extent the pool has filled up and how many percents each pipe has contributed with. All percent values must be formatted to an integer (without rounding).
 - "The pool is [x]% full. Pipe 1: [y]%. Pipe 2: [z]%."
- If the pool has overflown – with how many liters it has overflown for the given time – a floating-point number.
 - "For [x] hours the pool overflows with [y] liters."

Have in mind that due to **the rounding to an integer**, there is **data loss** and it is normal for the sum of the percents to be 99%, not 100%.

Sample Input and Output

Input	Output	Input	Output
1000 100 120 3	The pool is 66% full. Pipe 1: 45%. Pipe 2: 54%.	100 100 100 2.5	For 2.5 hours the pool overflows with 400 liters.

Hints and Guidelines

To solve the task, we read the input data, write a few conditional statements, do some calculations and print the result.

Processing The Input Data

From the task requirements, we note that our program must have **four lines** from which we read **the input data**. The first **three** consist of **integers** and that is why the **variables** that will store their values will be of **int** type. We know that the **fourth** line will be a **floating-point number**, therefore, the variable we use will be of **float** type:

```
volume = int(input())
pipe_1 = int(input())
pipe_2 = int(input())
hours = float(input())
```

Our next step is to **declare and initialize** a variable in which we are going to calculate how many **liters** the pool has **filled up** for the **time** the worker was **absent**. We do the calculations by **summing** the values of the flow rates of the **two pipes** and **multiplying** them by the **hours** that are given as input data:

```
water = (pipe_1 + pipe_2) * hours
```

Checking The Conditions and Processing Output Data

After we have **the value of the quantity** of water that has flown through the **pipes**, the next step is to **compare** that quantity with the volume of the pool itself.

We do that with a simple **if-else** statement, where the condition will be whether **the quantity of water is less than the volume of the pool**. If the statement returns **true**, we have to print one line that contains **the ratio** between the quantity of **water that has flown through the pipes** and **the volume of the pool**, as well as the **ratio of the quantity of the water** from each pipe to the volume of the pool.

The ratio has to be in **percentage**, that is why all the calculations so far will be **multiplied by 100**. The values will be printed using **placeholders**, and as there is a condition for **the result in percentage** to be formatted to **two digits after the decimal point without rounding**, we will use the method **math.trunc(...)**:

```
if water <= volume:
    print('The pool is {0}% full. Pipe 1: {1}%. Pipe 2: {2}%.' . format(
        math.trunc((water / volume) * 100),
        math.trunc((pipe_1 * hours / volume) * 100),
        math.trunc((pipe_2 * hours / volume) * 100)
    ))
else:
    print('For {0} hours the pool overflows with {1} liters.' . format(
        hours,
        water - volume
    ))
```

However, if **the condition** returns **false**, that means that **the quantity of water** is **more** than the **volume** of the pool, therefore, it has **overflowed**. Again, the output data has to be on **one line**, but this time it should contain only two values – one of the **hours** when the worker was

absent, and the **quantity of water**, which is the difference between the incoming water and the volume of the pool.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1050#1>.

Problem: Sleepy Tom Cat

Tom Cat likes to sleep all day but, unfortunately, his owner is always playing with him whenever he has free time. To sleep well, **the norm of games** that Tom has is **30 000 minutes per year**. The time for games he has **depends on the holidays** that his owner has:

- During **workdays**, his owner plays with him **63 minutes per day**.
- During **holidays**, his owner plays with him **127 minutes per day**.

Write a program that reads **the number of holidays** and prints whether **Tom can sleep well** and how much **the difference from the norm** for the current year is. It is assumed that **there are 365 days in one year**.

Example: 20 holidays -> the working days are 345 ($365 - 20 = 345$). The time for games is 24 275 minutes ($345 * 63 + 20 * 127$). The difference from the norm is 5 725 minutes ($30\,000 - 24\,275 = 5\,725$) or 95 hours and 25 minutes.

Input Data

The input is read from the console and consists of an integer – **the number of holidays** in the range of **[0 ... 365]**.

Output Data

Two lines have to be printed to the console:

- If Tom's time for games **is above the norm** for the current year:
 - On the first line print: "Tom will run away".
 - On the second line print the difference from the norm in the format: "{H} hours and {M} minutes more for play".
- If the time for games of Tom **is below the norm** for the current year:
 - On the first line print: "Tom sleeps well".
 - On the second line print the difference from the norm in the format: "{H} hours and {M} minutes less for play".

Sample Input and Output

Input	Output
20	Tom sleeps well 95 hours and 25 minutes less for play

Input	Output
113	Tom will run away 3 hours and 47 minutes more for play

Hints and Guidelines

To solve the problem, we will read the input data. Then, we will write a few conditional statements and do some calculations. Finally, we will print the result.

Processing The Input Data and calculating

From the task we see that **the input data** will be read only on **the first line** and will be an **integer** in the range of **[0 ... 365]**. This is why we will use a variable of **int** type:

```
holidays = int(input())
```

To solve the problem, **first**, we have to calculate **the total minutes** the owner of Tom is playing with him. We see that not only does the sleepy cat has to play with his owner during **the holidays**, but also during **the working days**. **The number** that we read from the console refers to **the holidays**.

Our next step is to **calculate**, with the help of that number, how many **the working days** of the owner are, as without them we cannot calculate **the total minutes for play**. As the total number of days per year is **365** and the number of holidays is **X**, that means that the number of working days is **365 - X**. We store **the difference** in a new variable that only stores this value:

```
working_days = 365 - holidays
```

Once we have **the number of days for playing**, we can calculate **the time for games** of Tom in minutes. Its **value** is equal to the **result of the multiplication of the working days by 63 minutes** (the task specifies that during working days, the time for play is 63 minutes per day), **summed with the result of the multiplication of the holidays by 127 minutes** (the task specifies that during holidays, the time for play is 127 minutes per day):

```
total_play_minutes = working_days * 63 + holidays * 127
```

In the task condition, we see that we have to **print the difference** between the two values in **hours** and **minutes** as output data. That is why we **subtract** the **total** time for play from the norm of **30 000 minutes** and **store** the result in a **new** variable. After that, we **divide** that variable by 60 to get the **hours**, and then, to find out how many the **minutes** are, we use **modular division with the operator %**, as again we divide the variable of the difference by 60.

Here we have to note that if the total **time for the playing** of Tom is **less** than **30,000** when **subtracting** the norm from it, we will obtain a **negative number**. To **neutralize** the number in the division, we use **the function `math.fabs(...)`** when finding the difference:

```
difference = math.fabs( (total_play_minutes - 30000) )
hours = difference // 60
minutes = difference % 60
```

Checking The Conditions

The time for games is already calculated, which leads us to the **next** step – **comparing** the **time for play** of Tom with the **norm** on which the good sleep of the cat depends. To do so, we will use an **if-else** conditional statement. In the **if** clause we will check whether **the time for play** is more than 30 000 (the norm).

Processing The Output Data

Whatever the result of the conditional statement is, we have to print how much **the difference in hours and minutes** is. We will do that with a **placeholder** and the variables that store the values of the hours and the minutes, as the formatting will be according to the task requirements for output:

```
if total_play_minutes > 30000:
    print('Tom will run away')
    print(f'{hours} hours and {minutes} minutes more for play')
else:
    print('Tom sleeps well')
    print(f'{hours} hours and {minutes} minutes less for play')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1050#2>.

Problem: Harvest

In a vineyard with an area of X square meters, 40% of the harvest goes for wine production. Y kilograms of grapes are extracted from a 1 square meter vineyard. 2,5 kg of grapes is needed for 1 liter of wine. The wanted quantity of wine for sale is Z liters.

Write a program that **calculates** how much wine can be produced and whether that quantity is enough. If it is enough, the rest is divided between the vineyard workers equally.

Input Data

The input data is read from the console and consists of **exactly 4 lines**:

- First line: **X m² – the vineyard size** – an integer in the range of [10 ... 5000].
- Second line: **Y grapes for one m²** – an integer in the range of [0.00 ... 10.00].
- Third line: **Z needed liters of wine** – an integer in the range of [10 ... 600].
- Fourth line: **number of workers** – an integer in the range of [1 ... 20].

Output Data

The following has to be printed to the console:

- If the produced wine is less than the needed quantity:
 - "It will be a tough winter! More {insufficient wine}} liters wine needed."
* The result has to be rounded down to the nearest integer.
- If the produced wine is more than the needed quantity:
 - "Good harvest this year! Total wine: {total wine} liters."
* The result has to be rounded down to the nearest integer.
 - "{Wine left} liters left -> {wine for one worker} liters per person."
* Both of the results have to be rounded up to the higher integer.

Sample Input and Output

Input	Output	Input	Output
650	Good harvest this year! Total	1020	
2	wine: 208 liters.	1.5	It will be a tough winter! More
175	33 liters left -> 11 liters per	425	180 liters wine needed.
3	person.	4	

Hints and Guidelines

To solve the problem, we will read the input data. Then, we will write a few conditional statements and do some calculations. Finally, we will print the result.

Processing The Input Data and performing the calculations

First, we have to **check** what the **input data** will be so that we can choose what **variables** we will use. The code below is blurred on purpose and it should be written by the reader:

```
vineyard_area =   
grape_per_square =   
needed_liters =   
workers = 
```

To solve the task, based on the input data, we have to **calculate** how many **liters of wine** will be produced. From the task requirements, we see that to **calculate** the quantity of **wine in liters**, we first, have to find **the number of grapes in kilograms**, which we will get from the harvest. For that, we will declare a variable that keeps a **value**, equal to **40%** of the result from the **multiplication** of the vineyard area by the number of grapes, which is extracted from 1 **m2**.

After having done these calculations, we are ready to **calculate** the **quantity of wine in liters** that will be produced from the harvest as well. For that, we declare one more variable that stores that **quantity**. To calculate, we have to **divide** the **number of grapes in kg** by **2.5**:

```
harvest_per_vine = (needed_liters * 100) / (workers * 100)
vine = (needed_liters * 100) / (workers * 100)
```

Checking The Conditions and Processing Output Data

After having done the necessary calculations, the next step is to **check** whether the liters of wine that have been produced **are enough**. For that, we will use a **simple conditional statement** of the **if-else** type and we will check whether the liters of wine from the harvest are **more than or equal to the needed liters**.

If the condition returns **true**, from the task requirement we see that **on the first line** we have to print **the wine that has been produced from the harvest**. That value has to be **rounded down to the nearest integer**, which we will do by using both the method **math.floor(...)** and a **placeholder** when printing it.

On the second line, we have to print the results by **rounding them up to the higher integer**, which we will do by using the method **math.ceil(...)**. The values that we have to print are **the quantity of wine left** and **the quantity that each worker gets**. The wine left is equal to **the difference** between the produced liters of wine and the needed liters of wine. We calculate the value of that quantity in a new variable, which we declare and initialize in the **if condition body**, before printing the first line. We calculate the quantity of wine that **each worker gets** by dividing the wine left by the number of workers:

```
if (harvest_per_vine >= needed_liters) :
    vine_left = (needed_liters * 100) - (workers * 100)
    print('Good harvest this year! Total wine: {0} liters.' . format(
        math.floor(vine_left)))
    print('{0} liters left -> {1} liters per person.' . format(
        math.ceil(vine_left / workers), math.ceil(vine_left / workers)))
```

If the condition returns **false**, we have to **print the difference** between **the needed liters** and the **liters of wine produced from the harvest**. There is a specification that the result has to be **rounded down to the nearest integer**, which we will do by using the method **math.floor(...)**:

```
else:
    print('It will be a tough winter! More '
        '{0} liters wine needed.' . format(
        math.floor((needed_liters * 100) - (workers * 100))))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1050#3>.

Problem: Firm

A firm gets a request for creating a project for which a certain number of hours are needed. The firm has a **certain number of days**. During 10% of the days, the workers are **being trained**

and cannot work on the project. A normal **working day** is 8 hours long. The project is important for the firm and every worker must work on it with **overtime** of 2 hours per day. The hours must be **rounded down to the nearest integer** (for example, 6.98 hours are rounded to **6 hours**).

Write a program that calculates whether the firm can finish the project on time and how many hours more are needed or left.

Input Data

The input data is read from the console and contains exactly three lines:

- On the first line are the needed hours – an integer in the range of [0 ... 200 000].
- On the second line are the days that the firm has – an integer in the range of [0 ... 20 000].
- On the third line are the number of all workers – an integer in the range of [0 ... 200].

Output Data

Print one line to the console:

- If the time is enough:
 - "Yes!{the hours left} hours left."
- If the time is NOT enough:
 - "Not enough time!{additional hours} hours needed."

Sample Input and Output

Input	Output	Input	Output
90 7 3	Yes!99 hours left.	99 3 1	Not enough time!72 hours needed.

Hints and Guidelines

To solve the problem, we will read the input data. Then, we will write a few conditional statements and do some calculations. Finally, we will print the result.

Processing The Input Data

First, we have to read the input data to solve the problem. The code below is blurred on purpose and it should be written by the reader:

```
project_hours =   
available_days =   
overtime_workers = 
```

Auxiliary Calculations

The next step is to calculate **the number of total working hours** by multiplying the working days by 8 (every working day is 8 hours long) with the number of workers and then sum them with the overtime. **The working days** equal **90% of the days** that the firm has. **The overtime** equals the result of the multiplication of the number of workers by 2 (the possible hours of overtime) and then it is multiplied by the number of days that the firm has. From the task requirements, we see that **the hours** should be **rounded down to the nearest integer**, which we will do with the method **math.floor(...)**:

```
work_days = (int) (days * 0.9)
overtime_hours = (int) (workers * 2 * days)
work_hours = (int) (work_days * 8 + overtime_hours)
total_hours = (int) (work_hours + overtime_hours)
```

Checking The Conditions

After having done the calculations that are needed to find the value of **the working hours**, now we have to check whether these hours are **enough**, or **some hours are left**.

If **the time is enough**, we print the result that is specified in the task requirements, which in this case is the difference between **the working hours** and **the hours needed** for finishing the project.

If **the time is not enough**, we print the additional hours that are needed for finishing the project. They equal the difference between **the hours for the project** and **the total working hours**:

```
if (total_hours <= hours_needed):
    print('Yes!{0} hours left.'.format(
        total_hours - hours_needed))
else:
    print('Not enough time!{0} hours needed.'.format(
        hours_needed - total_hours))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1050#4>.

Chapter 4.1. More Complex Conditions

In the **current** chapter, we are going to examine **nested conditional statements** in the Python language, by which our program can execute **conditions**, that contain other **nested conditional statements**. We call them "**nested**" because we put the **if** condition into another **if** condition. We are going to examine the **more complex logical conditions** through proper examples.

Nested Conditions

Pretty often the program logic requires the use of **if** or **if-else** statements, which are contained one inside another. They are called **nested if** or **if-else** statements. As implied by the title "**nested**", these are **if** or **if-else** statements that are placed inside other **if** or **else** statements.

```
if condition1:
    if condition2:
        # body
    else:
        # body
```

Nesting of **more than three conditional** statements inside each other is not considered a good practice and has to be avoided, mostly through optimization of the structure/the algorithm of the code and/or by using another type of conditional statement, which we are going to examine below in this chapter.

Problem: Personal Titles

Depending on **age** (decimal number and **gender** (m / f), print a personal title:

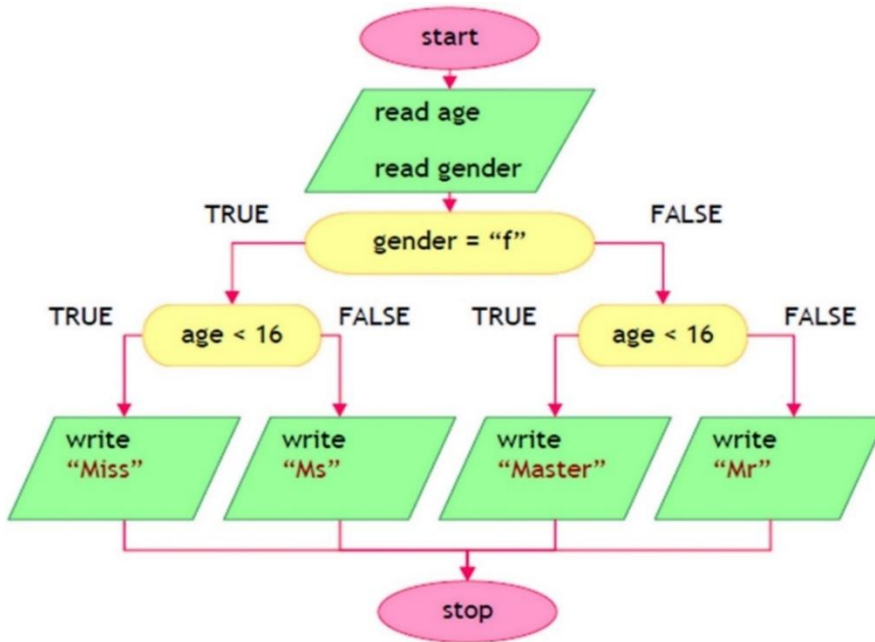
- "Mr." – a man (gender "m") – 16 or more years old.
- "Master" – a boy (gender "m") under 16 years.
- "Ms." – a woman (gender "f") – 16 or more years old.
- "Miss" – a girl (gender "f") under 16 years.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
12 f	Miss	17 m	Mr.	25 f	Ms.	13.5 m	Master

Solution

We should notice that the **output** of the program **depends on a few things**. First, we have to check what is the entered **gender** and **then** check the **age**. Respectively, we are going to use **a few if-else** blocks. These blocks will be **nested**, meaning from **the result** of the first, we are going to **define** which one of the **others** to execute.



After reading the input data from the console, the following program logic should be executed:

```
age = float(input())
gender = str(input())
```

```
if age < 16:
    if gender == 'm':
        print('Master')
    elif gender == 'f':
        print('Miss')
else:
    if gender == 'm':
        print('Mr.')
    elif gender == 'f':
        print('Ms.')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#0>.

Problem: Small Shop

A Bulgarian entrepreneur opens small shops in a few cities with different prices for the following products:

product / city	Sofia	Plovdiv	Varna
coffee	0.50	0.40	0.45
water	0.80	0.70	0.70
beer	1.20	1.15	1.10
sweets	1.45	1.30	1.35
peanuts	1.60	1.50	1.55

Calculate the price by the given **city** (string), **product** (string) and **quantity** (float).

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
coffee Varna 2	0.9	peanuts Plovdiv 1	1.5	beer Sofia 6	7.2	water Plovdiv 3	2.1

Solution

We **convert** all of the letters into **lowercase** using the function `.lower()`, to compare products and cities **no matter** what the letters are – small or capital ones.

```
product = input().lower()
town = input().lower()
quantity = float(input())

if town == 'sofia':
    if product == 'coffee':
        print(0.50 * quantity)
    # TODO: Finish this ...

if town == 'varna':
    # TODO Finish this ...

if town == 'plovdiv':
    # TODO Finish this ...
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#1>.

More Complex Conditions

Let's take a look at how we can create more complex logical conditions. We can use the logical "AND" (**and**), logical "OR" (**or**), logical **negation** (**not**) and **brackets** (**()**).

Logical "AND"

As we saw, in some tasks we have to make **many checks at once**. But what happens when to execute some code **more** conditions have to be executed and we **don't want to** make a **negation (else)** for each one of them? The option with nested **if blocks** is valid, but the code would look very **unordered** and for sure – **hard** to read and maintain.

The logical "AND" (operator **and**) means a few conditions have to be **fulfilled simultaneously**. The following table of truthfulness is applicable:

a	b	a and b
True	True	True
True	False	False
False	True	False
False	False	False

How Does The and Operator Work?

The **and** operator accepts a couple of Boolean (conditional) statements, which have a **True** or **False** value, and returns **one** bool statement as a **result**. Using it **instead** of a couple of nested **if** blocks, makes the code **more readable**, **ordered** and **easy** to maintain. But how does it **work**, when we put a **few** conditions one after another? As we saw above, the logical "AND" returns **True**, **only** when it accepts as **arguments statements** with the value **True**. Respectively, when we have a **sequence** of arguments, the logical "AND" **checks** either until one of the arguments is **over**, or until it **meets** an argument with value **False**.

Example:

```
a = True
b = True
c = False
d = True

result = a and b and c and d
# False (as d is not being checked)
```

The program will run in the **following** way: It **starts** the check form **a**, **reads** it and accepts that it has a **True** value, after which it **checks b**. After it has **accepted** that **a** and **b** return **True**, it **checks the next** argument. It gets to **c** and sees that the variable has a **False** value. After the program accepts that argument **c** has a **False** value, it calculates the expression **before c**, **independent** of what the value of **d** is. That is why the evaluation of **d** is being **skipped** and the whole expression is calculated as **False**.

Problem: Point in a Rectangle

Checks whether point $\{x, y\}$ is placed inside the rectangle $\{x_1, y_1\} - \{x_2, y_2\}$. The input data is read from the console and consists of 6 lines: the decimal numbers x_1, y_1, x_2, y_2, x and y (as it is guaranteed that $x_1 < x_2$ and $y_1 < y_2$).

Sample Input and Output

Input	Output	Visualization
2 -3 12 3 8 -1	Inside	

Solution

A point is internal for a given polygon if the following four conditions are applied **at the same time**:

- The point is placed to the right from the left side of the rectangle.
- The point is placed to the left from the right side of the rectangle.
- The point is placed downwards from the upper side of the rectangle.
- The point is placed upwards from the down side of the rectangle.

```
x1 = float(input())
y1 = float(input())
x2 = float(input())
y2 = float(input())

x = float(input())
y = float(input())

if x >= x1 and x <= x2 and y >= y1 and y <= y2:
    print('Inside')
else:
    print('Outside')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#2>.

Logical "OR"

The logical "OR" (operator **or**) means that **at least one** among a few conditions is fulfilled. Similar to the operator **and**, the logical "OR" accepts a few arguments of **bool** (conditional) type and returns **True** or **False**. We can easily guess that we **obtain** a value **True** whenever at least **one** of the arguments has a **True** value. This is shown at the truth table below:

In school, the teacher said: "Ivan or Peter should clean the board". For completing this condition (the board to be clean), it's possible only Ivan to clean it, only Peter to clean it or both of them to do it.

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

How Does The **or** Operator Work?

We have already learned what the logical "OR" **represents**. But how is it being achieved? Just like with the logical "AND", the program **checks** from left to right **the arguments** that are given. To obtain **True** from the expression, **just one** argument must have a **True** value. Respectively, the checking **continues** until an **argument** with **such** value is met or until the arguments **are over**.

Here is one **example** of the **or** operator in action:

```
a = False
b = True
c = False
d = True

result = a or b or c or d
# True (as c and d are not being checked)
```

The program **checks** **a**, accepts that it has a value **False** and continues. Reaching **b**, it understands that it has a **True** value and the whole **expression** is calculated as **True**, **without** having to check **c** or **d**, because their values **wouldn't change** the result of the expression.

Problem: Fruit or Vegetable

Let's check whether a given **product** is a **fruit** or a **vegetable**. The "fruits" are **banana**, **apple**, **kiwi**, **cherry**, **lemon** and **grapes**. The "vegetables" are **tomato**, **cucumber**, **pepper** and **carrot**. Everything else is "unknown".

Sample Input and Output

Input	Output	Input	Output	Input	Output
banana	fruit	tomato	vegetable	Java	unknown

Solution

We have to use a few conditional statements with logical "OR" (**or**):

```
s = input()

if (s == 'banana' or s == 'apple'
    or s == 'kiwi' or s == 'cherry'
    or s == 'lemon' or s == 'grapes'):
    print('fruit')
elif s == 'tomato' or s == 'cucumber' \
     or s == 'pepper' or s == 'carrot':
    print('vegetable')
else:
    print('unknown')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#3>.

Logical Negation

Logical negation (operator **not**) means that a given condition is **not** fulfilled.

a	not a
True	False

The operator **not** accepts as an **argument** a bool variable and **returns** its value. (the truth becomes a lie and the lie becomes a truth).

Problem: Invalid Number

A given number is **valid** if it is in the range [100 ... 200] or it is 0. Validate an **invalid** number.

Sample Input and Output

Input	Output	Input	Output	Input	Output
75	invalid	150	(no output)	220	invalid

Solution

```
num = float(input())
in_range = (num >= 100 and num <= 200 ) or num == 0
if not in_range:
    print('invalid')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#4>.

The Parenthesis () Operator

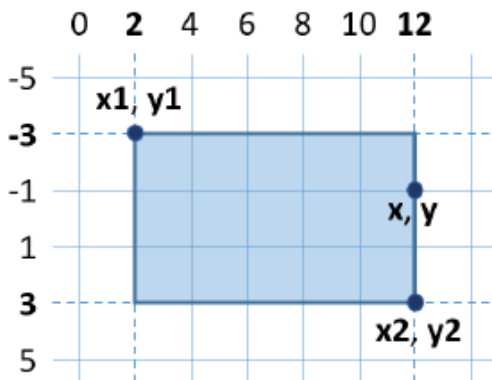
Like the rest of the operators in programming, the operators **and** and **or** have a priority, as in the case **and** is with higher priority than **or**. The operator **()** serves for **changing the priority of operators** and is being calculated first, just like in mathematics. Using parentheses also gives the code better readability and is considered a good practice.

More Complex Conditions - Problems

Sometimes the conditions may be **very complex**, so they can require a long bool expression or a sequence of conditions. Let's take a look at a few examples.

Problem: Point on Rectangle Border

Write a program that checks whether a point $\{x, y\}$ is placed **onto** any of the sides of a rectangle $\{x_1, y_1\} - \{x_2, y_2\}$. The input data is read from the console and consists of 6 lines: the decimal numbers x_1, y_1, x_2, y_2, x and y (as it is guaranteed that $x_1 < x_2$ and $y_1 < y_2$). Print "Border" (if the point lies on any of the sides) or "Inside / Outside" (in the opposite case).



Sample Input and Output

Input	Output	Input	Output
2	Border	2	Inside / Outside
-3		-3	
12		12	
3		3	
12		8	
-1		-1	

Solution

The point lies on any of the sides of the rectangle if:

- x coincides with x_1 or x_2 and at the same time y is between y_1 and y_2 or
- y coincides with y_1 or y_2 and at the same time x is between x_1 and x_2 .

```
if ((x == x1 or x == x2) and (y >= y1) and (y <= y2))
    or ((y == y1 or y == y2) and (x >= x1) and (x <= x2)):
    print('Border')
```

The previous conditional statement can be simplified by this way:

```
on_left_side = (x == x1) and (y >= y1) and (y <= y2)
on_right_side = (x == x2) and (y >= y1) and (y <= y2)
on_up_side = (y == y1) and (x >= x1) and (x <= x2)
on_down_side = (y == y2) and (x >= x1) and (x <= x2)

if on_left_side or on_right_side or on_up_side or on_down_side:
    print('Border')
else:
    print('Inside / Outside')
```

The second way with an additional boolean variable is longer but it's also more readable than the first, right? We advise you when writing boolean conditions to make them **easier for reading than understanding** and not short. If you are forced to, use additional variables with similar names. Names of the boolean variables should be with reasonable names. They should hint at what value will be stored in them.

All that's left is to write the code, that prints "Inside / Outside" if the point is not on one of the sides of the rectangle.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#5>.

Problem: Fruit Shop

A fruit shop during **weekdays** sells at the following **prices**:

Fruit	Price
banana	2.50
apple	1.20
orange	0.85
grapefruit	1.45
kiwi	2.70
pineapple	5.50
grapes	3.85

During the **weekend days** the prices are higher:

Fruit	Price
banana	2.70
apple	1.25
orange	0.90
grapefruit	1.60
kiwi	3.00
pineapple	5.60
grapes	4.20

Write a program that **reads** from the console a **fruit** (banana / apple / ...), a **day of the week** (Monday / Tuesday / ...) and a **quantity (a decimal number)** and **calculates the price** according to the prices from the tables above. The result has to be printed **rounded up to 2 digits after the decimal point**. Print **“error”** if it is an **invalid day** of the week or an **invalid name** of a fruit.

Sample Input and Output

Input	Output
orange Sunday 3	2.70

Input	Output
kiwi Monday 2.5	6.75

Input	Output
grapes Saturday 0.5	2.10

Input	Output
tomato Monday 0.5	error

Solution

```
if (day == "saturday" or day == "sunday"):
    if (fruit == "banana"):
        price = 2.70
    elif (fruit == "apple"):
        price = 1.25
    # TODO: more fruits come here ...
elif (day == "monday" or day == "tuesday"
      or day == "wednesday" or day == "thursday"
      or day == "friday"):
    if (fruit == "banana"):
        price = 2.50
    # TODO: more fruits come here ...
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#6>.

Problem: Trade Comissions

A company is giving the following **commissions** to its traders according to the **city**, in which they are working and the **volume of sales** s:

City	$0 \leq s \leq 500$	$500 < s \leq 1000$	$1000 < s \leq 10000$	$s > 10000$
Sofia	5%	7%	8%	12%
Varna	4.5%	7.5%	10%	13%
Plovdiv	5.5%	8%	12%	14.5%

Write a **program** that reads the name of a **city** (string) and the volume of **sales** (float) and calculates the rate of the commission fee. The result has to be shown rounded **up to 2 digits after the decimal point**. When there is an **invalid city or volume of sales** (a negative number), print "error".

Sample Input and Output

Input	Output	Input	Output	Input	Output
Sofia 1500	120.00	Plovdiv 499.99	27.50	Kaspichan -50	error

Solution

When reading the input, we could convert the city into small letters (with the function **.lower()**). Initially, we set the commission fee to **-1**. It will be changed if the city and the price range are found in the table of commissions. To calculate the commission according to the city and volume of sales, we need a few nested **if statements**, as in the sample code below:

```
if town == "sofia":
    if 0 <= sales and sales <= 500:
        |      comission = 0.05
    elif 500 < sales and sales <= 1000:
        |      comission = 0.07
    # TODO: check the other price ranges ...
elif town == "varna":
    # TODO: check the price ranges ...
elif town == "plovdiv":
    # TODO: check the price ranges ...

if comission >= 0:
    print("%.2f" % (sales * comission))
else:
    print("error")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#7>.

Problem: Day of Week

Let's write a program that prints the day of the week depending on the given number (1 ... 7) or "Error!" if invalid input is given.

Sample Input and Output

Input	Output	Input	Output	Input	Output
1	Monday	7	Sunday	-1	Error

Solution

```
day = int(input())

if day == 1:
    print("Monday")
elif day == 2:
    print("Tuesday")
...
elif day == 7:
    print("Sunday")
else:
    print("Error")
```




It is a good practice to put at the **first** place those **case** statements that process the most common situations and leave the **case** constructions processing the rarer situations at the end, before the **default** construction. Another good practice is to arrange the **case** labels in ascending order, regardless of whether they are integral or symbolic.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#8>.

Problem: Animal Type

Write a program that prints the type of the animal depending on its name:

- dog -> mammal
- crocodile, tortoise, snake -> reptile
- others -> unknown

Sample Input and Output

Input	Output
tortoise	reptile

Input	Output
dog	mammal

Input	Output
elephant	unknown

Solution

We can solve the example with a few **if-elif** conditional statements by doing so:

```
if animal == "dog":
    print("mammal")
elif animal == "crocodile" or animal == "tortoise" \
    or animal == "snake":
    print("reptile")
else:
    print("unknown")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#9>.

What Have We Learned from This Chapter?

Let's review the new constructions and program techniques we have met in this chapter:

Nested Conditions

```
if condition1:
    if condition2:
        # body
    else:
        # body
```

Complex Conditions with and, or, not, and ()

```
if (x == left or x == right) and (y >= top or y <= bottom):
    print(...)
```

Problems: More Complex Conditions

Let's practice using more complex conditions. We will solve a few practical exercises.

Problem: Cinema

In a cinema hall, the chairs are ordered in a **rectangle** shape in **r** rows and **c** columns. There are three types of screenings with tickets of **different** prices:

- **Premiere** – a premiere screening, with a price of **12.00** BGN.
- **Normal** – a standard screening, with a price of **7.50** BGN.
- **Discount** – a screening for children and students at a reduced price – **5.00** BGN.

Write a program that enters a **type of screening** (string), a number for **rows** and a number for **columns** in the hall (integer numbers) and calculates **the total income** from tickets from a **full hall**. The result has to be printed in the same format as in the examples below – rounded up to 2 digits after the decimal point.

Sample Input and Output

Input	Output	Input	Output
Premiere	1440.00 leva	Normal	2047.50 leva
10		21	
12		13	

Hints and Guidelines

While reading the input, we could convert the screening type into small letters (with the function **.lower()**). We create and initialize a variable that will store the calculated income. In another variable, we calculate the full capacity of the hall. We use an **if-elif** conditional statement to calculate the income according to the type of the projection and print the result on the console in the given format (look for the needed **Python** functionality on the internet).

Sample code (parts of the code are blurred with the purpose to stimulate your thinking and solving skills):

```
type = input().lower()
rows = 
columns = 

full = 
income = 

if type == "premiere":
    income = full * 12.00

elif type == "normal":
    income = full * 7.00

elif type == "weekend":
    income = full * 5.00

print( "The total income is: ", income )
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#10>.

Problem: Volleyball

Vladimir is a student, lives in Sofia and goes to his hometown from time to time. He is very keen on volleyball, but is busy during weekdays and plays **volleyball** only during **weekends** and on **holidays**. Vladimir plays in Sofia every **Saturday** when he is not working, and he is not traveling to his hometown and also during **2/3 of the holidays**. He travels to his hometown **h** times a year, where he plays volleyball with his old friends on **Sunday**. Vladimir is not working **3/4 of the weekends**, during which he is in Sofia. Furthermore, during **leap years** Vladimir plays **15% more** volleyball than usual. We accept that the year has exactly **48 weekends**, suitable for volleyball. Write a program that calculates **how many times Vladimir has played volleyball** throughout the year. **Round the result** down to the nearest whole number (e.g. 2.15 -> 2; 9.95 -> 9).

The input data is read from the console:

- The first line contains the word "**leap**" (leap year) or "**normal**" (a normal year with 365 days).
- The second line contains the integer **p** – the count of holidays in the year (which are not Saturday or Sunday).
- The third line contains the integer **h** – the count of weekends, in which Vladimir travels to his hometown.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
leap 5 2	45	normal 3 2	38	normal 11 6	44	leap 0 1	41

Hints and Guidelines

As usual, we read the input data from the console and, to avoid making mistakes, we convert the text into small letters with the function `.lower()`. Consequently, we calculate **the weekends spent in Sofia, the time for playing in Sofia and the common playtime**. At last, we check whether the year is a **leap**, we make additional calculations when necessary and we print the result on the console **rounded down** to the nearest **integer** (look for a **Python** class with such functionality).

A sample code (parts of the code are blurred on purpose to stimulate independent thinking and solving skills):

```
year = input().lower()
holidays = int(input())
weekends_home = int(input())

sofia_weekends = int(weekends_home)
play_sofia = float(weekends_home / 4 * 3.5 + holidays / 3)
play_total = float(weekends_home)

if year == "leap":
    play_total = float(play_total + 16 / 100 * play_sofia)
elif year == "normal":
    play_total = float(play_sofia)

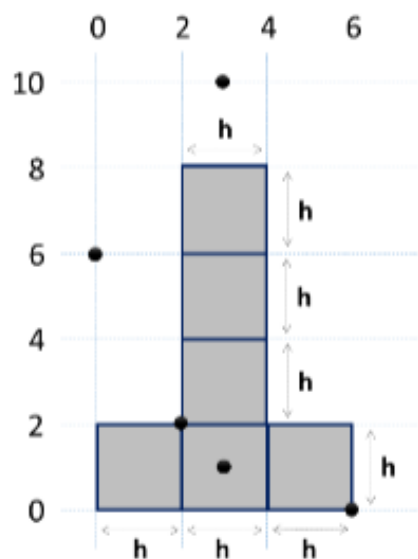
print(play_total)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#11>.

Problem: * Point in The Figure

The figure consists of **6 blocks with size $h * h$** , placed as in the figure below. The lower left angle of the building is on position $\{0, 0\}$. The upper right angle of the figure is on position $\{2*h, 4*h\}$. The coordinates given in the figure are for **$h = 2$** :



Write a program that enters an integer **h** and the coordinates of a given point **{x, y}** (integers) and prints whether the point is inside the figure (**inside**), outside of the figure (**outside**) or on any of the borders of the figure (**border**).

Sample Input and Output

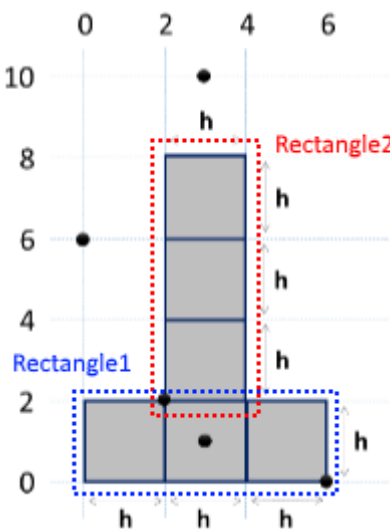
Input	Output	Input	Output	Input	Output	Input	Output
2	outside	2	inside	2	border	2	border
3		3		2		6	
10		1		2		0	

Input	Output	Input	Output	Input	Output	Input	Output
2	outside	15	outside	15	inside	15	outside
0		13		29		37	
6		55		37		18	

Hints and Guidelines

A possible logic for solving the task (not the only correct one):

- We might split the figure into **two rectangles** with a common side:



- A point is **outer (outside)** for the figure when it is **outside** both of the rectangles.
- A point is **inner (inside)** for the figure if it is inside one of the rectangles (excluding their borders) or lies on their common side.
- In **every other case**, the point lies on the border of the rectangle (**border**).

Sample code (parts of the code are blurred to stimulate logical thinking and solving skills):

```
h =   
x =   
y =   
  
out_rectangle1 =   
out_rectangle2 =   
  
in_rectangle1 =   
in_rectangle2 =   
  
common_border =   
  
if   
    print("outside")  
elif   
    print("inside")  
else:  
    print("border")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1051#12>.

Chapter 4.2. More Complex Conditions – Exam Problems

The previous chapter introduced you to **nested conditions** in **Python**. Via nested conditions, the program logic in a particular application can be represented using **if conditional statements** that are nested inside each other. We also explained the more complex **if-elif-else** conditional statement that allows selecting from a number of options. Now we are going to solve some practical exercises and make sure we have an in-depth understanding of the material, by discussing a set of more complex problems that had been given to students on exams. Before moving to the problems, let's first recall what nested conditions are:

Nested Conditions

```
if condition1:
    if condition2:
        # body
    else:
        # body
```



Remember that it is not a good practice to write **deeply nested conditional statements** (with more than three levels of nesting). Avoid nesting of more than three conditional statements inside one another. This complicates the code and makes its reading and understanding difficult.

If-Elif-Else Conditions

When the program operation depends on the value of a variable, we can do consecutive checks with multiple **if-elif-else** blocks:

```
if condition1:
    # body
elif condition2:
    # body
elif condition3:
    # body
else:
    # body
```

The body can consist of any code, as long as it corresponds to the syntactic particularity of the language and is indented one tab-press in.

Exam Problems

Now, after we refreshed our knowledge on how to use nested conditional statements to implement more complex conditions and program logic, let's solve some exam problems.

Problem: On Time for The Exam

A student has to attend **an exam at a particular time** (for example at 9:30 am). They arrive in the exam room at a particular **time of arrival** (for example 9:40 am). It is considered that the student has arrived **on time** if they have arrived **at the time when the exam starts or up to half an hour earlier**. If the student has arrived **more than 30 minutes earlier**, the student has come **too early**. If they have arrived **after the time when the exam starts**, they are **late**.

Write a program that inputs the exam starting time and the time of student's arrival, and prints if the student has arrived **on time**, if they have arrived **early** or if they are **late**, as well as **how many hours or minutes** the student is early or late.

Input Data

Four integers are read from the console (each on a new line):

- The first line contains the **exam starting time (hours)** – an integer from 0 to 23
- The second line contains the **exam starting time (minutes)** – an integer from 0 to 59.
- The third line contains the **hour of arrival** – an integer from 0 to 23.
- The fourth line contains **minutes of arrival** – an integer from 0 to 59.

Output Data

Print the following on the first line on the console:

- **"Late"**, if the student arrives **later** compared to the exam starting time.
- **"On time"**, if the student arrives **exactly** at the exam starting time or up to 30 minutes earlier.
- **"Early"**, if the student arrives more than 30 minutes **before** the exam starting time.

If the student arrives with more than one minute difference compared to the exam starting time, print on the next line:

- **"mm minutes before the start"** for arriving less than an hour earlier.
- **"hh:mm hours before the start"** for arriving 1 hour or earlier. Always print minutes using 2 digits, for example "1:05".
- **"mm minutes after the start"** for arriving less than an hour late.
- **"hh:mm hours after the start"** for arriving late with 1 hour or more. Always print minutes using 2 digits, for example "1:03".

Sample Input and Output

Input	Output	Input	Output	Input	Output
9 30 9 50	Late 20 minutes after the start	16 00 15 00	Early 1:00 hours before the start	9 00 8 30	On time 30 minutes before the start
Input	Output	Input	Output	Input	Output
9 00 10 30	Late 1:30 hours after the start	14 00 13 55	On time 5 minutes before the start	10 00 10 00	On time

Hints and Guidelines



It is recommended **to read the assignment a few times**, take notes and sketch the examples while thinking before you start writing code.

Processing The Input Data

According to the assignment, we expect **four** lines containing different **integers** to be passed. Examining the provided parameters, we can use the **int** type, as it is suitable for the expected values. We simultaneously **read** the input data and **parse** the string value to the selected data type for the **integer**.

```
exam_hours = int(input())
exam_minutes = int(input())
arrival_hours = int(input())
arrival_minutes = int(input())
```

Examining the expected output, we can create variables that contain the different output data types, to avoid using the so-called "**magic strings**" in the code.

```
late = "Late"
on_time = "On time"
early = "Early"
```

Calculations

After reading the input data, we can now start writing the logic for calculating the result. Let's first calculate the **start time** of the exam **in minutes** for easier and more accurate comparison:

```
exam_time = (exam_hours * 60) + exam_minutes
```

Let's also calculate the **student arrival time** using the same logic:

```
arrival_time = (arrival_hours * 60) + arrival_minutes
```

What remains is to calculate the difference between the two times, to determine **when** and **what time compared to the exam time** the student arrived at:

```
total_minutes_difference = arrival_time - exam_time
```

Our next step is to do the required **checks and calculations**, and finally, we are going to print the output. Let's separate the code into **two** parts.

- First, let's show when the student arrived – were they early, late or on time. To do that, we will use an **if-else** statement.
- After that, we're going to show the **time difference**, if the student arrives at a **different time** compared to the **exam starting time**.

To spare one additional check (**else**), we can, by default, assume that the student was late.

After that, according to the condition, we will check whether the difference in times is **more than 30 minutes**. If this is true, we assume that the student is **early**. If we do not match the first condition, we need to check if **the difference is less than or equal to zero** (≤ 0), by which we are checking the condition whether the student arrived within the range of **0 to 30 minutes** before the exam.

In all other cases, we assume that the student **was late**, which we set as **default**, and no additional check is needed:

```
student_arrival = late
if total_minutes_difference < -30:
    student_arrival = early
elif total_minutes_difference <= 0:
    student_arrival = on_time
```

Finally, we need to print **what is the time difference between exam start time and student arrival time**, as well as whether this time difference indicates the time of arrival **before or after the exam start**.

We check whether the time difference is **more than** one hour, to print hours and minutes in the required **format**, or **less than** one hour, to print **only minutes** as a format and description. We also need to do one more check – whether the time of student's arrival is **before** or **after** the exam start time.

```
result = ""
if total_minutes_difference != 0:
    hours_difference = abs(total_minutes_difference) // 60
    minutes_difference = abs(total_minutes_difference) % 60

    if hours_difference > 0:
        result = \
            f"{hours_difference}:{minutes_difference:02} hours"
    else:
        result = f"{minutes_difference} minutes"
```

```

if total_minutes_difference < 0:
    result += " before the start"
else:
    result += " after the start"

```

Printing The Result

Finally, what remains is to print the result on the console. According to the requirements, if the student arrived right on time (**not even a minute difference**), we do not need to print a second result. This is why we apply the following condition:

```

print(student_arrival)
if result:
    print(result)

```

Actually, for the task, printing the result **on the console** can be done at a much earlier stage – during the calculations. This, however, is not a very good practice. **Why?**

Let's examine the idea that our code is not 10 lines, but 100 or 1000! One day, printing the result will not be done on the console, but will be written in a **file** or displayed as a **web application**. Then, how many places in the code you will make changes at, due to such a correction? Are you sure you won't miss some places?



Always consider the code that contains logical calculations as a separate piece, different from the part that processes the input and output data. It has to be able to work regardless of how the data is passed to it and where the result will be displayed.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1052#0>.

Problem: Trip

It is strange, but most people start planning their vacations well in advance. A young programmer from Bulgaria has a **certain budget** and spare time in a particular **season**.

Write a program that accepts **as input the budget and season** and **as output** displays programmer's **vacation place** and **the amount of money they will spend**.

The **budget determines the destination**, and the **season determines what amount of the budget will be spent**. If the season is summer, the programmer will go camping, if it is winter – he will stay in a hotel. If it is in Europe, regardless of the season, the programmer will stay in a hotel. Each camp or hotel, according to the destination, has its price, which corresponds to a particular **percentage of the budget**:

- If 100 BGN or less – somewhere in **Bulgaria**.
 - Summer – 30% of the budget
 - Winter – 70% of the budget.

- If **1000 BGN or less** – somewhere in the **Balkans**.
 - **Summer** – 40% of the budget.
 - **Winter** – 80% of the budget.
- If **more than 1000 BGN** – somewhere in **Europe**.
 - Upon traveling in Europe, regardless of the season, the programmer will spend 90% of the budget.

Input Data

The input data will be read from the console and will consist of **two lines**:

- The **first line** holds the **budget** – a **floating-point(double) number** in the range [10.00 ... 5000.00].
- The **second line** holds one of two possible seasons that are **"summer"** or **"winter"**.

Output Data

Two lines must be printed on the console.

- On the **first line** – **"Somewhere in {destination}"** among **"Bulgaria"**, **"Balkans"** and **"Europe"**.
- On the **second line** – **"{Vacation type} – {Amount spent}"**.
 - The **Vacation** can be in a **"Camp"** or **"Hotel"**.
 - The **Amount** must be **rounded up to the second digit after the decimal point**.

Sample Input and Output

Input	Output
50 summer	Somewhere in Bulgaria Camp – 15.00

Input	Output
75 winter	Somewhere in Bulgaria Hotel – 52.50

Input	Output
312 summer	Somewhere in Balkans Camp – 124.80

Input	Output
1500 summer	Somewhere in Europe Hotel – 1350.00

Hints and Guidelines

Typically, as for the other tasks, we can separate the solution into the following parts:

- Reading the input data
- Doing calculations
- Printing the result

Processing The Input Data

While reading the requirements carefully, we understand that we expect **two** lines of input data. Our first parameter is a **real number**, for which we need to pick an appropriate variable type. We can pick **float** as a variable for the budget and – **string** for the season:

```
budget = float(input())
season = input()
```



Always take into consideration what **value type** is passed in the input data, as well as what type these need to be converted to, for the program conditions to work properly!

Calculations

Let's create and initialize the variables needed for applying the logic and calculations:

```
destination_result = ""
holiday_information = ""
money_spent = 0.00
```

Similar to the example in the previous task, we can initialize variables with some of the output results, to spare additional initialization.

When examining the problem requirements once again, we notice that the main distribution of where the vacation will take place is determined by the **value of the budget**, i.e. our main logic is divided into two cases:

- If the budget is **less than** a particular value.
- If it is **less than** another value or is **more than** the specified border value.

Based on the way we arrange the logical scheme (the order in which we will check the border values), we will have more or fewer conditions in the solution. **Think why!**

After that, we need to apply a condition to check the value of the **season**. Based on it, we will determine what percentage of the budget will be spent, as well as where the programmer will stay – in a **hotel** or a **camp**. This is a sample code that may be used to implement the above idea:

```
if budget <= 100:
    destination_result = "Bulgaria"
    if season == "summer":
        money_spent = 0.30 * budget
        holiday_information = f"Camp - {money_spent:.2f}"
    else:
        money_spent = 0.70 * budget
        holiday_information = f"Hotel - {money_spent:.2f}"
```

```
elif budget <= 1000:
    destination_result = "Balkans"
    if season == "summer":
        money_spent = 0.40 * budget
        holiday_information = f"Camp - {money_spent:.2f}"
    else:
        money_spent = 0.80 * budget
        holiday_information = f"Hotel - {money_spent:.2f}"
else:
    destination_result = "Europe"
    money_spent = 0.90 * budget
    holiday_information = f"Hotel - {money_spent:.2f}"
```

We can optimize the conditional check by assigning a **default value** and then checking one variant less. This saves one logical step. For example, this block:

```
if budget <= 100:
    destination_result = "Bulgaria"
    if season == "summer":
        money_spent = 0.30 * budget
        holiday_information = f"Camp - {money_spent:.2f}"
    else:
        money_spent = 0.70 * budget
        holiday_information = f"Hotel - {money_spent:.2f}"
```

can be shortened like so:

```
destination_result = "Bulgaria"
money_spent = 0.70 * budget
holiday_information = f"Hotel - {money_spent:.2f}"

if season == "summer":
    money_spent = 0.30 * budget
    holiday_information = f"Camp - {money_spent:.2f}"
```

Printing The Result

What remains is to display the calculated result on the console:

```
print("Somewhere in " + destination_result)
print(holiday_information)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1052#1>.

Problem: Operations

Write a program that reads **two integers (n1 and n2)** and an **operator** that performs a particular mathematical operation with them. Possible operations are: **summing up (+)**, **subtraction (-)**, **multiplying (*)**, **division (/)** and **modular division (%)**. Upon summing up, subtracting and multiplying, the console must print the result and display whether it is an **even** or an **odd** number. Upon regular division – **just the result**, and upon modular division – **the remainder**. You need to take into consideration the fact that **the divisor can be equal to zero (= 0)**, and dividing by zero is not possible. In this case, a **special notification** must be printed.

Input Data

3 lines are read from the console:

- N1 – integer within the range [0 ... 40 000].
- N2 – integer within the range [0 ... 40 000].
- Operator – one character among: "+", "-", "*", "/", "%". "+", "-", "\", "/", "%".

Output Data

Print the output as a **single line** on the console:

- If the operation is **summing up**, **subtraction** or **multiplying**:
 - "{N1} {operator} {N2} = {output} – {even/odd}".
- If the operation is **division**:
 - "{N1} / {N2} = {output}" – the result is formatted up to the second digit after the decimal point.
- If the operation is **modular division**:
 - "{N1} % {N2} = {remainder}".
- In case of **dividing by 0 (zero)**:
 - "Cannot divide {N1} by zero".

Sample Input and Output

Input	Output
10 1 -	10 - 1 = 9 - odd

Input	Output
10 0 %	Cannot divide 10 by zero

Input	Output
10 12 +	10 + 12 = 22 – even

Input	Output
123 12 /	123 / 12 = 10.25

Input	Output
112 0 /	Cannot divide 112 by zero

Input	Output
10 3 %	10 % 3 = 1

Hints and Guidelines

The problem is not complex, but there are a lot of lines of code to write.

Processing The Input Data

Upon reading the requirements, we understand that we expect **three** lines of input data. The first two lines are **integers** (within the specified range), and the third one – **an arithmetical symbol**.

```
n1 = int(input())
n2 = int(input())
n_operator = input()
```

Calculations

Let's create and initialize the variables needed for the logic and calculations. In one variable we will store **the calculations output**, and in the other one, we will use it for the **final output** of the program.

```
result = 0
output = ""
```

When carefully reading the requirements, we understand that there are cases where we don't need to do **any** calculations, and simply display a result. Therefore, we can first check if the second number is **0** (zero), as well as whether the operation is a **division** or a **modular division**, and then initialize the output.

```
if n2 == 0 and (n_operator == "/" or n_operator == "%"):
    output = f"Cannot divide {n1} by zero"
```

Let's place the output as a value upon initializing the **output** parameter. This way we can apply only **one condition** – whether it is needed to **recalculate** or **replace** this output.

Based on the approach that we choose, our next condition will be either a simple **elif** or a single **if**. In the body of this condition, using additional conditions regarding the manner of

calculating the output based on the passed operator, we can separate the logic based on the **structure** of the expected **output**.

From the requirements we can see that for **summing up (+)**, **subtraction (-)** or **multiplying (*)** the expected output has the same structure: "{n1} {operator} {n2} = {output} – {even/odd}", whereas for **division (/)** and **modular division (%)** the output has a different structure:

```
elif n_operator == "/":
    result = n1 / n2
    output = f"{n1} {n_operator} {n2} = {result:.2f}"
elif n_operator == "%":
    result = n1 % n2
    output = f"{n1} {n_operator} {n2} = {result}"
```

We finish the solution by applying conditions for summing up, subtraction and multiplying:

```
else:
    if n_operator == "+":
        result = n1 + n2
    elif n_operator == "-":
        result = n1 - n2
    elif n_operator == "*":
        result = n1 * n2

    output = f"{n1} {n_operator} {n2} = {result} " \
            f"- {'even' if result % 2 == 0 else 'odd'}"
```

For short and clear conditions, such as the above example for even and odd numbers, you can use a **ternary operator**. Let's examine the possibility to apply a condition **with** or **without** a ternary operator.

Without using a ternary operator the code is longer but easier to read:

```
number_is = ""
if result % 2 == 0:
    number_is = "even"
else:
    number_is = "odd"
```

Upon using a ternary operator the code is much shorter, but may require additional efforts to read and understand the logic:

```
number_is = 'even' if result % 2 == 0 else 'odd'
```

Printing The Output

Finally, what remains is to print the calculated result on the console:

```
print(output)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1052#2>.

Problem: Match Tickets

A group of football fans decided to buy tickets for Euro Cup 2016. The tickets are sold in two price categories:

- VIP – 499.99 BGN (Bulgarian leva).
- Normal – 249.99 BGN (Bulgarian leva).

The football fans have a shared budget, and the number of people in the group determines what percentage of the budget will be spent on transportation:

- 1 to 4 – 75% of the budget.
- 5 to 9 – 60% of the budget.
- 10 to 24 – 50% of the budget.
- 25 to 49 – 40% of the budget.
- 50 or more – 25% of the budget.

Write a program that calculates whether the money left in the budget will be enough for the football fans to buy tickets in the selected category, as well as how much money they will have left or be insufficient.

Input Data

The input data will be read from the console and contains exactly 3 lines:

- The first line contains the budget – a real number within the range [1 000.00 ... 1 000 000.00].
- The second line contains the category – "VIP" or "Normal".
- The third line contains the number of people in the group – an integer within the range [1 ... 200].

Output Data

Print the following on the console as one line:

- If the budget is sufficient:
 - "Yes! You have {N} leva left." – where N is the amount of remaining money for the group.
- If the budget is NOT sufficient:
 - "Not enough money! You need {M} leva." – where M is the insufficient amount.

The amounts must be formatted up to the second digit after the decimal point.

Sample Input and Output

Input	Output	Explanations
1000 Normal 1	Yes! You have 0.01 leva left.	1 person: 75% of the budget are spent on transport . Remaining amount: $1000 - 750 = 250$. Category Normal: The ticket price is $249.99 * 1 = 249.99$ $249.99 < 250$: they remain to him $250 - 249.99 = 0.01$

Input	Output	Explanations
30000 VIP 49	Not enough money! You need 6499.51 leva.	49 people: 40% of the budget are spent on transport . Remaining amount: $30000 - 12000 = 18000$. Category VIP: The ticket price is $499.99 * 49 = 24499.51$ $24499.51 > 18000$: Not enough money! Money needed $24499.51 - 18000 = 6499.51$

Hints and Guidelines

We will read the input data and perform the calculations described in the task requirements, to check if the money will be sufficient.

Processing The Input Data

Let's read the requirements carefully and examine what we expect to take as **input data**, what is expected to **return as a result**, as well as what the **main steps** for solving the problem are. For a start, let's process and save the input data in **appropriate variables**:

```
budget = float(input())
ticket_type = input()
people = int(input())
```

Calculations

Let's create and initialize the variables needed for doing the calculations:

```
transport_charges = 0.00
money_for_tickets = 0.00
money_difference = 0.00
```

Let's review the requirements once again. We need to perform **two** different block calculations. By the first set of calculations, we must understand what part of the budget has to be spent on **transportation**. You will notice that the logic for doing these calculations only depends on the **number of people in the group**. Therefore, we will do a logical breakdown according to the number of football fans. We will use a conditional statement – a sequence of **if-elif** blocks:

```
if people <= 4:
    transport_charges = 0.75 * budget
elif people <= 9:
    transport_charges = 0.60 * budget
elif people <= 24:
    transport_charges = 0.50 * budget
elif people <= 49:
    transport_charges = 0.40 * budget
elif people >= 50:
    transport_charges = 0.25 * budget
```

By the second set of calculations, we need to find out what amount will be needed to **purchase tickets** for the group. According to the requirements, this only depends on the type of tickets that we need to buy. Let's use **if-elif** conditional statement:

```
if ticket_type == "Normal":
    money_for_tickets = people * 249.99
elif ticket_type == "VIP":
    money_for_tickets = people * 499.99
```

Once we have calculated the **transportation costs** and **ticket costs**, what remains is to calculate the final result and understand if the group of football fans will **attend** Euro Cup 2016 or **not**, by the provided available parameters.

For the output, to spare one **condition** in the construction, we will assume that the group can, by default, attend Euro Cup 2016:

```
money_difference = \
    budget - transport_charges - money_for_tickets

result = f"Yes! You have {money_difference:.2f} leva left."

if money_difference < 0:
    result = "Not enough money! " \
        f"You need {abs(money_difference):.2f} leva."
```

Printing The Result

Finally, we need to display the calculated result on the console.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1052#3>.

Problem: Hotel Room

A hotel offers **two types** of rooms: studio and apartment.

Write a program that calculates **the price of the whole stay** for a studio and apartment. Prices depend on the **month** of the stay:

May and October	June and September	July and August
Studio – 50 BGN/night	Studio – 75.20 BGN/night	Studio – 76 BGN/night
Apartment – 65 BGN/night	Apartment – 68.70 BGN/night	Apartment – 77 BGN/night

The following **discounts** are also offered:

- For a **studio**, in the case of **more than 7** nights stayed in **May and October**: 5% discount.
- For a **studio**, in the case of **more than 14** nights stayed in **May and October**: 30% discount.
- For a **studio**, in the case of **more than 14** nights stayed in **June and September**: 20% discount.
- For an **apartment**, in the case of **more than 14** nights stayed, no limitation regarding the month: 10% discount.

Input Data

The input data will be read from the console and contains **exactly two lines**:

- The **first** line contains the **month** – May, June, July, August, September or October.
- The **second** line is the **amount of nights stayed** – integer within the range [0 ... 200].

Output Data

Print the following **two lines** on the console:

- On the **first** line: "Apartment: { price for the whole stay } lv."
- On the **second** line: "Studio: { price for the whole stay } lv."

The price for the whole duration of the stay must be formatted up to two symbols after the decimal point.

Sample Input and Output

Input	Output	Comments
May 15	Apartment: 877.50 lv. Studio: 525.00 lv.	In May , in case of more than 14 stays, the discount for a studio is 30% ($50 - 15 = 35$), and for the apartment is 10% ($65 - 6.5 = 58.5$).. The whole stay in the apartment : 877.50 lv. The whole stay in the studio : 525.00 lv.
Input	Output	
June 14	Apartment: 961.80 lv. Studio: 1052.80 lv	
Input	Output	
August 20	Apartment: 1386.00 lv. Studio: 1520.00 lv.	

Hints and Guidelines

We will read the input data and do the calculations according to the provided price list and the discount rules, and finally, print the result.

Processing The Input Data

According to the task requirements, we expect two lines of input data - the first line is the **month in which the stay is planned**, and the second - the **number of stays**. Let's process and store the input data in the appropriate parameters:

```
month = input()
nights = int(input())
```

Calculations

Now let's create and initialize the variables needed for the calculations:

```
studio_price = 50.00
apartment_price = 65.00
studio_rent = 0.00
apartment_rent = 0.00
```

When doing an additional analysis of the requirements, we understand that our main logic depends on what month is passed and what is the number of **stays**.

In general, there are different approaches and ways to apply the above conditions, but let's examine a basic **if-elif** conditional statement, as in each different **case** we will use **if** and **if-elif** conditional statements.

Let's start with the first group of months: **May** and **October**. For these two months, the price for a stay is the same for both types of accommodation - a **studio** or an **apartment**. Therefore, the only thing that remains is to apply an internal condition regarding the **number of stays** and recalculate the **relevant price** (if needed):

```
if month == "May" or month == "October":
    studio_price = 50.00
    apartment_price = 65.00

    studio_rent = studio_price * nights
    apartment_rent = apartment_price * nights

    if nights > 14:
        studio_rent *= 0.70
        apartment_rent *= 0.90
    elif nights > 7:
        studio_rent *= 0.95
```

To some extent, the **logic** and **calculations** will be **identical** for the following months:

```

elif month == "June" or month == "September":
    studio_price = 75.20
    apartment_price = 68.70

    studio_rent = studio_price * nights
    apartment_rent = apartment_price * nights

    if nights > 14:
        studio_rent *= 0.80
        apartment_rent *= 0.90
elif month == "July" or month == "August":
    studio_price = 76.00
    apartment_price = 77.00

    studio_rent = studio_price * nights
    apartment_rent = apartment_price * nights

    if nights > 14:
        apartment_rent *= 0.90

```

After calculating the relevant prices and the total amount for the stay, now let's prepare the formatted result. Before that, we should store it in our output **variables** - **studio_info** and **apartment_info**:

```

studio_info = f"Studio: {studio_rent:.2f} lv."
apartment_info = f"Apartment: {apartment_rent:.2f} lv."

```

To calculate the output parameters, we will use the **formatting .2f**. This formatting **rounds the decimal** number up to a specified number of characters after the decimal point. In our case, we will round the decimal number up to **2 digits** after the decimal point.

Printing The Result

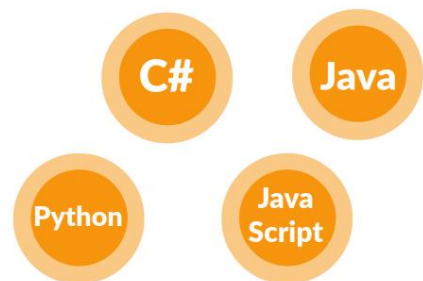
Finally, what remains is to print the calculated results on the console.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1052#4>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**

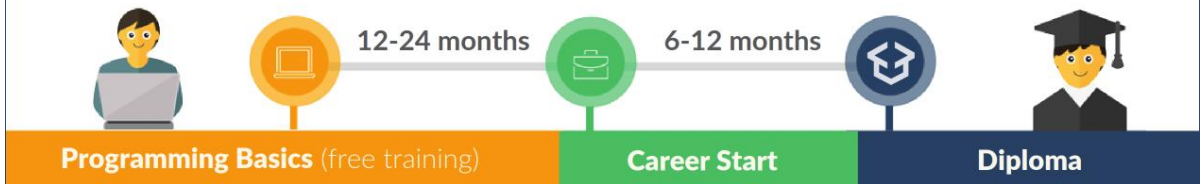


SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



[Apply now](#)

softuni.org/apply

Chapter 5.1. Loops

In this chapter, we will become familiar with how to **repeat blocks of commands**, known in programming as "**loops**". We will write a number of loops using the **for** operator in its classic form. Finally, we will solve some practical problems, which require repeating a series of actions, using loops.

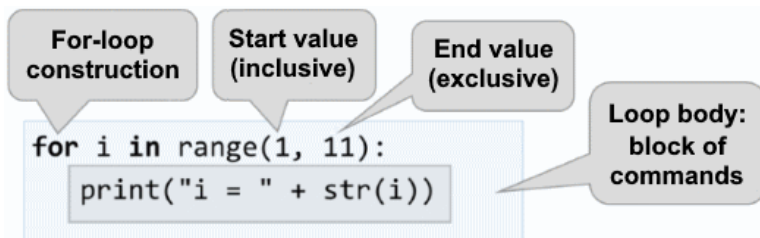
Repeating Blocks of Code (For Loop)

In programming, it is often required to **execute a block with commands multiple times**. For this reason, we are using **loops**. We can see an example of a **for** loop, which goes through the numbers from 1 to 10 and prints them:

```
for i in range(1, 11):
    print('i = ' + str(i))
```

The loop starts with the **operator for** and passes through all values of a particular variable in a given range, for example, all numbers from 1 to 10 included (without 11), and for each value, it performs a series of commands.

Upon declaring the loop, you can specify a **start value** and **end value**, while the end value is not included in the range. The **body of the loop** is a block with at least one command. The figure below shows the structure of a **for** loop:



In most cases, a **for** loop is run between **1** and **n** (for example from 1 to 10). The purpose of the loop is to go sequentially through the numbers 1, 2, 3, ..., n and for each of them to perform a particular action. In the example above, the variable **i** accepts values from 1 to 10 and the current value is printed in the body of the loop. The loop is repeated 10 times and each of these repetitions is called "**iteration**".

Problem: Numbers from 1 to 100

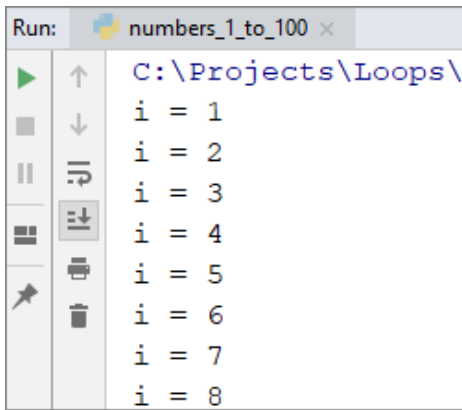
Write a program that **prints the numbers from 1 to 100**. The program does not accept input and prints the numbers from 1 to 100 sequentially, each on a separate line.

Hints and Guidelines

We can solve the problem using a **for** loop, via which we will pass through the numbers from 1 to 100 using the **i** variable, and print the numbers in the body of the loop:

```
for i in range(1, 101):
    print('i = ' + str(i))
```

Start the program with [Ctrl+Shift+F10] or with right-click - **Run** and **test it**:

A screenshot of a Java IDE window titled 'Run: numbers_1_to_100'. The window shows a list of numbers from 1 to 100, with the first eight numbers (1 through 8) visible. The numbers are printed in a monospaced font, and the window has a standard IDE interface with a toolbar on the left and a title bar at the top.

```
Run: numbers_1_to_100 x
C:\Projects\Loops\
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#0>.

You should get **100 points** (fully accurate solution).

Problem: Numbers Ending in 7

Write a program that finds all numbers in the range [1 ... 1000], that end in 7.

Hints and Guidelines

We can solve the problem by combining a **for loop** for passing through the numbers between 1 and 1000 and a condition to **check** if each of the numbers ends in 7. Of course, there are other solutions, but let's solve the problem using **loop + condition**:

```
for i in range(1, 1000):
    if i % 10 == 7:
        print(str(i))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#1>.

Problem: Latin Letters

Write a program that prints the letters from the Latin alphabet: a, b, c, ..., z.

Hints and Guidelines

It is good to know that the **for loops** don't work only with numbers. We can solve the task by running a **for loop**, that goes sequentially through all letters in the Latin alphabet:

```
for i in range(ord('a'), ord('z') + 1):
    print(chr(i))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#2>.

Problem: Sum Numbers

Write a program that reads **n** integers and finds their sum.

- The first line of the input holds the number of integers **n**.
- Each of the following **n** holds an integer.
- Sum up the numbers and print the result.

Sample Input and Output

Input	Output
2 10 20	30

Input	Output
3 -10 -20 -30	-60

Input	Output
4 45 -20 7 11	43

Input	Output
1 999	999

Input	Output
0	0

Hints and Guidelines

We can solve the problem by summing up numbers in the following way:

- We read the input number **n**.
- We initially start with a sum **sum = 0**.
- We run a loop from 0 to **n**. On each step of the loop, we read the number **num** and add it to the **sum**.
- Finally, we print the calculated amount **sum**.

Here is the source code for the solution:

```
n = int(input())
sum_nums = 0

for i in range(0, n):
    current_num = int(input())
    sum_nums = sum_nums + current_num

print("sum = " + str(sum_nums))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#3>.

Problem: Max Number

Write a program that enters **n integers** ($n > 0$) and finds **the max** among them. The first line of the input reads the number of integers. After that, the next lines read the integers, each on a separate line. Examples:

Sample Input and Output

Input	Output
2 100 99	100

Input	Output
3 -10 20 -30	20

Input	Output
4 45 -20 7 99	99

Input	Output
1 999	999

Input	Output
2 -1 -2	-1

Hints and Guidelines

We will first enter one number **n** (the number of integers that are about to be entered). We assign the current maximum **max** an initial neutral value, for example **-1000000**. Using a **for loop**, that is iterated **n-1 times**, we read one integer **num**. If the read number **num** is higher than the current maximum **max**, we assign the value of the **num** to the **max** value. Finally, in **max**, we must have stored the highest number. We print the number on the console.:

```
n = int(input('n = '))
max_num = -1000000000000000

for i in range(n):
    num = int(input())
    if num > max_num:
        max_num = num

print('max = ' + str(max_num))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#4>.

Problem: Min Number

Write a program that enters **n integers** ($n > 0$) and finds **the min** among them. The first line of the input reads the number of integers and after that, the next lines read the integers, each on a separate line.

Sample Input and Output

Input	Output	Input	Output	Input	Output
2		3		4	
100		-10		45	
99	99	20	-30	-20	-20
		-30		7	
				99	

Hints and Guidelines

The problem is completely identical to the previous one, except this time we will start with another neutral starting value:

```
n = int(input('n = '))
min_num = 1000000000000000000

# Use similar logic to previous problem

print('min = ' + str(min_num))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#5>.

Problem: Left and Right Sum

Write a program that reads an input **2 * n integers** and checks if **the sum of the first n numbers** (left sum) is equal to **the sum of the second n numbers** (right sum). In case the sums are equal, print **"Yes" + sum**, otherwise print **"No" + the difference**. The difference is calculated as a positive number (by absolute value). The format of the output must be identical to the one in the examples below.

Sample Input and Output

Input	Output
2	Yes, sum = 100
10	
90	
60	
40	

Input	Output
2	No, diff = 1
90	
9	
50	
50	

Hints and Guidelines

We will first input the number **n**, after that the first **n** numbers (**left** half) and we sum them up. We will then proceed with inputting more **n** numbers (**right** half) and sum them up. We calculate the **difference** between the sums by absolute value: **math.fabs(rightSum - leftSum)**. If the difference is 0, print "Yes" + sum, otherwise print "No" + the difference .

```
import math

n = int(input())
left_sum = 0
right_sum = 0

for i in range(0, n):
    numbersForLeftSide = float(input())
    left_sum += numbersForLeftSide

# Read the right part of the numbers and sum them
for i in range(0, n):
    numbersForRightSide = float(input())
    right_sum += numbersForRightSide

if left_sum == right_sum:
    print("Yes, sum = %d" % left_sum)
else:
    print("No, diff = %d" % math.fabs(right_sum - left_sum))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#6>.

Problem: Odd Even Sum

Write a program that inputs **n integers** and checks whether **the sum of the numbers on even positions** is equal to **the sum of the numbers on odd positions**. In case the sums are equal, print **"Yes" + sum**, otherwise print **"No" + the difference** . The difference is calculated by absolute value. The format of the output must be identical to the one in the examples below.

Sample Input and Output

Input	Output	Input	Output	Input	Output
4	Yes Sum = 70	4	No Diff = 1	3	No Diff = 2
10		3		5	
50		5		8	
60		1		1	
20		-2			

Hints and Guidelines

Input the numbers one by one and calculate the two **sums** (of the numbers on **even** positions and the numbers on **odd** positions). Identically to the previous problem, we calculate the absolute value of the difference and print the result ("**Yes**" + **sum** in case of difference of 0 or "**No**" + **the difference** in any other case):

```
import math

n = int(input())
even_sum = 0
odd_sum = 0

for i in range(1, n+1):
    num = int(input())
    if i % 2 == 0:
        even_sum += num
    else:
        odd_sum += num

# Print sum or difference
if even_sum == odd_sum:
    print("Yes", even_sum)
else:
    print("No", abs(even_sum - odd_sum))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#7>.

Problem: Vowels Sum

Write a program that inputs **text** (string), calculates and prints **the sum of the values of vowels** according to the table below:

a	e	i	o	u
1	2	3	4	5

Sample Input and Output

Input	Output
hello	6 (e+o = 2+4 = 6)
hi	3 (i = 3)

Input	Output
bamboo	9 (a+o+o = 1+4+4 = 9)
beer	4 (e+e = 2+2 = 4)

Hints and Guidelines

We read the input text **line**, null the sum, and run a loop, which goes through each symbol from the text. We check each letter **c** and verify if it is a vowel, and accordingly, add its value to the sum:

```
line = input().lower()
sum_vowels = 0

for c in line:
    if c == 'a':
        sum_vowels += 1
    elif c == 'e':
        sum_vowels += 2
    elif c == 'i':
        sum_vowels += 3
    elif c == 'o':
        sum_vowels += 4
    elif c == 'u':
        sum_vowels += 5

print(sum_vowels)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#8>.

What Have We Learned from This Chapter?

We can repeat a block of code with a **for** loop:

```
for i in range(1, 11):  
    print('i = ' + str(i))
```

We can read a sequence of **n** numbers from the console:

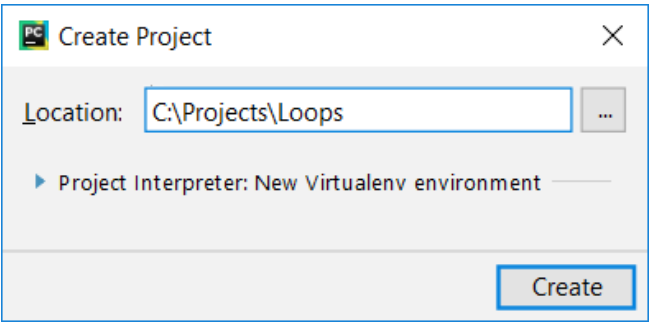
```
for i in range(n):  
    num = int(input())
```

Problems: Loops

Now, that we've become acquainted with the loops, it's time **to consolidate our knowledge in practice**, and as you know, -> this is done through writing lots of code. Let's solve a few problems for exercise.

Creating New Project in PyCharm

Create a new project in PyCharm (from [File] -> [New Project]), to organize better the exercise tasks. The purpose of this **project** is to contain **one Python file for each task** from the exercises:



Problem: Half Sum Element

Write a program that inputs **n integers** and checks whether there is a number among them, which is equal to the sum of all the rest. If there is such an element, print **"Yes" + its value**, otherwise print - **"No" + the difference between the largest element and the sum of the rest** (by absolute value).

Sample Input and Output

Input	Output	Comment	Input	Output	Comment
7 3 4 1 1 2	Yes Sum = 12	3 + 4 + 1 + 2 + 1 + 1 = 12	3 1 1 10	No Diff = 8	10 - (1 + 1) = 8

Input	Output	Comment	Input	Output	Comment
12 1					

Hints and Guidelines

We have to calculate **the sum** of all elements, find **the largest** of them, and check the condition.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#9>.

Problem: Odd / Even Positions

Write a program that reads **n numbers** and calculates **the sum**, **the min**, and **max** values of the numbers on **even** and **odd** positions (counted from 1). If there are no min / max elements, print "No".

Sample Input and Output

Input	Output	Input	Output	Input	Output
6 2 3 5 4 2 1	OddSum=9, OddMin=2, OddMax=5, EvenSum=8, EvenMin=1, EvenMax=4	2 1.5 -2.5	OddSum=1.5, OddMin=1.5, OddMax=1.5, EvenSum=-2.5, EvenMin=-2.5, EvenMax=-2.5	1 1	OddSum=1, OddMin=1, OddMax=1, EvenSum=0, EvenMin=No, EvenMax=No

Input	Output	Input	Output	Input	Output
3 -1 -2 -3	OddSum=-4, OddMin=-3, OddMax=-1, EvenSum=-2, EvenMin=-2, EvenMax=-2	1 -5	OddSum=-5, OddMin=-5, OddMax=-5, EvenSum=0, EvenMin=No, EvenMax=No	5 3 -2 8 11 -3	OddSum=8, OddMin=-3, OddMax=8, EvenSum=9, EvenMin=-2, EvenMax=11

Hints and Guidelines

This task combines some of the previous tasks: finding the **min**, **max** value, and **sum**, as well as processing of elements on **even and odd positions** . Check them out.

In the current task, it is better to work with **fractions** (not integers). The sum, the min, and the max will also be fractions. We must use a **neutral starting value** when finding the min / max value, for example 10000.0 and -10000.0. If the result is the neutral value, print "No".

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#10>.

Problem: Equal Pairs

There are $2 * n$ numbers. The first and the second number form a **pair**, the third and the fourth number as well, and so on. Each pair has a **value** – the sum of its numbers. Write a program that checks if all pairs have equal value. In case the value is the same, print "Yes, value=..." + the value, otherwise print the maximum difference between two neighboring pairs in the following format - "No, maxdiff=..." + the maximum difference. The input consists of the number n , followed by $2*n$ integers, all of them one per line.

Sample Input and Output

Input	Output	Comment	Input	Output	Comment
3 1 2 0 3 4 -1	Yes, value=3	values = {3, 3, 3} equal values	2 1 2 2 2	No, maxdiff=1	values = {3, 4} difference = {1} max difference = 1
2 -1 2 0 -1	No, maxdiff=2	values = {1, -1} difference = {2} max difference = 2	1 5 5	Yes, value=10	values = {10} one value → equal values

Hints and Guidelines

We read the input numbers **in pairs**. For each pair, we calculate its **sum**. While reading the input pairs, for each pair except the first one, we must calculate **the difference compared to the previous one**. To do that, we need to store as a separate variable the sum of the previous pair. Finally, we find the **largest difference** between two pairs. If it is 0, print "Yes" + the value, otherwise print - "No" + the difference.

Testing in The Judge System

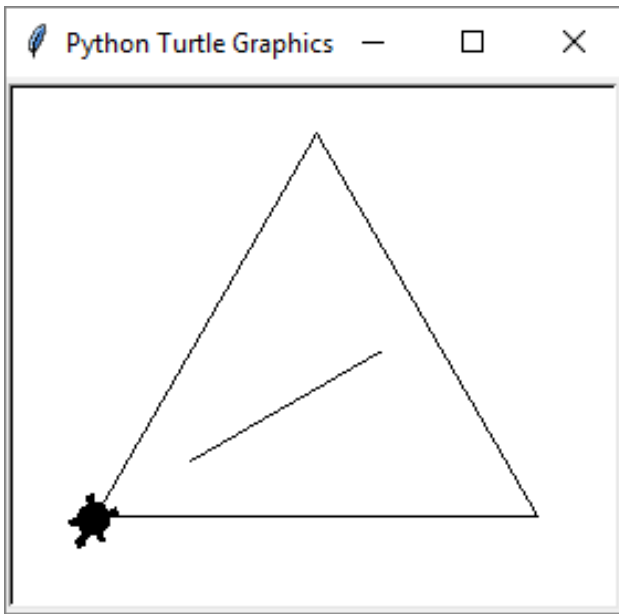
Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1053#11>.

Lab: Graphics and Web Applications

In the current chapter, we learned about **loops** as a programming construction that allows repeating a particular action or a group of actions multiple times. Now let's play with them. To do that, we will draw some figures that will consist of a large number made up of repeating graphical elements, but this time we will not do it on the console, but in a graphical environment using "**turtle graphics**". It will be interesting. And it is not hard at all. Try it!

Problem: Turtle Graphics GUI Application

The purpose of the following exercise is to play with a **drawing library**, also known as "**turtle graphics**". We will build a graphical application in which we will **draw various figures**, by moving our "**turtle**" across the screen via operations like "move 100 positions ahead", "turn 30 degrees to the right", "move 50 more positions ahead". The application will look approximately like this:

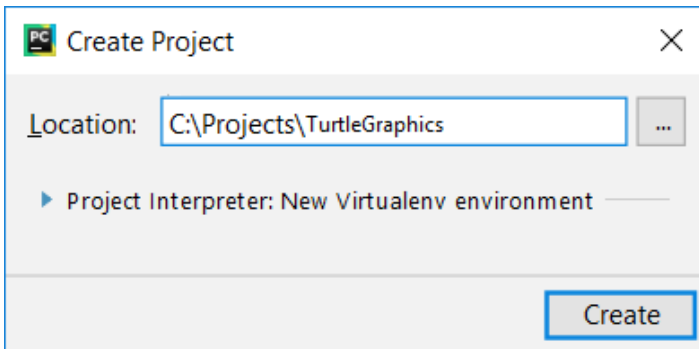


Let's first get familiar with the concept of drawing "Turtle Graphics". Take a look at the following sources:

- Definition of "turtle graphics": <https://wiki.c2.com/?TurtleGraphics>
- Article on "turtle graphics" in Wikipedia – https://en.wikipedia.org/wiki/Turtle_graphics

- Interactive online tool for drawing with a turtle – <https://blockly-games.appspot.com/turtle>

We will start by creating a new **project** in PyCharm:



In the newly created project, we add a new **Python File**. For the drawing, we will use the external library **turtle**. It defines class **Turtle**, which represents **drawing turtle**. To use it, we add the following code at the beginning of the Python file:

```
import turtle
```

```
my_turtle = turtle.Turtle()
```

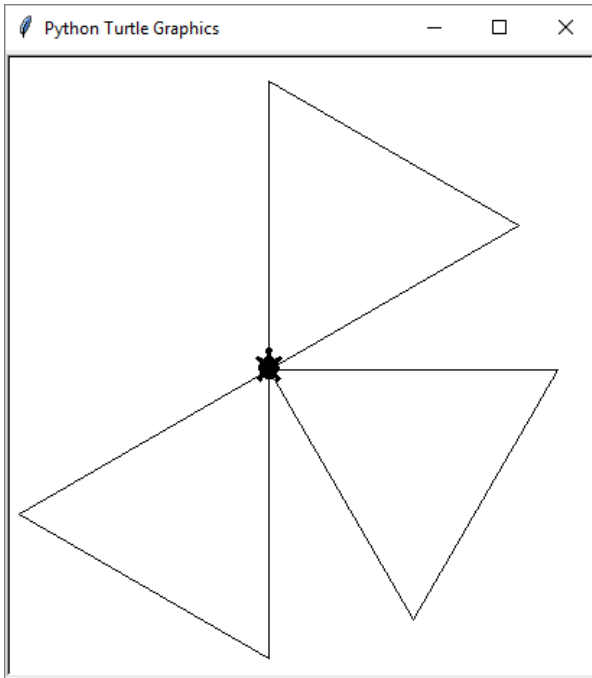
After that for the drawing we add the code:

```
# Change the shape of the turtle
my_turtle.shape("turtle")

for i in range(0, 3):
    # Draw a equilateral triangle
    my_turtle.left(30)
    my_turtle.forward(200)
    my_turtle.left(120)
    my_turtle.forward(200)
    my_turtle.left(120)
    my_turtle.forward(200)

turtle.done()
```

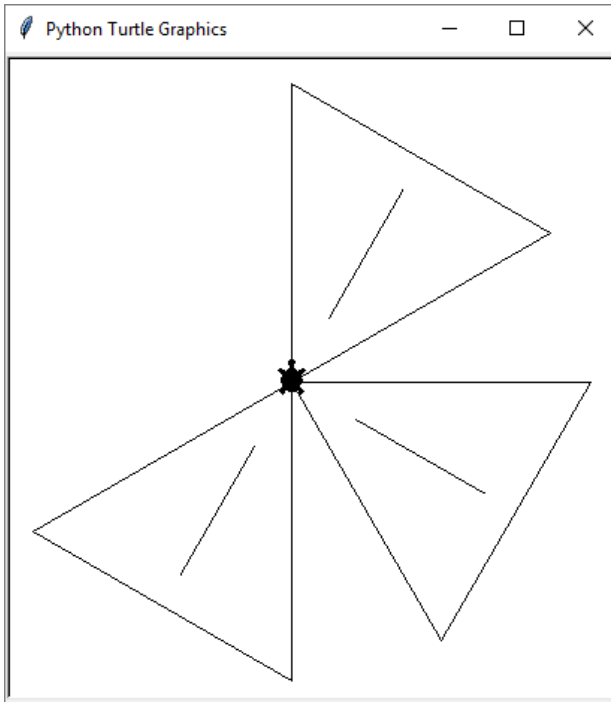
The above code moves and rotates the turtle, which is located in the center of the screen at the beginning (in the middle of the form) and draws an equilateral triangle. You can edit it and play with it. **Start** the app:



Now you can modify the turtle code and make it more complex, for the turtle to make a more complex figure:

```
for i in range(0, 3):  
    # Draw a equilateral triangle  
    my_turtle.left(30)  
    my_turtle.forward(200)  
    my_turtle.left(120)  
    my_turtle.forward(200)  
    my_turtle.left(120)  
    my_turtle.forward(200)  
  
    # Draw a line in the triangle  
    my_turtle.left(-30)  
    my_turtle.penup()  
    my_turtle.backward(50)  
    my_turtle.pendown()  
    my_turtle.backward(100)  
    my_turtle.penup()  
    my_turtle.forward(150)  
    my_turtle.pendown()  
    my_turtle.left(30)  
  
turtle.done()
```

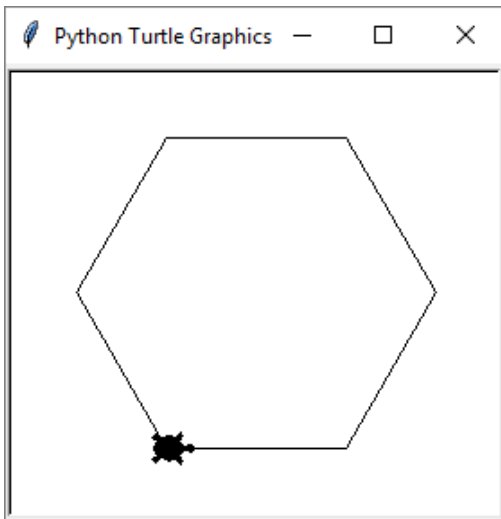
Again **start** the app to see the result:



Now our turtle draws more complex figures via a nice animated motion.

Problem: * Draw a Hexagon with The Turtle

Add a new Python file, with the name **hexagon**, which will draw a hexagon:



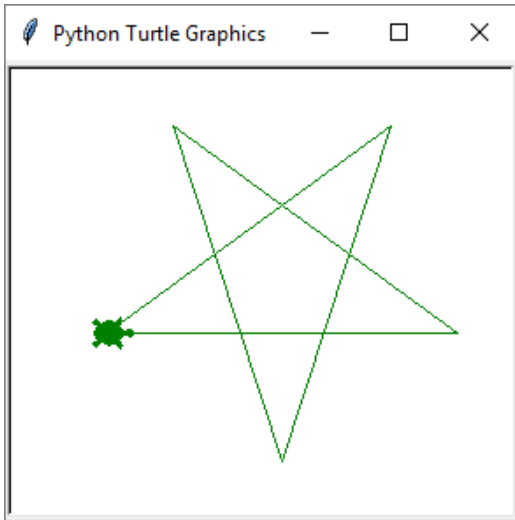
Hint:

With loop repeat the following 6 times:

- 60 degrees rotation.
- Forward step of 100.

Problem: * Draw a Star with The Turtle

Add new Python file **star.py**, which draws a star with 5 beams (**pentagram**), as on the figure below:



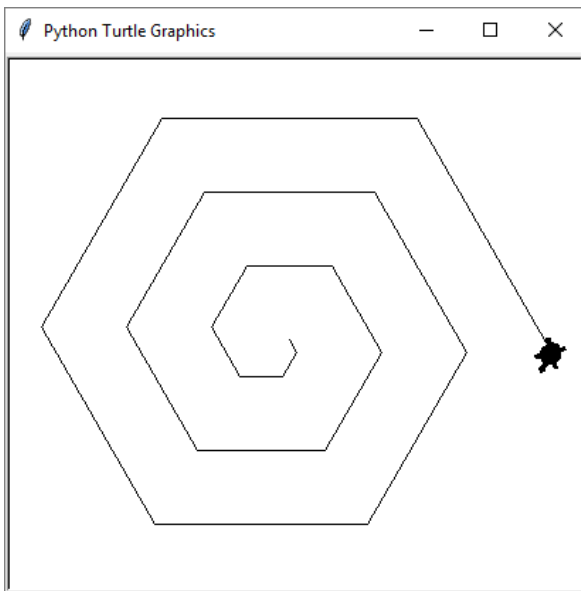
Hints: Change the colour: `my_turtle.color("green")`.

Repeat 5 times the following in a loop:

- Forward step of 200.
- 144 degrees rotation.

Problem" * Draw a Spiral with The Turtle

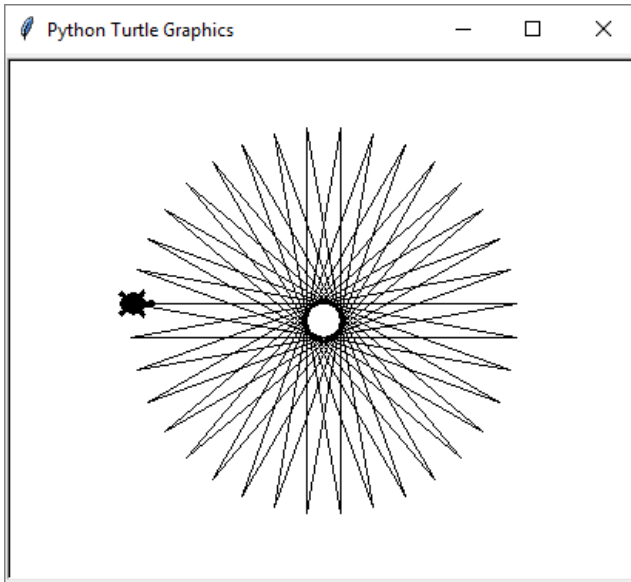
Add new Python file **spiral.py**, which draws spiral with 20 beams as on the figure below:



Hint: Draw in a loop by moving ahead and rotating. In each step, decrease the length gradually of the forward step and rotate 60 degrees.

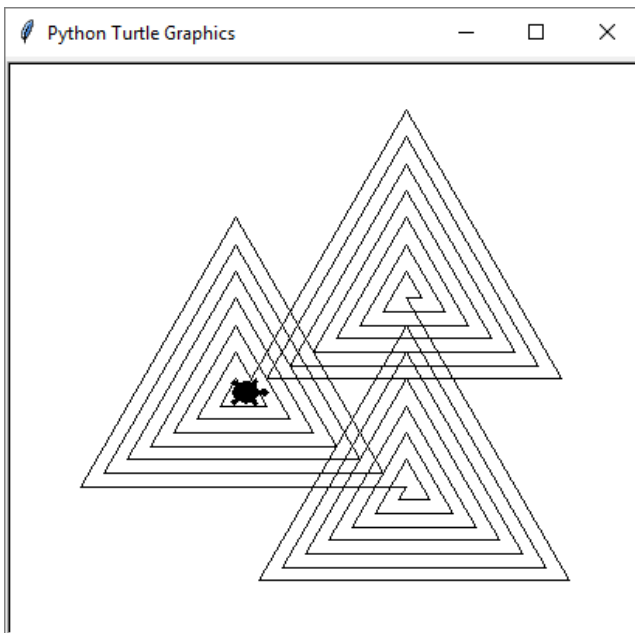
Problem: * Draw a Sun with The Turtle

Add a new Python file `sun.py`, which draws a sun with 36 beams, as on the figure below:



Problem: * Draw a Spiral Triangles with The Turtle

Add new Python file `triangle.py`, which draws three triangles with 22 beams each, as on the figure below:

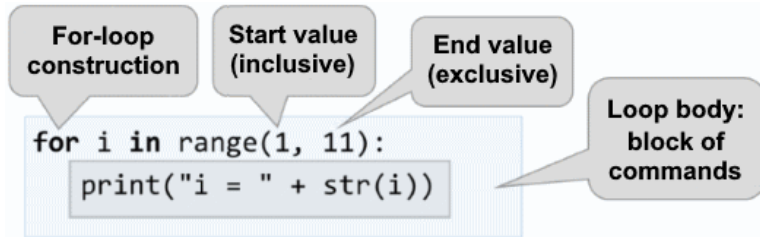


Hint: Draw in a loop by moving forward and rotating. In each step, increase the length of the forward step by 10 and rotate 120 degrees. Repeat 3 times for the three triangles.

If you have a problem with the above exercises, ask for help in the **SoftUni's Reddit Community**: <https://www.reddit.com/r/softuni/>.

Chapter 5.2. Loops – Exam Problems

In the previous chapter, we learned how to run a command block **more than once**. That's why we implemented **for loop** and covered some of its main applications. Our task in the current chapter is to hone our knowledge by solving a couple of more complex problems with loops, which appear in exams. For some of them, we'll show detailed solved examples, while for others there'll be tips only. Before we begin, we'll recall the **for loop** construction:



for loops consist of:

- **Initialization block**, where the variable-counter (**i**) is declared, and with the help of the **range(...)** function built into Python, we define what its starting and ending value will be.
- **Updating the counter** – we implement it as a third parameter in the **range(...)** function, and it shows with how many steps the variable-counter should be updated

Problem: Histogram

We're given **n-count integers** in the range `[1 ... 1000]`. A percent of them, **p1**, is under 200, **p2** percent are between 200 and 399, **p3** percent are between 400 and 599, **p4** percent are between 600 and 799, and the remaining **p5** percent begin at 800. Write a program that calculates and prints the percentages **p1**, **p2**, **p3**, **p4**, and **p5**.

Example: we have $n = 20$ integers: 53, 7, 56, 180, 450, 920, 12, 7, 150, 250, 680, 2, 600, 200, 800, 799, 199, 46, 128, 65. We get the following distribution and visualization:

Group	Numbers	Number count	Percentage
< 200	53, 7, 56, 180, 12, 7, 150, 2, 199, 46, 128, 65	12	$p1 = 12 / 20 * 100 = 60.00\%$
200 ... 399	250, 200	2	$p2 = 2 / 20 * 100 = 10.00\%$
400 ... 599	450	1	$p3 = 1 / 20 * 100 = 5.00\%$

Group	Numbers	Number count	Percentage
600 ... 799	680, 600, 799	3	$p4 = 3 / 20 * 100 = 15.00\%$
≥ 800	920, 800	2	$p5 = 2 / 20 * 100 = 10.00\%$

Input Data

On the first line of the input is an integer **n** ($1 \leq n \leq 1000$), which stands for the number of lines with numbers, which will be given to us. On the next **n lines**, there is **one integer** in the range [**1 ... 1000**] – the numbers that the histogram will be based on.

Output Data

In the console, print a histogram of **5 lines**, each of them containing a number between 0% and 100%, formatted with two-digit precision after the decimal point (for example, 25.00%, 66.67%, 57.14%).

Sample Input and Output

Input	Output	Input	Output	Input	Output
9	33.33%	14	57.14%	7	14.29%
367	33.33%	53	14.29%	800	28.57%
99	11.11%	7	7.14%	801	14.29%
200	11.11%	56	14.29%	250	14.29%
799	11.11%	180	7.14%	199	28.57%
999		450		399	
333		920		599	
555		12		799	
111		7			
9		150			
		250			
		680			
		2			
		600			
		200			

Input	Output	Input	Output
3	66.67%	4	75.00%
1	0.00%	53	0.00%
2	0.00%	7	0.00%
999	0.00%	56	0.00%
	33.33%	999	25.00%

Hints and Guidelines

The program that solves this problem can be divided theoretically into three parts:

- **Reading the input data** – in the current problem, this means reading the integer **n**, followed by a **count of n integers**, each on a new line.
- **Processing The Input Data** – in this case, this means dividing the numbers into groups and calculating the division percentage by those groups.
- **Outputting the final result** – printing the histogram in the console, in the given format.

Processing The Input Data

Before we transition to the real reading of the input, we have to **declare our variables**, in which the data will be stored:

```
#Variables, in which we'll save
#the percentage division for each group
p1_percentage = 0
p2_percentage = 0
p3_percentage = 0
p4_percentage = 0
p5_percentage = 0

#Variables, storing the amount of numbers by groups
cnt_p1 = 0
cnt_p2 = 0
cnt_p3 = 0
cnt_p4 = 0
cnt_p5 = 0
```

We declare variables **p1_percentage**, **p2_percentage**, etc., in which we'll store the percentages, as well as **cnt_p1**, **cnt_p2**, etc., in which we'll keep the count of numbers for the respective group.

After we've declared the needed variables, we can move on to reading the number **n** from the console:

```
n = int(input())
```

Processing The Output Data

To read and assign each number to its respective group, we'll use a **for-loop** from 0 to **n** (the count of the numbers). Each iteration of the cycle will read and assign **only one number** (**current_number**) to its respective group. So that we can decide if a selected number belongs to a group, **we check its range**. If it passes, we increase the count of this group's numbers (**cnt_p1**, **cnt_p2**, etc.) by 1:

```
for index in range(n):
    current_number = int(input())

    if current_number < 200:
        cnt_p1 += 1
    elif 200 <= current_number <= 399:
        cnt_p2 += 1
    elif 400 <= current_number <= 599:
        cnt_p3 += 1
    elif 600 <= current_number <= 799:
        cnt_p4 += 1
    else:
        cnt_p5 += 1
```

After we've found out how many numbers there are in each group, we can move on to calculating the percentages, which is also the main part of the problem. We'll use the following formula:

(Group percentage) = (Group number count) * 100 / (Count of all numbers)

It doesn't matter whether we'll divide by 100 (an **integer** type), or 100.0(a **float** type) since the **division** will take place and the result will be saved to the variable. Example: 5 / 2 = 2.5, and 5 / 2.0 = 2.5. In **Python 3**, there's no difference whether we'll be dividing by an integer or a real number - if the result is a real number itself, then it will be saved in the variable as a floating-point number. But in **Python 2.7** we have to convert the numbers to a **float** type, to get the correct result – a real number. Having that in mind, the first variable's formula will look like this:

```
p1_percentage = cnt_p1 * 100 / n
#TODO: Add the formulas for the other variables
```

To better understand what's happening, let's look at the following example:

Input	Output
3	66.67%
1	0.00%
2	0.00%
999	0.00%
	33.33%

In this case, **n = 3**. The cycle consists of:

- **i = 0** - we read the number 1, which is lower than 200 and belongs to the first group, thus increasing the counter of the number (**cnt_p1**) by 1.
- **i = 1** - we read the number 2, which, again, belongs to the first group and we increase the group's counter(**cnt_p1**) by 1.
- **i = 2** - we read the number 999, which belongs to the last group(**p5**), because it's bigger than 800, and we increase its group counter (**cnt_p5**) by 1.

After reading the numbers, we have two of them in the first group, and we have only one in the last group. There are **no numbers** in the other groups. After we apply the aforementioned formula, we calculate the percentage of each group. It doesn't matter whether we multiply by **100** or **100.0** - we'll get the same result: the **first** group has 66.67%, and the **last** group - 33.33%. We have to mention that this is valid only for **Python 3**.

Printing The Final Result

The last step is to print the calculated results. In the problem's description, it's said that the percentages have to be with **2-digit precision after the decimal point**. To achieve this, we have to write **.2f** after the placeholder.

```
print("%.2f" % p1_percentage + "%")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1054#0>.

Problem: Smart Lily

Lily is **N years old**. Each **birthday** she receives a gift. For her **odd** birthdays (1, 3, 5, ..., n) she receives **toys**, and for each **even** birthday (2, 4, 6, ..., n) she receives **money**. For her **second birthday** she receives **10.00 USD**, and **the sum increases by 10 USD with each following even birthday** (2 -> 10, 4 -> 20, 6 -> 30, etc.). Lily has secretly been saving the money for years. Her **brother**, in the years when she **receives money**, **takes 1.00 USD**. Lily **sold the toys** received with the years, **each for P USD**, and added the sum to the saved money. With them, she wants to **buy a washing machine for X USD**. Write a program that calculates **how much money she has saved** and whether **it's enough to buy a washing machine**.

Input Data

3 numbers are read from the console, each on a new line:

- Lily's age - integer in the range [1 ... 77].
- The price of the washing machine - a number in the range [1.00 ... 10 000.00].
- The price of a single toy - integer in the range [0 ... 40].

Output Data

Print a single line in the console:

- If Lily's money is enough:
 - "Yes! {N}" – where N is the remaining money after the purchase
- If it is not:
 - "No! {M}" – where M is the amount of money lacking
- The numbers N and M should be formatted with 2-digit precision after the decimal point.

Sample Input and Output

Input	Output	Comments
10 170.00 6	Yes! 5.00	On the first birthday she receives a toy; Second -> 10 USD; 3rd -> toy; 4th -> 10 + 10 = 20 USD; 5th -> toy; 6th -> 20 + 10 = 30 USD; 7th -> toy; 8th -> 30 + 10 = 40 USD; 9th -> toy; 10th -> 40 + 10 = 50 USD She has saved -> 10 + 20 + 30 + 40 + 50 = 150 USD. She has sold 5 toys by 6 USD = 30 USD. Her brother took 1 USD for 5 years = 5 USD. Remaining money -> 150 + 30 – 5 = 175 USD. 175 >= 170 (washing machine's price) She has succeeded buying it and there remain 175-170 = 5 USD.
21 1570.98 3	No! 997.98	She has saved 550 USD. She's sold 11 toys by 3 USD each = 33 USD. Her brother has taken 1 USD for 10 years = 10 USD. There remain 550 + 33 – 10 = 573 USD. 573 < 1570.98 – She's failed buying a washing machine. She needs 1570.98–573 = 997.98 USD more.

Hints and Guidelines

Solving this problem, like the previous one, again can be divided into three parts – **reading** the input data, **processing** it, and **outputting** a result.

As we already know, like in most scripting languages, in Python as well, we don't bother defining the types of the variables that we declare. The interpreter decides on its own what it'll be. For Lily's (**age**) and a single toy's price (**present_price**) in the problem's description, it's said that they'll be **integers**. That's why we'll use the **built-in function int()** to convert the read value from string to integer. When the **input()** function is used, the input's value in the console is always (**string**), that's why if a conversion to another type is needed, we can use the **built-in functions of Python** for this problem. For the washing machine's price,

(**price_of_washing_machine**), we know that it's a fractional number and we choose the **float** type. In the code below we declare and initialize (assign a value) to the variables:

```
age = int(input())
price_of_washing_machine = float(input())
present_price = int(input())
```

To solve the problem, we'll need a couple of helper variables – for the toys' count (**number_of_toys**) for the saved money (**saved_money**) and the money received on each birthday (**money_for_birthday**). We initially assign 10 to **money_for_birthday**, because in the description it's said that the first sum received by Lily is 10 USD:

```
number_of_toys = 0
saved_money = 0
money_for_birthday = 10
```

With a **for** loop, we go through each of Lily's birthdays. When a loop variable is an **even** number, it means that Lily has **received money** and we add them to her savings. At the same time, we **subtract 1 USD** - the money taken by her brother. After that we **increase** the value of the variable **money_for_birthday**, meaning we increase the sum by 10 for the next time she receives money for her birthday. Contrary, when the loop variable is an **odd** number, we increase the **toys'** count. Checking whether it's even or odd happens with a **division with the remainder (%)** by 2 – when the remainder is 0, the number is even, and when the remainder's 1 - it's odd:

```
for current_year in range(1, age + 1):
    if current_year % 2 == 0:
        saved_money += (money_for_birthday - 1)
        money_for_birthday += 10
    else:
        number_of_toys += 1
```

We add the money from the sold toys to Lily's savings:

```
saved_money += number_of_toys * present_price
```

At the end we print the results, taking into account the required formatting, meaning the sum has to be **rounded to 2 digits after the decimal point**:

```
print("Yes! %.2f " % (saved_money - price_of_washing_machine)
      if saved_money >= price_of_washing_machine
      else "No! %.2f" % (price_of_washing_machine - saved_money))
```

In some programming languages there's a construction called **conditional operator** (**?:**) (also known as ternary operator), as it's shorter to write. It has the following syntax in Python: **operand1 if operand2 else operand3**. The second operand is our condition and it has to be of **bool** type (meaning it has to return **true/false**). If **operand2** returns **true**, it'll execute **operand1**, and if it returns **false** – **operand3**. In our case, we check whether Lily's **saved money** is enough to buy a washing machine. If it's higher or equal to its price, the check **saved_money >= price_of_washing_machine** will return **true** and it'll print "Yes! ...", while if it's lower – the result will be **false**, and "No! ..." will be printed. Of course, instead of the ternary operator, we can use simple **if** expressions.

More about ternary operators:
https://book.pythontips.com/en/latest/ternary_operators.html.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1054#1>.

Problem: Back to The Past

John is **18 years old** and receives an inheritance of **X USD** and a **time-traveling machine**. He decides to **travel back to 1800**, but he doesn't know **whether the money is enough** to live without working. Write a **program that calculates** whether John **will have enough money**, to live without working **until a given year, including the year itself**. We accept that **each even year** (1800, 1802, etc.) he'll **spend 12 000 dollars**. For each **odd year** (1801, 1803, etc.) he'll spend **12 000 + 50 * [John's age in the given year]**.

Input Data

The input is read from the console and **contains exactly 2 lines**:

- The **inherited money** – a real number in the range [1.00 ... 1 000 000.00].
- The **year until he has to live (inclusive)** – a real number in the range [1801 ... 1900].

Output Data

Print to the console **1 line**. The sum has to be formatted with **2-digit precision after the decimal point**:

- If the money is enough:
 - "Yes! He will live a carefree life and will have {N} dollars left." – where **N** is the remaining money.
- If the money is NOT enough:
 - "He will need {M} dollars to survive." – where **M** is the amount lacking.

Sample Input and Output

Input	Output	Explanation
50000 1802	Yes! He will live a carefree life and will have 13050.00 dollars left.	1800 → even → Spends 12000 dollars → Left 50000 - 12000 = 38000 1801 → odd → Spends 12000 + 19*50 = 12950 dollars → Left 38000 - 12950 = 25050 1802 → even

Input	Output	Explanation
		→ Spends 12000 dollars → Left $25050 - 12000 = 13050$
100000.15 1808	He will need 12399.85 dollars to survive.	1800 → even → Left $100000.15 - 12000 = 88000.15$ 1801 → odd → Left $88000.15 - 12950 = 75050.15$... 1808 → even → $-399.85 - 12000 = -12399.85$ 12399.85 lacking

Hints and Guidelines

The method of solving this problem isn't unlike the previous ones, so we begin by **declaring and initializing** the needed variables. In the problem's description, it's said that John's age is 18, so we declare the variable **years** with the initial value of **18**. We read other variables from the console:

```
heritage = float(input())
year_to_live = int(input())
years = 18
```

With the help of a **for loop**, we loop through all years. We begin at **1800** – the year when John travels back in time, and we end at **the year until he has to live**. In the loop, we check whether the current year is **even** or **odd**. We check it with **division with a remainder (%)** by 2. If the year is **even**, from the inheritance (**inheritance**) we subtract **12000**, while if it's **odd**, from the inheritance (**inheritance**) we subtract **12000 + 50 * (John's age)**:

```
for current_year in range(1800, year_to_live + 1):
    if current_year % 2 == 0:
        heritage -= 12000
    else:
        heritage -= (12000 + 50 * years)
    years += 1
```

In the end, we print the results, and we do a **check whether the inheritance (inheritance)** has been enough for him to live without working or not. If the inheritance (**inheritance**) is a positive number, we print: **"Yes! He will live a carefree life and will have {N} dollars left."**, while if it's a negative number: **"He will need {M} dollars to survive."**. We don't forget to format the sum with 2-digit precision after the decimal point.

Hint: Think about using the function **abs(...)** when printing the output and the inheritance is not enough.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1054#2>.

Problem: Hospital

For a given amount of time, patients arrive for a checkup in the hospital every day. She initially has 7 doctors. Each of them can check one patient a day only, but sometimes there's a shortage of doctors, so the other patients are sent to other hospitals. Every third day the hospital calculates whether the count of patients that haven't been examined is higher than those that have been and if so, an additional doctor is assigned. The assignment happens before the start of the day.

Write a program that calculates the count of treated and untreated patients for the given period.

Input Data

The input is read from the console and contains:

- On the first line – the period for which we'll be calculating. Integer in the range [1 ... 1000].
- On the following lines (equal to the number of days) – the count of patients that arrive for a checkup on the current day. Integer in the range [0 ... 10 000].

Output Data

Print to the console 2 lines:

- On the first line: "Treated patients: {count of treated patients}."
- On the second line: "Untreated patients: {count of untreated patients}."

Sample Input and Output

Input	Output	Explanations
4 7 27 9 1	Treated patients: 23. Untreated patients: 21.	Day 1: 7 treated and 0 untreated patients for the day Day 2: 7 treated and 20 untreated patients for the day Day 3: To this moment the treated patients are 14, and the untreated – 20 -> New doctor is hired -> 8 treated and 1 untreated patient for the day Day 4: 1 treated and 0 untreated patients for

Input	Output	Explanations
		the day In total: 23 treated and 21 untreated patients.

Input	Output
6	Treated patients: 40.
25	Untreated patients: 87.
25	
25	
25	
25	
2	

Input	Output
3	Treated patients: 21.
7	Untreated patients: 0.
7	
7	

Hints and Guidelines

Again, we begin by **declaring and initializing** the needed variables. The period, for which we have to do our calculations, we read from the console and assign to the variable **period**. We'll need a couple of additional variables: the count of treated patients (**treated_patients**), the count of untreated patients (**untreated_patients**), and the count of doctors (**count_of_doctors**), which is initially 7:

```
period = int(input())

treated_patients = 0
untreated_patients = 0
count_of_doctors = 7
```

With the help of the **for loop**, we go through all days in the given period (**period**). For each day we read the number of patients from the console (**current_patients**). The addition of doctors is said in the problem's description to happen **every third day**, BUT only if the untreated patients' count is **higher** than the treated's count. That's why we check whether the day is the third one – with the arithmetic operator for division with a remainder (%): **day % 3 == 0**.

Example:

- If the day is a **third** one, the remainder of division by 3 will be 0 (**3 % 3 = 0**) and the check **day % 3 == 0** will return **true**.
- If the day is a **second** one, the remainder of division by 3 will be 2 (**2 % 3 = 2**) and the check will return **false**.
- If the day is a **fourth** one, the remainder of the division will be 1 (**4 % 3 = 1**) and the check will again return **false**.

If the conditional check `day % 3 == 0` returns `true`, there'll also be a check whether the count of untreated patients is higher than the treated's count: `untreated_patients > treated_patients`. If the result is again `true`, then the count of doctors (`count_of_doctors`) will increase.

After that, we check whether the patients' count for the current day (`current_patients`) is higher than the doctors' count (`count_of_doctors`). If the patients' count is higher:

- We increase the value of the variable `treated_patients` with the doctors' count (`count_of_doctors`).
- We increase the value of the variable `untreated_patients` with the count of patients left, which we calculate by subtracting the doctors' count from the patients' count (`current_patients - count_of_doctors`).

If the patients' count is lower, we increase only the variable `treated_patients` with the current day's count of patients (`current_patients`):

```
for day in range(1, period + 1):
    current_patients = int(input())

    if (day % 3 == 0) and (untreated_patients > treated_patients):
        count_of_doctors += 1

    if current_patients > count_of_doctors:
        treated_patients += count_of_doctors
        untreated_patients += current_patients - count_of_doctors
    else:
        treated_patients += current_patients
```

In the end, we only have to print the count of treated and untreated patients.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1054#3>.

Problem: Division

We're given `n` integers in the range `[1 ... 1000]`. A percentage of them, percent `p1`, are divided by 2 without remainder, percent `p2` are divided by 3 without remainder, percent `p3` are divided by 4 without remainder. Write a program that calculates and prints the percentages `p1`, `p2`, and `p3`. **Example:** we have `n = 10` integers: 680, 2, 600, 200, 800, 799, 199, 46, 128, 65. We get the following distribution and visualization:

Division without remainder by:	Numbers	Count	Percentage
2	680, 2, 600, 200, 800, 46, 128	7	$p1 = (7 / 10) * 100 = 70.00\%$
3	600	1	$p2 = (1 / 10) * 100 = 10.00\%$
4	680, 600, 200, 800, 128	5	$p3 = (5 / 10) * 100 = 50.00\%$

Input Data

On the first line of the input stands the integer **n** ($1 \leq n \leq 1000$) – count of numbers. On the next **n lines**, each stands a **single integer** in the range **[1 ... 1000]** – the numbers for which we'll check their divisors.

Output Data

Print to the console **3 lines**, each of them containing a percentage between 0% and 100%, with 2-digit precision after the decimal point, for example: 25.00%, 66.67%, 57.14%.

- On the **first line** – the percentage of numbers **divisible by 2**.
- On the **second line** – the percentage of numbers **divisible by 3**.
- On the **third line** – the percentage of numbers **divisible by 4**.

Sample Input and Output

Input	Output	Input	Output	Input	Output
10	70.00%	3	33.33%	1	100.00%
680	10.00%	3	100.00%	12	100.00%
2	50.00%	6	0.00%		100.00%
600		9			
200					
800					
799					
199					
46					
128					
65					

Hints and Guidelines

For this and the next problem, you'll have to write the program's code on your own, with the help of the following advice.

The program that solves the current problem is similar to the one from the problem **Histogram**, which we viewed previously. That's why we can begin by declaring our needed variables. For example, a couple of variable names can be **n** – count of numbers (which we'll read from the console) and **divisible_by_2**, **divisible_by_3**, **divisible_by_4** – additional variables, storing the count of numbers in their respective groups.

To read and distribute each number in its respective group, we'll have to start our **for** loop from **0** and end at **n** (the numbers' count). Each of the loop's iterations has to read and distribute **a single number**. The difference is that **a single number can be distributed to more than one group simultaneously**, so we have to do **three different if** checks for each number – whether it's divided by 2, 3, and 4, and to increase the value of the variable that stores the count of numbers in the respective group.

Warning: an **if-elif** construction won't be of use to us, since when it detects a match, the loop gets broken before checking the following conditions.

In the end, print the found results while keeping the given in the problem's description format.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1054#4>.

Problem: Logistics

You're responsible for the logistics of different cargos. Depending on the weight of each load, a different vehicle is needed and costs a different price per ton:

- Up to and including **3 tons** – **van** (200 USD per ton).
- Over 3 and up to **11 tons** – **truck** (175 USD per ton).
- Over **11 tons** – **train** (120 USD per ton).

Your task is to calculate the **average price per ton of transported load**, as well as **what percentage of the overall cargo** is transported by each vehicle.

Input Data

A sequence of numbers is read from the console, each on a different line:

- First line: count of loads for transportation – **integer** in the range [1 ... 1000].
- On each following line, the **weight in tons of the current load** is written – **integer** in the range [1 ... 1000].

Output Data

Print 4 lines to the console, as given:

- **Line #1** – the **average price per ton of transported cargo** (rounded to the second digit after the decimal point).
- **Line #2** – the **percentage** of cargo, transported with a **van** (between 0.00% and 100.00%, rounded to the second digit after the decimal point).
- **Line #3** – the **percentage** of cargo, transported with a **truck** (between 0.00% and 100.00%).
- **Line #4** – the **percentage** of cargo, transported with a **train** (between 0.00% and 100.00%).

Sample Input and Output

Input	Output	Explanations
4 1 5 16 3	143.80 16.00% 20.00% 64.00%	Two of the cargoes are transported by van 1 + 3 , in total 4 tons. One of the shipments is transported by lorry: 5 tons. One of the shipments is transported by train: 16 tons. The sum of all the cargo is: $1 + 5 + 16 + 3 = 25$ tons. Percentage cargo by van : $4/25 * 100 = 16.00\%$ Percentage cargo by lorry : $5/25 * 100 = 20.00\%$ Percentage cargo by train : $16/25 * 100 = 64.00\%$ Average price per ton of transported cargo: $(4 * 200 + 5 * 175 + 16 * 120) / 25 = 143.80$

Input	Output	Input	Output
5	149.38	4	120.35
2	7.50%	53	0.00%
10	42.50%	7	0.63%
20	50.00%	56	99.37%
1		999	
7			

Hints and Guidelines

First, we'll read the weight of each load and we'll sum how many tons are being transported by a **van**, **truck**, and **train** respectively, and we'll additionally calculate the **total tons** of transported cargos. We'll calculate the **prices of each transport type** according to the total tons and the **total price**. In the end, we'll calculate and print the **total average price per ton** and **what part of the overall load is transported by each transport type**, in percentages.

We declare our variables, for example: **count_of_loads** – the count of cargos to be transported (we read them from the console), **sum_of_tons** – the sum of the overall load's weight,

microbus_tons, **truck_tons**, **train_tons** – variables, holding the sum of the weights transported respectively by a van, truck, and train.

We'll need a **for** loop from **0** to **count_of_loads - 1**, to go through all loads. For each load, we read its weight (in tons) from the console and assign it to our variable, for example, **tons**. To the sum of all loads (**sum_of_tons**), we add our current load's weight (**tons**). After we've read our current load's weight, we have to decide which transport vehicle will be used for it (van, truck or train). For this we'll need some **if-elif** checks:

- If the value of the variable **tons** is lower than 3, we increase the value of the variable **microbus_tons** by the value of **tons**:
- **microbus_tons += tons;**
- Else, if the value of **tons** is up to 11, we increase **truck_tons** by **tons**.
- If **tons** is higher than 11, we increase **train_tons** by **tons**.

Before we print our output, we have to calculate the percentage of tons transported by each vehicle and the average price per ton. For the average price, we'll declare another additional variable **total_price**, in which we'll sum the total price of all transported loads (with a van, truck, and train). The average price will be calculated by dividing **total_price** by **sum_of_tons**. You're left with calculating by yourself the percentage of tons transported by each vehicle and printing the output, adhering to the format as shown in the problem's description.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1054#5>.

Chapter 6.1. Nested Loops

In the current chapter, we will be looking at **nested loops** and how to use **for** loops to **draw** various **figures on the console**, that contain symbols and signs, ordered in rows and columns on the console. We will use **single** and **nested loops** (loops that stay in other loops), **calculations**, and **checks**, to print on the console simple and not so simple figures by specified sizes.

Problem: Rectangle of 10 x 10 Stars

Print on the console a rectangle made out of **10 x 10** asterisks.

Input	Output
(None)	<pre> ***** ***** ***** ***** ***** ***** ***** ***** ***** ***** </pre>

Hints and Guidelines

```

for i in range(10):
    print('*' * 10)

```

How does the example work? We initialize a **loop with a variable i**. The default value of the variable is **i = 0**. With each iteration of the loop, the variable increases by **1** while it is **less than 10**. This way the code in the body of the loop is executed **10 times** - from **0** to **9** included. In the body of the loop, we print a new line on the console **'*' * 10**, which creates a string of 10 asterisks.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#0>.

Problem: Rectangle of N x N Stars

Write a program that receives a positive number **n** and prints on the console a **rectangle** made out of **N x N** asterisks.

Input	Output	Input	Output	Input	Output
2	<pre>** **</pre>	3	<pre>*** *** ***</pre>	4	<pre>**** **** **** ****</pre>

Hints and Guidelines

The task is similar to the previous one:

```
n = int(input())
for i in range(n):
    print('*' * n)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#1>.

Nested Loops

A nested loop is a construction where the body of one loop (the outer one) stays inside another loop (the inner one). In each iteration of the outer loop, the whole inner loop is executed. This happens in the following way:

- When nested loops start executing, the outer loop starts first: initialization of its control variable is performed and after checking for the end of the loop, the code in its body is executed.
- After that, the inner loop is executed. The control variable's start position is initialized, a check for the end of the loop is made and the code in its body is executed.
- When the set value for the end of the loop is reached, the program goes back one step up and continues executing the previous outer loop. The control variable of the outer loop changes with one step, a check is made to see if the condition for the end of the loop is met and a new execution of the nested (inner) loop is started.
- This is repeated until the variable of the outer loop meets the condition to end the loop.

Here is an example that illustrates nested loops. The aim is again to print a rectangle made of N x N asterisk, in which for each row a loop iterates from 1 to N, and for each column a nested loop is executed from 1 to N:

```
n = int(input())
for row in range(1, n + 1):
    for col in range(1, n + 1):
        print("*", end="")
    print()
```

In **Python**, when the standard initial value of the variable in the loop (**i = 0**) does not work for us, we can change it with the above syntax. I.e. when we want the loop to start from **1** and rotate to **n** inclusive, we write: **for i in range (1, n + 1)**. The first value in parentheses indicates the beginning of the loop, and the second - the end of the loop, but not including, i.e. the loop ends before it is reached.

Let's look at the example above. After the initialization of the **first (outer) loop**, its **body**, which contains **the second (nested) loop**, starts to run. It prints one line **n** of asterisks. After the **internal loop completes** its execution in the first iteration of the external one, then **the external one will continue**, i.e. will print a blank line on the console. Then the **first loop will be updated** and the whole **second (nested) loop will be executed again**. The inner loop will be executed as many times as the body of the outer loop is executed, in this case, **n**** times.

Problem: Square of Stars

Print on the console a square made of **N x N** asterisks:

Input	Output	Input	Output	Input	Output
2	<pre>* * * *</pre>	3	<pre>* * * * * * * * *</pre>	4	<pre>* * * * * * * * * * * * * * * *</pre>

Hints and Guidelines

The problem is similar to the last one. The difference here is that we need to figure out how to add a whitespace after the asterisks so that there aren't any excess white spaces at the beginning and the end:

```
n = int(input())
for row in range(1, n + 1):
    print('*', end='')
    for col in range(1, n):
        print(' *', end='')
    print()
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#2>.

Problem: Triangle of Dollars

Write a program that receives an integer **n** and prints a **triangle made of dollars** of size **n**.

Input	Output	Input	Output
3	\$ \$ \$ \$ \$ \$	4	\$ \$ \$ \$ \$ \$ \$ \$ \$ \$

Hints and Guidelines

The problem is **similar** to those for drawing a **rectangle** and **square**. Once again, we will use **nested loops**, but there is a **catch** here. The difference is that **the number of columns** that we need to print depends on **the row**, on which we are and not on the input number **n**. From the example input and output data, we see that **the count of dollars depends** on which **row** we are on at the moment of the printing, i.e. 1 dollar means the first row, 3 dollars mean the third row, and so on. Let's see the following example in detail. We see that **the variable** of **the nested** loop is connected with the variable of **the outer** one. This way our program prints the desired triangle:

```
n = int(input())
for row in range(n):
    print('$', end='')
    for col in range(row):
        print(' $', end='')
    print()
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#3>.

Problem: Square Frame

Write a program that receives a positive integer **n** and draws on the console a **square frame** with a size of **N x N**.

Input	Output	Input	Output	Input	Output
4	+ - - + - - - - + - - +	5	+ - - - + - - - - - - - - - + - - - +	6	+ - - - - + - - - - - - - - - - - - - - - - + - - - - +

Hints and Guidelines

We can solve the problem in the following way:

- We read from the console the number `n`.
- We print **the upper part**: first a `+` sign, then `n-2` times `-` and in the end a `+` sign.
- We print **the middle part**: we print `n-2` rows, as we first print a `|` sign, then `n-2` times `-` and in the end again a `|` sign. We can do this with nested loops.
- We print **the lower part**: first a `+` sign, then `n-2` times `-` and in the end a `+` sign.

Here is an example implementation of the above idea with nested loops:

```
n = int(input())

# printing the top row: + - - - +
print('+', end='')
for i in range(n - 2):
    print('-', end='')
print(' +')

# printing the mid rows: | - - - |
for row in range(n - 2):
    print('|', end='')
    for col in range(n - 2):
        print('-', end='')
    print(' |')

# printing the bottom row: + - - - +
print('+', end='')
for i in range(n - 2):
    print('-', end='')
print(' +')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#4>.

Problem: Rhombus of Stars

Write a program that receives a positive integer `n` and prints a rhombus made of asterisks with size `N`.

Input	Output	Input	Output	Input	Output	Input	Output
1	*	2	<pre> * * * *</pre>	3	<pre> * * * * * * * * *</pre>	4	<pre> * * * * * * * * * * * * * * *</pre>

Hints and Guidelines

To solve this problem, we need to mentally **divide the rhombus** into **two parts** – the **upper** one, which **also** includes the middle row, and the **lower** one. For **the printing** of each part, we will use **two** separate loops, as we leave the reader to decide the dependency between **n** and the variables of the loops. For the first loop we can use the following guidelines:

- We print **n-row** whitespaces.
- We print *****.
- We print **row-1** times *****.

The **second** (lower) part will be printed **similarly**, which again we leave to the reader to do.

```
n = int(input())
```

```
for row in range(1, n + 1):
    for col in range(1, n - row + 1):
        print(' ', end='')
    print('*', end='')

    for col in range(1, row):
        print(' *', end='')
    print()
```

TODO: print the bottom side of the rhombus



In Python, the standard step of the **for** loop is positive and is equal to 1. If we want to change it, we must use a third parameter in the arguments of the loop: **for i in range (0, 100, 2)**. The third parameter, in this case, shows that the variable will increase from 0 to 99 inclusive, with step 2.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#5>.

Problem: Christmas Tree

Write a program that receives a number **n** ($1 \leq n \leq 100$) and prints a Christmas tree with a height of **N + 1**.

Hints and Guidelines

From the examples, we see that **the Christmas tree** can be **divided** into **three** logical parts. **The first** part is **the asterisks** and the **whitespaces** before and after them, **the middle** part is **|**, and **the last** part is again **asterisks**, but this time there are **whitespaces** only **before** them. The printing can be done with only **one loop** and the operation **string multiplication**, which we will use once for the asterisks and once for the whitespaces:


```
n = int(input())

for i in range(n + 1):
    stars = '*' * i
    spaces = ' ' * (n - i)
    print(spaces, end='')
    print(stars, end='')
    print(' | ', end='')
    print(stars, end='')
    print(spaces)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#6>.

Drawing More Complex Figures

Let's look at how to **draw figures** using **nested loops** with more complex logic, for which we need to think more before coding.

Problem: Sunglasses

Write a program that receives an integer **n** ($3 \leq n \leq 100$) and prints sunglasses with a size of **5*N x N** as found in the examples:

Input	Output	Input	Output
3	<pre>***** ***** */////* */////* ***** *****</pre>	4	<pre>***** ***** *////////* *////////* *////////* *////////* ***** *****</pre>
Input	Output		
5	<pre>***** *////////* *////////* *////////* *////////* *////////* *****</pre>		

Hints and Guidelines

From the examples, we can see that the sunglasses can be divided into **three parts** – upper, middle, and lower. Below is part of the code with which the problem can be solved. When drawing the upper and lower rows we need to print **2 * n** asterisks, **n** whitespaces, and **2 * n** asterisks:

```

n = int(input())

# printing the top part
print('*' * (2 * n), end='')
print(' ' * n, end='')
print('*' * (2 * n))

for i in range(n - 2):
    # TODO: print the middle part

# printing the bottom part
print('*' * (2 * n), end='')
print(' ' * n, end='')
print('*' * (2 * n))

```

When drawing the middle part, we need to check if the row is $(n-1) / 2 - 1$ because in the examples we can see that in this row we need to print vertical slashes instead of whitespaces:

```

for i in range(n - 2):
    # TODO: print */////////*

    if i == (n - 1) // 2 - 1:
        print('|' * n, end='')
    else:
        print(' ' * n, end='')

    # TODO: print */////////*
print()

```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#7>.

Problem: House

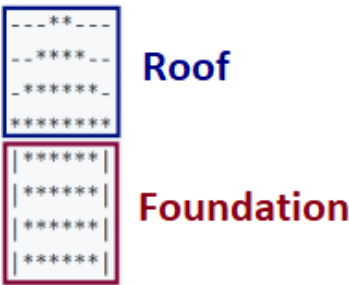
Write a program that receives a number n ($2 \leq n \leq 100$) and prints a house with size $N \times N$, just as in the examples:

Input	Output	Input	Output	Input	Output	Input	Output
2	<pre> ** </pre>	3	<pre> - * - *** * </pre>	4	<pre> - ** - ***** ** ** </pre>	5	<pre> - - * - - - *** - ***** *** *** </pre>

Hints and Guidelines

We understand from the description of the problem that the house is with a size of `n x n`. What we see from the example input and output is that:

- The house is divided into two parts: **roof** and **base**.



- When `n` is an even number, the point of the house is "dull".
- When `n` is odd, the **roof** has a sharp point and is one row larger than the **base**.

The Roof

- It comprises **asterisks** and **dashes**.
- At its highest part, there are one or two asterisks, depending on whether `n` is even or odd, as well as dashes.
- At its lowest part, there are many asterisks and little to no dashes.
- With each lower row, the **asterisks** increase by 2, and the **dashes** decrease also by 2.

The Base

- The height is `n` rows.
- It is made out of **asterisks** and **vertical slashes**.
- Each row comprises 2 **vertical slashes** – one in the beginning and one at the end of the row, and also **asterisks** between the vertical slashes with a string length of `n - 2`.

We read the input number `n` from the console and write its value in a variable:

```
n = int(input())
```



It is very important to check if the input data is correct! In these tasks, it is not a problem to directly convert the data from the console into type `int`, because it is said that we will be given valid integers. If you are making more

complex programs, it is a good practice to check the data. What will happen if instead of a number the user inputs the character "A"?

To draw **the roof**, we write down how many **asterisks** we start with a variable called **stars**:

- If **n** is an **odd** number, there will be 1 star.
- If it is **even**, there will be 2.

```
stars = 1
if n % 2 == 0:
    stars += 1
```

Calculate the length of **the roof**. It equals half of **n**. Write the result in the variable **roof_length**:

```
roof_length = math.ceil(n / 2)
```

Note: To use **math.ceil()**, which rounds to the larger integer, regardless of the fractional part, it is necessary to import the library **math**. This is done with the command **import math**. It is recommended to write **import math** (as well as all other imports) at the beginning of the file.

It is important to note that when **n** is an odd number, the length of the roof is one row more than that of the **base**.

In **Python**, when two integer types are divisible and there is a remainder, the result will be a number with a remainder. If we want to perform a pure integer division without a remainder, it is necessary to use the operator **//**.

Example:

```
result1 = 3 / 2    # result 1.5
result2 = 3 // 2   # result 1
```

If we want to round up to the next largest integer number, we need to use the method **math.ceil(...): result = math.ceil(3 / 2)**.

After we have calculated the length of the roof, we make a loop from 0 to **roof_length**. On each iteration we will:

- Calculate the number of **dashes** we need to draw. The number will be equal to **(n - stars) / 2**. We store it in variable **padding**.

```
padding = (n - stars) // 2
```

- We print on the console: "dashes" (**padding** times) + "asterisks" (**stars** times) + "dashes" (**padding** times):

```
padding = (n - stars) // 2
line = '-' * padding \
      + '*' * stars \
      + '-' * padding
print(line)
```

- Before the iteration is over, we add 2 to **stars** (the number of the asterisks).

```
stars += 2
```

After we have finished with the **roof**, it is time for **the base**. It is easier to print:

- We start with a loop from 0 to **n** (exclusive).
- We print on the console: `| + * (n - 2 times) + |`.

```
for i in range(n // 2):
    line = '|' + '*' * (n - 2) + '|'
    print(line)
```

If we have written everything correctly, our problem should be solved.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#8>.

Problem: Diamond

Write a program that receives an integer **n** ($1 \leq n \leq 100$) and prints a diamond with size **N**, as in the following examples:

Input	Output	Input	Output	Input	Output	Input	Output	Input	Output
1	*	2	**	3	-*- *-* -*-	4	-**- *-** -**-	5	--*-- -*-*- *---* -**- --*--

Hints and Guidelines

What we know from the problem's description is that the diamond is **n x n** in size. From the example input and output we can conclude that all rows contain exactly **n** symbols, and all the rows, except for the top and bottom ones, have **2 asterisks**. We can mentally divide the diamond into 2 parts:

- **Upper** part. It starts from the upper tip down to the middle.
- **Lower** part. It starts from the row below the middle one and goes down to the lower tip (inclusive).

Upper Part

- If **n** is an **odd** number, it starts with **1 asterisk**.
- If **n** is an **even** number, it starts with **2 asterisks**.
- With each row down, the asterisks get further away from each other.
- The space between, before, and after **the asterisks** is filled up with **dashes**.

Lower Part

- With each row down, the asterisks get closer to each other. This means that space (**the dashes**) between them is getting smaller and space (**the dashes**) on the left and the right is getting larger.
- The bottom-most part has 1 or 2 **asterisks**, depending on whether **n** is an even or odd number.

Upper and Lower Parts of The Diamond

- On each row, except the middle one, the asterisks are surrounded by inner and outer **dashes**.
- On each row, there is space between the two **asterisks**, except on the first and the last row (sometimes **the asterisk is 1**).

We read the value of **n** from the console, converting it to type **int**:

```
n = int(input())
```

We start drawing the upper part of the diamond. The first thing we need to do is to calculate the number of the outer **dashes left_right** (the dashes on the outer side of **the asterisks**). It is equal to **(n - 1) / 2**, rounded down:

```
left_right = (n - 1) // 2
```

After we have calculated **left_right**, we start drawing **the upper part** of the diamond. We can start by running a **loop** from **(n + 1) // 2** (i.e. rounded down).

At each iteration of the loop the following steps must be taken:

- We print on the console the left **dashes** (with length **left_right**) and right after them the first **asterisk**:

```
print('-', * leftRight, end='')  
print('*', end='')
```

- We will calculate the distance between the two **asterisks**. We can do this by subtracting from **n** the number of the outer **dashes**, and the number 2 (the number of **the asterisks**, i.e. the diamond's outline). We need to store the result of the subtraction in a variable **mid**.

```
mid = n - 2 * left_right - 2
```

- If the **mid** is lower than 0, we know that on the row there should be only 1 star. If it is higher or equal to 0 then we have to print **dashes** with length **mid** and one **asterisk** after them.
- We print on the console the right outer **dashes** with length **left_right**.

```
print('-' * left_right)
```

- At the end of the loop, we decrease **left_right** by 1 (the **asterisks** are moving away from each other).

We are ready with the upper part.

Printing the lower part is very similar to that of the upper part. The difference is that instead of decreasing **left_right** with 1 at the end of the loop, we will increase it with 1 at the beginning of the loop. Also, the loop will iterate from 0 to **(n - 1) // 2**:

```
for i in range((n - 1) // 2):
    left_right += 1

    print('-' * left_right, end='')
    print('*', end='')

    mid = n - 2 * left_right - 2

    if mid >= 0:
        print('-' * mid, end='')
        print('*', end='')

    print('-' * left_right)
```

Repeating a code is considered **bad practice** because the code becomes very hard to maintain. Let's imagine that we have a piece of code (e.g. the logic for drawing a row from the diamond) in a few more places and we decide to change it. To do this, we will have to go through all the places and change it everywhere. Now let's imagine that you need to reuse a piece of code not 1, 2, or 3 times but tens of times. A way to overcome this problem is to use **functions**. You can search for more information about them on the Internet or view [Chapter “10” \(Functions\)](#).

If we have written all correctly, then the problem is solved.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1055#9>.

What Have We Learned from This Chapter?

We learned about one of the ways to create strings:

```
print_me = '*' * 5
```

We learned how to draw figures using nested **for** loops:

```
for row in range(5):
    print('*', end='')

    for col in range(4):
        print(' *', end='')

    print()
```

Lab: Drawing Figures in a Web Environment

Now that we got used to the **nested loops** and the way to use them to draw figures on the console, we can get into something even more interesting: we can see how loops can be used to **draw in a Web environment**. We will make a web application that visualizes a number rating (a number from 0 to 100) with stars. This kind of visualization is common in e-commerce sites, reviews of products, event rating, rating of apps, and others.

Don't worry if you don't understand all of the code, how exactly it is written and how the project works. It is normal, now we are learning to write code and we are a long way from the web development technologies. If you are struggling to write your project by following the steps, ask for help in the **SoftUni's Reddit Community**:

<https://www.reddit.com/r/softuni/>.

Problem: Ratings – Visualization in a Web Environment

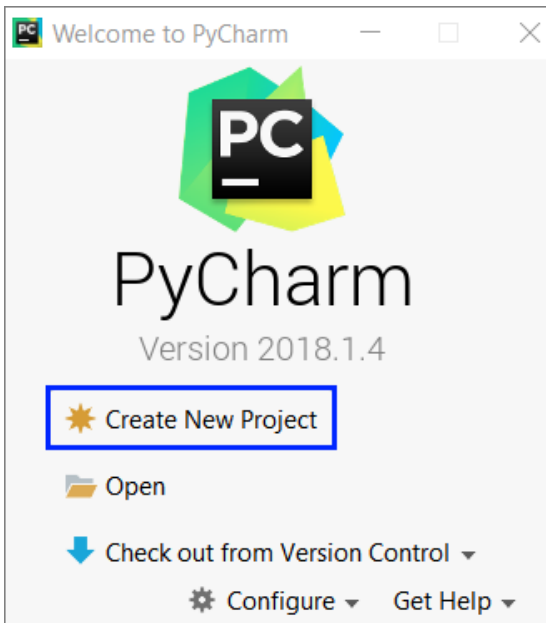
Develop a web application for rating visualization (number from 0 to 100). Draw from 1 to 10 stars (with halves). Stars to be generated with a **for** cycle.

Rating

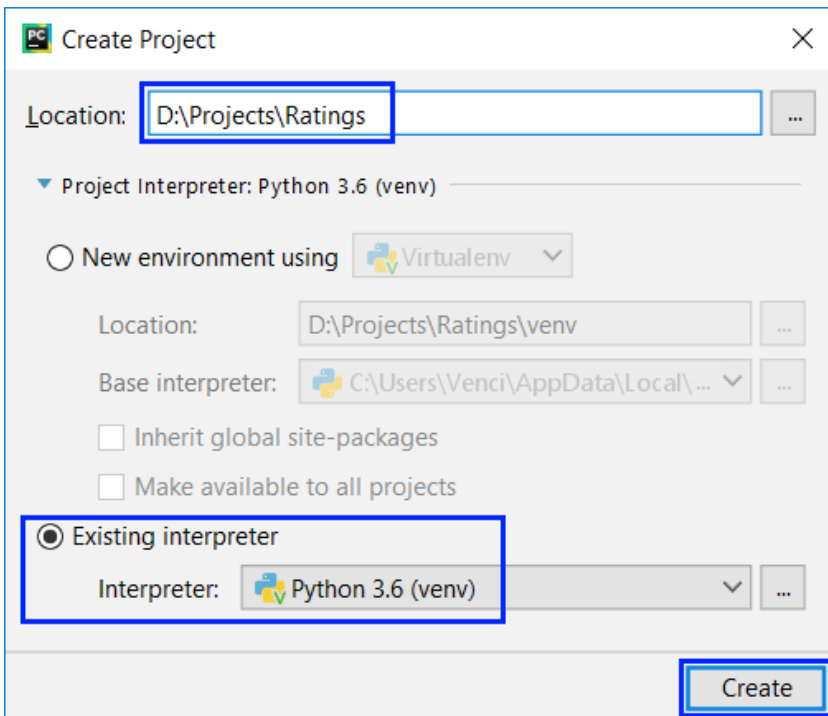


Hints and Guidelines

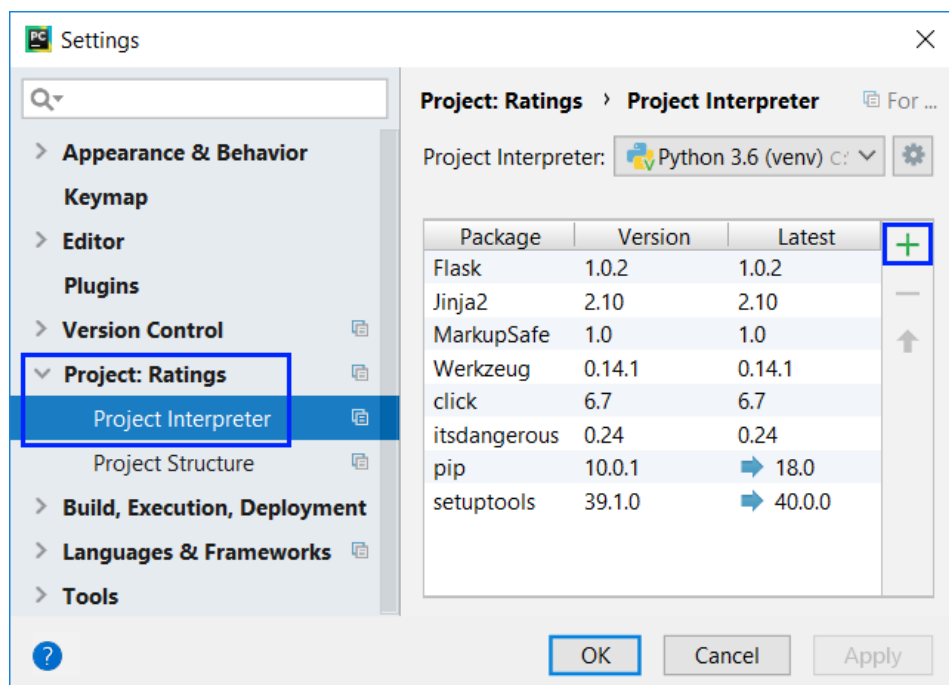
We start by creating a new project in **PyCharm** from **[File] -> [New Project]** (or from the start window):



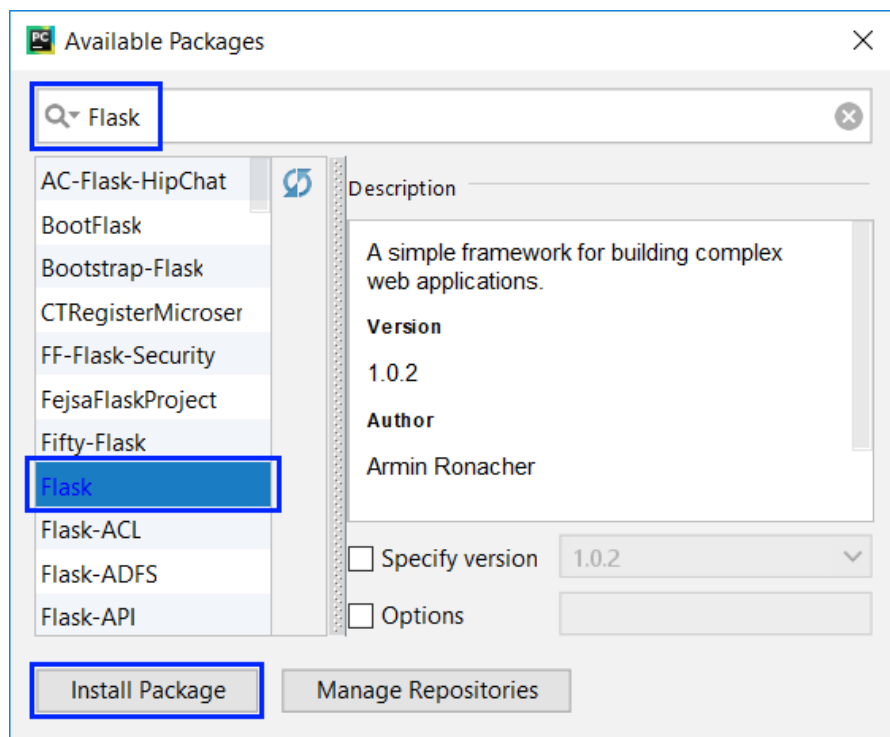
We give a meaningful name to the project, for example "Ratings". We choose the type of the current **Python interpreter**. Let this be the default:



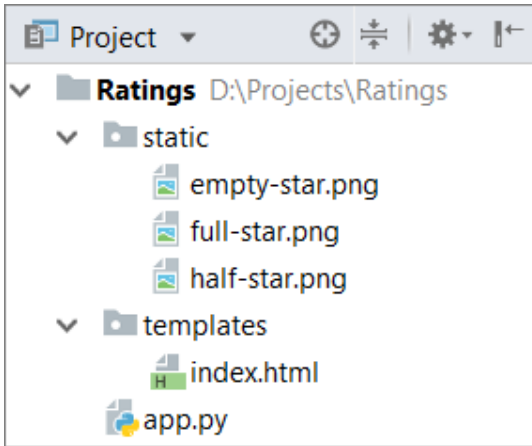
We will again use the Flask library, which is used to create web applications. Before we can start coding, we need to install Flask. Let's recall how to do this. We go to the settings of PyCharm [File] -> [Settings] -> [Project: Ratings] -> [Project Interpreter]. There, we press the **+** button:



Look for **Flask** in the window that appears and click the [Install Package] button:



We are now adding the **structure** of the project (the assignment files for this project can be downloaded from [here](#)). Copy them from Windows Explorer and paste them in the **Ratings** project folder with Copy/Paste:



For everything to work, we need to add the code. First, we go to the file **index.html** (from the templates folder) and look for the **TODO** sections. In their place we enter the following code:

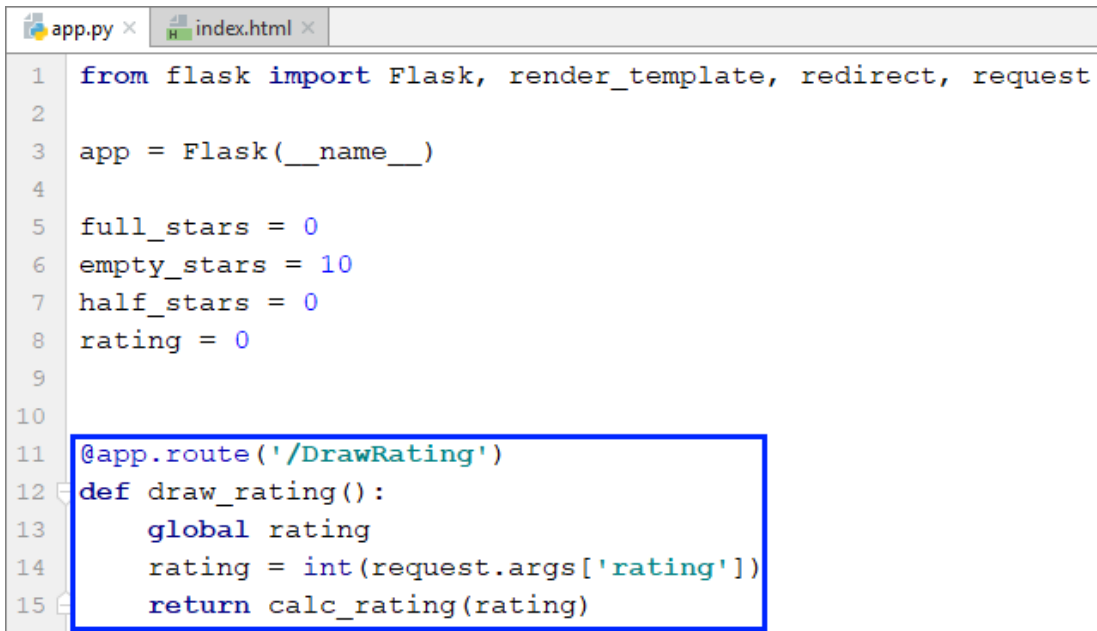
```
<h1>Rating</h1>
<form action="/DrawRating" method="GET">
    <input type="number" min="0"
        max="100" name="rating" value="{{rating}}"/>
    <input type="submit" value="Rate"/>
</form>
<br />
{% for star in range(full_stars) %}
    
{% endfor %}

{% for star in range(half_stars) %}
    
{% endfor %}

{% for star in range(empty_stars) %}
    
{% endfor %}
```

The above code creates a web form **<form>** with one field **" rating "** for entering a number in the interval **[0... 100]** and a button **[Rate]** to send the data from the form to the server. Then, draw with three separate **for** loops the corresponding number of stars - filled, half-empty and empty.

The action that will process the data is called **/DrawRatings**, which means the function **draw_ratings ()** in the file **app.py**:



```

1 from flask import Flask, render_template, redirect, request
2
3 app = Flask(__name__)
4
5 full_stars = 0
6 empty_stars = 10
7 half_stars = 0
8 rating = 0
9
10
11 @app.route('/DrawRating')
12 def draw_rating():
13     global rating
14     rating = int(request.args['rating'])
15     return calc_rating(rating)

```

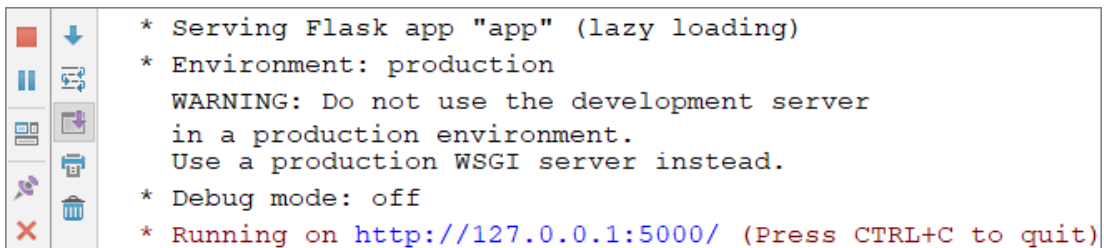
The code from the function **draw_ratings ()** takes the entered number **rating** from the form and passes it to the function **calc_rating (...)**. The **calc_rating (...)** function computes and calculates the number of **full stars**, the number of **empty stars**, and the number of **halves of stars**, then reload the page, but with new ones submitted values of the variables for the stars. We implement it as follows:

```

def calc_rating(rating):
    global full_stars
    full_stars = rating * 10 // 100
    global empty_stars
    empty_stars = (100 - rating) * 10 // 100
    global half_stars
    half_stars = 10 - full_stars - empty_stars
    return redirect('/')

```

We start the project with [Ctrl+Shift+F10] (or with [Right button] -> [Run 'app']) and wait for it to load:

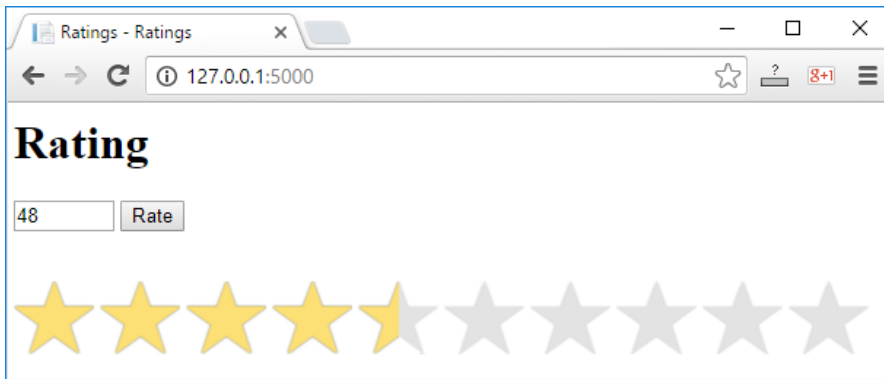


```

* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: Do not use the development server
  in a production environment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

```

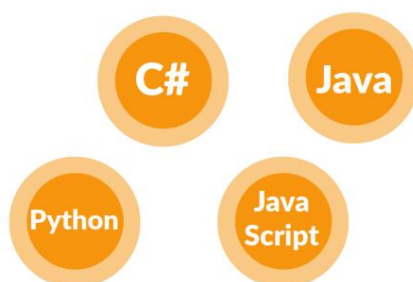
We go to the specified address and enjoy the finished project:



If you have problems with the sample project above, you can ask questions in the **SoftUni's** Reddit Community: <https://www.reddit.com/r/softuni/>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 6.2. Nested Loops – Exam Problems

In the previous chapter, we introduced **nested loops** and how to use them for **drawing** various kinds of **figures on the console**. We've learned how to print figures with different sizes, establishing suitable logic construction by using **single and nested for** loops in combination with different calculations and program logic:

```
for r in range(5):
    print("*", end="")
    for c in range(5):
        print(" *", end="")
    print("")
```

We also learned the **operator ***, which lets you for defined by us a **number** of times, a **given string** to be printed:

```
print_me = ('abc' * 2)
```

Now let's solve some Exam Problems to consolidate what we have learned and to develop our algorithmic thinking.

Problem: Draw Fort

Write a program, that reads from the console an **integer n** and draws a **fortress** with a width of $2 * n$ columns and height of **n rows**, as in the below-given examples. The left and the right inner columns have a width of $n / 2$.

Input Data

The program input consists one element (argument) - **integer n** within the range [3 ... 1000].

Output Data

Print on the console **n** text lines, depicting the **fortress**, just as in the examples below.

Sample Input and Output

Input	Output	Input	Output	Input	Output
3		4		5	

Hints and Guidelines

By the set task condition, we can see that the **input data** will contain only one **integer** within the range [3 ... 1000]. Therefore, we will convert the input value into **int** type:

```
n = int(input())
```

After we've declared and initialized the input data, we have to divide the **fortress** into three parts:

- roof
- body
- base

We can see from the examples, that the **roof** consists of **two towers** and a **middle part**. Each tower has a beginning **/**, middle part **^** and an end ****.

By the set task condition the left and the right inner columns have a width of **n / 2**, therefore we can save this value as a separate **variable**, keeping in mind, that if we receive an **odd number** as input, the result of dividing by two will be a number with a whole and fractional part. In this case, we need **only the whole part** (in the set task condition we can see, that when the input is equal to **3** the count of **^** in the inner part column is equal to **1**, and the input of **5** it is **3**), we can separate it with the function **math.trunc(...)** and to save only its value in our new variable:

```
col_size = math.trunc(n / 2)
```



It's always a good practice, whenever we have an expression with the value we intend to use **more than once**, to keep it in a variable. In this way, on the one hand, our code will be **easier to read**, and on the other hand, it will be **easier to correct possible errors**, as we will not have to look for each use of the expression separately.

We also declare a second **variable**, which will keep the **value** of the part **between the two towers**. By the set task condition, we know that the total width of the fortress is **n * 2**. In addition, we have two towers with one slash for a start and one slash for an end (a total of 4 characters), and width of **col_size**. Therefore, to get the number of characters in the middle part, we have to subtract the size of the towers from the width of the entire fortress:
 $2 * n - 2 * col_size - 4$.

```
mid_size = (2 * n - 2 * col_size - 4)
```

To print the **roof** part, on the console we will use the **format(...)** method in combination with the operator *****, which joins a given string **n** number of times:

```
print("/{0}\\{1}\\{0}\\".format("^" * col_size,
                                "_" * mid_size))
```




`\` is a special symbol in Python and using it solely in the method `print(...)`, the console will not print it out, so with `\\` we indicate on the console, that we want to print out exactly this character, without interpreting it as a special character (**character escaping it**).

The **fortress body** consists of beginning `|`, middle part (**white spaces**), and an end `|`. The middle part of white spaces has a width of $2 * n - 2$. The number of **rows** for the walls can be determined from the given examples: $n - 3$:

```
for row in range(n - 3):
    print("|{0}|".format(" " * (2 * n - 2)))
```

To draw a penultimate row, which is part of the base, we need to print a beginning `|`, middle part (**white space**)(**white space**), and an end `|`. To do this, we can use already declared variables `col_size` and `mid_size` because as we see from the examples they are equal to the number of `_` in the roof:

```
print("|{0}{1}{0}|".format(" " * (col_size + 1),
                           "_" * mid_size))
```

We add to the value of **white spaces + 1** because in the examples we have **one** white space more.

The structure of the **fortress base** is the same as the one in the **roof**. It includes **two towers** and a **middle part**. Each **tower** begins with `\`, followed by a middle part `_`, and an end `/`.

```
print("\\{0}/{1}\\{0}/".format("_" * mid_size,
                              "_" * mid_size))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1056#0>.

Problem: Butterfly

Write a program, that takes an integer n from the console and draws **butterfly** with a width of $2 * n - 1$ columns and height of $2 * (n - 2) + 1$ rows as in the examples below. The left and the right part have a width of $n - 1$.

Input Data

The input consists of one element (argument) - integer n in the range $[3 \dots 1000]$.

Output Data

Print on the console $2 * (n - 2) + 1$ text rows, representing the **butterfly**, exactly as shown in the examples.

Sample Input and Output

Input	Output	Input	Output	Input	Output
3	<pre>* \ /* @ */ *</pre>	5	<pre>*** \ /*** --- \ /--- *** \ /*** @ *** / *** --- / \--- *** / ***</pre>	7	<pre>***** \ /***** ----- \ /----- ***** \ /***** ----- \ /----- ***** \ /***** @ ***** / ***** ----- / \----- ***** / ***** ----- / \----- ***** / *****</pre>

Hints and Guidelines

Similar to the previous task, we can see from the condition, that the **input data** will consist of only one **integer** in the range [3 ... 1000]. That's why we will convert the input value into **int** type:

```
n = Integer.parseInt(sc.next());
```

We can divide the figure into 3 parts - **upper wing**, **body**, and **lower wing**. To draw the upper wing of the butterfly, we have to divide it into three parts - a beginning with *****, a middle part with **\ /**, and an end with *****. After looking at the examples, we can say that the upper wing of the butterfly is with a size of **n - 2**:

```
half_row_size = (n - 2) / 2;
```

To draw the upper wing we make a loop repeated **half_row_size** number of times:

```
for (int i = 0; i < half_row_size; i++)
```

We can see in the examples, that on an **even** row we have a beginning *****, middle part **\ /** and an end *****, on the other hand on an **odd** row we have a beginning **-**, middle part **\ /** and an end **-**. Therefore, at each iteration of the loop, we have to do an **if-else** check to see whether the row that we print is even or odd. From the examples given in the set condition, we can see that the number of star characters and dashes on each row is equal to **n - 2**, i. e. we can use again the variable **half_row_size** to print them:

Input	Output
	..._____//... _____//.... _____//.....

Hints and Guidelines

As in the previous examples, the input data will be on one line, which will contain one int in the range [3 ... 1000]:

```
n = 1000
```

We can divide the figure into 3 parts - upper, middle, and lower. The upper part consists of two subparts - a starting line and lines in which the sign widens. The starting line consists of beginning ., middle part _ and an end .. After looking at the examples, we can see that the beginning has a size of n + 1 and it's better to keep this value as a separate variable. We also have to create a second variable, in which we will keep the value of the first-row middle part which has a size of 2 * n + 1:

```
dots = 1001
underscores = 2001
```

Once we have declared and initialized the two variables, we can print the first row on the console:

```
print("{0}{1}{0}".format(dots, underscores))
```

To draw the rows in which the sign is getting "wider", we have to create a loop, that iterates n number of times. The row structure consists of a beginning ., // + middle part _ + \\ and an end .. To reuse the already created variables, we have to decrease dots by 1 and underscores by 2, because we've already printed the first row, and the dots and underscores in the top part of the figure are decreasing on each row:

```
underscores -= 2
dots -= 1
```

At each subsequent iteration the beginning and the end decrease by 1, and the middle part increases by 2:

```
for i in range(n):
    print("{0}://{1}\\\\\\\\{0}".format(dots, underscores))
    underscores += 2
    dots += 1
```

The middle part of the figure begins with `// + _`, middle part **STOP!** and an end `_ + \\`. The count of the underscores `_` is $(\text{underscores} - 5) / 2$:

```
print("//{0}STOP!{0}\\\\\\"
      .format((underscores - 5) / 2)))
```

The lower part of the figure, in which the width of the sign **decreases**, can be done by creating a **loop**, that iterates **n** number of times. The structure of a row should have a beginning `. + \\`, middle part `_` and an end `// + .`. The number of the **dots** in the first loop iteration has to be 0 and each subsequent has to **increase** by one. Therefore we can say that the **dots in the lower part of the figure** are equal to **i**. To ensure proper operation of our program, on each **loop** iteration, we have to **decrease** the number of `_` by 2:

```
for i in range(n):
    print("{0}\\\\\\\\{1}//{0}".format("." * i,
                                   "_" * underscores))
    underscores -= 2
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1056#2>.

Problem: Arrow

Write a program that receives from the console an odd integer **n** and draws a vertical arrow with size as in the examples below.

Input Data

The input is an odd integer **n** (argument) within the range [3 ... 79].

Output Data

Print on the console a vertical arrow, in which **"#"** (hash sign) marks the outline of the arrow, and **"."** - the rest.

Sample Input and Output

Input	Output	Input	Output
3	<pre> .###. .#.#. ##.## .#.#. ..#..</pre>	5	<pre> ..#####.. ..#...#.. ..#...#.. ..#...#.. ###...### .#.....# ..#...#.. ...#.#...#....</pre>

Hints and Guidelines

From the explanation we see that the input data will be read from one input line only, which will contain an integer within the range [3 ... 1000]. That's why we will convert the input value into **int** type:

```
n = int(input())
```

We can divide the figure into 3 parts - upper, middle, and lower. The upper part consists of two subparts - the first row and the body of the arrow. We can see from the examples, that the count of the outer dots in the first row and the body of the arrow are equal to $(n - 1) / 2$. We can keep this value in a variable **outer_dots**:

```
outer_dots = int((n - 1) / 2)
```

The count of the inner dots in the body of the arrow is equal to $(n - 2)$. We have to create the variable **inner_dots**, which will keep this value:

```
inner_dots = n - 2
```

We can see from the examples the structure of the first row. We can use the declared and initialize by us variables **outer_dots** and **n**, to print the first row:

```
print("{0}{1}{0}".format("." * outer_dots,
                          "0" * n))
```

To draw the body of the arrow, we have to create a loop, which iterates $n - 2$ number of times:

```
for i in range(n - 2):
    print("{0}#{1}#{0}".format("." * outer_dots,
                                "0" * inner_dots))
```

The middle part of the figure is made of a beginning #, middle part . and an end #. The count of # is equal to **outer_dots** and that is why we will use the same variable:

```
print("{0}{1}{0}".format("#" * outer_dots + "." * inner_dots + "#",
                          "0" * n))
```

To draw the lower part of the arrow, we have to assign new values of the two variables **outer_dots** and **inner_dots**.

```
outer_dots = 1
inner_dots = 2 * n - 5
```

On each loop iteration **outer_dots** increase by 1, and **inner_dots** decrease by 2. We can notice, that on the penultimate row the **inner_dots** value will be 1 and on each subsequent loop iteration will be a negative number. Since the operator ***** cannot concatenate symbol 0 or a negative number of times in a string, it will print nothing on the console. To avoid that we can print the last row of the figure separately. The height of the lower part of the arrow

is $n - 1$, therefore the loop, that will print all the rows, except the last one, have to iterate $n - 2$ number of times:

```
for i in range(n - 2):
    print(
        outer_dots + '#' + inner_dots
        + '#' + inner_dots + outer_dots
    )
    outer_dots += 1
    inner_dots += 2
```

The last row of our figure is made of a beginning `.`, middle part `#` and an end `.`. The count of `.` is equal to **outer_dots**:

```
print("{0}#{0}".format("." * outer_dots))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1056#3>.

Problem: Axe

Write a program, that receives an integer n and draws an axe with size as in the example below. The width of the axe is $5 * n$ columns.

Input Data

The input consists one element (argument) - integer n within range $[2..42]$.

Output Data

Print on the console **axe**, as in the examples.

Sample Input and Output

Input	Output	Input	Output
2	<pre> -----**----- -----*_*----- -----*_*----- *****_*_*----- -----***-----</pre>	5	<pre> -----**----- -----*_*----- -----*_*----- -----*_*----- -----*_*----- *****_*_*----- *****_*_*----- *****_*_*----- -----*_*----- -----*****-----</pre>

Hints and Guidelines

From the explanation we see that the input data will be read from one input line only, which will contain an integer within the range $[2 \dots 42]$. That's why we will use **int** type. After that,

for the solution we need to calculate the **dashes in the left**, the **dashes in the middle**, the **dashes in the right**, and the **whole figure length**:

```
n = int(input())
width = 5 * n
left_dashes = 3 * n
middle_dashes = 0
right_dashes = width - left_dashes - middle_dashes - 2
```

Once we have declared and initialized the **variables**, we can draw the figure, starting with the **upper part**. We can see from the examples what the structure of the **first row** is and we can create a loop that iterates **n** number of times. At each loop iteration, the **middle dashes** are increasing by 1, and the **right dashes** are decreasing by 1:

```
for i in range(n):
    print("{0}*{1}*{2}".format("-" * left_dashes,
                                "-" * middle_dashes,
                                "-" * right_dashes))
```

```
middle_dashes += 1
right_dashes -= 1
```

Now we have to draw the **handle of the axe**. To be able to use the newly created **variables**, when drawing the handle of the axe, we have to decrease the **middle dashes** by 1 and increase **these on the right and left** by 1.

```
middle_dashes -= 1
right_dashes += 1
left_dashes += 1
```

The **handle of the axe** we can draw, by iterating a loop that repeats **n / 2** number of times. We can set this value into a separate **variable**, considering that when dividing **odd number** inputs by 2 the result will be a **real number** with a whole and fractional part. Since in this case, we need **only the whole part** (from the example condition we see that at input **5** the height of the axe handle is **2**), we can use the **math.trunc(...)** method, to save only its value in our new variable **axe_height**. We get the structure of the handle from the examples given:

```
axe_height = int(n / 2)

for i in range(axe_height):
    print("{0}{1}*{2}".format("*" * left_dashes,
                                "-" * middle_dashes,
                                "-" * right_dashes))
```

The **lower part** of the figure should be divided into two subparts - the **head of the axe** and the **last row of the figure**. We will print on the console the **head of the axe**, by making a self iterating loop **axe_height - 1** number of times. On each iteration, the **left dashes** and the **right dashes** decrease by 1, and the **middle dashes** increase by 2:


```
left_dashes -= 1
```

```

for i in range(1, height + 1):
    print("{}*{}*{}".format(" " * left_dashes,
                              "*" * middle_dashes,
                              "-" * right_dashes))

    middle_dashes += 2
    left_dashes -= 1
    right_dashes -= 1

```

For the last row of the figure, we can use again, the already declared variables **left_dashes**, **middle_dashes**, **right_dashes**.

```

print("{}*{}*{}".format(" " * left_dashes,
                        "*" * middle_dashes,
                        "-" * right_dashes))

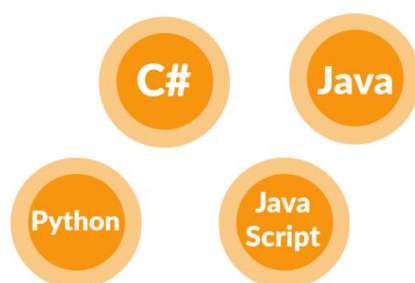
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1056#4>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 7.1. Complex Loops

Since we have learned what **for** loops are and their function in code, now is the time to take a look at **other loop types**, as well as some **more complex loop constructs**. These will expand our knowledge and help us solve more challenging problems. In particular, we will discuss how to use the following programming constructs:

- loops with step
- **while** loops
- **while** + **break** loops
- infinite loops

In the current chapter, we will also take a look at the **break** operator and **how** to use it to **interrupt** a loop. We will also learn how to process **errors during the execution** of the program, using the **try-except** statement.

Loops with a Step

In the [“Loops \(Repetitions\)”](#) chapter we learned how the **for** loop works and we already know when and for what purpose it is used. In the present chapter we will **take a look** at a particular and a very important **part of this structure** – its **step**.

Explanation

The **step** is **part** of the **‘range’** function, which specifies the **amount** used to **increment** or **decrement** the value of the **main** variable. The step is declared as the last argument in the **range** function.

By default, range in Python uses a step **value of 1** and is not added to the **range** function. If we want our step to be **different from 1**, when writing the **range** function, we add another number as the last parameter, which is our step. With a step of **10**, the loop would appear as below:

```
n = int(input())
for i in range(1, n + 1, 10):
    print(i)
```

Here is a series of sample problems, the solution of which will help us better understand the use of a **step** in a **for** loop.

Problem: Numbers 1...N with Step 3

Write a program that prints the numbers **from 1 to n** with a **step of 3**. For example, if **n = 100**, then the output would be: **1, 4, 7, 10, ..., 94, 97, 100**.

We can solve the problem using the following sequence of actions (algorithm):

- We read **n** from the console.
- We run a **for** loop from **1** to **n** (including **n**) with a step of **3**.

- In the **body of the loop**, we print the value of the current step.

```
n = int(input())
for i in range(1, n + 1, 3):
    print(i)
```

Testing in The Judge System

You can test your solution at the following link:

<https://judge.softuni.org/Contests/Practice/Index/1057#0>.

Problem: Numbers N...1

Write a program that prints the numbers **from n to 1 in reverse** (step -1). For example, if $n = 100$, then the output will be: 100, 99, 98, ..., 3, 2, 1.

We can solve the problem in the following manner:

- We read **n** from the console.
- We create a **for loop**, from **n** to 0.
- We define the step size: -1.
- In the **body of the loop**, we print the value of the current step.

```
n = int(input())
for i in range(n, 0, -1):
    print(i)
```

Testing in The Judge System

You can test your solution at the following link:

<https://judge.softuni.org/Contests/Practice/Index/1057#1>.

Problem: Powers of Two

In the following example, we will use the standard size 1 step.

Write a program that prints the numbers **from 1 to 2^n** (two to the power of n). For example, if $n = 10$, then the output would be 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024.

```
n = int(input())
num = 1
for i in range(0, n + 1):
    print(num)
    num = num * 2
```

Testing in The Judge System

You can test your solution at the following link:

<https://judge.softuni.org/Contests/Practice/Index/1057#2>.

Problem: Even Powers of 2

Print the **even** powers of 2 until 2^n : 2^0 , 2^2 , 2^4 , 2^8 , ..., 2^n . For example, if $n = 10$, then the output would be 1, 4, 16, 64, 256, 1024.

Here is how we can solve the problem:

- We declare a variable **num** that will hold the current number and we assign it the initial **value of 1**.
- For the **step** of the loop, we set a value of 2.
- In the **body of the loop**: we print the value of the current number and **increase the current number num 4 times** (as per the problem description).

This is the code behind the above idea:

```
n = int(input())
num = 1
for i in range(0, n + 1, 2):
    print(num)
    num = num * 2 * 2
```

Testing in The Judge System

You can test your solution at the following link:

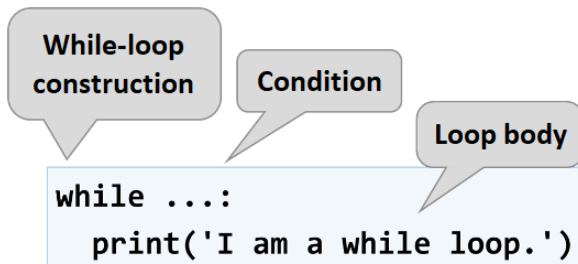
<https://judge.softuni.org/Contests/Practice/Index/1057#3>.

While Loop

The next type of loops that we will become familiar are called **while loops**. The special thing about them is that they repeat a command block, **while a condition is met**. Their structure is a bit different than that of the **for** loops, however, they boast a simpler syntax.

Explanation

In programming, the **while loop** is used when we want to **repeat** the execution of a specific logic block until **a specific condition is met**. Any expression that returns either **true** or **false** (a Boolean) can be used as a “condition”. When the **condition** becomes **invalid**, the **while loop is interrupted** and the program **proceeds** to execute the code after the loop. The **while loop** structure looks like this:



Here is a series of sample problems, the solutions of which will help us better understand the use of the **while** loop.

Problem: Sequence $2k+1$

Write a program that prints all numbers $\leq n$ in the series: 1, 3, 7, 15, 31, ..., assuming that each number is generated according to the following formula $\text{next_number} = \text{previous_number} * 2 + 1$.

Here is how we can solve the problem:

- We declare a variable **num** that will hold the current number and we assign it the initial value of 1.
- For loop condition, we use the current number $\leq n$.
- In the **body of the loop**: we print the value of the current number and increase the current number by using the formula above.

Here is a sample implementation of this idea:

```
n = int(input())
num = 1
while num <= n:
    print(num)
    num = 2 * num + 1
```

Testing in The Judge System

You can test your solution here:

<https://judge.softuni.org/Contests/Practice/Index/1057#4>.

Problem: Number in Range [1 ... 100]

Enter an integer in the range [1 ... 100]. If the entered number is **invalid**, enter **again**. In this case, an invalid number would be any number that is **outside** the given range.

To solve the problem, we can use the following algorithm:

- We declare a variable **num**, to which we assign the integer, received from the console.
- For a loop condition, we put a **True** expression, if the number is **not** in the range specified in the problem description.
- In the **body of the loop**: we print the message "Invalid number!" on the console, afterwards we assign a new value to **num** (the next line from the console input).
- Once we have validated the entered number, we print its value outside the body of the loop.

Here is a sample implementation of the algorithm using a **while** loop:

```

num = int(input())
while num < 1 or num > 100:
    print('Invalid number!')
    num = int(input())
print('The number is: %d' % num)

```

Testing in The Judge System

You can test your solution here:

<https://judge.softuni.org/Contests/Practice/Index/1057#5>.

Greatest Common Divisor (GCD)

Before proceeding to the next problem, we should become familiar with the definition of the greatest common divisor (GCD).

Definition of GCD: the greatest common divisor of two **natural** numbers **a** and **b** is the largest number that **divides both a and b** without remainder. For example:

a	b	GCD
24	16	8
67	18	1
12	24	12

a	b	GCD
15	9	3
10	10	10
100	88	4

The Euclidean Algorithm

In the next problem we will use one of the first published algorithms for finding the GCD – **Euclid's algorithm**:

Until we have a remainder of 0:

- We divide the greater number by the smaller one.
- We take the remainder of the division.

Euclid's algorithm **pseudo-code**:

```

while b ≠ 0
    oldB = b
    b = a % b
    a = oldB
print a

```

Problem: Greatest Common Divisor (GCD)

Enter the **integers a** and **b** and find **GCD(a, b)**.

We will solve the problem by implementing **Euclid's algorithm**:

- We declare variables **a** and **b**, to which we assign the **integer** values, passed by the console input.
- For loop condition, we use a **True** expression, if the number **b** is different than 0.
- In the **body of the loop** we follow the instructions from the pseudo-code:
 - We create a temporary variable to which we assign the **current** value of **b**.
 - We assign a new value to **b**, which is the remainder of the division of **a** and **b**.
 - On variable **a**, we assign the **previous** value of variable **b**.
- Once the loop is complete and we have found the GCD, we print it on the screen.

```
a = int(input())
b = int(input())

while b != 0:
    old_b = b
    b = a % b
    a = old_b
print('GCD = %d' % a)
```

Testing in The Judge System

You can test your solution here:

<https://judge.softuni.org/Contests/Practice/Index/1057#6>.

While True + break loop

The next type of loop construction that we will become familiar with while studying programming is the **while True + break** loop (for short **while + break** loop). Its idea is to repeat a block of code over and over again until **explicit termination of the loop**, usually after an **if** statement in the body of the loop. Here's what this loop looks like in Python code:

```
while True:
    print('I am a do...while loop.')
    if not ...:
        break
```

The above example is called “**inverted while loop**”, because the condition for exiting the loop is at the end, not at the beginning. In essence, the above construction is an “infinite loop” with a check, of a given condition, for an exit inside the body of the cycle.

In programming, the **break** operator unconditionally terminates a loop and proceeds to the first instruction after it. In the example above, a condition is checked at the end of the cycle, and if it is not true, the cycle is terminated.

The construct for the **while + break** loop in many other programming languages is implemented with the **do-while** construct, but there is no direct equivalent of **do-while** in Python. To do the same in Python, we can use an infinite loop (**while True**) and when an exit condition is reached, the loop is interrupted (with **break**).

The **while + break** construct provides more flexibility than **while** loops because it allows to break the loop at **any place** in the body of the loop (for example at the beginning, in the middle, or at the end), it even allows to have exit of the loop in several different places (with several **break** operators).

The structure of **while + break** loop is very similar to the classic **while** loop, but there is a significant difference: **while** is executed 0 or more times (according to the entry condition of the loop), whilst **while + break** will execute its body **at least once**. Why is this? In the **while True + break** loop structure, the **condition** is always checked **in the body of the loop**, whilst with the classic **while** loop the condition, which we check to exit the loop is always at the beginning, before its body.

Once we are familiar with the concept of a **while + break** loop with an exit condition, which is not required to be at the beginning, now we should proceed with the usual set of practical problems, with which we can master the new knowledge.

Problem: Factorial

For **n** number calculate $n! = 1 * 2 * 3 * \dots * n$. For example, if **n = 5**, then the result would be: $5! = 1 * 2 * 3 * 4 * 5 = 120$.

Here is how we can calculate factorial in more detail:

- We declare the variable **n**, to which we assign the integer value, received from the input of the console.
- We declare another variable – **fact**, with an initial value of 1. We will use it in the calculation and store the factorial value.
- For the loop condition we will use **n > 1**, because each time we perform the calculations in the body of the loop, we will decrease the value of **n** by 1.
- In the body of the loop:
 - We assign a new value to the **fact**, which value is the product of multiplying the current **fact** with the current **n**.
 - We decrement **n** with 1.
- Outside the body of the loop, we print the final factorial value.

```

n = int(input())
fact = 1
while True:
    fact = fact * n
    n -= 1
    if not n > 1:
        break

```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#7>.

Problem: Sum Digits

Sum up the digits of the integer **positive** number **n**. For example, if **n = 5634**, then the output would be: $5 + 6 + 3 + 4 = 18$.

We can use the following idea to solve the problem:

- We declare the variable **n**, to which we assign a value equal to the number received by the user.
- We create a second variable – **sum**, with an initial value of 0. We will use it for the calculation and storage of the result.
- As a loop condition, we will use **n > 0**, since, after each iteration of the loop, we will be removing the last digit from **n**.
- In the body of the loop:
 - We assign a new value to **sum**, which is the result of the sum of the current value of **sum** with the last digit of **n**.
 - We assign a new value to **n**, which is the result of removing the last digit of **n**.
- outside the body of the loop, we print the final value of the sum.

```

n = int(input())
sum = 0

while True:
    sum = sum + (n % 10)
    n = n // 10
    if not n > 0:
        break

print('Sum of digits: %d' % sum)

```



n % 10: returns the last digit of the number **n**.
n // 10: deletes the last digit of **n**.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#8>.

Infinite Loops and the `break` Operator

So far, we were introduced to various types of loops, learning what structures they have and how they are applied. Now, we need to understand what an **infinite loop** is, when it occurs, and how we can **break** it using the **break** operator.

Infinite Loop – Explanation

An **infinite loop** runs **infinitely** the code of its body. With the infinite **while** loops, the end check is a conditional expression that **always** returns **true**. Here is an example of an **infinite while** loop:

```
while True:
    print('Infinite loop')
```

The `break` Operator

We already know that the infinite loop executes a certain code infinitely, but what if we want at some point under a given condition to interrupt and exit the loop? The **break** operator comes in handy in this situation.



The operator **break** stops a loop's execution at the point it is called and the execution continues from the first line after the end of the loop. This means that the current iteration of the loop will not be completed accordingly and the rest of the code in the body of the loop will not be executed.

Problem: Check Prime

The next problem we are going to solve is to **check whether a given number is prime**, but before that, we should remember what prime numbers are.

Definition: An integer is considered **prime** if it is divisible only by itself and by 1. By definition, the prime numbers are positive and greater than 1. The smallest prime number is 2.

We can assume that an integer **n** is a prime number if **n > 1** and **n** is not divisible by a number between 2 and **n-1**.

The first few prime numbers are: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, ...

By contrast, **composite numbers** are integers, which can be obtained by multiplying several prime numbers.

Here are some examples of composite numbers:

- $10 = 2 * 5$
- $42 = 2 * 3 * 7$
- $143 = 11 * 13$

An algorithm to check whether a given number is **prime**: we check if $n > 1$ and if n is divisible by 2, 3, ..., $n-1$ without remainder.

- If it is divisible by any of the numbers, it is **composite**.
- If it is not divisible by any of the numbers, then it is **prime**.



We can optimize the algorithm by instead of checking until $n-1$, to check the divisors only until $\sqrt{n}$. Think of the reasons why this is so.

Problem: Enter Even Number

You are tasked to write a function that takes a single input n integer and checks if it is prime. This can be implemented by checking if n is divisible by any numbers in the range between 2 and $\sqrt{n}$.

The steps of the “prime checking algorithm” are given below in bigger detail:

- We declare the variable n , to which we assign the integer passed by the console.
- We create a **is_prime** boolean with an initial value of **True**. We assume that a number is prime until proven otherwise.
- We create a **for** loop, with the initial value set to 2, for a condition the **current value** $\leq \sqrt{n}$. The step is set to 1.
- In the **body of the loop**, we check if n , divided by the **current value** has a remainder. If there is **no remainder** from the division, then we change **is_prime** to **False** and exit the loop through the **break** operator.
- Depending on the value of **is_prime** we print whether the input number is prime (**True**) or composite (**False**).

Here is a sample implementation of the prime checking algorithm, described above:

```
import math

n = int(input())
prime = True

for i in range(2, int(math.sqrt(n) + 1)):
    if n % i == 0:
        prime = False
        break

if prime:
    print('Prime')
else:
    print('Not prime')
```

What remains is to add a **condition** that checks if the input number is greater than 1, because, by definition numbers such as 0, 1, -1, and -2 are not prime.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#9>.

The `break` Operator in an Infinite Loop

Write a function, which checks whether a given number `n` is even and if so – print it on the console. An even number can be divided by 2 without a remainder. If the number is invalid, we will print that the current number is not even and the user will need to input a new number.

Here is an idea for the solution:

- We declare a variable `n`, with an initial value of 0.
- We create an infinite `while` loop, with a condition set to `True`.
- In the body of the loop:
 - We take the integer value, passed to our function, and assign it to `n`.
 - If the number is even, we exit the loop by a `break`.
 - Otherwise, we print a message stating that the number is not even. Iterations continue until an even number is entered.
- We print the even number on the console.

Here is an example implementation of the idea:

```
while True:
    print('Enter even number: ', end='')
    n = int(input())
    if n % 2 == 0:
        break
    print('The number is not even.')
print('Even number entered: %d' % n)
```

Note: although the code above is correct, it will not work if the user enters text instead of a number, for example “Invalid number”. Then the parse of the text to a number will break and the program will display an **error message (exception)**. We will learn very soon how to deal with this problem and how to catch and handle exceptions using the **try-except** construct.

Testing in The Judge System

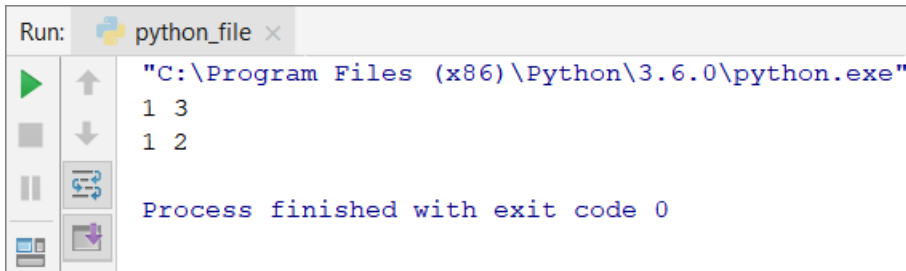
Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#10>.

Nested Loops and the `break` Operator

Now since we know what **nested loops** are and how the `break` operator works, it is time to figure out how they work together. To get a better idea, we should write a **function** step by step, that should make all possible combinations of **number pairs**. The first number in the

pair is increasing from 1 to 3, while the second one is decreasing from 3 to 1. Our solution must continue running until **$i + j$ is not equal to 2** (i.e. **$i = 1$** and **$j = 1$**).

The desired result is:

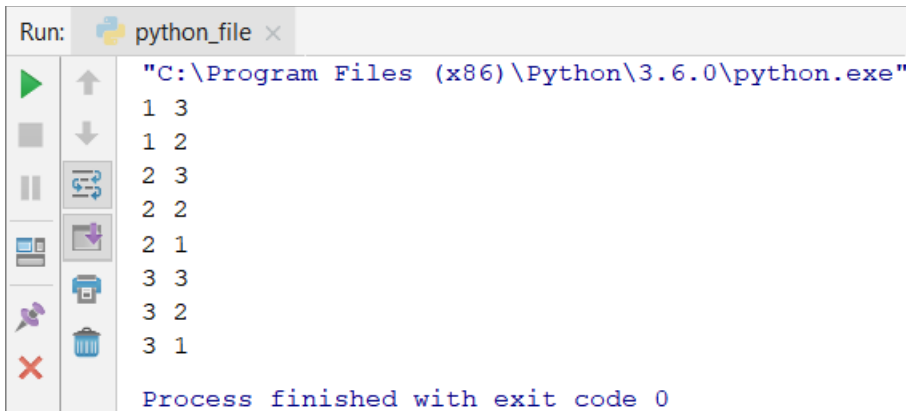


```
Run: python_file x
"C:\Program Files (x86)\Python\3.6.0\python.exe"
1 3
1 2
Process finished with exit code 0
```

Here is a **wrong implementation**, that looks right at first glance:

```
for i in range(1, 4):
    for j in range(3, 0, -1):
        if i + j == 2:
            break
        print('%d %d' % (i, j))
```

If we leave our function that way, our result will be as follows:



```
Run: python_file x
"C:\Program Files (x86)\Python\3.6.0\python.exe"
1 3
1 2
2 3
2 2
2 1
3 3
3 2
3 1
Process finished with exit code 0
```

Why is this so? As we can see, the result is **missing "1 1"**. When the function reaches the point when **$i = 1$** and **$j = 1$** , it enters the **if** check and executes the **break** operation. This way, it **exits the inner loop**, but then continues the execution of the **outer loop**. **i** increases, the function enters the inner loop and prints the result.



When we use the **break** operator in a **nested loop**, it interrupts **only** the execution of the inner loop.

What is the **correct solution**? One way to solve the problem is by declaring a **bool** variable, to keep track if the loop iteration has to continue. If we have to exit (leave all nested loops), we set the variable to **True** and exit the inner loop with a **break**, and in the next check, we exit the outer loop. Here is an example implementation of this idea:

```

has_to_end = False
for i in range(1, 4):
    if has_to_end == False:
        for j in range(3, 0, -1):
            if i + j == 2:
                has_to_end = True
                break
            print('%d %d' % (i, j))

```

In this manner, when $i + j = 2$, the program will set the variable `has_to_end = True` and will exit the inner loop. Upon the next iteration of the outer loop, via the `if` check, the function will not reach the inner loop and will interrupt its execution.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#11>.

Dealing with Exceptions: `try-except`

The last thing we'll look at in this chapter is how to “catch” errors (exceptions) using the `try-except` construction.

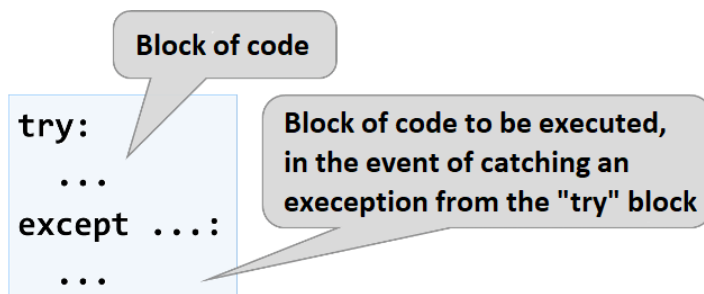
What is `try-except`?

The `try-except` construction is used for catching and handling exceptions (errors) during program execution.

In programming, **exceptions** are notifications of an event that disrupts the normal operation of a program. Such events **interrupt the execution** of the program, which starts looking for someone to handle the situation. If it is not found, the exception is printed on the console (i.e. the program stops). If found, **the exception is processed** and the program continues normal execution. In a bit, we will see exactly how this happens.

The `try-except` Construction

The `try-except` construction has different options, but for now, we will get acquainted only with the most basic of them



In the next task, we will see how to deal with a situation, in which the user enters an input other than a number (for example **string** instead of **int**), using **try-except**.

Problem: Break Sum

Write a program that checks that a number **n** is even and if so, print it on the screen. In case of an **invalid entered** number, a message should be displayed that the entered input is not a valid number and the input should continue.

Here's how we can solve the problem:

- We create an infinite **while** loop, as a condition we will set **True**.
- In the body of the loop:
 - We create a **try-except** construction.
 - In the block **try** we write the program logic for reading the user input, its parsing to a number, and the parity check.
 - At an **even number**, we print it and leave the loop (with **break**). The program has done its job and is coming to an end.
 - At an **odd number**, we print a message that an even number is required without leaving the loop (because we want it to be repeated).
 - If we **catch an exception** during the execution of the **try** block, we print a message for an invalid entered number (and the loop repeats, because we do not explicitly leave it).

Here is an example implementation of the described idea:

```
while True:
    try:
        print('Enter even number: ', end='')
        n = int(input())
        if n % 2 == 0:
            print('Even number entered: %d' % n)
            break
    except ValueError:
        print('Invalid number.')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#10>.

Now the solution must always work: whether we enter integers, invalid numbers (for example, numbers with too many digits), or text that does not contain numbers.

Problems with Loops

In this chapter, we got familiar with a few new types of loops that can perform repetitions with more complex programming logic. Let's solve a few practical problems using these new constructs.

Problem: Fibonacci

Fibonacci's numbers in mathematics form a sequence that looks like this: 1, 1, 2, 3, 5, 8, 13, 21, 34,

The formula to derive the next member of Fibonacci's sequence is:

```
F0 = 1
F1 = 1
Fn = Fn-1 + Fn-2
```

Sample Input and Output

Input (n)	Output	Comment
10	89	F(11) = F(9) + F(8)
5	8	F(5) = F(4) + F(3)
20	10946	F(20) = F(19) + F(18)
0	1	
1	1	

Enter an integer number **n** and calculate the **n-th Fibonacci number**.

Hints and Guidelines

An idea to solve the problem:

- We declare a variable **n**, which will hold the integer value received from the console input.
- We create the variables **f0** and **f1**, to which we assign the value to **1** since this is the start of Fibonacci's sequence.
- We create a **for** loop starting from 0 to the final value **n - 1**.
- In the **body of the loop**:
 - We create a **temporary** variable **f_next**, to which we assign the next number in the Fibonacci sequence.
 - To **f0** we assign the current value of **f1**.
 - To **f1** we assign the value of the temporary variable **f_next**.
- Out of the loop we print the **n-th** number of Fibonacci.

Example implementation:

```
n = int(input())
f0 = 1
f1 = 1
for i in range(0, n - 1):
    f_next = f0 + f1
    f0 = f1
    f1 = f_next
print(f1)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#12>.

Problem: Number Pyramid

Print the numbers 1 ... n in a pyramid as per the below example. On the first row, we print one number, at the second we print two, at the third, we print three, and so on, until the numbers are over. On the last line, we print as many numbers as we get until we get to n.

Sample Input and Output

Input	Output	Input	Output	Input	Output
7	1 2 3 4 5 6 7	5	1 2 3 4 5	10	1 2 3 4 5 6 7 8 9 10

Hints and Guidelines

We can solve the problem with **two nested loops** (by rows and columns) by printing in them and leaving when the last number is reached. Here is the idea, written in more detail:

- We declare a variable **n**, to which we assign the integer value, received from the console input.
- We declare a variable **num** with an initial value of 1. It will hold the count of printed numbers. At each iteration, we will **increment** it by **1** and will add it to the current row.
- We create an **outer for** loop, which will be responsible for the **rows** in the table. The loop variable will be named **row** and we assign it an initial value of 1. For condition, we set **n + 1**.
- In the body of the loop, we create an **inner for** loop, which will be responsible for the **columns** in the table. We name the variable **col** and assign it an initial value of 1. For condition we set **row + 1** (**row** = number of digits per line).

- In the body of the nested loop:
 - We check whether `col > 1`, if true we print space. If we do not make this check and instead print the space every time, each resulting line will start with a space.
 - We **print** the number `num` in the current cell of the table and **increment it by 1**.
 - We check if `num > n`. If the `num` is greater than `n`, we **break** the inner loop.
- We print an **empty line**, to move on to the next.
- Again, we check if `num > n`. If it is greater, we **interrupt** the execution of our program with a **break** operator.

Here is an example implementation:

```
n = int(input())
num = 1
for row in range(1, n + 1):
    for col in range(1, row + 1):
        print(num, end=' ')
        num += 1
        if col >= row:
            print()
        if num > n:
            break
    if num > n:
        break
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#13>.

Problem: Number Table

Print the numbers 1 ... n in a table as per the examples below:

Sample Input and Output

Input	Output	Input	Output
3	1 2 3 2 3 2 3 2 1	4	1 2 3 4 2 3 4 3 3 4 3 2 4 3 2 1

Hints and Guidelines

We can solve the problem by using **two nested loops** and some simple calculations in them:

- We take the size of the table from the integer value of the variable **n**, which is received from the console input.
- We create a **for** loop, that will be responsible for the rows of the table. We name the loop variable **row** and assign it an initial **value of 0**. For condition, we set **n**. The size of the step is 1.
- In **the body of the loop** we create a nested **for** loop, that will be responsible for the columns in the table. We name the loop variable **col** and assign it an initial **value of 0**. For condition, we set **n**. The size of the step is set to 1.
- In **the body of the nested loop**:
 - We create a variable **num**, to which we assign the result of **the current row + the current column + 1** (+1, is needed since we count from 0).
 - We check whether **num > n**. If **num** is **greater than n**, we assign **num** a new value equal to **two times n** – the current value of **num**. We do this in order not to exceed **n** in any of the cells of the table.
 - We print the number from the current table cell.
- We print an **empty line** in the outer loop, to move on to the next line.

Here is an example implementation:

```
n = int(input())
for row in range(n):
    for col in range(n):
        num = row + col + 1
        if num > n:
            num = 2 * n - num
        print('%d ' % num, end='')
    print()
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1057#14>.

What Have We Learned from This Chapter?

We can use **for** loops with a **step**:

```
for i in range(1, n + 1, 3):
    print(i)
```

The **while** loop is repeated while a condition is **True**:

```
num = 1
```

```
while num <= n:
    print(num)
    num += 1
```

If we have to **interrupt** the loop execution, we use the **break** operator:

```
n = 0
while True:
    n = int(input())
    if n % 2 == 0:
        break # even number -> exit from the loop
    print("The number is not even.")
print("Even number entered: {}".format(n))
```

We already know how to catch **errors** during the execution of our program:

```
try:
    print("Enter even number: ", end="")
    n = int(input())
except ValueError:
    print("Invalid number.")
# If int(...) gets an error, it will complete the exception block of
code
```

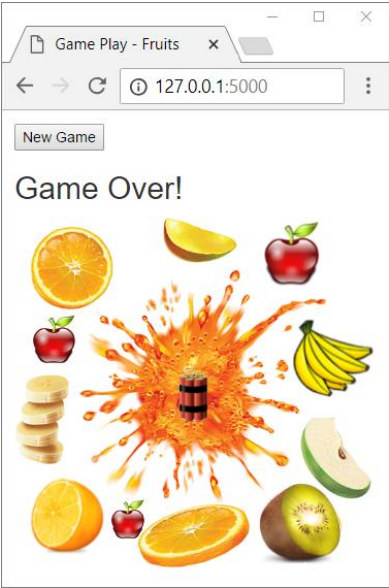
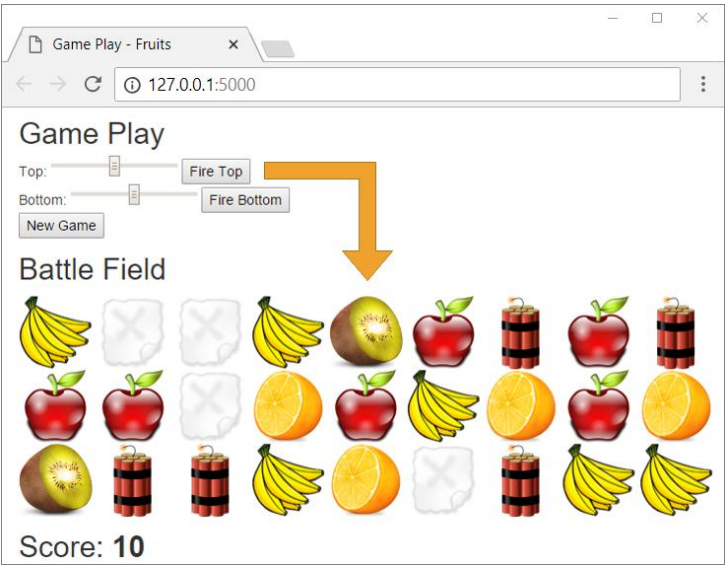
Problem: Web Applications with More Complex Loops

Now we know how to repeat a group of actions using **loops**. Let's do something interesting: a **web-based game**. Yes, a real game, with graphics and game logic. Let's have fun. It will be complicated, but if you do not understand exactly how it works, that is no problem. We are still just entering programming. There is time, you will advance with this technology. For now, follow the steps.

Problem: Web Game "Shoot the Fruits!"

Condition: Develop a **Flask web application** – a game in which the player **shoots at fruits**, arranged in a table. Successfully hit fruits disappear, and the player receives points for each fruit that was hit. The shooting is performed on columns, from top to bottom or vice-versa, and the location of the hit (the column under fire) is set by a scroll bar. Due to the inaccuracy of the scroller, the player is not quite sure which column to shoot at. So with each shot, there is a chance not to hit and this makes the game more interesting (similar to the slingshot in Angry Birds).

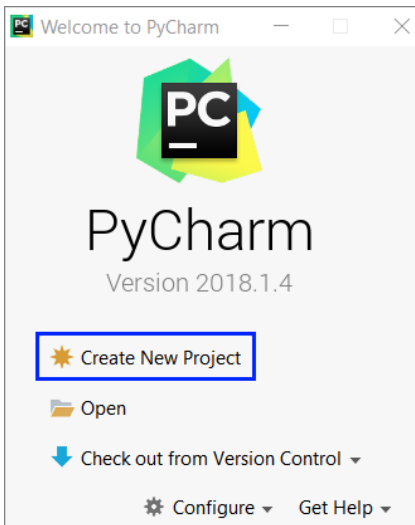
Our game should look like this:



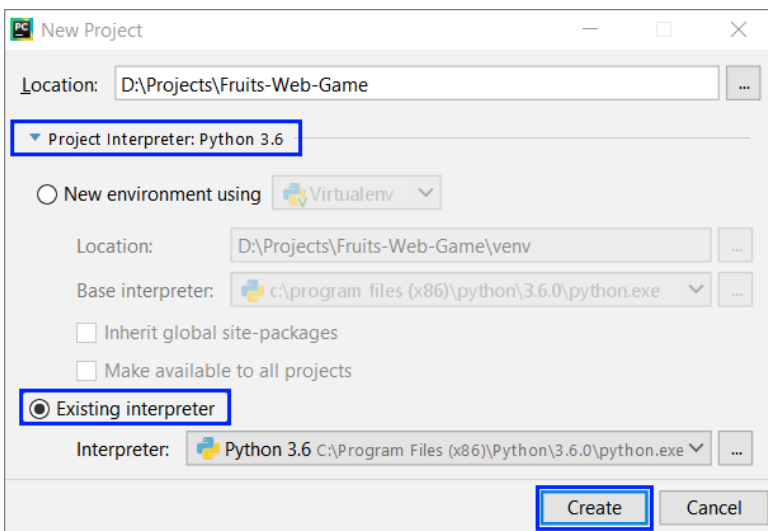
Shown below, are the steps for implementing the “Shoot the Fruits!” Web application.

Empty PyCharm Solution

We create an empty solution in PyCharm to organize the code from the application:



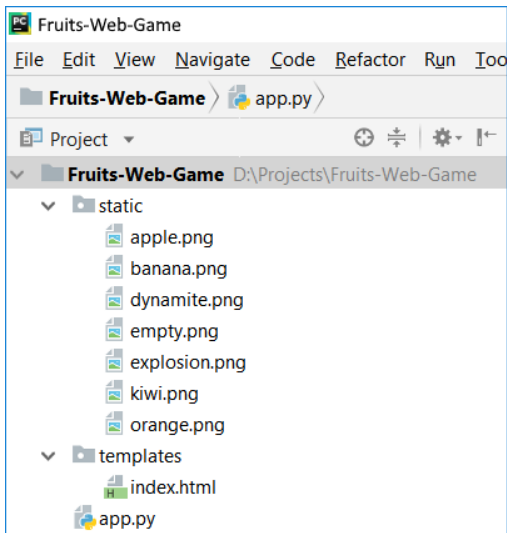
Then, we give a meaningful name to the project, for example “Fruits-Web-Game”. Also, set the Python interpreter to the default one:



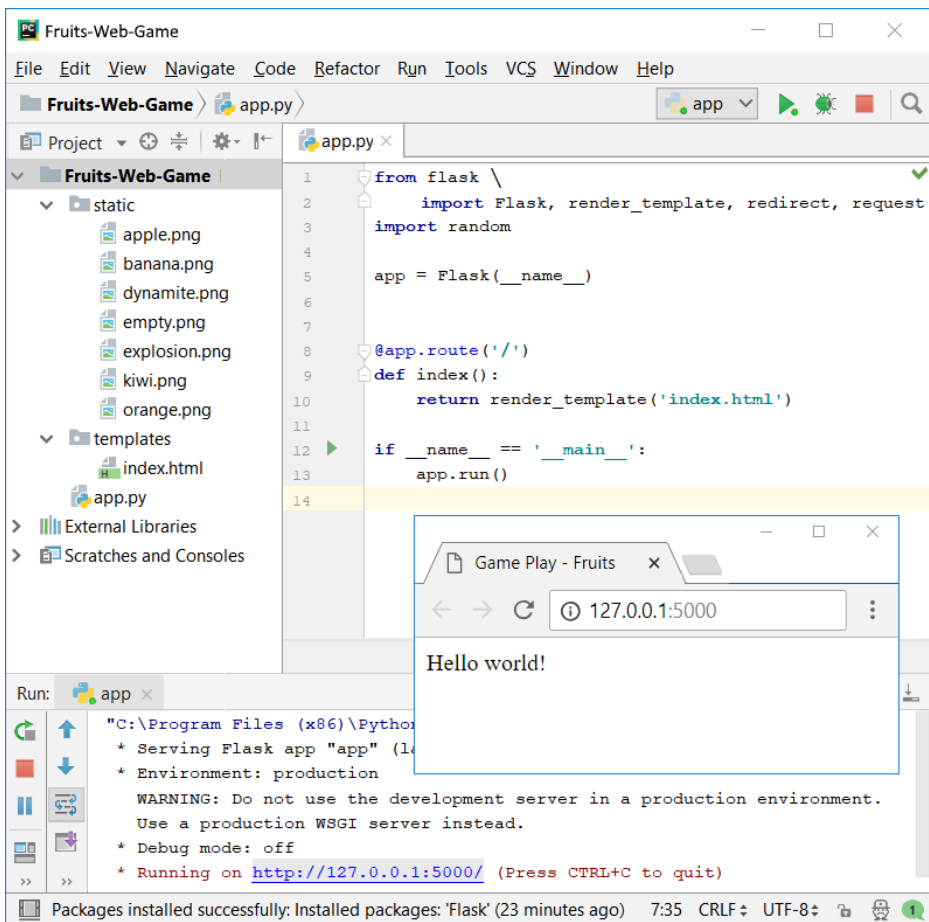
We will create our web application using the **Flask** library, which we have already met. As we already know, before we start writing code, we need to install it. We go to PyCharm settings [File] -> [Settings] and then to [Project: Fruit-Web-Game] -> [Project Interpreter]. There, press the **+** button, search for and install **Flask**.

Now we need to take the structure of the project from the resources provided to us.

Then, we add the **resources** for the game (they are part of the task files for this project and can be downloaded from [here](#)). We copy them from Windows Explorer and paste them into the project folder in PyCharm with **copy/paste**. Once we have placed the resources, the structure of the project should look like this:



If we open `app.py` and launch it with the right button -> [Run 'app'], we should open the application, then click on the link in the console and open our web browser :



Now we create the controls for the game. The goal is to add **scroll bars**, with which the player can aim, and a button to start a **new game**. Therefore, we need to edit the `templates/index.html` file. Delete “Hello World” and enter the following code in its place:

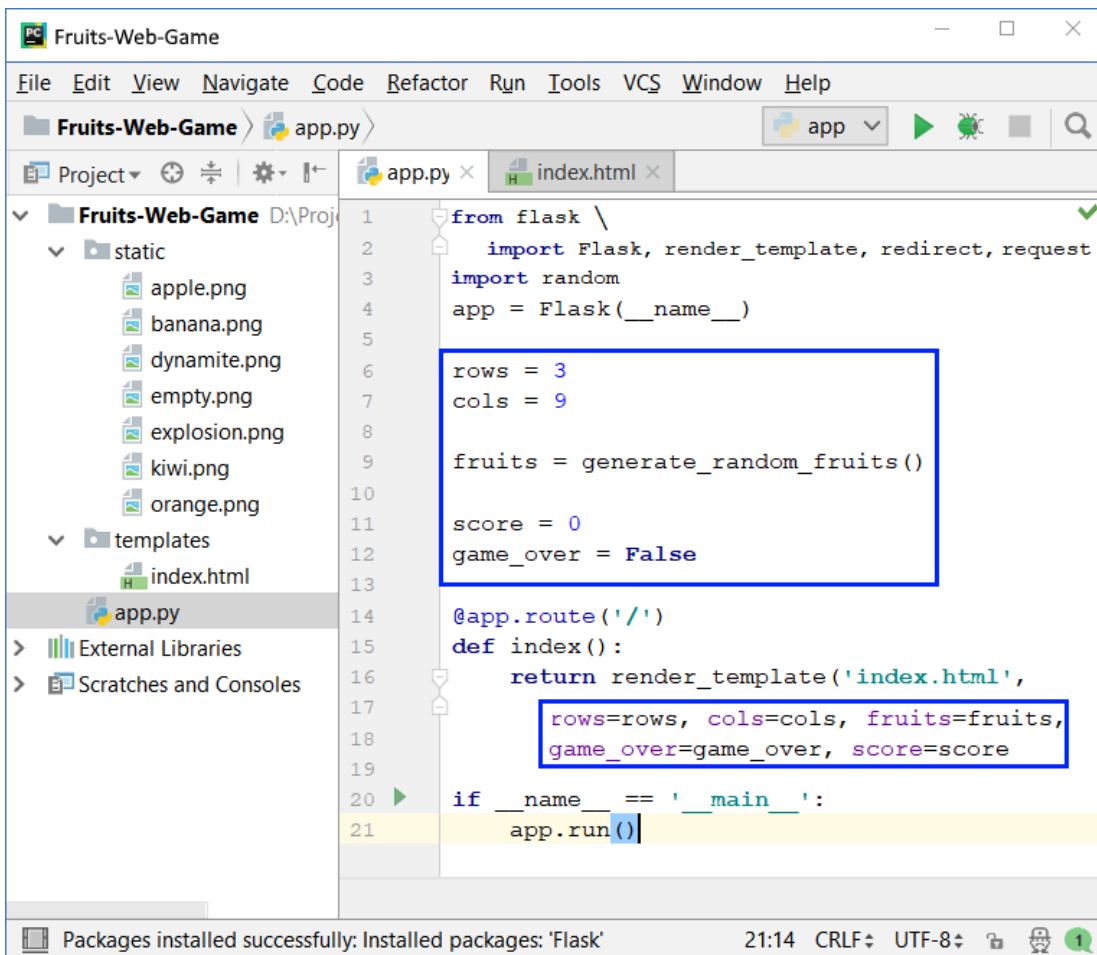
```

1 <html>
2 <head>
3   <title>Game Play - Fruits</title>
4 </head>
5 <body>
6   <h2>Game Play</h2>
7
8   <form action="/FireTop">
9     Top: <input type="range"
10           name="position" min="0" max="100"/>
11     <input type="submit" value="Fire Top">
12   </form>
13   <form action="/FireBottom">
14     Bottom: <input type="range"
15             name="position" min="0" max="100"/>
16     <input type="submit" value="Fire Bottom"/>
17   </form>
18   <form action="/Reset">
19     <input type="submit" value="New Game"/>
20   </form>
21 </body>
22 </html>

```

This code creates a web form `<form>` with a scroller (field) **position** to set a number in the interval `[0 ... 100]` and a button `[Fire Top]` to send the form data to the server. The action that will process the data is called `/FireTop`, which means the `fire_top()` function, which is located in the `app.py` file. There are two more similar forms with buttons `[Fire Bottom]` and `[New Game]`.

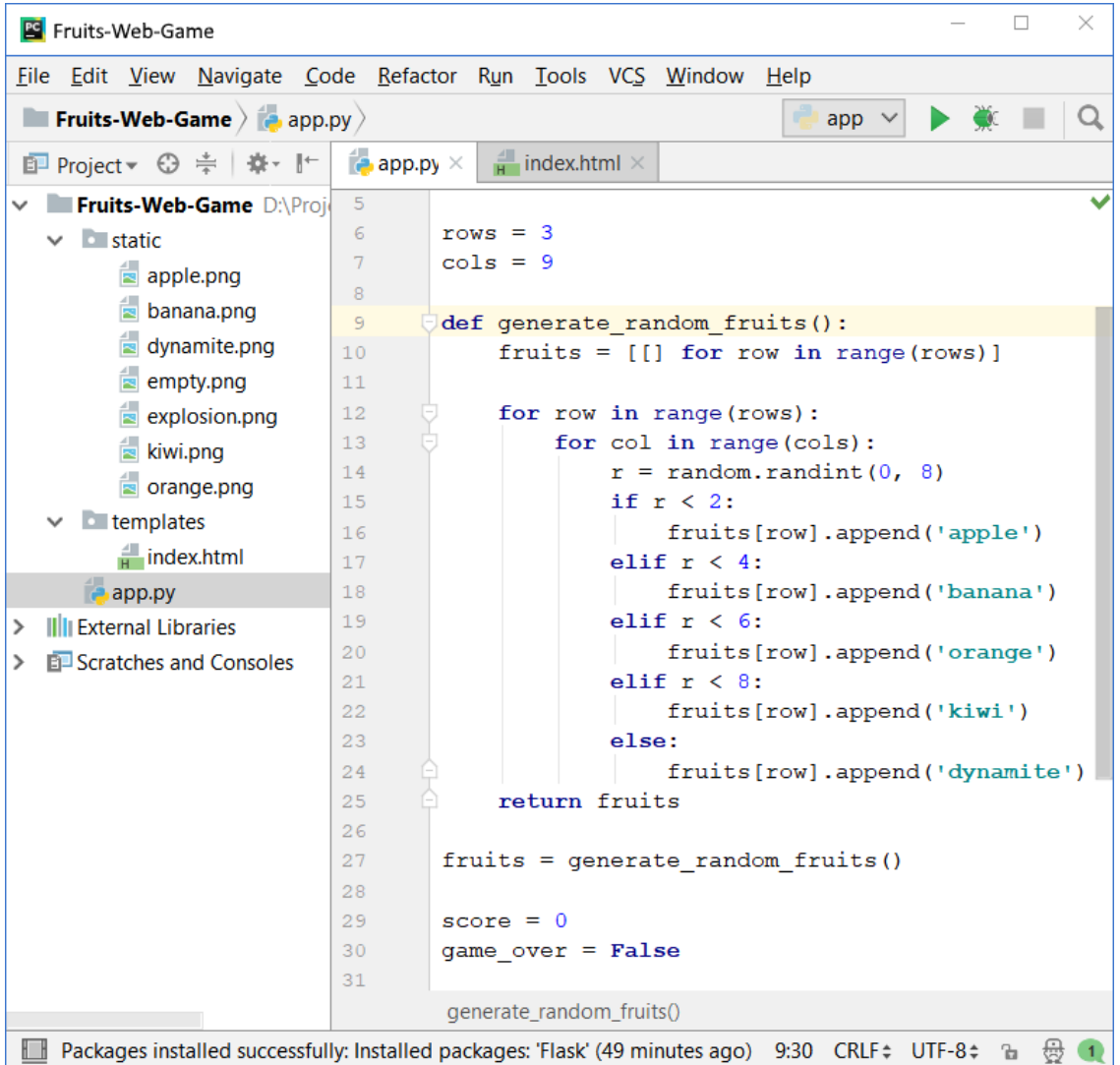
Now we need to prepare the fruits for drawing in the view. Add the following code to the `app.py` file:



The above code defines the fields for a number of rows, a number of columns, for fruit table (playing field), for the points accumulated by the player, and information whether the game is active or finished (field **gameOver**). The playing field is 9 columns in 3 rows and contains the text for each field: **apple, banana, orange, kiwi, empty, or dynamite**. The main action **index()** prepares the playing field for drawing by saving the elements of the game and calling the view that draws them on the game page (in the web browser as HTML).

We need to generate random fruits. To do this, we need to write the method **generate_random_fruits()** with the code from the picture below. This code writes in the table (matrix) **fruits** names of different pictures and thus builds the playing field. One of the following values is written in each cell of the table: **apple, banana, orange, kiwi, empty, or dynamite**. Then, to draw the corresponding image in the view, **.png** will be pasted to the text from the table and thus the name of the image file will be obtained, which will be inserted into the HTML page as part of the playing field. Filling the playing field (9 columns with 3 rows) is done in the view **index.html**, with two nested **for** loops (per row and column).

To generate random fruits, a **random number** between 0 and 8 is generated for each cell (see class **random** in Python). If the number is 0 or 1, put **apple**, if it is between 2 and 3, put **banana**, etc. If the number is 8, set **dynamite**. Thus, fruits appear 2 times more often than dynamite. Here is the code:



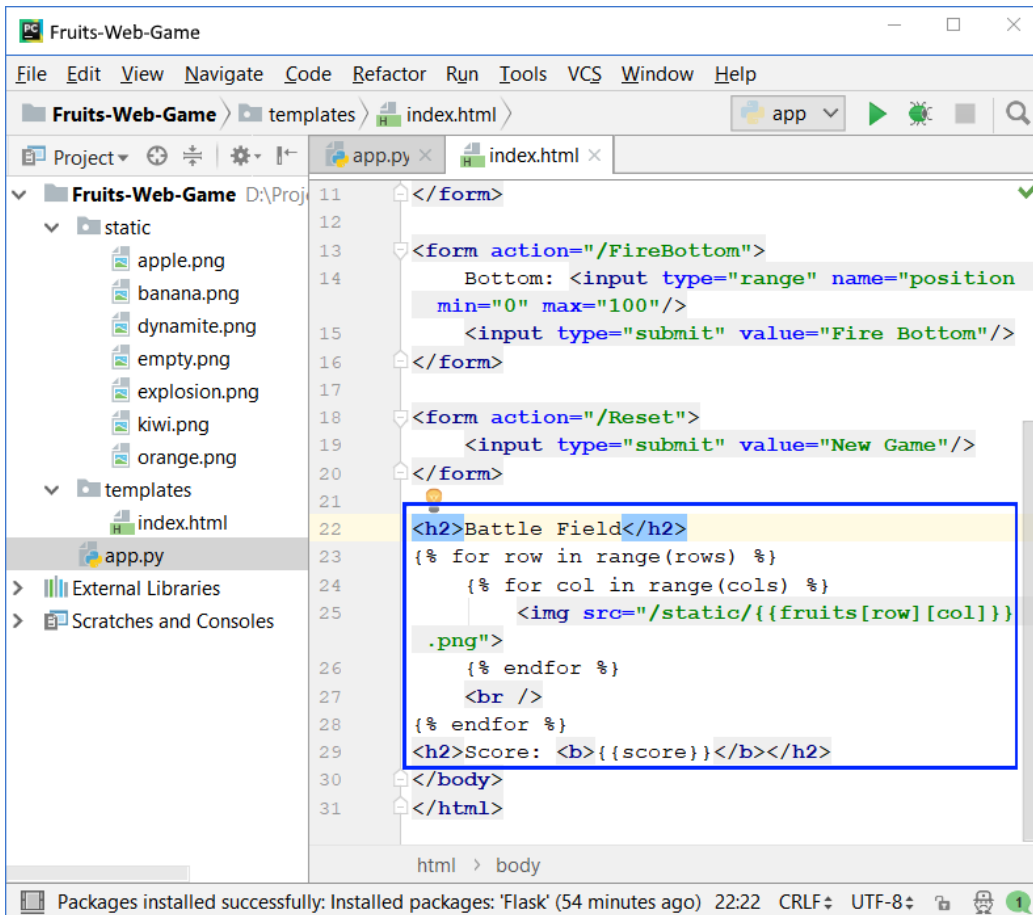
```

5
6     rows = 3
7     cols = 9
8
9     def generate_random_fruits():
10         fruits = [[] for row in range(rows)]
11
12         for row in range(rows):
13             for col in range(cols):
14                 r = random.randint(0, 8)
15                 if r < 2:
16                     fruits[row].append('apple')
17                 elif r < 4:
18                     fruits[row].append('banana')
19                 elif r < 6:
20                     fruits[row].append('orange')
21                 elif r < 8:
22                     fruits[row].append('kiwi')
23                 else:
24                     fruits[row].append('dynamite')
25         return fruits
26
27     fruits = generate_random_fruits()
28
29     score = 0
30     game_over = False
31
32     generate_random_fruits()

```

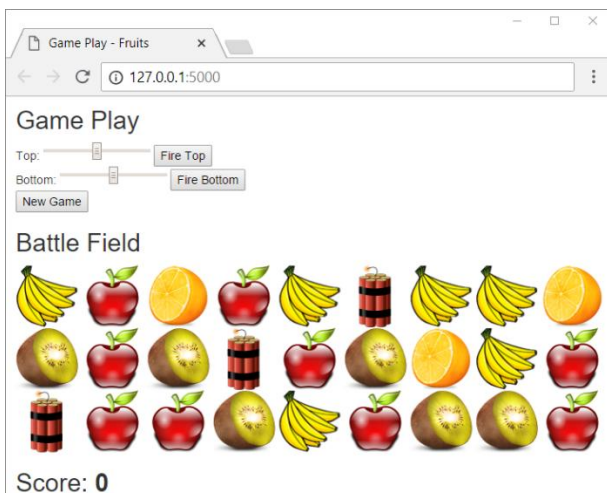
Drawing the fruit in **index.html**:

To fill the **playing field** with the fruits, we have to iterate **two nested loops** (for the rows and the columns). Each line consists of 9 pictures, each of which contains an **apple**, **banana**, or another fruit, or an empty field **empty**, or **dynamite**. Images are drawn by printing an HTML tag to insert an image: ``. Nine pictures are arranged one after the other on each of the lines, and then they are moved to a new line with `<br>`. This is repeated three times for the three lines. Finally, the player's points are printed. Here's how the **code** for drawing the playing field and points looks like:



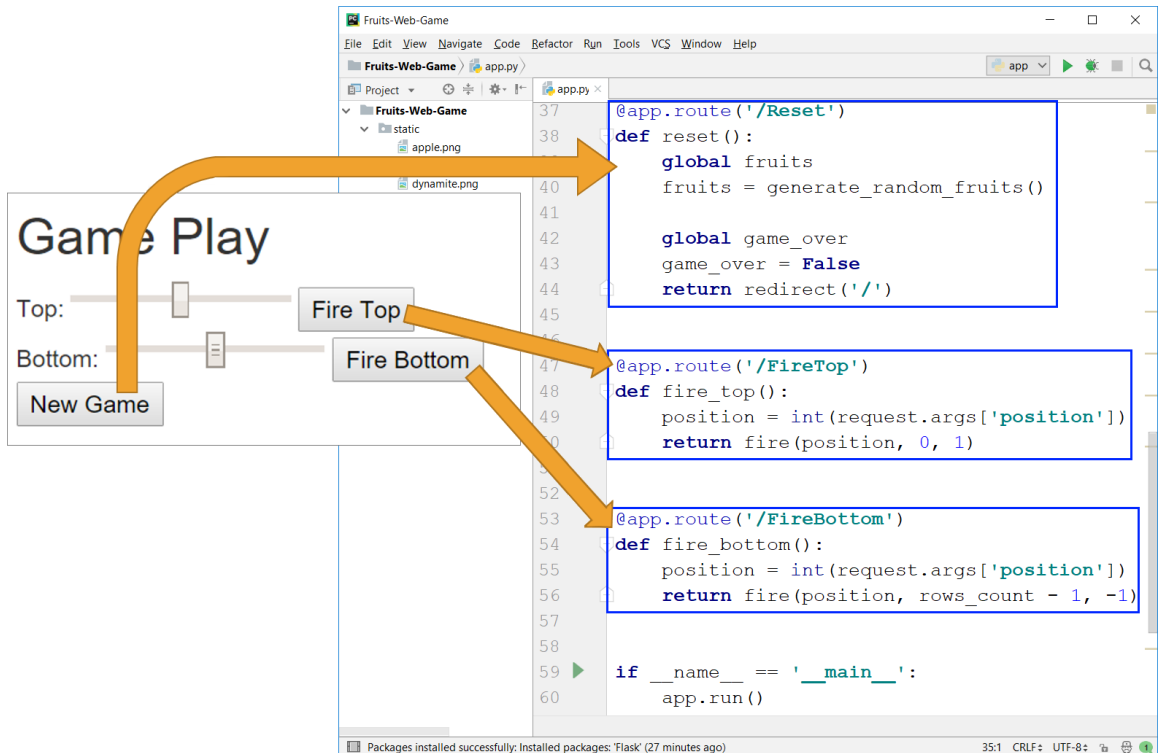
Note the curly braces – they are used to switch between the languages – HTML and Python and come from Jinja2 syntax for drawing dynamic web pages.

We start the project with [Shift+F10]. It is expected that a random fruit field with sizes 9 by 3 will be generated and visualized on the web page through a series of pictures:



Now the game is somewhat done: the playing field is randomly generated and visualized successfully (if you haven't made a mistake somewhere). It remains to realize the essence of the game: **shooting at the fruits**.

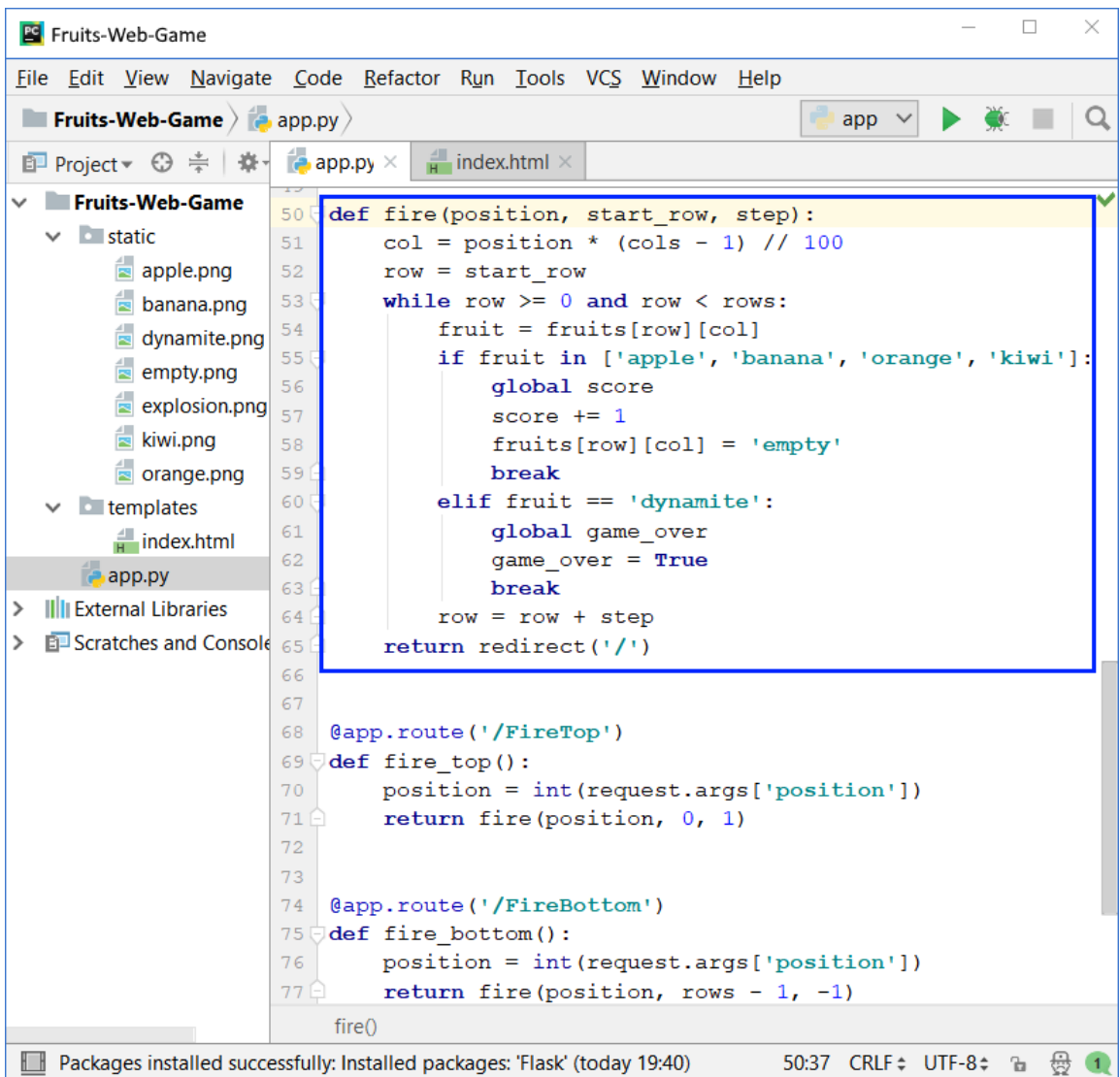
To do this, add the actions [New Game], [Fire Top], and [Fire Bottom] to the file `app.py`:



Using the code above we define three actions:

- **reset()** – launches a new game by generating a new random playing field with fruits and explosives, resets the player's points, and makes the game valid (**gameOver = false**). This action is quite simple and can be tested immediately with [Shift+F10] before writing the others.
- **fire_top()** – shoots in line 0 at **position** (number from 0 to 100), taken by the user. A **down** (+1) shooting from row 0 (top) is called. The shooting itself is more complex in logic and will be discussed shortly.
- **fire_bottom()** – shoots in line 2 at **position** (number from 0 to 100), taken by the user. A shoot in the **up** (-1) direction from row 2 (bottom) is called.

We implement the “shooting” method **fire(position, start_row, step)**:



```

50 def fire(position, start_row, step):
51     col = position * (cols - 1) // 100
52     row = start_row
53     while row >= 0 and row < rows:
54         fruit = fruits[row][col]
55         if fruit in ['apple', 'banana', 'orange', 'kiwi']:
56             global score
57             score += 1
58             fruits[row][col] = 'empty'
59             break
60         elif fruit == 'dynamite':
61             global game_over
62             game_over = True
63             break
64         row = row + step
65     return redirect('/')
66
67
68 @app.route('/FireTop')
69 def fire_top():
70     position = int(request.args['position'])
71     return fire(position, 0, 1)
72
73
74 @app.route('/FireBottom')
75 def fire_bottom():
76     position = int(request.args['position'])
77     return fire(position, rows - 1, -1)
78
79 def fire()

```

Package installed successfully: Installed packages: 'Flask' (today 19:40) 50:37 CRLF UTF-8

Shooting works as follows: first, the number of the column **col**, to which the player is aiming is calculated. The input number from the scroller (between 0 and 100) is reduced to a number between 0 and 8 (for each of the 9 columns). The line number **row** is either 0 (if the shot is at the top) or the number of lines minus one (if the shot is at the bottom). Accordingly, the direction of firing (the step) is **1** (down) or **-1** (up).

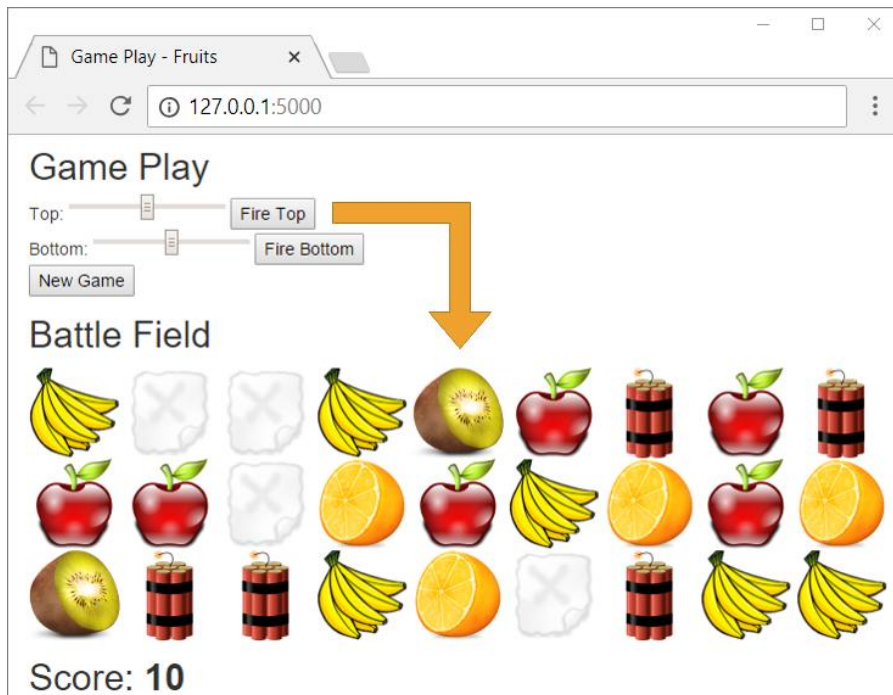
To find where the shot hits a fruit or dynamite, go through a loop through all the cells of the playing field in the target column and from the first to the last attacked row. If the fruit is found, it disappears (replaced by **empty**) and points are given to the player. If **dynamite** is encountered, the game is marked as over.

We leave it to more avid readers to implement more complex behaviors, for example, to give different points when hitting a different fruit, to realize an animation with an explosion (this

is not too easy), to take away points when shooting unnecessarily in an empty column, and similar.

We test what works so far by running the application with [Ctrl + Shift + F10]:

- **New game** ; the new game button must generate a new playing field with randomly placed fruits and explosives and reset the player's points.
- **Shooting from above** ; shooting from above must remove the top fruit in the hit column or cause the end of the game in dynamite. In fact, at the end of the game, nothing will happen yet, because in the view this case is not yet considered.
- **Shooting from below** ; shooting from below must remove the bottom fruit in the hit column or stop the game when hitting dynamite.



For now, nothing is happening at “End of Game”. If the player hits dynamite, the application notes that the game is over (**game_over = True**), but this fact is not visualized in any way. For the game to end, we need to add a few checks to the view:

```

6 <h2>Game Play</h2>
7 {% if not game_over %}
8   <form action="/FireTop">
9     Top: <input type="range"
10         name="position" min="0" max="100"/>
11     <input type="submit" value="Fire Top">
12   </form>
13   <form action="/FireBottom">
14     Bottom: <input type="range"
15         name="position" min="0" max="100"/>
16     <input type="submit" value="Fire Bottom"/>
17   </form>
18 {% endif %}
19 <form action="/Reset">
20   <input type="submit" value="New Game"/>
21 </form>
22 {% if game_over %}
23   <h2>Game Over!</h2>
24   
25 {% else %}
26   <h2>Battle Field</h2>
27   {% for row in range(rows) %}
28     {% for col in range(cols) %}
29       
30     {% endfor %}
31   <br />
32   {% endfor %}
33   <h2>Score: <b>{{score}}</b></h2>
34 {% endif %}
35 </body>
36 </html>

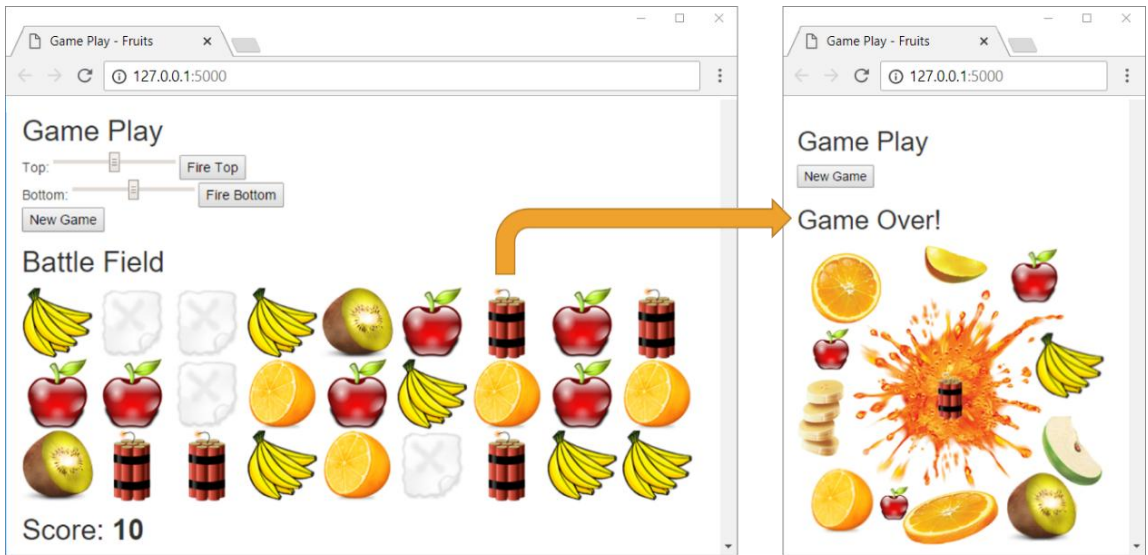
```

html > body

Packages installed successfully: Installed packages: 'Flask' (today 19:40) 7:23 CRLF UTF-8

The code above checks if the game is over and shows respectively the shooting controls and the playing field (during active play) or a picture with exploded fruits at the end of the game.

After changing the code of the view, we start with [Ctrl + Shift + F10] and **test** the game again:

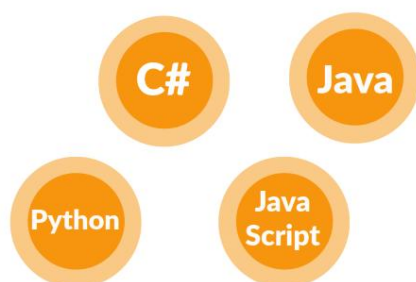


This time when hitting dynamite, the right picture should appear and only the “new game” action should be allowed (the **[New Game]** button).

Was it complicated? Did you manage to make the game? If you fail, don’t worry, this is a relatively complex project that involves a large amount of unstudied material. If you encounter any difficulties, you can ask in the **Softuni’s Reddit Community**:
<https://www.reddit.com/r/softuni/>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 7.2. Complex Loops – Exam Problems

We already learned how to execute a block of commands more than once using a **for** loop. In the previous chapter, we reviewed some loop structures that would help us solve more complex problems like:

- loops with a step
- nested loops
- **while** loops
- **do-while** loops
- infinite loops and breaking out of the loop (**break** operator)

Let's start work on solving the following practical exam problems.

Problem: Stupid Password Generator

Write a program that enters two integers **n** and **l** and generates in alphabetical order all possible "dumb" passwords, that consist of the following 5 characters:

- Character 1: digit from 1 to n.
- Character 2: digit from 1 to n.
- Character 3: small letter among the first l letters of the Latin alphabet.
- Character 4: small letter among the first l letters of the Latin alphabet.
- Character 5: digit from 1 to n, bigger than first 2 digits.

Input Data

The input is read from the console and consists of two integers: n and l within the range [1 ... 9], one per line.

Output Data

Print on the console all "dumb" passwords in alphabetical order, separated by space.

Sample Input and Output

Input	Output	Input	Output
2 4	11aa2 11ab2 11ac2 11ad2 11ba2 11bb2 11bc2 11bd2 11ca2 11cb2 11cc2 11cd2 11da2 11db2 11dc2 11dd2	3 1	11aa2 11aa3 12aa3 21aa3 22aa3

Input	Output	Input	Output
	11aa2 11aa3 11aa4 11ab2 11ab3 11ab4 11ba2 11ba3 11ba4 11bb2 11bb3 11bb4 12aa3 12aa4 12ab3 12ab4 12ba3 12ba4 12bb3 12bb4 13aa4 13ab4 13ba4 13bb4		11aa2 11aa3 11ab2 11ab3 11ba2 11ba3 11bb2 11bb3
4	21aa3 21aa4 21ab3 21ab4 21ba3 21ba4	3	12aa3 12ab3
2	21bb3 21bb4 22aa3 22aa4 22ab3 22ab4 22ba3 22ba4 22bb3 22bb4 23aa4 23ab4 23ba4 23bb4 31aa4 31ab4 31ba4 31bb4 32aa4 32ab4 32ba4 32bb4 33aa4 33ab4 33ba4 33bb4	2	12ba3 12bb3 21aa3 21ab3 21ba3 21bb3 22aa3 22ab3 22ba3 22bb3

Hints and Guidelines

We can split the solution of the problem into 3 parts:

- **Reading the input** – in the current problem, this includes reading two numbers **n** and **l**, each on a separate line.
- **Processing The Input Data** – using nested loops to iterate through every possible symbol for each of the five password symbols.
- **Printing the output** – printing every "dumb" password that meets the requirements.

Reading and Processing The Input Data

For **reading** of **input** data, we will declare two integer variables **int: n** and **l**.

```
n = int(input())  
l = int(input())
```

Printing Output

One of the ways to find a solution for this problem is to create **five for** nested loops, one for each variable. To ensure that the last digit is **greater** than the first two, we will use the built-in method **max(...)**:

```
result = ''  
for s1 in range(1, n + 1):  
    for s2 in range(1, n + 1):  
        for s3 in range(97, l + 97):  
            for s4 in range(97, l + 97):  
                for s5 in range(max(s1,s2) + 1, n + 1):
```

```

result += ('' + str(s1) + str(s2)
          + chr(s3) + chr(s4)
          + str(s5) + ' ')
print(result.strip())

```

Do you know that...?

- For convenience we can use the ASCII value of the symbol "a" directly in the declarations of the loops - **97**:

```

for s3 in range(97, 1 + 97):
    for s4 in range(97, 1 + 97):

```

- If we are not sure about a given value of a symbol or we want to take the values of dynamically generated symbols, we can use the built-in function **ord(char)**, which returns the integer representation of the character:

```

for s3 in range(ord('a'), 1 + ord('a')):
    for s4 in range(ord('a'), 1 + ord('a')):

```

- Or the opposite, we can generate a character from a given integer, using a built-in function in Python **chr(number)**:

```

result += ('' + str(s1) + str(s2)
          + chr(s3) + chr(s4)
          + str(s5) + ' ')

```

- Python has a wide range of built-in functions and methods for working with strings. Look for more information on the following functions: **str(number)**, **.lower()**, **.format(*args, kwargs)****, ****.swapcase()****, ****.upper()****.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1058#0>.

Problem: Magic Numbers

Write a program that enters a single integer **magic** number and produces all possible **6-digit** numbers for which the product of their digits is equal to the magical number.

Example: "Magic number" → 2

- 111112 → $1 * 1 * 1 * 1 * 1 * 2 = 2$
- 111121 → $1 * 1 * 1 * 1 * 2 * 1 = 2$
- 111211 → $1 * 1 * 1 * 2 * 1 * 1 = 2$
- 112111 → $1 * 1 * 2 * 1 * 1 * 1 = 2$
- 121111 → $1 * 2 * 1 * 1 * 1 * 1 = 2$
- 211111 → $2 * 1 * 1 * 1 * 1 * 1 = 2$

Input Data

The input is from the console and consists of **one integer** within the range [1 ... 600 000].

Output Data

Print on the console **all magic numbers**, separated by **space**.

Sample Input and Output

Input	Output	Input	Output	Input	Output
2	111112 111121 111211 112111 121111 211111	8	111118 111124 111142 111181 111214 111222 111241 111412 111421 111811 112114 112122 112141 112212 112221 112411 114112 114121 114211 118111 121114 121122 121141 121212 121221 121411 122112 122121 122211 124111 141112 141121 141211 142111 181111 211114 211122 211141 211212 211221 211411 212112 212121 212211 214111 221112 221121 221211 222111 241111 411112 411121 411211 412111 421111 811111	53 14 41	999999

Hints and Guidelines

The **solution** of this problem follows the **same** conception (we have to generate all n combinations of the element). Follow the steps and try to solve the problem on your own:

- Declare and initialize an integer type **variable** **int** and read the **input** from the console.
- Nest **six for loops** one for each digit.
- In the last loop, using an **if** statement checks if the **product** of the six digits is **equal** to the **magic** number.

Here is an example implementation of the idea:

```

n = int(input())
result = ''
for a in range(1, 10):
    for b in range(1, 10):
        for c in range(1, 10):
            for d in range(1, 10):
                for e in range(1, 10):
                    for f in range(1, 10):
                        if a * b * c * d * e * f == n:
                            result += f'{a} {b} {c} {d} {e} {f} '
print(result)

```

In the previous chapter, we reviewed other loop constructions. Let's look at the sample solution of the same problem using the **while** loop. First, we need to store the **input magical number** in a suitable variable:

```

n = int(input())
result = ''
a = 1

```

Then we will start writing **while** loops.

- We will initialize the **first digit**: **a = 1**.
- We will set a **condition for each** loop: the digit will be less than or equal to 9.
- At the **beginning** of each loop, we set a value of the **next** digit, in this case: **b = 1**. In the nested **for** loops, we initialize the variables in the inner loops at each increment of the outer ones. We want to do the same here.
- At the **end** of each loop, we will **increase** the digit by one: **a += 1, b += 1, c += 1** etc.
- In the **innermost** loop, we will make **the check** and if necessary, we will print on the console.

```

while a <= 9:
    b = 1
    while b <= 9:
        c = 1
        while c <= 9:
            d = 1
            while d <= 9:
                e = 1
                while e <= 9:
                    f = 1
                    while f <= 9:

```

```

    if a * b * c * d * e * f == n:
        result += ('' + str(a) + str(b)
                  + str(c) + str(d)
                  + str(e) + str(f) + ' ')
    b += 1
a += 1
print(result)

```

As we can see, we can solve a problem using different types of loops. Of course, each task has its most appropriate choice. To practice each type of loop – try to solve each of the following problems with all the learned loops.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1058#1>.

Problem: Stop Number

Write a program that prints on the console all numbers from **N** to **M**, that are **divisible by 2 and 3 without remainder**, in **reversed order**. We will read **one more** "stop" number from the console – **S**. If any of the numbers divisible by 2 and 3 is equal to the stop number, it should **not be printed** and the program should end. **Otherwise print all numbers up to N**, that meet the condition.

Input Data

Read from the console 3 numbers, each on a single line:

- **N** - integer number: $0 \leq N < M$.
- **M** - integer number: $N < M \leq 10000$.
- **S** - integer number: $N \leq S \leq M$.

Output Data

Print on the console on a single line all numbers, that meet the condition, separated by space.

Sample Input and Output

Input	Output	Comments
1 30 15	30 24 18 12 6	The numbers from 30 to 1 that are divisible by both 2 and 3 without remainder are: 30, 24, 18, 12 and 6. The number 15 is not equal to any of them, so the sequence continues.

Input	Output	Comments
1 36 12	36 30 24 18	The numbers from 36 to 1 that are divisible by both 2 and 3 without remainder are: 36, 30, 24, 18, 12 and 6. The number 12 is equal to the stop number, so we stop at 18.

Hints and Guidelines

The problem can be divided into **four** logical parts:

- **Reading** the input from the console.
- **Checking** all numbers in the given range, and then running a **loop**.
- **Checking** the conditions of the problem according to every number in the given range.
- **Printing** the numbers.

The **first** part is ordinary - we read **three** integer numbers from the console

We have already seen examples of the **second** part – initialization of the **for** loop. It is a bit **tricky** - the explanation mentions that the numbers have to be printed in **reversed order**. This means that the **initial** value of the variable **i** will be **bigger**, and from the examples, we can see that it is **M**. Thus, the **final** value of **i** should be **N**. The fact that we will print the results in reversed order and the values of **i** suggest that the step would be **decreased by 1**:

```
for i in range(m, n - 1, -1):
```

After we have initialized the **for** loop, it is time for the **third** part of the problem - **checking** the condition if the given **number is divisible both by 2 and 3 without remainder**. We will do this using one simple **if** condition that we will leave to the reader to do by themselves.

Another **tricky** part of this problem is that apart from the above check we need to do **another** one – whether the **number is equal to the "stop" number**, entered from the console on the third line. To do this check, the previous one has to be passed. For this reason, we will add another **if** statement that we will **nest in the previous one**. If the condition is **true**, we need to stop the program from printing. We can do this using a **break** operator, and it will lead us **out of the for loop**.

If the **condition** that checks whether the number is equal with the "stop" number returns a **false**, result, our program should **continue to print**. This covers the **fourth and last** part of our program.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1058#2>.

Problem: Special Numbers

Write a program that reads one integer number N and generates all possible "special numbers" from 1111 to 9999. To be considered "special" a number must correspond to the following condition:

- N to be divisible by each of its digits without remainder.

Example: upon N = 16, 2418 is a special number:

- 16 / 2 = 8 without remainder
- 16 / 4 = 4 without remainder
- 16 / 1 = 16 without remainder
- 16 / 8 = 2 without remainder

Input Data

The input is read from the program and consists of one integer within the range [1 ... 600 000].

Output Data

Print on the console all special numbers, separated by space.

Sample Input and Output

Input	Output	Comments
3	1111 1113 1131 1133 1311 1313 1331 1333 3111 3113 3131 3133 3311 3313 3331 3333	3 / 1 = 3 without remainder 3 / 3 = 1 without remainder 3 / 3 = 1 without remainder 3 / 3 = 1 without remainder

Input	Output	Input	Output
11	1111	16	1111 1112 1114 1118 1121 1122 1124 1128 1141 1142 1144 1148 1181 1182 1184 1188 1211 1212 1214 1218 1221 1222 1224 1228 1241 1242 1244 1248 1281 1282 1284 1288 1411 1412 1414 1418 1421 1422 1424 1428 1441 1442 1444 1448 1481

Input	Output	Input	Output
			1482 1484 1488 1811 1812 1814 1818 1821 1822 1824 1828 1841 1842 1844 1848 1881 1882 1884 1888 2111 2112 2114 2118 2121 2122 2124 2128 2141 2142 2144 2148 2181 2182 2184 2188 2211 2212 2214 2218 2221 2222 2224 2228 2241 2242 2244 2248 2281 2282 2284 2288 2411 2412 2414 2418 2421 2422 2424 2428 2441 2442 2444 2448 2481 2482 2484 2488 2811 2812 2814 2818 2821 2822 2824 2828 2841 2842 2844 2848 2881 2882 2884 2888 4111 4112 4114 4118 4121 4122 4124 4128 4141 4142 4144 4148 4181 4182 4184 4188 4211 4212 4214 4218 4221 4222 4224 4228 4241 4242 4244 4248 4281 4282 4284 4288 4411 4412 4414 4418 4421 4422 4424 4428 4441 4442 4444 4448 4481 4482 4484 4488 4811 4812 4814 4818 4821 4822 4824 4828 4841 4842 4844 4848 4881 4882 4884 4888 8111 8112 8114 8118 8121 8122 8124 8128 8141 8142 8144 8148 8181 8182 8184 8188 8211 8212 8214 8218 8221 8222 8224 8228 8241 8242 8244 8248 8281 8282 8284 8288 8411 8412 8414 8418 8421 8422 8424 8428 8441 8442 8444 8448 8481 8482 8484 8488 8811 8812 8814 8818 8821 8822 8824 8828 8841 8842 8844 8848 8881 8882 8884 8888

Hints and Guidelines

Solve the problem by yourself using what you learned from the previous two problems. Keep in mind the difference between operators for **integer division** `//` and **division with remainder** `%` in Python.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1058#3>.

Problem: Digits

Write a program that reads from the console 1 integer within the range [100 ... 999], and then prints it a predefined number of times – modifying it before each print, as follows:

- If the number is divisible by 5 without remainder, **subtract** from it **its first digit**.
- If the number is divisible by 3 without remainder, **subtract** from it **its second digit**.

- If none of the above-mentioned conditions is valid, **add** to it **its third digit**.

Print on the console **N lines**, and each line has **M numbers**, that are the result of the above actions. Let:

- N = sum of the first and second digits of the number.
- M = sum of the first and third digits of the number.

Input Data

The input is read from the console and is **an integer** within the range [100 ... 999].

Output Data

Print on the console **all integer numbers**, that is the result of the above-mentioned calculations in the respective number of rows and columns as in the examples.

Sample Input and Output

Input	Output	Comments
376	382 388 394 400 397 403 409 415 412 418 424 430 427 433 439 445 442 448 454 460 457 463 469 475 472 478 484 490 487 493 499 505 502 508 514 520 517 523 529 535 532 538 544 550 547 553 559 565 562 568 574 580 577 583 589 595 592 598 604 610 607 613 619 625 622 628 634 640 637 643 649 655 652 658 664 670 667 673 679 685 682 688 694 700 697 703 709 715 712 718	10 lines with 9 numbers on each Input number 376 → neither by 5, nor by 3 → $376 + 6 \rightarrow = 382 \rightarrow$ neither by 5, nor by 3 → $382 + 6 = 388 + 6 = 394 + 6 = 400 \rightarrow$ division by 5 → $400 - 3 = 397$

Input	Output	Comments
132	129 126 123 120 119 121 123 120 119 121 123 120	$(1 + 3) = 4$ and $(1 + 2) = 3 \rightarrow$ lines with 3 numbers on each Input number 132 $132 \rightarrow$ division by 3 $\rightarrow 132 - 3 =$ $= 129 \rightarrow$ division by 3 $\rightarrow 129 - 3 =$ $= 126 \rightarrow$ division by 3 $\rightarrow 126 - 3 =$ $= 123 \rightarrow$ division by 3 $\rightarrow 123 - 3 =$ $= 120 \rightarrow$ division by 5 $\rightarrow 120 - 1 =$ $121 \rightarrow$ neither by 5, no- by 3 $\rightarrow 121 + 2 = 123$

Hints and Guidelines

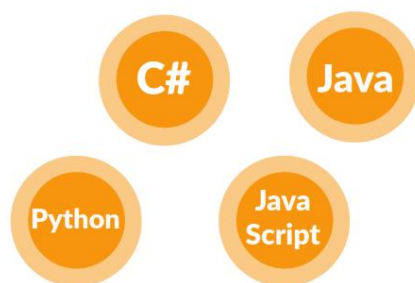
Solve the problem **by yourself**, using what you learned from the previous ones. Remember that you will need to define **different** variables for each digit of the input number.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1058#4>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 8.1. Practical Exam Preparation – Part I

In the present chapter, we will examine a few **problems** with a level of **difficulty** that can be expected in the **problems** of the practical **exam** in “Programming Basics”. We will **review** and **practice** all the knowledge that was gained from this book and through the “Programming Basics” course.

The “Programming Basics” Practical Exam

The **Practical Exam** contains **6** problems included, and you will have **4 hours** to solve them. **Each** of the exam problems will **cover** one of the studied **topics** during the course. Problem topics are as follows:

- Problem with simple calculations (without conditions)
- Problem with simple condition
- Problem with more complex conditions
- Problem with a simple loop
- Problem with nested loops (drawing a figure on the console)
- Problem with nested loops and more complex logic

The Online Evaluation System (Judge)

All **exams** and **problems** are automatically **tested** through the online **Judge system**: <https://judge.softuni.org>. For **each** of the problems, there are **visible** (zero points) tests that help you understand what is expected of the problem and fix your mistakes, as well as **competition** tests that are **hidden** and check if your solution is working properly.

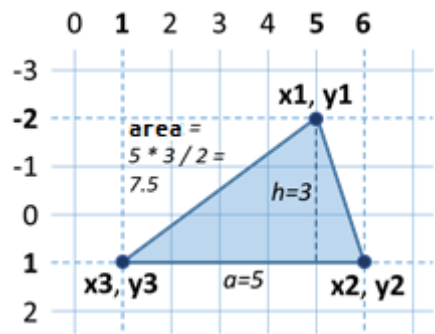
How does the testing in the **Judge** system work? **You upload** the source code and from the menu below you choose to compile as a **Python** program. The program is being **tested** with a series of tests, giving **points** for each **successful** test.

Problems with Simple Calculations

The **first** problem of the “Programming Basics” Practical Exam covers **simple calculations** without checks and loops. Here are a few examples:

Problem: Triangle Area

Triangle in the plain is defined by the coordinates of its three vertices. First, the vertex (x1, y1) is set. Then the other two vertices are set: (x2, y2) and (x3, y3) which lie on a common horizontal line i.e. they have the same Y coordinates). Write a program that calculates the triangle area by the coordinates of its three vertices.



Input Data

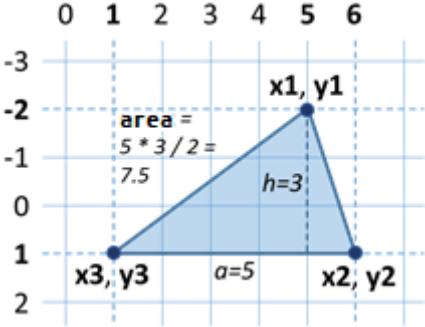
From the console 6 integers are read (one per line): x1, y1, x2, y2, x3, y3.

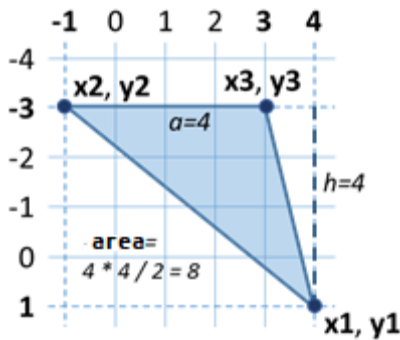
- All input numbers are in the range [-1000 ... 1000].
- It's guaranteed that y2 = y3.

Output Data

Print on the console the triangle area.

Sample Input and Output

Input	Output	Visualization	Comments
5 -2 6 1 1 1	7.5		The side of the triangle $a = 6 - 1 = 5$ The height of the triangle $h = 1 - (-2) = 3$ The area of the triangle $S = a * h / 2 = 5 * 3 / 2 = 7.5$

Input	Output	Visualization	Comments
4 1 -1 -3 3 -3	8		The side of the triangle $a = 3 - (-1) = 4$ The height of the triangle $h = 1 - (-3) = 4$ The area of the triangle $S = a * h / 2 = 4 * 4 / 2 = 8$

Hints and Guidelines

It is extremely important in these types of tasks, in which some coordinates are given, to pay attention to the **order** in which they are submitted, as well as to correctly understand which of the coordinates we will use and in what way. In this case, the input is in order **x1, y1, x2, y2, x3, y3**. If we do not follow this sequence, the solution becomes wrong. First, we write the code that reads the input data:

```
x1 = int(input())
y1 = int(input())
x2 = int(input())
y2 = int(input())
x3 = int(input())
y3 = int(input())
```

We have to calculate **the side** and **the height** of the triangle. From the examples and the condition **y2 = y3** we notice that one **side** is always parallel to the horizontal axis. It means that its **length** is equal to the length of the segment between its coordinates **x2** and **x3**, which is equal to the difference between the larger and the smaller coordinates. Similarly, we can calculate **the height**. It will always be equal to the difference between **y1** and **y2** (or **y3**, as they are equal). Since we do not know if **x2** is greater than **x3**, or **y1** will be below or above the triangle side, we will use **the absolute values** of the difference to always get positive numbers because one segment cannot have a negative length.

```
a = abs(x2 - x3)
h = abs(y2 - y1)
```

We will use the formula familiar to us from school for finding the **area of a triangle** to calculate the area.

```
s = a * h / 2
```

The only thing left is to print the area on the console.

```
print(s)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#0>.

Problem: Bricks

Construction workers have to transfer a total of **x bricks**. The **workers** are **w** and work simultaneously. They transport the bricks in trolleys, each with a **capacity of m** bricks. Write a program that reads the integers **x**, **w**, and **m**, and calculates **what is the minimum number of courses** the workers need to do to transport the bricks.

Input Data

From the console **3 integers** are read (one per line):

- The number of bricks **x** is read from the first line.
- The number of workers **w** is read from the second line.
- The capacity of the trolley **m** is read from the third line.

All input numbers are integers in the range [1 ... 1000].

Output Data

Print on the console **the minimum number of courses** needed to transport the bricks.

Sample Input and Output

Input	Output	Comments
120 2 30	2	We have 2 workers, each transporting 30 bricks per course. In total, workers are transporting 60 bricks per course. To transport 120 bricks, exactly 2 courses are needed.
Input	Output	Comments
355 3 10	12	We have 3 workers, each transporting 10 bricks per course. In total, workers are transporting 30 bricks per course. To transport 355 bricks, exactly 12 courses are needed: 11 complete courses carry 330 bricks and the last twelfth course carries the last 25 bricks.
Input	Output	Comments
5 12 30	1	We have 5 workers, each transporting 30 bricks per course. In total, workers are transporting 150 bricks per course. To transport 5 bricks, only 1 course is sufficient (although incomplete, with only 5 bricks).

Hints and Guidelines

The input is standard, and we only have to pay attention to the sequence in which we read the data:

```
x = int(input())
w = int(input())
m = int(input())
```

We calculate how many **bricks** the workers transport in a single course:

```
bricks_in_course = w * m
```

By dividing the total number of **bricks transported for 1 course**, we will obtain the number of **courses** needed to carry them. We will use the **math.ceil (...)** function to round the result up. **Remember** to add **import math** at the beginning of the working file so that the **math.ceil (...)** function can work. When the bricks can be transferred with **an exact number of courses**, the division will return a whole number and there will be nothing to round. Accordingly, if not, the result of the division will be **the number of exact courses** but with a decimal fraction. The decimal part will be rounded up and we will get the required **1 course** for the remaining bricks.

```
total_courses = math.ceil(x / bricks_in_course)
```

In the end, we print the result on the console:

```
print(total_courses)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#1>.

Problems with Simple Conditions

The **second** problem of the “Programming Basics” Practical Exam covers **conditional statements** and **simple calculations**. Here are a few examples:

Problem: Point on Segment

On a horizontal line, a **horizontal segment** is placed, set with the **x** coordinates at both ends: **first** and **second**. A **point** is located **on** the same horizontal line and is set with its **x** coordinate. Write a program that checks whether the point is **inside or outside of the segment** and calculates **the distance to the nearest end** of the segment.

Input Data

From the console **3 integers** are read (one per line):

- On the first line is the number **first** – **one end of the segment**.
- On the second line is the number **second** – **the other end of the segment**.
- On the third line is the number **point** – **the location of the point**.

All inputs are integers in the range **[-1000 ... 1000]**.

Output Data

Print the result on the console:

- On the first line, print “in” or “out” – whether the point is inside or outside the segment.
- On the second line, print the distance from the point to the nearest end of the segment.

Sample Input and Output

Input	Output	Visualization
10 5 7	in 2	
Input	Output	Visualization
8 10 5	out 3	
Input	Output	Visualization
1 -2 3	out 2	

Hints and Guidelines

First, we read the input:

```
first = int(input())
second = int(input())
point = int(input())
```

Since we do not know which **point** is on the left and which is on the right, we will create two variables to mark this. Since **the left point** is always the one with the smaller **x coordinate**, we will use the **min(...)** function to find it. Accordingly, **the right point** is always the one with a larger **x coordinate** and we will use the **max(...)** function. We will also find the distance from **the point x** to **the two points**. Since we do not know their position relative to each other, we will use the **abs(...)** function to get a positive result:

```

left = min(first, second)
right = max(first, second)

distance_left = abs(left - point)
distance_right = abs(right - point)

```

The shorter of the two **distances** we will find by using **min(...)**:

```
min_distance = min(distance_left, distance_right)
```

What remains is to find whether **the point** is on or out of the line. The point will be **on the line** always when it **matches** one of the other two points or its x coordinate lies **between** them. Otherwise, the point is **outside the line**. After checking, we display one of the two messages ("in" or "out"), depending on which condition is satisfied:

```

if left <= point <= right:
    print('in')
else:
    print('out')

```

Finally, we print **the distance** found before.

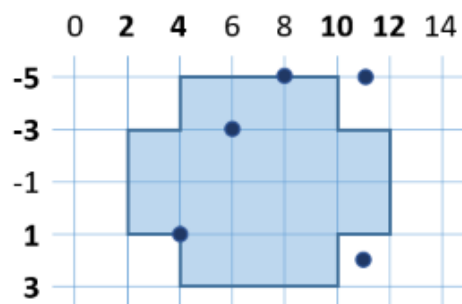
```
print(min_distance)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#2>.

Problem: Point in Figure

Write a program that checks if a point (with coordinates **x** and **y**) is **inside** or **outside** of the given figure:



Input Data

From the console are read **two integers** (one per line): **x** and **y**.

All inputs are integers in the range [-1000 ... 1000].

Output Data

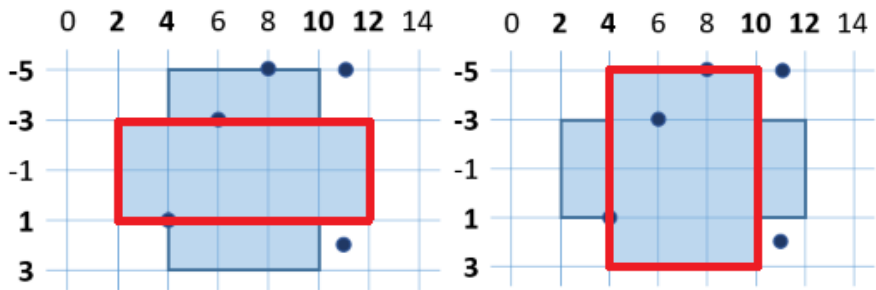
Print on the console "in" or "out" – whether the point is **inside** or **outside** the figure (the outline is inside).

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
8 -5	in	6 -3	in	11 -5	out	11 2	out

Hints and Guidelines

To find out if **the point** is inside of the figure, we will divide **the figure** into 2 rectangles:



A sufficient condition is the **point** to be located in one of them and be in the **figure**.

We read the input from the console:

```
x = int(input())
y = int(input())
```

We will initialize two variables that will mark whether **the point** is in one of the rectangles:

```
pointInRect1 = 2 <= x <= 12 and -3 <= y <= 1
pointInRect2 = 4 <= x <= 10 and -5 <= y <= 3
```

When printing the message, we will check whether any of the variables has accepted a value of **True**. It's enough **only one** of them to be **True** so that the point is in the figure.

```
if pointInRect1 or pointInRect2:
    print('in')
else:
    print('out')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#3>.

Problems with Complex Conditions

The **third** problem of the “Programming Basics” Practical Exam includes **several nested checks** combined with simple calculations. Here are a few examples:

Problem: Date After 5 days

There are two numbers **d** (day) and **m** (month) that form a **date**. Write a program that prints the date that will be **after 5 days**. For example, 5 days after **28.03** is the date **2.04**. We assume that the months: April, June, September, and November have 30 days, February has 28 days, and the rest have 31 days. Months to be printed with **leading zero** when they are single-digit numbers (e.g. 01, 08).

Input Data

The input is read from the console and consists of two lines:

- On the first line, there is an integer **d** in the interval **[1... 31]** – day. The number of the day does not exceed the number of days in the respective month (e.g. 28 for February).
- On the second line, there is an integer **m** in the interval **[1... 12]** – month. Month 1 is January, month 2 is February,..., month 12 is December. The month may contain a leading zero (e.g. April may be written as 4 or 04).

Output Data

Print a single line containing the date after 5 days in the format **day.month** on the console. The month must be a two-digit number with a leading zero, if necessary. The day must be written without a leading zero.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
28 03	2.04	27 12	1.01	25 1	30.01	26 02	3.03

Hints and Guidelines

We read the input data from the console:

```
d = int(input())
m = int(input())
```

To make our checks easier, we will create a variable that will contain the **number of days** that we have in the month we set:

```
days_in_month = 31
```

```
if m == 2:
    days_in_month = 28
```

```
if m == 4 or m == 6 or m == 9 or m == 11:
    days_in_month = 30
```

We increase the **day** by 5:

```
d += 5
```

We check if **the day** has exceeded the number of days in **the month**. If so, we must deduct the days of the month from the obtained day to calculate which day of the next month our day corresponds to:

```
if d > days_in_month:
    d -= days_in_month
```

After we have passed to **the next month**, this should be noted by increasing the initial one by 1. We need to check if it has become greater than 12 and if it has, to adjust it. Because we cannot skip more than **one month** when we increase by 5 days, the following check is enough:

```
if d > days_in_month:
    d -= days_in_month

    m += 1

    if m > 12:
        m = 1
```

The only thing that remains is to print the result on the console. It is important to **format the output** correctly to display the leading zero in the first 9 months. This is done by adding a **formatted string** `%02d` for the second element:

```
print('%d.%02d' % (d, m))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#4>.

Problem: Sums 3 Numbers

There are **3 integers** given. Write a program that checks if **the sum of two of the numbers is equal to the third one**. For example, if the numbers are **3, 5, and 2**, the sum of two of the numbers is equal to the third one: **2 + 3 = 5**.

Input Data

From the console are read **three integers**, one per line. The numbers are in the range [**1 ... 1000**].

Output Data

- Print a single line on the console containing the solution of the problem in the format **"a + b = c"**, where **a**, **b** and **c** are among the three input numbers and **a ≤ b**.
- If the problem has no solution, print **"No"** on the console.

Sample Input and Output

Input	Output
3	2 + 3 = 5
5	
2	

Input	Output
2	2 + 2 = 4
2	
4	

Input	Output
1	No
1	
5	

Input	Output
2	No
6	
3	

Hints and Guidelines

We take the input from the console:

```
a = int(input())
b = int(input())
c = int(input())
```

We have to check if **the sum** of a pair of numbers is equal to the third number. We have three possible cases:

- $a + b = c$
- $a + c = b$
- $b + c = a$

We will write **a template**, which will later be complemented by the required code. If none of the above three conditions is met, we will make our program print “No”:

```
if a + b == c:
    # TODO

elif a + c == b:
    # TODO

elif b + c == a:
    # TODO

else:
    print('No')
```

Now it remains to understand the order in which **the two addends** will be displayed at the output of the program. For this purpose, we will make a **nested condition**, which checks which of the two numbers is the greater. In the first case, it will be as follows:

```
if a + b == c:
    if a > b:
        print('%d + %d = %d', b, a, c)
    else:
        print('%d + %d = %d', a, b, c)
```

Similarly, we will supplement the other two cases. The full code of the program will look like this:

```
if a + b == c:
    if a > b:
        print('%d + %d = %d', b, a, c)
    else:
        print('%d + %d = %d', a, b, c)

elif a + c == b:
    if a < c:
        print('%d + %d = %d', a, c, b)
    else:
        print('%d + %d = %d', c, a, b)

elif b + c == a:
    if b < c:
        print('%d + %d = %d', b, c, a)
    else:
        print('%d + %d = %d', c, b, a)

else:
    print('No')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#5>.

Problems with Simple Loops

The fourth problem of the “Programming Basics” Practical Exam includes a simple loop with simple logic in it. Here are a few examples:

Problem: Sums Step 3

There are given n integers $a_1, a_2, \dots, a_n$. Calculate the sums:

- $\text{sum1} = a_1 + a_4 + a_7 + \dots$ (the numbers are summed, starting from the first one with step of 3).
- $\text{sum2} = a_2 + a_5 + a_8 + \dots$ (the numbers are summed, starting from the second one with step of 3).

- $\text{sum3} = a_3 + a_6 + a_9 + \dots$ (the numbers are summed, starting from the third one with step of 3).

Input Data

The input data is read from the console. The first line contains an integer n ($0 \leq n \leq 1000$). The next n lines contain n integers in the interval $[-1000 \dots 1000]$: $a_1, a_2, \dots, a_n$.

Output Data

On the console, we should print 3 lines containing the 3 sums in a format such as in the example.

Sample Input and Output

Input	Output	Input	Output	Input	Output
2	sum1 = 3	4		5	
3	sum2 = 5	7	sum1 = 19	3	sum1 = 10
5	sum3 = 0	-2	sum2 = -2	5	sum2 = 13
		6	sum3 = 6	2	sum3 = 2
		12		7	
				8	

Hints and Guidelines

We will take the count of numbers from the console and will declare starting values of the three sums.

```
n = int(input())
sum1 = sum2 = sum3 = 0
```

Since we do not know in advance how many numbers we will process, we will take them one at a time in a loop which will be repeated n times and we will process them in the body of the loop.

```
for i in range(0, n):
    a = int(input())
```

```
# TODO
```

To find out which of the three sums we need to add the number, we will divide its sequence number into three and we will use the remainder. We'll use the variable i which tracks the number of runs of the loop, to find out which sequence number we are at. When the remainder of $i/3$ is zero, it means we will add this number to the first sum, when it's 1 to the second, and when it's 2 to the third:

```
for i in range(0, n):
    a = int(input())

    if i % 3 == 0:
        sum1 += a

    if i % 3 == 1:
        sum2 += a

    if i % 3 == 2:
        sum3 += a
```

Finally, we will print the result on the console in the required format:

```
print('sum1 = %d' % sum1)
print('sum2 = %d' % sum2)
print('sum3 = %d' % sum3)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#6>.

Problem: Increasing Elements

A series of **n** numbers is given: **a1, a2, ..., an**. Calculate the length of the longest increasing sequence of consecutive elements in the series of numbers.

Input Data

The input data is read from the console. The first line contains an integer **n** ($0 \leq n \leq 1000$). The next **n** lines contain **n** integers in the interval **[-1000 ... 1000]**: **a1, a2, ..., an**.

Output Data

Print on the console one number – the length of the longest increasing sequence.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
3 5 2 4	2	4 2 8 7 6	2	4 1 2 4 4	3	4 5 6 7 8	4

Hints and Guidelines

To solve this problem, we need to think in a bit **more algorithmic way**. A **sequence of numbers** is given to us, and we need to check whether each **subsequent one** will be **larger than the previous one** and if so, we count how long is the sequence in which this condition is fulfilled. Then we have to find **which sequence** of these is **the longest one**. To do this, let's create some variables that we will use during solving the problem:

```
n = int(input())
count_current_longest = count_longest = a_prev = 0
```

The variable **n** is the count of numbers we get from the console. In **count_current_longest** we'll keep the number of elements in the increasing sequence we are currently counting. For example, in the sequence: 5, 6, 1, 2, 3 **count_current_longest** will be 2 when we reach the second element of the counting (5, 6, 1, 2, 3) and will become 3 when we have reached the last element (5, 6, 1, 2, 3) because the Increasing row 1, 2, 3 has 3 elements. We will use **count_longest** to keep the longest increasing sequence. The other variables we will use are **a** – the number we are at at the moment, and **a_prev** – the previous number, which we will compare with **a** to find out whether the row is growing.

We begin to run the numbers and check if the present number **a** is larger than the previous **a_prev**. If this is true, then the row **is growing**, and we need to increase its number by **1**. This is stored in the variable that tracks the length of the sequence we are currently in – **count_current_longest**. If the number **a** isn't bigger than the previous one, it means that a new sequence starts, and we have to start the count from **1**. Finally, after all the checks are done, **a_prev** becomes the number that we're currently using, and we start the loop from the beginning with the next entered **a**.

Here is a sample implementation of the algorithm described:

```
for i in range(0, n):
    a = int(input())

    if a > a_prev:
        count_current_longest += 1
    else:
        count_current_longest = 1

    a_prev = a
```

What remains is to see which of all sequences is **the longest**. We will do this by checking in the loop if the sequence we are currently in has become longer than the longest one by now. The whole loop will look like this:

```
for i in range(0, n):
    a = int(input())

    if a > a_prev:
        count_current_longest += 1
    else:
        count_current_longest = 1

    if count_current_longest > count_longest:
        count_longest = count_current_longest

a_prev = a
```

Finally, we print the length of the longest sequence found:

```
print(count_longest)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#7>.

Problems for Drawing Figures on The Console

The fifth problem of the “Programming Basics” Practical Exam requires using one or several nested loops for drawing a figure on the console. Logical reasoning, simple calculations, and conditional statements may be required. The problem tests the ability of students to think logically and invent simple algorithms for solving problems, i.e. to think algorithmically. Here are some examples of Exam Problems:

Problem: Perfect Diamond

Write a function that takes as a parameter **n** and draws a perfect diamond with size **n** as in the examples below.

Input Data

The input is an integer **n** in the interval [1... 1000].

Output Data

The diamond should be printed on the console as in the examples below.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
2	<pre> * *-*</pre>	3	<pre> * *-* *-**</pre>	5	<pre> * *-* *-** *-*** *-**** *-***** </pre>	4	<pre> * *-* *-** *-*** *-****</pre>

Hints and Guidelines

In tasks for drawing figures, the most important thing to consider is **the sequence** in which we will draw. Which elements are **repeated** and with what **steps**? We can see that **the top and the bottom** parts of the diamond are **the same**. The easiest way to solve the problem is by creating **a loop** that draws **the upper part**, and then **another loop** that draws **the bottom part** (opposite to the top one).

First, we read the number **n** from the console:

```
n = int(input())
```

We start drawing **the upper half** of the diamond. We see that **each line** starts with some **whitespaces** and *****. If we take a closer look, we will notice that **the whitespaces** are always equal to **n - index of row - 1** (the first row is n-1, the second - n-2, etc.). We will start by drawing the number of **whitespaces** and **the first asterisk**. Let's not forget to use **print(..., end='')** instead of just **print(...)** to stay on **the same line**. At the end of the line, we write **print ()** to move to a **new line**. Notice that we start counting from **1**, not **0**. Then all we have to do is add **-*** a few times to **complete the line**.

Here is the complete code for drawing **the upper part** of the diamond:

```
for i in range(1, n + 1):
    print((' ' * (n - i)) + '*', end='')

    # TODO: Draw the rest of the line

    print()
```

What remains is to **complete each line** with the required number of **-*** elements. On each row we have to add **i - 1** such **items** (on the first 1-1 -> 0, the second -> 1, etc.)

Here is the complete code for drawing **the upper part** of the diamond:

```
for i in range(1, n + 1):
    print((' ' * (n - i)) + '*', end='')

    for j in range(0, i - 1):
        print('-', end='')

    print()
```

To draw the bottom part of the diamond, we have to reverse the upper part. We'll count from $n - 2$ because if we start from $n - 1$, we will draw the middle row twice. Be sure to add **reversed (...)** to **range (...)** of the main loop.

Here is the code for drawing the bottom part of the diamond:

```
for i in reversed(range(1, n)):
    print(" " * (n - i) + "*", end='')

    for j in range(1, i):
        print('-', end='')

    print()
```

What remains is to assemble the whole program by first reading the input, printing the top part of the diamond, and then the bottom part of the diamond.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#8>.

Problem: Rectangle with Stars

Write a function that takes as a parameter an integer n and draws a rectangle with size n with 2 asterisks in the center as in the examples below.

Input Data

The parameter is an integer n in the range $[2 \dots 1000]$.

Output Data

The rectangle should be printed on the console as in the examples below.

Sample Input and Output

Input	Output	Input	Output	Input	Output	Input	Output
2	%% %*% %%	3	%% % % % ** % % % %%	4	%% % % % ** % % % %% %%	5	%% % % % % % ** % % % % % %%

Hints and Guidelines

We read the input data from the task:

```
n = int(input())
```

The first thing we can easily notice is that the first and the last rows contain $2 * n$ symbols `%`. We will start with this and then draw the middle part of the rectangle:

```
print('%' * (2 * n))
```

```
# TODO: Draw middle of the rectangle
```

```
print('%' * (2 * n))
```

From the examples, we see that the middle part of the figure always has an odd number of rows. Note that when an even number is set, the number of rows is equal to the previous odd number (2 -> 1, 4 -> 3, etc.). We create a variable that represents the number of rows that our rectangle will have, and correct it if the number `n` is even. Then we will draw a rectangle without the asterisks. Each row has for the beginning and at the end the symbol `%` and between them $2 * n - 2$ whitespaces (the width is $2 * n$ and we subtract 2 for the two percent at the end). Do not forget to move the code or the last line after the loop:

```
num_rows = n

if n % 2 == 0:
    num_rows -= 1

for i in range(0, num_rows):
    print('%', end='')

    # TODO: Place stars and spaces

    print('%', end='')
```

We can start and test the code so far. Everything without the two asterisks in the middle should work correctly.

Now, in the **body** of the loop let's add the **asterisks**. We'll check if we're on the **middle row**. If we are in the middle, (we will check this by comparing **i** with **floor(num_rows / 2)**), we will draw **the row together with the asterisks**, if not – we will draw **a normal row**. Remember to add **from the math import floor** at the beginning of the work file so that we can use the **floor(...)** function. The line with the asterisks has **n-2 whitespaces** (**n** is half the length and we remove the asterisk and the percentage), **two asterisks**, and again **n-2 whitespaces**. We leave out of the check the two percent at the beginning and the end of the row:

```
for i in range(0, num_rows):
    print('%', end='')

    if i == floor(num_rows / 2):
        print((' ' * (n - 2)) + '**' + (' ' * (n - 2)))
    else:
        print(' ' * (n * 2 - 2))

    print('%', end='')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#9>.

Problems with Nested Loops

The last (sixth) problem of the “Programming Basics” Practical Exam requires using of **several nested loops and more complex logic inside them**. The problems examine participants' ability to think algorithmically and to solve non-trivial coding problems that require nested loops. Here are some examples of exam problems.

Problem: Increasing 4 Numbers

For a given pair of numbers, **a** and **b** generate all four numbers **n1, n2, n3, n4**, for which $a \leq n1 < n2 < n3 < n4 \leq b$.

Input Data

The input contains two integers **a** and **b** in the interval **[0... 1000]**, one per line.

Output Data

The output contains all **numbers in batches of four**, in ascending order, one per line.

Sample Input and Output

Input	Output
3	3 4 5 6
7	3 4 5 7
	3 4 6 7
	3 5 6 7
	4 5 6 7

Input	Output
5	No
7	

Input	Output
10	10 11 12 13
13	

Hints and Guidelines

We read the input data from the console. We also create the additional variable **count**, which will keep track of whether **an existing number of ranges**:

```
a = int(input())
b = int(input())
```

```
count = 0
```

We will most easily solve the problem if we logically divide it **into parts**. If we are required to draw all the rows from a number between **a** and **b**, we will do it using **one loop** that takes all the numbers from **a** to **b**. Let's think about how to do this with a **series of two numbers**. The answer is easy – we will use **nested loops**:

```
for i in range(a, b + 1):
    for j in range(i + 1, b + 1):
        print('%d %d' % (i, j))
```

We can test the incomplete program to see if it's accurate so far. It must print all pairs of numbers **i, j** for which **$i \leq j$** .

Since each **next number** of the row must be **greater** than **the previous one**, the second loop will run around **$i + 1$** (the next greater number). Accordingly, if **there is no sequence** of two incremental numbers (**a** and **b** are equal), the second loop **will not be fulfilled**, and nothing will be printed on the console.

Similarly, what remains is to implement **the nested loops** for four numbers. We will add an **increase of the counter (count)** that we initialized to know if **there is such a sequence**:

```
for i in range(a, b + 1):
    for j in range(i + 1, b + 1):
        for k in range(j + 1, b + 1):
            for l in range(k + 1, b + 1):
                print('%d %d %d %d' % (i, j, k, l))
                count += 1
```

Finally, we will check if **the counter** is equal to **0** and we will print **"No"** on the console accordingly, if so.

```
if count == 0:
    print('No')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#10>.

Problem: Generating Rectangles

By a given number **n** and a **minimum area m**, generate all possible rectangles with integer coordinates in the range $[-n \dots n]$ with an area of at least **m**. The generated rectangles must be printed in the following format:

(left, top) (right, bottom) -> area

Rectangles are defined using the top left and bottom right corners. The following inequalities are in effect:

- $-n \leq \text{left} < \text{right} \leq n$
- $-n \leq \text{top} < \text{bottom} \leq n$

Input Data

Two numbers are entered from the console, one per line:

- Integer **n** in the range $[1 \dots 100]$ – sets the minimum and maximum coordinate of a vertex.
- Integer **m** in the interval $[0 \dots 50\,000]$ – sets the minimum area of the generated rectangles.

Output Data

- The described rectangles should be printed on the console in a format such as in the examples below.
- If there are no rectangles for the specified **n** and **m**, then print “No”.
- The order of rectangles in the output is not important.

Sample Input and Output

Input	Output	Input	Output	Input	Output
2 17	No	3 36	(-3, -3) (3, 3) -> 36	1 2	(-1, -1) (0, 1) -> 2 (-1, -1) (1, 0) -> 2 (-1, -1) (1, 1) -> 4 (-1, 0) (1, 1) -> 2 (0, -1) (1, 1) -> 2

Hints and Guidelines

Let's read the input from the console. Again, we will create a **counter** in which we will store the number of rectangles found:

```
n = int(input())
m = int(input())

count = 0
```

It is very important to be able to imagine the problem before we begin to solve it. In our case, it is required to search for rectangles in a coordinate system. The thing we know is that the **left point** will always have the coordinate **x**, smaller than the **right** one. Accordingly, the **upper one** will always have a smaller **y** coordinate than the **lower one**. To find all the rectangles, we'll have to create a **loop** similar to the previous problem, but this time, **not every next loop** will start from the next number because some of the **coordinates** can be equal (for example **left** and **top**):

```
for left in range(-n, n):
    for top in range(-n, n):
        for right in range(left + 1, n + 1):
            for bottom in range(top + 1, n + 1):
                # TODO
```

With the variables **left** and **right** we will follow the coordinates along the **horizontal**, and with **top** and **bottom** – along the **vertical**. The important thing here is knowing the corresponding coordinates so we can correctly calculate the sides of the rectangle. Now we have to find the **area of the rectangle** and check if it is **greater than or equal to m**. One side will be the **difference between left and right** and the other one – between **top and bottom**. Since the coordinates may be eventually interchanged, we will use **absolute values**. Again, we add the **counter** in the loop, counting **only the rectangles** we write. It is important to note that the writing order is **left, top, right, bottom**, as it is set in the problem's description:

```
for left in range(-n, n):
    for top in range(-n, n):
        for right in range(left + 1, n + 1):
            for bottom in range(top + 1, n + 1):
                area = abs(right - left) * abs(bottom - top)

                if area >= m:
                    print('%d, %d (%d, %d) -> %d'
                          % (left, top, right, bottom, area))

                    count += 1
```

Finally, we print "No", if there are no such rectangles.

```
if count == 0:
    print('No')
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1059#11>.

Chapter 8.2. Practical Exam Preparation – Part II

In the current chapter, we will review a **practical exam in Programming Basics**, conducted at SoftUni on December 18, 2016. The problems will give you a good overview of what you can expect at an admission exam in Programming at SoftUni. The exam covers the material studied in the current book.

Traditionally, the admission exam at SoftUni consists of **6 practical programming problems**:

- Simple problems (no conditions).
- A problem with a single condition.
- A problem with more complex conditions.
- A problem with a single loop.
- A problem with nested loops (drawing a figure on the console).
- A problem with nested loops and more complex logic.

Let's examine a **real exam topic**, the problems it contains, and their solutions.

Problem: Distance

Write a program that calculates **what is the distance passed by a car (in kilometers)**, if we know **the initial speed (km/h)**, **the initial time frame in minutes**, then the **speed is increased by 10%**, **the second time frame**, then the **speed is decreased by 5%**, and the **time until the end of the trip**. To calculate the distance, you need to **convert the minutes into hours** (for example 70 minutes = 1.1666 hours).

Input Data

Read **4 lines** from the console:

- The **start speed in km/h** – an integer within the range [1 ... 300].
- The **first time in minutes** – an integer within the range [1 ... 1000].
- The **second time in minutes** – an integer within the range [1 ... 1000].
- The **third time in minutes** – an integer within the range [1 ... 1000].

Output Data

Print a number on the console: **the kilometers passed**, formatted up to the **second digit after the decimal point**.

Sample Input and Output

Input	Output	Comments
90 60 70 80	330.90	Distance with initial speed: 90 km/h * 1 hour (60 min) = 90 km After speed increase: 90 + 10% = 99.00 km/h * 1.166 hours (70 min) = 115.50 km After speed decrease: 99 - 5% = 94.05 km/h * 1.33 hours (80 min) = 125.40 km Total number of km passed: 330.9 km

Input	Output	Comments
140 112 75 190	917.12	Distance with initial speed: 140 km/h * 1.86 hours (112 min) = 261.33 km After speed increase: 140 + 10% = 154.00 km/h * 1.25 hours (75 min) = 192.5 km After speed decrease: 154.00 - 5% = 146.29 km/h * 3.16 hours (190 min) = 463.28 km Total number of km passed: 917.1166 km

Hints and Guidelines

Such a description may look **misleading** and incomplete at first glance, which **adds** to the **complexity** of a relatively easy task. Let's **separate** the problem into a few **sub-problems** and try to **solve** each of them one by one, which will lead us to the final result:

- **Receiving** the input data.
- **Execution** of the main programming logic.
- **Calculation** and shaping up the final result.

The **main** part of the programming logic is to calculate what will be the **distance passed after all speed changes**. As during **execution** of the program, part of the data that we have is modified, we could **separate** the program code into a few logically separated parts:

- **Calculation** of the **distance** passed with initial speed.
- Change of **speed** and calculation of the **distance** passed.
- Last change of **speed** and **calculation**.
- **Summing up**.

On condition for **input** will be submitted **four** separate rows, for this reason, we will have to call the function **input()** four times:


```
initial_speed_string = input()
# first_interval_string = TODO
# second_interval_string = TODO
# third_interval_string = TODO
```

Next, we have to **convert the input data** to a suitable **type** so we can do the needed computations. We choose type **int** because in the description of the problem is mentioned that the input data is going to be in a **determined interval**, for which this data type is enough. We perform the data **conversion** as follows:

```
initial_speed = int(initial_speed_string)
# first_interval = TODO
# second_interval = TODO
# third_interval = TODO
```

In this way, we solved successfully the **first sub-problem** – receiving the input data.

We initially **store** one **variable** that will be used multiple times. This centralization approach gives us **flexibility** and the **possibility** to **modify** the final result of the program with minimum effort. In case we need to change the value, we must do it in **only one place in the code**, which saves us time and effort:

```
minutes_per_hour = 60
```



Avoiding repetitive code (centralization of the program logic) in the tasks that we examine in the present book may look unnecessary at first glance, but this approach is very important upon building large applications in a real work environment, and its exercising in an initial stage of training will help you build a quality programming style.

We calculate the **travel time** (in hours) by **dividing the time by 60** (minutes in an hour). The **travel distance** is calculated by **multiplying the starting speed by the time passed** (in hours). After that, we change the speed by increasing it by **10%**(on condition), as per the task description. Calculating the **percentage**, as well as the following **distances** passed, is done in the following way:

- The **time frame** (in hours) is calculated by **dividing** the provided time frame in minutes by the minutes that are contained in an hour (60).
- The **distance passed** is calculated by **multiplying** the time frame (in hours) by the speed that is obtained after the increase.
- The next step is to **decrease the speed** by **5%**, as per the problem description.
- We calculate the **remaining distance** in the manner described in the first two points.

```
first_interval_hours = first_interval / minutes_per_hour
first_distance = initial_speed * first_interval_hours

speed_after_increase = initial_speed + ((initial_speed * 10) / 100)
second_interval_hours = second_interval / minutes_per_hour
second_distance = speed_after_increase * second_interval_hours
```

Up until now, we were able to **solve two** of the **most important sub-problems**, namely the **receiving data input** and **their processing**. What remains is to **calculate the final result**. As by the description, we are required to **format it** up to **2** symbols after the decimal point, we can do this in the following **manner**:

```
final_distance = \
    first_distance + second_distance + third_distance
final_result = "{:.2f}".format(final_distance)
print(final_result)
```

In case you worked accurately and wrote the program using the input data given in the task description, you will be convinced that it works properly.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1060#0>.

Problem: Change Tiles

Haralambi has some **savings** that he wants to use to **change the tiles** on the bathroom floor. The **floor is rectangular**, and the **tiles are triangular**. Write a program that **calculates** if his **savings will be sufficient**. The **width and length of the floor** are submitted, as well as **one of the sides of the triangle with its height towards it**. We must **calculate how many tiles are needed**, to cover the floor. The **number of tiles must be rounded up to the higher integer** and **5 more tiles must be added** as spare tiles. Also, we have submitted – the **price per tile** and the **amount paid for the work of a workman**.

Input Data

Read **7 lines** from the console:

- The **savings**.
- The **width** of the floor.
- The **length** of the floor.
- The **side** of the triangle.
- The **height** of the triangle.
- The **price** of a tile.
- The **sum** for the workman.

All numbers are real numbers within the range [0.00 ... 5000.00].

Output Data

In this problem, our currency will be **lv**, which is BGN (Bulgarian lev). The following must be printed on the console as a **single line**:

- If the money **is sufficient**:
 - "{Remaining funds} left."
- If the money **IS NOT sufficient**:
 - "You'll need {Insufficient funds} lv more."

The result must be **formatted up to the second symbol** after the decimal point.

Sample Input and Output

Input	Output	Comments
1000 5.55 8.95 0.90 0.85 13.99 321	You'll need 1209.65 lv more.	Floor area $\rightarrow 5.55 * 8.95 = 49.67249$ Tile area $\rightarrow 0.9 * 0.85 / 2 = 0.3825$ Needed tiles $\rightarrow 49.67249 / 0.3825 = 129.86... = 130 + 5$ spare tiles = 135 Total amount $\rightarrow 135 * 13.99 + 321$ (workman) = 2209.65 $2209.65 > 1000 \rightarrow 1209.65$ lv are insufficient

Input	Output	Comments
500 3 2.5 0.5 0.7 7.80 100	25.60 lv left.	Floor area $\rightarrow 3 * 2.5 = 7.5$ Tile area $\rightarrow 0.5 * 0.7 / 2 = 0.175$ Needed tiles $\rightarrow 7.5 / 0.175 = 42.857... = 43 + 5$ spare tiles = 48 Total amount $\rightarrow 48 * 7.8 + 100$ (workman) = 474.4 $474.4 < 500 \rightarrow 25.60$ lv left

Hints and Guidelines

The following task requires our problem to accept more input data and to perform a larger amount of calculations, even though the solution is **identical**. Accepting the input data is done **familiarly**. Pay attention that in the **input** part of the description is said that all input data will be **real numbers** and for this reason, we will use **float** type.

Now that we already have everything for executing the programming logic, we can move to the following part. How can we **calculate** what is the **needed** number of tiles that will be sufficient to cover the entire floor? The requirement is that tiles have a **triangular** shape, which can confuse, but practically, the task needs just **simple calculations**. We can calculate

the **common part of the floor** by the formula for finding a rectangle area, as well as the **area of a single tile** using the relevant formula for the triangle area.

To calculate the **number of tiles** that are needed, **we divide the floor area by the area of a single tile** (we should not forget to add the 5 additional tiles, that were mentioned in the requirements).



Pay attention that the requirements state that we should round up the number of tiles, obtained upon the division, up to the higher number, and then we should add 5. Find more information about the system functionality that does that: `math.ceil(...)`.

We can find the final result by **calculating the total amount** that is needed to cover the entire floor, by **summing up the tile price and the price that will be paid to the workman**, that we have from the input data. We can figure out that **the total costs** for tiles can be calculated by **multiplying the number of tiles by the price per tile**. We will find out whether the amount that we have will be sufficient by comparing the savings (based on the input data) and the total costs:

```
if budget >= total_cost:
    # TODO: Print message
else:
    # TODO: Print message
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1060#1>.

Problem: Flowers

A flowers shop offers 3 types of flowers: **chrysanthemums**, **roses**, and **tulips**. The prices depend on the season. In this problem, our currency will be **lv**, which is BGN (Bulgarian lev).

Season	Chrysanthemums	Roses	Tulips
spring / summer	2.00 lv./pc.	4.10 lv./pc.	2.50 lv./pc.
autumn / winter	3.75 lv./pc.	4.50 lv./pc.	4.15 lv./pc.

On holidays, the prices of all flowers are **increased by 15%**. The following **discounts** are offered:

- For purchasing more than 7 tulips in spring – **5% of the price** of the whole bouquet.
- For purchasing 10 or more roses in winter – **10% of the price** of the whole bouquet.
- For purchasing more than 20 flowers in total in any season – **20% of the price** of the whole bouquet.

Discounts are made in the above-described order and can be combined! All discounts are valid after increasing the price on a holiday!

The price for arranging a bouquet is always **2 lv** (BGN). Write a program that calculates the price of a bouquet.

Input Data

The input is read from the **console** and contains **exactly 5 arguments**:

- The number of the purchased **chrysanthemums** – integer number inside the interval of **[0 ... 200]**.
- The number of the purchased **roses** – integer number inside the interval of **[0 ... 200]**.
- The number of the purchased **tulips** – integer number inside the interval of **[0 ... 200]**.
- The season – **[Spring, Summer, Autumn, Winter]**.
- If the day is a holiday – **[Y – yes / N – no]**.

Output Data

Print on the console 1 number – **the price of flowers**, formatted up to the second digit after the decimal point.

Sample Input and Output

Input	Output	Comments
2 4 8 Spring Y	46.14	Price: $2 \cdot 2.00 + 4 \cdot 4.10 + 8 \cdot 2.50 = 40.40$ lv Holiday: $40.40 + 15\% = 46.46$ lv 5% discount for more than 7 tulips in spring: 44.14lv The flowers are in total 20 or less: no discount $44.14 + 2$ for arranging the bouquet = 46.14 lv

Input	Output	Comments
3 10 9 Winter N	69.39	Price: $3 \cdot 3.75 + 10 \cdot 4.50 + 9 \cdot 4.15 = 93.60$ lv Not a holiday: no increase in price 10% discount for 10 or more roses in winter: 84.24lv Total flowers more than 20: 20% discount = 67.39lv $67.39 + 2$ for arranging the bouquet = 69.39 lv

Input	Output
10	101.20
10	
10	
Autumn	
N	

Hints and Guidelines

After carefully reading the requirements, we understand that once again we need to do **simple calculations**, however, this time we will need **additional** logical **conditions**. We need to pay more **attention** to the moment of **making changes** in the final price, to be able to properly build the logic of our program. Again, the bold text gives us sufficient **guidelines** on how to proceed. For a start, we will separate the already **defined** values in **variables**, as we did in the previous tasks:

```
# Initial price list
rose_autumn_winter_price = 4.50
rose_spring_summer_price = 4.10
tulip_autumn_winter_price = 4.15
tulip_spring_summer_price = 2.50
chrysantemum_autumn_winter_price = 3.75
chrysantemum_spring_summer_price = 2.00
arrange_price = 2.00
```

We will also do the same for the rest of the defined values:

```
# Price increases
price_increase_percentage = 15

# Price decreases
tulip_price_decrease_percentage = 5
rose_price_decrease_percentage = 10
total_price_decrease_percentage = 20

# Price decrease thresholds
tulip_price_decrease_threshold = 7
rose_price_decrease_threshold = 10
total_price_decrease_threshold = 20
```

Our next sub-task is to **read** properly the **input** data from the console. We will do that in an already familiar manner but this way **we combine two** separate functions – one to **read** a line from the console and another to **convert** in a numeric data type:

```
chrysantemums_purchased = int(input())
# roses_purchased = TODO
# tulips_purchased = TODO
# season = TODO
# is_special_day = TODO
```

Let's think of the most appropriate way to **structure** our programming logic. By the requirements, it becomes clear that the path of the program is divided mainly into two parts: **spring / summer** and **autumn / winter**. We can do the separation by a conditional statement, by storing variables in advance for the **prices** of the individual flowers, as well as for the **final** result:

```
if season == 'Winter' or season == 'Autumn':
    roses_price = roses_purchased * rose_autumn_winter_price
    chrysantemums_price = chrysantemums_purchased * \
        chrysantemum_autumn_winter_price
    tulips_price = tulips_purchased * tulip_autumn_winter_price
    total_costs = roses_price + chrysantemums_price + tulips_price
else:
    # TODO
```

What remains is to perform a **few checks** regarding the **discounts** of the different types of flowers, depending on the season, and to modify the final result.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1060#2>.

Problem: Grades

Write a program that **calculates statistics for grades** in an exam. In the beginning, the program receives the **number of students** who attended the exam and for **each student** – **their grade**. In the end, the program must **print the percentage of students** that have grades between 2.00 and 2.99, between 3.00 and 3.99, between 4.00 and 4.99, 5.00 or more, as well as the **average grade** of the exam.

Input Data

On the console are being read a **sequence of numbers**, each on a different row:

- On the first line – **the number of students who attended the exam** – an integer within the range [1 ... 1000].
- For **each student** on a separate line – **the grade on the exam** – a real number within the range [2.00 ... 6.00].

Output Data

Print on the console **5 lines** that hold the following information:

- “Top students: {percentage of students with a grade of 5.00 or more}%”.
- “Between 4.00 and 4.99: {between 4.00 and 4.99 included}%”.
- “Between 3.00 and 3.99: {between 3.00 and 3.99 included}%”.
- “Fail: {less than 3.00}%”.
- “Average: {average grade}%”.

The results must be **formatted up to the second symbol** after the decimal point.

Sample Input and Output

Input	Output	Comments
10		
3.00		5 or more: 3 students = 30% of 10
2.99		Between 4.00 and 4.99 – 3 students = 30% of 10
5.68	Top students: 30.00%	
3.01	Between 4.00 and 4.99: 30.00%	Between 3.00 and 3.99 – 2 students = 20% of 10
4	Between 3.00 and 3.99: 20.00%	
4	Fail: 20.00%	Under 3 – 2 students = 20% of 10
6.00	Average: 4.06	The average grade is: 3 + 2.99 + 5.68 + 3.01 + 4 + 4 + 6 + 4.50 + 2.44 + 5 = 40.62 / 10 = 4.06
4.50		
2.44		
5		

Input	Output
6	
2	Top students: 33.33%
3	Between 4.00 and 4.99: 16.67%
4	Between 3.00 and 3.99: 16.67%
5	Fail: 33.33%
6	Average: 3.70
2.2	

Hints and Guidelines

By requirements, we see that **first**, we will receive **the number** of students, and then, **their grades**. For that reason, **first**, we will obtain **the number** of students, as we will convert the

value we have read to **int**. To process the grades themselves, we will use a **for** loop. Every iteration of the loop will read and process one grade:

```
number_of_students = int(input())
for i in range(number_of_students):
    # TODO
```

Before executing the code of the **for** loop, we will create variables where we will store **the number of students** for each group: poor results (up to 2.99), results from 3 to 3.99, from 4 to 4.99, and grades above 5. We will also need one more variable, where we will store **the sum of all grades**, via which we will calculate the average grade of all students:

```
number_of_failed_students = 0
number_of_average_students = 0
number_of_good_students = 0
number_of_excellent_students = 0
total_result = 0
```

We run the **loop** and inside it, we **declare one more** variable, in which we will store the **currently** entered grade. The variable will be of the type **float** and on every iteration, we check **what is its value**. According to this value, **we increase** the number of students in the relevant group by **1**, as we should not forget to also increase the **total** amount of the grades, which we also track:

```
for i in range(number_of_students):
    grade = float(input())
    total_result += grade
    if grade < 3:
        number_of_failed_students += 1
    else:
        # TODO: check other groups
```

We can calculate what **percentage** is taken by a particular **group of students** from the total number by **multiplying the number of students** in the relevant group by **100** and then dividing this by the **total number of students**.

The **final result** is formed in a well-known way **up to the second symbol** after the decimal point.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1060#3>.

Problem: Christmas Hat

Write a program that reads from the console an **integer n** and draws a **Christmas hat** with a width of $4 * n + 1$ columns and a height of $2 * n + 5$ rows, as in the examples below.

Input Data

The input is read from the console – one **integer n** within the range [3 ... 100].

Output Data

Print on the console a **Christmas hat**, exactly like in the examples.

Sample Input and Output

Input	Output	Input	Output
4	<pre>...../ \.....\ /..... *** *-*-* *-*-* *-*-* *-*-* *-*-* *-*-* *-----* ***** *.*.*.*.*.*.*.*.* *****</pre>	7	<pre>...../ \.....\ /..... *** *-*-* *-*-* *-*-* *-*-* *-*-* *-*-* *-----* ***** *.*.*.*.*.*.*.*.* *****</pre>

Hints and Guidelines

In tasks requiring **drawing** on the console, most often the user inputs **an integer** that is related to the **total size of the figure** that we need to draw. As the task requirements mention how the total length and width of the figure are calculated, we can use them as **starting points**. In the examples, it is clear that regardless of the input data, we always have the **first two rows** that are almost identical.

```
...../|\.....
.....\|/.....
```

We also notice that the **last three rows** are always present, as **two** of them are completely the same.

```
*****
*.*.*.*.*.*.*.*
*****
```

By these observations, we can come up with the **formula** for the **height of the variable part** of the Christmas hat. We use the formula specified in the task to calculate the total height, by subtracting the size of the unchangeable part. We obtain $(2 * n + 5) - 5$ or $2 * n$.

To **draw** the **dynamic** or the variable part of the figure, we will use a **loop**. The size of the loop will be from **0** to the **width** that we have by requirements, namely $4 * n + 1$. Since we will use this formula in **a few places** in the code, it is a good practice to declare it in a **separate variable**. Before running the loop, we should **declare variables** for the **number** of individual symbols that participate in the dynamic part: **dots** and **dashes**. By analyzing examples, we can also prepare formulas for the **starting values** of these variables. Initially, the **dashes** are **0**, but we can calculate the number of **dots** by subtracting **3** from the **total width** (the number of symbols that are building the top of the Christmas hat) and then **dividing by 2**, as the number of dots on both sides of the hat is the same.

```
.....***.....
.....*-*-*.....
.....*--*--*.....
.....*---*---*.....
.....*----*----*.....
.....*-----*-----*.....
.....*-----*-----*.....
.....*-----*-----*.....
.....*-----*-----*.....
```

What remains is to execute the body of the loop, as **after each** drawing we **decrease** the number of dots by **1** and **increase the number of dashes** by **1**. Let's not forget to draw one **star** between each of them. The sequence of drawing in the body of the loop is the following:

- Symbol string of dots
- Star
- Symbol string of dashes
- Star
- Symbol string of dashes
- Star
- Symbol string of dots

In case we have worked properly, we will obtain figures identical to those in the examples.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1060#4>.

Problem: Letters Combinations

Write a program that prints on the console **all combinations of 3 letters** within a specified range, by skipping the combinations **containing a certain letter**. Finally, print the number of printed combinations.

Input Data

The input is read from the console and contains **exactly 3 lines**:

- A small letter from the English alphabet – between ‘a’ and ‘z’.
- A small letter from the English alphabet – between the **first letter** and ‘z’.
- A small letter from the English alphabet – from ‘a’ to ‘z’ – as the combinations containing this letter are skipped.

Output Data

Print on one line **all combinations** corresponding to the requirements, followed by **their number**, separated by a space.

Sample Input and Output

Input	Output	Comments
a c b	aaa aac aca acc caa cac cca ccc 8	All possible combinations with letters ,a‘ ,b‘ and ,c‘ are: aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca bcb bcc caa cab cac cba cbb cbc cca ccb ccc The combinations containing ,b‘ are not valid. 8 valid combinations remain.

Input	Output
a c z	aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca bcb bcc caa cab cac cba cbb cbc cca ccb ccc 27

Input	Output
f k h	fff ffg ffi ffj ffk fgf fgg fgi fgj fgk fif fig fii fij fik fjf fjg fji fjf fjk fkf fkg fki fkj fkk gff gfg gfi gfj gfk ggf ggg ggi ggj ggk gif gig gii gjj gik gjf gjg gji gjj gjk gkf gkg gki gkj gkk iff ifg ifi ifj ifk igf igg igi igj igk iif iig iii iij iik ijf ijg iji ijj ijk ikf ikg iki ikj ikk jff jfg jfi jfj jfk jgf jgg jgi jgj jgk jif jig jii jij jik jjf jgg jji jjj jjk jkf jkg jki jkj jkk kff kfg kfi kfj kfk kgf kgg kgi kgj kgk kif kig kii kij kik kjf kjg kji kjj kjk kkf kkg kki kkj kkk 125

Hints and Guidelines

By requirements, we have input data on **3 rows**, each of which is represented by one character of the **ASCII table** (<https://www.asciitable.com/>). We could use the already defined function in Python, **ord(...)** through which we will receive the ASCII code of the symbol:

```
start_letter = ord(input()[0])
# TODO
```

Let's think of how we can achieve the **final result**. In case the task requirement is to print all characters, from the starting to the end one (by skipping a particular letter), what should we do?

The easiest and most efficient way is to use a **loop**, by passing through **all characters** and printing those that are **different** from the **letter** that we need to skip. In Python, we can go around all the symbols from 'a' to 'z' the following way:

```
for i in range(ord('a'), ord('z') + 1):
    print(chr(i), end=' ')
```

The function **chr(...)** will convert the received ASCII code into a symbol. The result of running the code is all letters from **a** to **z** included, printed on a single line and separated by spaces. Does this look like the final result of our task? We must find a **way** to print **3 characters**, as required, instead of **1**. Running such a program very much looks like a slot machine. We often win in slots, if we arrange a few identical characters in a row. Let's say that the machine has space for three characters. When we **stop** on a particular **character** in the first place, the other two places will **continue** rolling characters among all possible ones. In our case, **all possible characters** are the letters from the starting to the end one, entered by the user, and the solution of our program is identical to the way a slot machine works.

We use a **loop** that runs through **all characters** from the starting to the end letter (included). On **each iteration** of the **first** loop, we run a **second** one with the same parameters (but **only if** the letter of the first loop is valid, i.e. does not match the one that we must exclude, by requirements). In each iteration of the **second** loop, we run **one** more with the **same parameters** and the same **condition**. This way we have three nested loops and in the body of **the latter** we will add the symbols to the final result:

```
for i in range(ord(start_letter), ord(end_letter) + 1):  
    if chr(i) != except_letter:  
        # TODO
```

Let's not forget that we also need to print the **total number of valid combinations** that we have found, and they must be printed on the **same line**, separated by a space. We leave this sub-task to the reader.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1060#5>.

Chapter 9.1. Problems for Champions – Part I

In this chapter, we will offer the reader a few **more difficult tasks**, that aim to develop **algorithmic skills** and acquire **programming techniques** to solve tasks of higher complexity.

More Complex Problems on The Studied Material

Together we will solve several programming problems that cover the material studied in the book, but are more difficult than the usual problems of the practical exams at SoftUni. If you want to **become a programming basics champion**, we recommend practicing solving such complex tasks to make it easier for you to take exams.

Problem: Crossing Sequences

We have two sequences:

- a **sequence of Tribonacci** (by analogy with the Fibonacci sequence), where each number is **the sum of the previous three** (with given three numbers)
- a sequence generated by a **numerical spiral**, defined by looping like a spiral (right, bottom, left, top, right, bottom, left, top, etc.) of a matrix of numbers starting from its center with a given starting number and incremental step, by storing the current numbers in the sequence each time we make a turn.

Write a program that finds the first number that appears **in both sequences** defined in an **aforementioned way**.

Problem

Let the **Tribonacci sequence** start with **1, 2 and 3**. This means that **the first sequence** will contain the numbers 1, 2, 3, 6, 11, 20, 37, 68, 125, 230, 423, 778, 1431, 2632, 4841, 8904, 16377, 30122, 55403, 101902, and so on.

At the same time, let the **numbers in the spiral** begin with **5**, and the spiral increases by **2** at each step.

Then **the second sequence** will contain the numbers 5, 7, 9, 13, 17, 23, 29, 37, and so on. We see that **37** is the first number to be found both in the Tribonacci sequence and in the spiral one, and that is the desired solution to the problem.

45	...	...	...	...	55
...	17	19	21	23	...
...	15	5	7	...	...
...	13	11	9	...	...
37	...	...	...	29	...
...	...	...	...	...	65

Input Data

Input data should be read from the console.

- On the first three lines of the input, we will read **three integers**, representing **the first three numbers** in the Tribonacci sequence.
- On the next two lines of the input, we will read **two integers**, representing **the first numbers** and **the step** for each cell of the matrix for the spiral of numbers.

Input data will always be valid and will always be in the format described. No need to check.

Output Data

The result should be printed on the console.

On the single line of the output, you must print **the lowest number that occurs in both sequences**. If there is no number in the **range [1 ... 1 000 000]**, which can be found in both sequences, print **“No”**.

Constraints

- All numbers in the input will be in the range [1 ... 1 000 000].
- Allowed program time: 1.5 seconds.
- Allowed memory: 16 MB.

Sample Input and Output

Input	Output
1	37
2	
3	
5	
2	

Input	Output
13	13
25	
99	
5	
2	

Input	Output
99	No
99	
99	
2	
2	

Input	Output
1	23
4	
7	
23	
3	

Hints and Guidelines

The problem seems quite complicated, so we will break it into simpler sub-problems: processing the input, generating Tribonacci sequence, generating numerical spiral, and sending the number for the sequences.

Processing The Input Data

The first step in solving the problem is to read and process the input. Input data consists of **5 integers**: **3** for the Tribonacci sequence and **2** for the numerical spiral:


```
tribonacci_first = int(input())
tribonacci_second = int(input())
tribonacci_third = int(input())
```

```
spiral_current = int(input())
spiral_step = int(input())
```

Once we have the input data, we need to think about how we will generate the numbers in the two sequences.

Generating Tribonacci Sequence

For the Tribonacci sequence, we will always **collect the previous three values** and then move the values of those numbers (the three previous ones) to one position in the sequence, i.e. the value of the first one must accept the value of the second one, and so on. When we are done with the number, we will store its value in **an array**. Since the problem description states that the numbers in the sequences do not exceed 1,000,000, we can stop generating this range at exactly 1,000,000.

```
tribonacci_numbers = [tribonacci_first,
                      tribonacci_second,
                      tribonacci_third]

tribonacci_current = tribonacci_third
while tribonacci_current < 1000000:
    tribonacci_current = tribonacci_first + \
                        tribonacci_second + \
                        tribonacci_third
    tribonacci_numbers.append(tribonacci_current)

    tribonacci_first = tribonacci_second
    tribonacci_second = tribonacci_third
    tribonacci_third = tribonacci_current
```

Generating Numerical Spiral

We need to think of **a relationship** between numbers in the numerical spiral so we can easily generate every next number without having to look at matrices and loop through them. If we carefully look at the picture from the description, we will notice that **every 2 “turns” in the spiral, the numbers we skip are increased by 1**, i.e. from 5 to 7 and from 7 to 9, not a single number is skipped, but we directly **add with the step** with the sequence. From 9 to 13 and from 13 to 17 we skip a number, i.e. we add the step twice. From 17 to 23 and from 23 to 29 we skip two numbers, i.e. we add the step three times and so on.

Thus, we see that for the first two we have the **last number + 1 * step**, the next two we add with the **2 * step**, and so on. Every time we want to get to the next number of the spiral, we will have to make similar calculations.

```
spiral_current += spiral_step * spiral_step_mul
```

What we have to take care of is for every two numbers, our multiplier (let's call it "coefficient") must increase by 1 (`spiral_step_mul++`), which can be achieved with a simple check (`spiral_count % 2 == 0`). The whole code from the generation of the spiral in an array is given below:

```
spiral_numbers = [spiral_current]
spiral_count = 0
spiral_step_mul = 1

while spiral_current < 1000000:
    spiral_current += spiral_step * spiral_step_mul
    spiral_numbers.append(spiral_current)
    spiral_count += 1

    if spiral_count % 2 == 0:
        spiral_step_mul += 1
```

Finding Common Number for The Sequences

Once we have generated the numbers in both sequences, we can proceed to unite them and build the final solution. How will it look? For **each of the numbers** in the first sequence (starting from the smaller one) we will check if it exists in the next one. The first number that meets this criteria will be **the answer** to the problem.

We will do a **linear** search in the second array, and we will leave the more curious participants to optimize it using the technique called **binary search** because the second array is generated in sorted form, i.e. it meets the requirement to apply this type of search. The code for finding our solution will look like this:

```
found = False
for i in range(0, len(tribonacci_numbers)):
    for j in range(0, len(spiral_numbers)):
        if tribonacci_numbers[i] == spiral_numbers[j] \
            and tribonacci_numbers[i] <= 1000000:
            print(tribonacci_numbers[i])
            found = True
            break
    if found:
        break

if not found:
    print("No")
```

The previous solution to the problem uses arrays to store the values. Arrays are not needed to solve the problem. There is an **alternative solution**, that generates the numbers and works

directly with them instead of keeping them in an array. On **every step**, we can check whether **the numbers in the two sequences match**. If this is the case, we will print the number on the console and terminate the execution of our program. Otherwise, we will see the current number of which sequence is the smaller one and we will generate the next one where we are “lagging”. The idea is that **we will generate numbers from the sequence that is “behind”** until we skip the current number of the other sequence and then vice versa, and if we find a match in the meantime, we will terminate the execution:

```
while tribonacci_current <= 1000000 and spiral_current <= 1000000:
    if tribonacci_current == spiral_current:
        print("TODO: Print and stop execution")
    elif tribonacci_current < spiral_current:
        print("TODO: Generate next Tribonacci number")
    else:
        print("TODO: Generate next Spiral number")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1061#0>.

Problem: Magic Dates

Date is given in a “dd-mm-yyyy” format, e.g. 17-04-2018. We calculate **the weight of that date**, by taking all of its digits, multiplying each digit with the ones after it, and finally summing up all the results obtained. In our case, we have 8 digits: 17032007, so the weight is $1*7 + 1*0 + 1*3 + 1*2 + 1*0 + 1*0 + 1*7 + 7*0 + 7*3 + 7*2 + 7*0 + 7*0 + 7*7 + 0*3 + 0*2 + 0*0 + 0*0 + 0*7 + 3*2 + 3*0 + 3*0 + 3*7 + 2*0 + 2*0 + 2*7 + 0*0 + 0*7 + 0*7 = 144$.

Our task is to write a program that finds all the **magical dates between two specific years (inclusively) corresponding to the given weight**. Dates must be printed in ascending order (by date) in the format “dd-mm-yyyy”. We will only use the valid dates in the traditional calendar (the leap years have 29 days in February).

Input Data

Input data should be read from the console. It consists of 3 lines:

- The first line contains an integer: **start year**.
- The second line contains an integer: **end year**.
- The third line contains an integer: **the search weight** for the dates.
- Input data will always be valid and will always be in the format described. No need to check.

Output Data

The result should be printed on the console as consecutive dates in **dd-mm-yyyy” format**”, sorted by date in ascending order. Each string must be in a separate line. If there are no existing magic dates, print “No”.

Constraints

- The start and final years are integer numbers in the range [1900 – 2100].
- Magic weight is an integer in the range [1 ... 1000].
- Allowed program time: 0.75 seconds.
- Allowed memory: 16 MB.

Sample Input and Output

Input	Output	2012 2014 80	Input	Output
2007	17-03-2007			
2007	13-07-2007			09-01-2013
144	31-07-2007			17-01-2013
				23-03-2013
Input	Output			11-07-2013
2003	No			01-09-2013
2004				10-09-2013
1500				09-10-2013
				17-10-2013
Input	Output			07-11-2013
2011	01-01-2011			24-11-2013
2012	10-01-2011			14-12-2013
14	01-10-2011			23-11-2014
	10-10-2011			13-12-2014
			31-12-2014	

Hints and Guidelines

We start with inserting the necessary functionality and reading the input from the console. In this case, we insert the type **date** (with new name **datatype**), the functions **timedelta** and **floor** and read 3 integers:

```
from datetime import date as datatype
from datetime import timedelta
from math import floor

first_year = int(input())
second_year = int(input())
number_to_search_for = int(input())
```

Having the start and the end year, it is nice to know how we will go through every date, not to worry about how many days there are in the month and whether it is a leap year, and so on.

Loop Through Dates

For looping through the dates, we will take advantage of the functionality that gives us the **date** type in **Python**. We will define a **start date variable**, that we can do using the constructor that accepts a year, month, and day. We know the year is the starting year we read from the console and the month and the day must be January and 1st respectively:

```
date = datetime(first_year, 1, 1)
```

Once we have the start date, we want to create a **loop that runs until we exceed the final year** (or until we pass December 31 in the final year if we compare the full dates), increasing each day by 1 day.

To increase by one day in each rotation, we will use the method **timedelta()** in **Python** for creating a 1-day difference, which will be added to **date**. Adding 1 day to the date will help **Python** to take care instead of us, to decide where to skip the next month, how many days there is a month, and everything around the leap years:

```
date = date + timedelta(days=1)
```

In the end, our loop may look like this:

```
while date.year <= second_year:
```

```
    # TODO: Check the date for number_to_search_for
```

```
    date = date + timedelta(days=1)
```

Calculating Date Weight

Each date consists of exactly 8 characters (digits) – 2 for the day (**d1**, **d2**), 2 for the month (**d3**, **d4**), and 4 for the year (**d5** to **d8**). This means that we will always have the same calculation every time, and we can benefit from this to **define the formula statically** (i.e. not to use loops, referring to different numbers from the date, but write the whole formula). To be able to write it, we will need **all digits from the date** in individual variables to make all the necessary multiplications. By using the division and partition operations on the individual components of the date, using the **day**, **month** and **year**, properties, we can retrieve each digit. We have to be careful with the integer division by 10 (**/ 10**), which won't be an integer division, that's why after each operation we have to round down to the lowest integer number by using **floor(...)**:

```

d1 = floor(date.day / 10)      # First day digit
d2 = date.day % 10            # Second day digit

d3 = floor(date.month / 10)    # First month digit
d4 = date.month % 10          # Second month digit

d5 = floor(date.year / 1000)   # First year digit
d6 = floor(date.year / 100) % 10 # Second year digit
d7 = floor(date.year / 10) % 10 # Third year digit
d8 = date.year % 10            # Fourth year digit

```

Let's also explain one of the more interesting lines here. Let's take the second digit of the year for example (**d6**). We divide the year by 100, and we take a remainder of 10. What do we do? First, we eliminate the last 2 digits of the year by dividing by 100 (Example: **2018 / 100 = 20**). With the remainder of 10, we take the last digit of the resulting number (**20 % 10 = 0**) and so we get 0, which is the second digit of 2018.

What remains is to do the calculation that will give us the magical weight of a given date. In order **not to write all multiplications**, as shown in the example, we will simply apply a **grouping**. What we need to do is multiply each digit with those that follow it. Instead of typing **d1 * d2 + d1 * d3 + ... + d1 * d8**, we can shorten this expression to **d1 * (d2 + d3 + ... + d8)**, for grouping when we have multiplication and summing up. Applying the same simplification for the other multiplications, we get the following formula:

```

weight = d1 * (d2 + d3 + d4 + d5 + d6 + d7 + d8) + \
          d2 * (d3 + d4 + d5 + d6 + d7 + d8) + \
          d3 * (d4 + d5 + d6 + d7 + d8) + \
          d4 * (d5 + d6 + d7 + d8) + \
          d5 * (d6 + d7 + d8) + \
          d6 * (d7 + d8) + \
          d7 * d8

```

Printing The Output

Once we have the weight calculated on a given date, we need **to check and see if it matches the magical weight we want**, to know if it should be printed or not. Checking can be done using a standard **if** operator, taking care to print the date in the correct format. Fortunately, we already have each one of the digits that we need to print from **d1** to **d8**. Here we have to be careful with the data types. Since the concatenation of strings and the summation are done by the same operator, we have to convert numbers to strings:

```

if weight == number_to_search_for:
    print(str(d1) + str(d2) + "-" + str(d3) + str(d4) +
          "-" + str(d5) + str(d6) + str(d7) + str(d8))
    found = True

```

Caution: as we go through the dates from the start of the year to the end of it, they will always be arranged in ascending order as per the description.

Finally, if we have not found an eligible date, we will have a **False** value in the **found** variable and we will be able to print **"No"**:

```
if not found:
    print("No")
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1061#1>.

Problem: Five Special Letters

Two numbers are given: **start** and **end**. Write a program that generates all combinations of 5 letters, each among the sets of {'a', 'b', 'c', 'd', 'e'}, so that the weight of these 5 letters is a number in the range [**start** ... **end**], inclusive. Print them in alphabetical order, in a single row, separated by a space.

The weight of the letters is calculated as follows:

```
weight('a') = 5;
weight('b') = -12;
weight('c') = 47;
weight('d') = 7;
weight('e') = -32;
```

The weight of the sequence of the letters **c1, c2, ..., cn** is calculated by removing all the letters that are repeated (from right to left) and then calculating the formula:

$$\text{weight}(c1, c2, \dots, cn) = 1 * \text{weight}(c1) + 2 * \text{weight}(c2) + \dots + n * \text{weight}(cn)$$

For example, the weight of **bcddc** is calculated as follows:

First we remove the repeating letters and get **bcd**. Then we apply the formula: $1 * \text{weight}('b') + 2 * \text{weight}('c') + 3 * \text{weight}('d') = 1 * (-12) + 2 * 47 + 3 * 7 = 103$.

Another example: $\text{weight}(\text{"cadae"}) = \text{weight}(\text{"cade"}) = 1 * 47 + 2 * 5 + 3 * 7 + 4 * (-32) = -50$.

Input Data

The input data is read from the console. It consists of two numbers:

- The number for **start** is on the first line.
- The number for **end** is on the second line.

Input data will always be valid and will always be in the format described. No need to check.

Output Data

The result should be printed on the console as a sequence of strings, **arranged in alphabetical order**. Each string must be separated from the next one by a single space. If the weight of any of the 5 letter strings does not exist within the specified range, print “No”.

Constraints

- Numbers for **start** and **end** are integers in the range [-10000 ... 10000].
- Allowed program time: 0.75 seconds.
- Allowed memory: 16 MB.

Sample Input and Output

Input	Output	Input	Output
-1 1	bcdea cebda eaaad eaada eaadd eaade eaaed eadaa eadad eadae eadda eaddd eaddde eadea eaded eadee eaead eaeda eaedd eaede eaeed eeaad eeada eeadd eeade eeaed eeead	200 300	baadc babdc badac badbc badca badcb badcc badcd baddc bbadc bbdac bdaac bdabc bdaca bdacb bdacc bdacd bdadc bdbac bddac beadc bedac eabdc ebadc ebdac edbac

Input	Output	Comment	Input	Output
40 42	bcead bdcea	weight(“bcead”) = 41 weight(“bdcea”) = 40	300 400	No

Hints and Guidelines

Like every problem, we start the solution by **Processing The Input Data**:

```
first_number = int(input())
second_number = int(input())
```

We have several main points in the problem – **generating all combinations** with a length of 5 including the 5 letters, **removing repeating letters**, and **calculating weight** for a simplified word. The answer will consist of every word whose weight is within the given range **[first_number, second_number]**.

Generating All Combinations

To generate **all combinations with a length of 1** using 5 symbols, we would use a **loop from 0..4**, as we want each number of the loop to match one character. To generate **all**

combinations with a length of 2 using 5 characters (i.e. “aa”, “ab”, “ac”, ..., “ba”, ...), we would create two nested loops each running through the digits from 0 to 4, as we will once again make sure that each digit matches a specific character. We will repeat this step 5 times, so we will finally have 5 nested loops with indexes **i1**, **i2**, **i3**, **i4**, and **i5**:

```
for i1 in range(0, 5):
    for i2 in range(0, 5):
        for i3 in range(0, 5):
            for i4 in range(0, 5):
                for i5 in range(0, 5):
```

Now that we have all 5-digit combinations, we must find a way to “turn” the five digits into a word with the letters from ‘a’ to ‘e’. One of the ways to do that is to **predefine a simple string that contains the letters**, that we have:

```
pattern = "abcde"
```

and for each digit, we take the letter from the particular position. This way the number 00000 will become “aaaaa”, the number 02423 will become “acecd”. We can create the 5-letter string in the following way:

```
full_word = pattern[i1] + \
            pattern[i2] + \
            pattern[i3] + \
            pattern[i4] + \
            pattern[i5]
```

Another solution: we can convert the digits to letters by using their arrangement in the **ASCII table**. The expression **chr(ord(‘a’) + i)** returns the result ‘a’ in case **I = 0**, ‘b’ in case **I = 1**, ‘c’ in case **I = 2**, etc.

This way we already have generated all 5-letter combinations and can proceed with the following part of the task.

Caution: as we have chosen a **pattern**, that takes into consideration the alphabetical arrangement of the letters, and cycles are run appropriately, the algorithm will generate the works in alphabetical order and there is no need for additional sorting before printing the output.

Removing Repetitive Letters

Once we have the finished string, we have to remove all the repeating symbols. We will do this by adding the letters from left to right in a new string and each time before adding a letter, we will check if it already exists – if it does, we will skip it and if it doesn’t, we will add it. To begin with, we will add the first letter to the starting string.:

```
word = pattern[i1]
```

Then we will do the same with the other 4, checking each time with the following condition and the **find(...)** method. This can be done with a loop by **full_word** (leaving it to the reader for exercise), and it can be done lazily by copy-paste.:

```
word = pattern[i1]

if word.find(pattern[i2]) == -1:
    word += pattern[i2]
if word.find(pattern[i3]) == -1:
    word += pattern[i3]
if word.find(pattern[i4]) == -1:
    word += pattern[i4]
if word.find(pattern[i5]) == -1:
    word += pattern[i5]
```

The **find(...)** method returns the index of the particular element if it is found or **-1** if the item is not found. Therefore, every time we get **-1**, it means that we still do not have this letter in the new string with unique letters and we can add it, and if we get a value other than **-1**, this will mean we already have the letter and we'll not add it.

Calculating Weight

Calculating the weight is simply going through the unique word (**word**), obtained in the last step, and for each letter, we need to take its weight and multiply it by the position. For each letter, we need to calculate what value we will multiply its position by, for example by using **if** constructions:

```
multiplier = 0
if word[i] == 'a':
    multiplier = 5
if word[i] == 'b':
    multiplier = -12
if word[i] == 'c':
    multiplier = 47
if word[i] == 'd':
    multiplier = 7
if word[i] == 'e':
    multiplier = -32
```

Once we have the value of that letter, we should **multiply it by its position**. Because the indexes in the string differ by 1 from the actual positions, i.e. index 0 is position 1, index 1 is position 2, etc., we will add 1 to the indexes.

```
weight += multiplier * (i + 1)
```

All intermediate results obtained must be added to the **total amount for each letter of the 5-letter combination**.

Preparing The Output

Whether a word needs to be printed is determined by its weight. We need a condition to determine if **the current weight is in the range [start ... end]**, passed to the input at the start of the program. If this is the case, we print the **full word (full_word)**.

Be careful not to print the word with unique letters. It was only needed to calculate the weight!

The words are **separated with a space** and we'll accumulate them in an intermediate variable **result**, which is defined as an empty string at the beginning.:

```
if first_number <= weight <= second_number:
    result += full_word + " "
```

Final Touches

The condition is met **unless we do not have a single word in the entered range**. To find out if we have found a word, we can simply check whether the string **result** has its initial value (i.e., an empty string), if it does, we print **"No"**, otherwise we print the whole string without the last space (using the **.strip(...)** method):

```
if result == "":
    print("No")
else:
    print(result.strip())
```

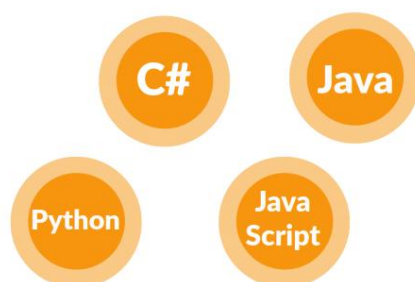
For the curious ones, we will leave as an exercise to optimize the loop's function by reducing the number of iterations in nested loops.

Testing in The Judge System

Test your solution here <https://judge.softuni.org/Contests/Practice/Index/1061#2>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**



SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 9.2. Problems for Champions – Part II

In this chapter, we will review three additional tasks that belong to the category "For Champions", i.e. they are more complex than the rest of the tasks in this book.

More Complex Tasks on The Studied Material

Before we move on to particular tasks, we must clarify that these can be solved more easily with **additional knowledge in programming with Python** (functions, arrays, collections, recursion, etc.), but each solution that will be provided now only uses the material covered in this book. The goal is to learn how to construct **more complex algorithms** based on your knowledge collected up to the present moment.

Problem: Passion Days

Lina has a real shopping passion. When she has some money, she immediately goes to the closest shopping center (mall) and tries to spend as much as she can on clothes, bags, and shoes. But her favorite thing is winter sales. Our task is to analyze her strange behavior and **calculate the purchases** that Lina does when she enters the mall, as well as the **money she has left** when the shopping is over.

The **first line** of the input will pass the **amount** that Lina has **before** she starts shopping. After that, upon reading the **"mall.Enter"** command, Lina enters the mall and starts shopping until the **"mall.Exit"** command is given. When Lina starts shopping, **on each line** of the input you will be given strings that are **actions performed by Lina**. Each **symbol** in the string is a **purchase or another action**. String commands contain only symbols of the **ASCII table**. The ASCII code of each sign is **related to what Lina must pay** for each of the goods. You need to interpret the symbols in the following way:

- If the symbol is a **capital letter**, Lina gets a **50% discount**, which means that you must decrease the money she has by 50% of the numeric representation of the symbol from the ASCII table.
- If the symbol is a **small letter**, Lina gets a **70% discount**, which means that you must decrease the money she has by 30% of the numeric representation of the symbol from the ASCII table.
- If the symbol is **"%"**, Lina makes a **purchase** that decreases her money in half.
- If the symbol is **"*"**, Lina **withdraws money from her debit card** and adds 10 BGN to her available funds.
- If the symbol is **different from all of the aforementioned**, Lina just purchases without discount, and in this case, you should simply subtract the value of the symbol from the ASCII table from her available funds.

If a certain value of her purchases is **higher** than her current available funds, Lina **DOES NOT** make the purchase. Lina's funds **cannot be less than 0**.

The shopping ends when the `"mall.Exit"` command is given. When this happens, you need to **print the number of purchases made and the money** that Lina has left.

Input Data

The input data must be read from the console. The **first line** of the input will indicate the **amount that Lina has before starting to purchase**. On each of the following lines, there will be a particular command. After you read the command `"mall.Enter"`, on each of the following lines you will be given strings holding **information regarding the purchases / actions** that Lina wants to perform. These strings will keep being passed until the `"mall.Exit"` command is given.

Always only one `"mall.Enter"` command will be given, as well as only one `"mall.Exit"` command.

Output Data

When shopping is over, you must print on the console a particular output depending on what purchases have been made:

- If **no purchases have been made** – `"No purchases. Money left: {remaining funds} lv."`
- If **at least one purchase is made** – `"{number of purchases} purchases. Money left: {remaining funds} lv."`

The funds must be printed with an accuracy of up to 2 symbols after the decimal point.

Constraints

- Money is a **float** number within the range: $[0 - 7.9 \times 10^{28}]$.
- The number of strings between `"mall.Enter"` and `"mall.Exit"` will be within the range: $[1 - 20]$.
- The number of symbols in each string that represents a command will be within the range: $[1 - 20]$.
- Allowed execution time: **0.1 seconds**.
- Allowed memory: **16 MB**.

Sample Input and Output

Input	Output	Comment
110 mall.Enter d mall.Exit	1 purchases. Money left: 80.00 lv.	'd' has an ASCII code of 100. 'd' is a small letter, this is why Lina gets a 70% discount. She spends 30% of 100, which is 30 lv. After this purchase, she has: $110 - 30 = 80$ lv.

Input	Output
110 mall.Enter % mall.Exit	1 purchases. Money left: 55.00 lv.

Input	Output
100 mall.Enter Ab ** mall.Exit	2 purchases. Money left: 58.10 lv.

Hints and Guidelines

We will separate the solution of the problem into three main parts:

- **Processing** of the input.
- **Algorithm** for solving the problem.
- **Formatting** the output.

Let's examine each of the parts in detail.

Processing The Input Data

The input of our task consists of a few components:

- In the **first argument**, we have all the money that Lina has for shopping.
- In the **second argument**, we have an array that is a series of commands.

The first part is simple:

```
shopping_money = float(input())
```

But in the second part is a detail that we need to take into consideration. The requirements state the following:

Each of the next rolls will contain specific command. When we receive the command "mall.Enter", on each of the next rolls the element will be a string containing information regarding the purchases/actions that Lina wants to perform.

This is where we need to take into consideration the fact that inside our array we should start processing commands, but only after we receive the command "mall.Enter". How can we do that? Using a **while** or a **do-while** loop is a good option. Here is an exemplary solution of how to skip all commands before processing the command "mall.Enter":

```
command = input()
```

```
while command != "mall.Enter":
    command = input()
```

```
command = input()
```

Here is the place to say that by calling `input()` after the end of the loop is used to pass from the first command for processing.

Algorithm for Solving The Problem

The algorithm for solving the problem is a direct one – we continue **reading commands** from the console **until the command "mall.Exit" is passed**. In the meantime, we **process** each symbol (**char**) of each one of the commands according to the rules specified in the task requirements, and in parallel, we **modify the amount** that Lina has and **store the number of purchases**.

Let's examine the first two problems for our algorithm. The first problem concerns the way we read the commands until we reach the **"mall.Exit"** command. The solution that we previously saw uses a **while-loop**. The second problem for the task is to **access each symbol** of the command passed. Keeping in mind that the input data with the commands is a **string** type, the easiest way to access each symbol inside the strings is via a **for-in loop**.

This is how the code may look like using two loops like this:

```
while command != "mall.Exit":  
    for action in command:  
        pass  
  
command = input()
```

The next part of the algorithm is to **process the symbols from the commands**, according to the following rules in the requirements:

- If the symbol is a **capital letter**, Lina gets a 50% discount, which means that you must decrease the money she has by 50% of the numeric representation of the symbol from the ASCII table.
- If the symbol is a **small letter**, Lina gets a 70% discount, which means that you must decrease the money she has by 30% of the numeric representation of the symbol from the ASCII table.
- If the symbol is **"%"**, Lina purchases and that decreases her money in half.
- If the symbol is **"**"**, Lina withdraws money from her debit card and adds 10 BGN to her available funds.
- If the symbol is **different from all of the aforementioned**, Lina just purchases without discount, and in this case, you should simply subtract the value of the symbol from the ASCII table from her available funds.

Let's examine the problems that we will be facing in the first condition. The first one is how to distinguish if a particular **symbol is a capital letter**. We can use one of the following ways:

- Keeping in mind the fact that the letters in the alphabet have a particular order, we can use the following condition **action >= 'A' && action <= 'Z'**, to check if our symbol is within the capital letters range.
- We can use the function **.isupper(...)**.

The other problem is how **to skip a particular symbol** if it is not an operation that requires more money than Lina has. This is doable using the **continue** construction.

An exemplary condition for the first part of the requirements looks like this:

```
if action >= 'A' and action <= 'Z':
    price = ord(action) * 0.5
    if shopping_money < price:
        continue

    shopping_money -= price
    purchases += 1
```

Note: the variable **purchases** is of **int** type, in which we store the number of all purchases. We believe the reader should not have difficulties implementing all the other conditions because they are very similar to the first one.

Formatting The Output

At the end of our task we must **print** a particular **output**, depending on the following condition:

- If no purchases have been made – "No purchases. Money left: {remaining funds} lv."
- If at least one purchase is made – "{number of purchases} purchases. Money left: {remaining funds} lv."

The printing operations are simple, as the only thing we need to take into consideration is that **the amount has to be printed with an accuracy of up to 2 symbols after the decimal point**.

How can we do that? We will leave the answer to this question to the reader.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1062#0>.

Problem: X Expression

Bonny is an exceptionally powerful witch. As her natural power is not sufficient to successfully fight vampires and werewolves, she has started to master the power of Expressions. An expression is very hard to master because the spell relies on the ability to **quickly solve mathematical expressions**.

To use an "Expression spell", the witch must know the result of a mathematical expression in advance. An **Expression spell** consists of a few simple mathematical expressions. Each mathematical expression can contain operators for **summing up**, **subtraction**, **multiplying**, and/or **division**.

The expression is solved **without considering the mathematical rules** for calculating numerical expressions. This means that the priority is applied according to the sequence of the operators, and not the type of calculation that they do. The expression **can contain**

brackets, as **everything inside the brackets is calculated first**. Every expression can contain multiple brackets, but no nested brackets:

- An expression containing **(...(…)...)** is an invalid one.
- An expression containing **(...)(...)** is a valid one.

Problem

The expression

$$4 + 6 / 5 + (4 * 9 - 8) / 7 * 2$$

is solved in the following way:

$$\begin{aligned} 4 + 6 / 5 + (4 * 9 - 8) / 7 * 2 &= \\ 10 / 5 + (4 * 9 - 8) / 7 * 2 &= \\ 2 + (4 * 9 - 8) / 7 * 2 &= \\ 2 + (36 - 8) / 7 * 2 &= \\ 2 + 28 / 7 * 2 &= \\ 30 / 7 * 2 &= \\ 4.285714285714286 * 2 &= \\ 8.571428571428571 &= \\ 8.57 & \end{aligned}$$

Bonny is very pretty, but not as wise, so she will need our help to master the power of Expressions.

Input Data

The input data consists of a single text line, passed from the console. It contains a **mathematical expression for calculation**. The line **always ends with the "=" symbol**. The "=" symbol means **the end of the mathematical expression**.

The input data is always valid and always in the described format. No need to validate it.

Output Data

The output data must be printed on the console. The output consists of one line: the result of the **calculated mathematical expression**.

The result must be rounded up to the **second digit after the decimal point**.

Constraints

- The expressions will consist of **a maximum of 2500 symbols**.
- The numbers of each mathematical expression will be within the range **[1 ... 9]**.
- The operators in the mathematical expressions will always be among **+** (summing up), **-** (subtraction), **/** (division) or ***** (multiplying).

- The result of the mathematical expression will be within the range $[-100000.00 \dots 100000.00]$.
- Allowed execution time: **0.1 seconds**.
- Allowed memory: **16 MB**.

Sample Input and Output

Input	Output
$4+6/5+(4*9-8)/7*2=$	8.57

Input	Output
$3+(6/5)+(2*3/7)*7/2*(9/4+4*1)=$	110.63

Hints and Guidelines

As usual, we will first read and process the input, after that, we will solve the problem, and finally, we will print the result, formatted as required.

Processing The Input Data

The input data will consist of exactly one text line read from the console. To have access to each expression, we have to read the input with `input()` function and then convert the string into an array of symbols. Using the function `pop(...)` we can process each symbol:

```
expression = input()
symbols = list(expression)
symbol = symbols.pop(0)
```

Algorithm for Solving The Problem

For the tasks of our problem we need to use some variables:

- One variable will store our **current result**.
- And our final variable will store the **current operator** from our expression.

```
result = 0
expression_operator = "+"
```

We have to clarify that in the code above, by default the operator is `+`, so that the first meet number can be summed with our result.

Now that we already have our starting variables, we must decide **what will be the main structure** of our program. By the requirements, we understand that **each expression ends with =**, i.e. we must read and process symbols until we reach a `=`. This is followed by an accurately written **while** loop.

```
while symbol != "=":

    symbol = symbols.pop(0)
```

Our next step is to process our **symbol** variable. We have 3 possible cases for it:

- If the symbol is a **start of a sub-expression placed in brackets** i.e. the found symbol is a **(**.
- If the symbol is a **digit between 0 and 9**. But how can we check this? How can we check if our symbol is a digit? We can use for assistance the **ASCII code** of the symbol, via which we can use the following formula: **[ASCII code of our symbol] - [ASCII code of the symbol 0] = [the digit that represents the symbol]**. If the result of this condition is between 0 and 9, then our symbol is a number.

Note: to get the ASCII code of a certain symbol, we will use the function **ord(...)**.

- If the symbol is an **operator**, i.e. it is **+**, **-**, ***** or **/**.

```
while symbol != "=":
    if symbol == "(":
        # TODO

    elif 0 <= ord(symbol) - ord('0') \
         and ord(symbol) - ord('0') <= 9:
        # TODO

    elif (symbol == '+' or
          symbol == '-' or
          symbol == '/' or
          symbol == '*'):
        # TODO
```

Let's examine the actions that we need to undertake in the relevant cases that we defined:

- If our symbol is an **operator**, then the only thing we need to do is to **set a new value for the expression_operator** variable.
- If our symbol is a **digit**, then we need to **change the current result of the expression** depending on the current operator, i.e. if **expression_operator** is a **-**, then we must decrease the result by the numerical representation of the current symbol. We can get the numerical representation of the current symbol via the formula that we used upon checking the condition for this case (**[ASCII code of our symbol] - [the ASCII code of the symbol 0] = [the digit that represents the symbol]**).

```

elif 0 <= ord(symbol) - ord('0') \
    and ord(symbol) - ord('0') <= 9:
    if expression_operator == "+":
        result += ord(symbol) - ord('0')
    elif expression_operator == "-":
        result -= ord(symbol) - ord('0')
    elif expression_operator == "*":
        result *= ord(symbol) - ord('0')
    elif expression_operator == "/":
        result /= ord(symbol) - ord('0')

```

```

elif (symbol == '+' or
      symbol == '-' or
      symbol == '/' or
      symbol == '*'):
    expression_operator = symbol

```

- If our symbol is a (, this indicates the beginning of a sub-expression (an expression in brackets). By definition, the sub-expression must be calculated before modifying the result of the whole expression (the actions in brackets are performed first). This means that we will have a local result for the sub-expression and a local operator.

```

if symbol == "(":
    inner_result = 0
    inner_operator = '+'
    symbol = symbols.pop(0)

    # TODO: Calculate the sub-expression

```

After that, to calculate the sub-expression value, we will use the same methods that we used for calculating the main expression – we use a **while loop** to read symbols (until we reach an **)** symbol). Depending on whether the read symbol is a number or an operator, we modify the result of the sub-expression. The implementation of these operations is identical to the above-described implementation for calculating expressions. This is why we believe the reader will be able to easily handle it.

After finishing the result calculation for our sub-expression, we modify the result of the whole expression depending on the value of the **expression_operator**.

```

if expression_operator == "+":
    result += ord(symbol) - ord('0')
elif expression_operator == "-":
    result -= ord(symbol) - ord('0')
elif expression_operator == "*":
    result *= ord(symbol) - ord('0')
elif expression_operator == "/":
    result /= ord(symbol) - ord('0')

```

Formatting The Output

The only output that the program must print on the console is the **result of solving the expression with an accuracy of up to two symbols after the decimal point**. How can we format the output this way? We will leave the answer to this question to the reader.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1062#1>.

Problem: Bulls and Cows

We all know the game called "Bulls and Cows" (https://en.wikipedia.org/wiki/Bulls_and_cows). Upon having a particular 4-digit **secret number** and a 4-digit **suggested number**, the following rules are applied:

- If a digit in the suggested number matches a digit in the secret number and is located at the **same position**, we have a **bull**.
- If a digit in the suggested number matches a digit in the secret number but is located at a **different position**, we have a **cow**.

Here are a few examples of the game:

Secret number	1	4	8	1	Comment
Suggested number	8	8	1	1	Bulls = 1 Cows = 2
Secret number	2	2	4	1	Comment
Suggested number	9	9	2	4	Bulls = 0 Cows = 2

Upon having a particular secret number and the bulls and cows pertaining to it, our task is **to find all possible suggested numbers** in ascending order. If there are **no suggested numbers** that match the criteria provided from the console, we must print "No".

Input Data

The input data is read from the console. We have 3 lines in the input data:

- The first contains **the secret number**.
- The second contains **the number of bulls**.
- The third contains **the number of cows**.

The input data will always be valid. There is no need to verify them.

Output Data

The output data must be printed on the console. The output must consist of **a single line - all suggested numbers**, space-separated. If there are **no suggested numbers** that match the criteria provided from the console, we must **print “No”**.

Constraints

- The secret number will always consist of **4 digits in the range [1..9]**.
- The number of **cows and bulls** will always be in the range **[0..9]**.
- Allowed execution time: **0.15 seconds**.
- Allowed memory: **16 MB**.

Sample Input and Output

Input	Output
2228 2 1	1222 2122 2212 2232 2242 2252 2262 2272 2281 2283 2284 2285 2286 2287 2289 2292 2322 2422 2522 2622 2722 2821 2823 2824 2825 2826 2827 2829 2922 3222 4222 5222 6222 7222 8221 8223 8224 8225 8226 8227 8229 9222

Input	Output
1234 3 0	1134 1214 1224 1231 1232 1233 1235 1236 1237 1238 1239 1244 1254 1264 1274 1284 1294 1334 1434 1534 1634 1734 1834 1934 2234 3234 4234 5234 6234 7234 8234 9234

Hints and Guidelines

To complete our task we can do the followings steps:

- We will generate all possible **four-digit combinations** (candidates for verification).
- For each generated combination we will calculate **how many bulls** and **how many cows** it has according to the secret number. Upon matching the needed bulls and cows, we will **print the combination**.

Processing The Input Data

We have 3 lines in the input data:

- The first contains **the secret number**.
- The second contains **the number of bulls**.
- The third contains **the number of cows**.

Reading the input data is simple:

```
guess_number = input()
target_bulls = int(input())
target_cows = int(input())
```

Algorithm for Solving The Problem

Before starting to write the algorithm for solving our problem, we must **declare a flag** that indicates whether a solution is found:

```
solution_found = False
```

If after finishing our algorithm this flag is still **False**, then we will print **No** on the console, as specified in the requirements:

```
if not solution_found:
    print("No")
```

Let's start analyzing our problem. What we need to do is **analyze all numbers from 1111 to 9999**, excluding those that contain zeroes (for example **9011**, **3401**, etc. are invalid). What is the easiest way to **generate** all these **numbers**? We will **use nested loops**. As we have a **4-digit number**, we will have **4 nested loops**, as each of them will generate an **individual digit** in our number for testing:

```
for digit_1 in range(1, 10):
    for digit_2 in range(1, 10):
        for digit_3 in range(1, 10):
            for digit_4 in range(1, 10):
```

Thanks to these loops, **we have access to every digit** of all numbers that we need to check. Our next step is to **separate the secret number into digits**. This can be achieved very easily by **slicing the string (string slicing)**. An alternative solution is to access the symbols by index:

```
guess_digit_1 = int(guess_number[0:1])
guess_digit_2 = int(guess_number[1:2])
guess_digit_3 = int(guess_number[2:3])
guess_digit_4 = int(guess_number[3:4])
```

Only two last steps remain until we start analyzing how many cows and bulls there are in a particular number. Accordingly, the first one is **the declaration of counter variables** in the nested loops, to **count the cows and bulls** for the current number. The second step is to make **copies of the digits of the current number** that we will analyze, to prevent problems upon working with nested loops, in case we make changes to them.

```
digit_to_check_1 = digit_1
digit_to_check_2 = digit_2
digit_to_check_3 = digit_3
digit_to_check_4 = digit_4
```



```
current_bulls = 0
current_cows = 0
```

We are ready to start analyzing the generated numbers. What logic can we use? The easiest way to check how many cows and bulls there are inside a number is via a **sequence of if-elif conditions**. Yes, this is not the most optimal way, but to stick to what is covered in the current book, we will use this approach.

What conditions do we need?

The condition for the bulls is very simple – we check whether the **first digit** of the generated number matches the **same digit** in the secret number. We remove the digits that are already checked to avoid repetitions of bulls and cows:

```
# Find all bulls, count them and remove them (assign -1 and -2)
if digit_to_check_1 == guess_digit_1:
    # Bull at position #1 found -> count it and remove it
    current_bulls += 1
    guess_digit_1 = -1
    digit_to_check_1 = -2
```

We repeat the action for the second, third, and fourth digits.

Our conditional statement for cows can be done by this method - we check if the **first digit** from the generated number **matches with the second, the third, or the fourth** digit of the secret number. Example:

```
# Find all cows for digitToCheck1, count them and
# and remove them (assign -1)
if digit_to_check_1 == guess_digit_2:
    # Cow at position #2 found -> count it and remove it
    current_cows += 1
    guess_digit_2 = -1
elif digit_to_check_1 == guess_digit_3:
    # Cow at position #3 found -> count it and remove it
    current_cows += 1
    guess_digit_3 = -1
elif digit_to_check_1 == guess_digit_4:
    # Cow at position #4 found -> count it and remove it
    current_cows += 1
    guess_digit_4 = -1

# TODO: Check other digits
```

After that, we sequentially check whether the **second digit** of the generated number **matches the first one**, the **third one**, or the **fourth digit** of the secret number; whether the **third digit** of the generated number matches the **first one**, the **second one**, or the **fourth digit** of the

secret number; and finally, we check whether the **fourth digit** of the generated number matches the **first one**, the **second one** or the **third digit** of the secret number.

Printing The Output

After completing all conditions, we just need to check whether the bulls and cows in the currently generated number match the desired bulls and cows read from the console. If this is true, we print the current number on the console:

```
if current_bulls == target_bulls \
    and current_cows == target_cows:
    if solution_found:
        print(" ", end="")

    print("%d%d%d%d" % (digit_1, digit_2
                        digit_3, digit_4), end="")
    solution_found = True
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1062#2>.

Chapter 10. Functions

In this chapter, the topic of **functions** will be introduced, together with cases in which they might be useful, as well as the **primary concepts** when working with functions. It will be demonstrated why it is a **good practice** to use functions, how to **define** them and how to **call** them. The concept of a **parameter** and a **returned value** of a function, as well as how we can further use the returned value from the function, will be introduced. At the end of the chapter, we'll look at what the **best practices** are when dealing with functions.

What is a Function?

So far, we learned that dividing our code into **smaller units** (functions), each of which is responsible for a specific functionality is more practical in general. Each unit is concerned with **distinct functionality**, in which case not only is it easier to process the problem we are facing, but also improves the **debugging** and **readability** of the code.

When we think about writing code, the function in practice can be considered as an **encapsulated set of instructions** that implement the required functionality. This set of instructions (i.e., the function) has its structure, separated from the rest of the code. Throughout the code, we can **always** call the function when we need its functionality, simply by typing its name and giving appropriate arguments for the parameters if there are any.

A function can be called as many times as we need to solve our problem. This prevents us from typing one functionality several times, which in turn reduces the risk of error if the function is changed afterward.

We will look at two types of functions – "**basic** functions" (without parameters) and "**complex**" functions (with parameters).



In Object-Oriented Programming (which is out of the scope of this book) the functions are part of higher-level objects called **classes**, where functions are called **methods**. In other programming languages, the functions are called **procedures**.

Basic Functions

The **basic** functions are responsible for the execution of an **action** which **contributes** to the solving of a particular problem. Such an action can be printing a string to the console, performing a conditional statement check, going through a loop, etc.

Now, let's look at the following **example of a basic function**:

```
def print_header():
    print("-----")
```

The purpose of this function is to print a headline, which, in this case, is a string of dashes. Because of this, it is called '**print_header**'. The parentheses **always** follow the name of the function, irrespective of its name. The names of our functions must describe the purpose of the function. Later in this chapter, we will learn more in-depth about the naming conventions regarding functions.

The **function declaration** is always followed by a **colon**, which is followed by the set of instructions (an algorithm), used for solving a particular problem, described by the name of the function. This part of the code is also referred to as the **body** of the function which contains the machine code (the instructions). It is positioned on a new line after the function's declaration. It is written with an indentation compared to the function-defining line. The body of the function is always indented four spaces or a single Tab from the level at which the function declaration begins. This separates the body of the function as a single block of instructions.

If the given snippet of code is present in a program that is to be executed, the function will not affect the program at this stage, because the function is declared, but not called yet.

Why Should We Use Functions?

So far, we found that the functions help with the logical differentiation of a longer code to less complicated, easier-to-digest pieces. Working with functions gives our code advantages, such as better structure and greater cohesion.

The usage of functions **prevents us from code repetition**. Repeating code is **bad** practice as it heavily **impedes** the code's maintenance and leads to errors. If a single code functionality is present in our program several times throughout the code, and we need to change something to this functionality afterward, we need to correct the same piece of code as many times as we have it in the program. The likelihood of missing one spot in the redaction of the code, in this case, is high, which results in unpredictable behavior. This is the reason why it is a good practice if we do use a fragment of a code **more than once**, to define it as a **separate function**.

The functions **allow** us to use a piece of code **several** times. Working on more and more problems, you will notice that with the use of already existing functions you will save a lot of time and effort.

Declaration of Functions

A **function declaration** can be considered as a registration of a given functionality within a program so that this functionality can be recognized and later used in the rest of the program where necessary. In the next example, we will explore the element of which a function is composed:

```
def calculate_square(num):  
    return num * num
```

- **def.** The keyword **def** in Python expresses the intention to declare a new function.
- **Name of the function.** The name of the function is **defined by the developer** and follows after the keyword **def**. It is important to describe the **functionality** which is bound to be executed by the function's instructions listed in its body. In our example, the name of the function is **calculate_square** which hints to us that the given problem aims to calculate the squared value of a number.

- **List of parameters.** It is introduced in between the parentheses, which we put after the name of the function. Here we list the parameters which we want to include in the function. There might be a single parameter, several parameters, or zero parameters (basic function). If we don't have a parameter, we leave the parentheses empty. In the function `calculate_square` there is only one parameter called `num`.
- **Colon.** After the closing bracket, we put a colon sign which initiates the beginning of the function's body.
- **Implementation of function (body).** The body of the function is the place to put our algorithm (instructions) which solves the problem we deal with. The body, per se, implements the **logic** of the function. In the particular example, we estimate the squared value of a given number which is `num*num`. the body is always indented and on a new line after the colon.

Within the definition of a function, it is important to follow this exact **structure** of the function's elements.

When declaring a given variable in the body of a function, it is called a local variable for the function. The scope of the variable in which it exists and can be used starts with the line on which we have defined it and spans to the last instruction, part of the function's body. This space of definition is called the **field of scope** for the variable.



Some programming languages such as C or C++ distinguish between the **declaration** and the **definition** of a function. The **declaration** of a function informs the compiler or the interpreter that a function with its given name exists without imposing implementation. The **definition** of a function brings in the implementation of the function's body. Within the **Python** language, this division into two steps does not exist, which in turn means that each function that is declared must also include some functionality (in the body).

Invoking a Function

The invoking of the function is regarded as the **beginning of the execution** of the code which is in the **function's body**. This is accomplished as the name of the function is called, followed by the parentheses. If the function expects input arguments for its parameters, they must be inserted in the parentheses in the same order in which the parameters were defined in the parentheses. An example that follows:

```
# define function
def print_header():
    print("-----")
```

```
# invoke function
print_header()
```

The given function can be called plenty of times from **any** location in the code. It's important to mention that if a function in **Python** is defined somewhere inside the code, the program will only recognize it if it is called after its definition within the code of the program.

As the invoking of a function is a command by itself, we can call a function from inside another function:

```
# define function
def print_header():
    # invoking function from
    # another function
    print_header_top()
    print_header_bottom()
```

There is also a possibility the function to be called in **its own body**. This is called **recursion**, for which additional information can be found in [Wikipedia](#) and we strongly encourage you to do your research on the internet.

Problem: Bank Receipt

Write a function that prints an empty receipt. The function should call another three functions: function which prints the headline of the receipt, function which prints the body of the receipt, and function which prints the bottom of the receipt.

Part of receipt	Text
Upper part	CASH RECEIPT -----
Middle part	Charged to_____ Received by_____
Lower part	----- (c) SoftUni

Sample Input and Output

Input	Outpput
(none)	CASH RECEIPT ----- Charged to_____ Received by_____ ----- (c) SoftUni

Hints and Guidelines

First, we need to create the function which **prints the header** of the receipt. Its name should be short and sensible which describes the task the function will execute. For example, it can be called **print_receipt_header** and its body will contain the following code:

```
def print_receipt_header():
    print("CASH RECEIPT")
    print("-----")
```

Similar to the first function, we will create another two functions which will print out the center part of the receipt (body) and the bottom of the receipt (footer). We will call them **print_receipt_body** and **print_receipt_footer**.

Next, we will create **another function** that wraps the three functions and call them one after another. Finally, we will call the function **print_receipt** in our program:

```
def print_receipt():
    print_receipt_header()
    print_receipt_body()
    print_receipt_footer()
```

```
print_receipt()
```

Testing in The Judge System

The program which contains four functions called inside another function is ready and we can run it and test it locally after which we should send it for a quality check to the Judge platform: <https://judge.softuni.org/Contests/Practice/Index/1063#0>.

Functions with Parameters (Complex Functions)

To solve our problem using the function, often in our practice we need **additional knowledge** which varies with the problem we need to solve. The **parameters of a function** are supplementing us with this knowledge; hence the behavior of the function depends on these parameters.

Use of Parameters in a Function

As we just mentioned, the parameters **can be zero or as many as needed**. When they are declared, commas are used to separate them from each other. The parameters can be of any type (number, string, etc.). The following is an example that demonstrates how exactly the parameters are being used in a body of a function.

This is an example of a **definition** of a function that uses a **set of parameters**:

```
def print_numbers(start, end):
    for i in range(start, end):
        print(str(i))
```

In this example, we will be using two parameters called **start** and **end**, respectively. After the definition of the function, it can be used in the program – we **call** the function including **arguments** for our parameters:

```
print_numbers(5, 10)
```

In this case, the numerical value of the argument is declared directly in the function (e.g., **5** for **start** and **10** for the **end**).

When declaring **parameters**, they can be of **any** type (integer, string, etc.) and each parameter has to have a meaningful name. It's important to notice that when the function is called, the **arguments** must be put in the **same order** as the parameters, declared in the definition of the function.

Let's look at another example of function definition which has a few parameters, each of a different type:

```
def print_student(name, age, grade):  
    print("Student: " + name + ", age: "  
          + str(age) + ", grade: " + str(grade))
```

Problem: Sign of Integer Number

Create a function that prints whether the given integer is positive, negative, or zero.

Sample Input and Output

Input	Output
2	The number 2 is positive.
-5	The number -5 is negative.
0	The number 0 is zero.

Hints and Guidelines

The first step is to **create** a function and give it a descriptive name such as **print_sign**. This function will have only one parameter – integer number, which sign we want to check:

```
def print_sign(n):  
    # your code here
```

Next, we want to **implement** the logic which will check whether the input value is positive, negative, or zero. Looking at the examples we can conclude there are three cases to consider – the integer is bigger than zero, the integer is equal to zero and the integer is less than zero, which means we need to check **three conditions** in the function's body.

Then we need to read the input value and call our function:


```
print_sign(input("Enter number: "))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#1>.

Optional Parameters and Default Values

Python allows us to use **optional parameters**. This in turn allows us to **skip** initialization of the arguments if **default** ones are given to the parameters during the function declaration.

The following example will demonstrate the use of default arguments:

```
def print_numbers(start=0, end=100):
    for i in range(start, end):
        print(str(i))
```

The given function **print_numbers(...)** can be called in several ways:

```
print_numbers(5, 10)
print_numbers(15)
print_numbers()
print_numbers(end = 40, start = 35)
```

On the first line, the parameters use the following arguments – **start = 5, end = 10**. With the second line the arguments will be – **start = 0, end = 15**. With the third line we have **start = 0, end = 100**. The last line will use **start = 35, end = 40**.

Problem: Printing Triangle

Create a function that prints triangles as shown in the examples.

Sample Input and Output

Input	Output	Input	Output
3	1 1 2 1 2 3 1 2 1	4	1 1 2 1 2 3 1 2 3 4 1 2 3 1 2 1 2 1

Hints and Guidelines

Before creating a function, that prints numbers on a single line with initial and final values, we must read the input from the console. Next, we choose a meaningful name for our function which shortly describes its purpose. This might be **print_line** and we can now build it:

```
def print_line(start, end):
    for i in range(start, end + 1):
        print(str(i)+" ", end="")
    print("")
```

Looking at the problems regarding console drawing, we remember that it is a good practice to **divide the completion of the figure** into several steps. We can simplify the problem by dividing the triangle into three sections – upper, mid, and lower segments.

Our next step will be to print the **upper segment** of the triangle's body:

```
for i in range(0, n):
    print_line(1, i)
```

After that we need to print the **centerline**:

```
print_line(1, n)
```

Finally, we print the **bottom segment** of the triangle, as this time the step of the for loop is a negative one:

```
for i in range(n - 1, 0, -1):
    print_line(1, i)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#2>.

Problem: Draw a Filled Square

Draw a filled square to the console with a side length N as it is shown in the examples.

Sample Input and Output

Input	Output	Input	Output	Input	Output
4	<pre>----- -\///- -\///- -----</pre>	3	<pre>----- -\///- -\///- -----</pre>	2	<pre>----- -----</pre>

Hints and Guidelines

The first step will be to read the input from the console. Next, we need to create a function that prints the first and the last line as they are identical. Let's not forget that we need to choose a **descriptive name** and define the length of the side of the square as a **parameter**:

```
def print_header_footer(n):
    for i in range(0, (n * 2)):
        print("-", end="")
    print("")
```

The next step is to create a function that draws to the console the lines in the middle. Again, we think of a descriptive enough name, say **print_middle_row**:

```
def print_middle_row(n):
    print("-", end="")
    for i in range(0, n - 1):
        print("\\/", end="")
    print("-")
```

Finally, we call the created functions to draw the square:

```
side = int(input("Enter size:"))
print_header_footer(side)
# TODO: Draw the rest of the square
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#3>.

Returned Value by a Function

Up until now, we were looking at functions that execute a particular action such as printing some text, value, or a string of characters on the console. Other than such functions, there also can be functions that **return** a value as a **result** of their execution – for example, this could be the multiplication of two numbers. It will be these functions that will be the topic of our discussion in the next few lines.

The **return** Keyword

To return a value that is the result of the function's execution we use the **return** keyword. It must be **used in the body** of the function and indicates to the program to **stop the execution** of the function and **returns value** to the caller of the function which is written as an argument after the keyword 'return'. In the example that follows, there is a function that reads two names from the console, concatenates them, and returns them as a result:

```
def read_full_name():
    first_name = input("First name:")
    last_name = input("Last name:")
    return first_name + " " + last_name
```

The **return** keyword can be used in functions that do not return anything (the **return** keyword must be last at the end of the function). The presence of the keyword allows the function to stop its execution without returning a value. In this case, the use of the keyword is helpful only to terminate the execution of the function. It is also possible for the **return** keyword to be used in more than one place in the function's body.

In the following example, we have a function that compares two numbers and returns as a result either **-1**, **0**, or **1**, depending on whether the first argument is smaller, equal, or bigger than the second argument of the function. The function uses the **return** keyword on three

different instances to return different values depending on the arguments fed to the function.

```
def compare_to(number1, number2):
    if number1 > number2:
        return 1
    elif number1 == number2:
        return 0
    else:
        return -1
```

It is important to notice that the returned value by the function can be of a **different type** compared to the argument's type - string, integer, floating-point number, etc.

The Code After `return` is Unreachable

After the **return** keyword in a given function, the execution of the function is terminated, and the execution of the program continues where the function was last called. If there are other instructions written after the **return** keyword, they will not be executed. Some IDEs (including PyCharm) will inform you of the following warning:

```
def foo(n):
    return n * n
    # the following instruction will not be executed
    print("foo was executed.")
```

This code is unreachable more... (Ctrl+F1)



In programming, it is meaningless to place two **return** keywords one after another, as the execution of the first **return** will not allow the execution of the second **return**.

Use Case of a Returned Value from a Function

After a function is executed and the value is returned by its execution, this value can be used in **several** ways. For once, we can **assign this value to a variable**:

```
maximum = get_max(5, 10)
```

Another way to accommodate the returned value is to use it directly in an **expression**:

```
total = get_price() * quantity * 1.20
```

The third method of using the returned value of our function is to **pass** this value to **another function** via a parameter:

```
age = int(input())
```

Problem: Calculate Triangle Area

Write a function that calculates the area of a triangle, given the base and height, and returns it as an argument.

Sample Input and Output

Input	Output
3 4	6

Hints and Guidelines

First, create a function that calculates the surface area based on two input parameters – length of the side **a** and length of the height **h**:

```
def get_triangle_area(length, height):  
    return (length * height) / 2
```

The next step will be to read the input values and **call the function** using them. The result is **formatted in an appropriate variable** and appears on the screen:

```
a = float(input("Enter length:"))  
h = float(input("Enter height:"))  
area = get_triangle_area(a, h)  
print("Area: "+str(area))
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#4>.

Problem: Math Power

First, we need to read the input values from the console. The next step will be to create a function that accepts two arguments as its parameters (a number and its power) and returns, as a result, a float type number.

Sample Input and Output

Input	Output	Input	Output
2 8	256	3 4	81

Hints and Guidelines

```
def calculate_power(number, power):  
    result = 0  
    # TODO: calculate power  
    # by using a loop (or the ** operator)  
    return result
```

After we have done the necessary calculation, we must call the defined function and print the result.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#5>.

Functions, Returning Several Values

Up to this point, we looked at functions that **do not return any value** and functions which **return only one value**. In practice, it is often sensible for a function to return **more than one value**.

To achieve this in **Python**, after the **return** keyword we list the values which are bound to be returned, separated by a comma. The following function accepts as parameters two integers (**x** and **y**) and **returns two values** - the integer and the remainder of their division:

```
def divide(x, y):  
    result = x // y  
    reminder = x % y  
    return result, reminder
```

```
a, b = divide(19, 5)
```

Invoking the function works as with functions that do not return a value or return a single value. To use the returned values, we can **assign** them to variables whose names are separated by commas as demonstrated with **a** and **b**. After the execution of this example, **a** will hold the value **3** while **b** will hold the value **4**.

Variations of Functions

In many programming languages **identically** named functions can be used throughout the code using only different parameters which are considered **function overloading**. Python does not support this functionality, but a similar effect can be achieved with the unconditional parameters introduced earlier.

Function's Signature

In programming, the way a function is **recognized** is by looking at **both** its fundamental components - **name** and **list** of parameters, declared during the definition of the function. These components define its **specification**, and that's also known as the **signature** of the function:

```
def print_name(name):
    print("Name: "+name)
```

In this example, the signature of the function is its name **print_name** as well as its parameter **name**.

If in a program we have functions with **identical names** but **different parameters**, we are looking at function overloading. This functionality is not implemented in Python.

Variations of Functions in Python

Different options to call functions in **Python** can be achieved with the use of **default arguments** or more precisely with the dynamic allocation of arguments to declared parameters of the function. Let's look at the example from earlier with a function using a couple of default arguments:

```
def print_numbers(start=0, end=100):
    for i in range(start, end):
        print(str(i))
```

As we already noticed, we can call this function using a different combination of arguments that recall closely matched variants of the function:

```
print_numbers(5, 10)
print_numbers(15)
print_numbers()
print_numbers(end = 40, start = 35)
```

Problem: Greater of Two Values

The input is two values of the same type. The values can be of **int**, **chr**, or **str** type. Create a function **getMax()** that returns, as a result, the greater of the two values.

Sample Input and Output

Input	Output	Input	Output	Input	Output
int 2 16	16	char a z	z	string Ivan Tod	Tod

Creating the Functions

We need to create three functions with the same name and different signatures. First, we create a function that will compare integers.

Following the logic of the previous function, we create another one with the same name, but this one will compare characters.

Testing in the Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#6>.

Nested Functions (Local Functions)

Let's consider the following function which calculates the area of a circle:

```
def circle_circumference(radius):  
    pi = 3.14  
  
    def circle_diameter(r):  
        return r * 2  
  
    return pi * circle_diameter(radius)
```

What is a Local Function?

We see that in this piece of code, in the function `circle_circumference(...)` there is another function called `circle_diameter(...)`. This function is also known as a **nested function** or a **local function**. The nested functions can be declared in every other function. They are only visible and can be called in the scope of the function in which they are declared. In the last example, the function `circle_diameter(...)` can be called only inside the body of the function `circle_circumference(...)` rather than outside its body.

Why Using Local Functions?

With practice, we will find that when we write a program, often it appears that we need to use our function **only once**. Other often faced scenarios are when the function of interest gets too long. We already mentioned that when a function uses too many lines of code, the code gets hard to maintain. In these cases, we can be assisted by the **nested functions**. They present us with the possibility to declare a function within a function, where a nested function can be used independently. This contributes to the function's code as it makes it **cleaner** and **improves its readability**. This in turn improves the correction time if a bug is spotted later and minimizes the probability of error when changes are made to the code.

Declaration of a Nested Function

Let's again have a look at the example from before:

```
def circle_circumference(radius):
    pi = 3.14

    def circle_diameter(r):
        return r * 2

    return pi * circle_diameter(radius)
```

In this example, the function `circle_diameter(...)` is nested as it is declared inside the body of the function `circle_circumference(...)`. This means that the function `circle_diameter(...)` can be used only inside the function `circle_circumference(...)` but not outside of it. If we try to call the function `circle_diameter(...)` outside the function `circle_circumference(...)` this will cause an error during the execution of the program.

Nested functions have access to all variables which are declared inside the parenting function. In the discussed example `pi` can be used in the body of the function `circle_diameter(...)`. This asset of the nested functions can make them a preferred choice to solve a particular problem. This functionality saves writing time and decreases the amount of code that would otherwise be used to declare parameters and variables, already used in the parenting functions.

Classic Approaches in The Use of Functions

In this section of the chapter, we'll look at some **classic approaches** regarding the functions used in a program such as naming conventions, appearance, and structure of the function's code.

Naming Conventions

When naming a given function, it is beneficial to use **meaningful names**. As each function is **in charge** of some functionality of the problem, it must be taken into account **what task it will execute** inside the program which makes it a good practice to **name the function after the task it executes**.

In **Python**, the convention suggests that the functions must be written in lower cases, as well as that the words used to name the function must be separated by an **underscore** (`_`) (if there's more than one word). A good practice is that the function's name consists of a verb, or a pair of a verb and a noun, respectively.

Some examples of **appropriate** naming of functions:

- `find_student`
- `load_report`
- `sine`

Some examples of **inappropriate** naming of functions:

- **Method1**
- **DoSomething**
- **Handle_Stuff**
- **SampleMethod**
- **DIRTYHack**

If you can't think of an appropriate name, there is a good possibility that the function is responsible for more than one function or lacks a clearly defined goal. In such cases, it is better to think of how the function can be subdivided into smaller functions.

Naming Function's Parameters

In the naming of the **parameters** of the functions, the same rules apply to the naming of functions. The differences are that it is nice to name a parameter using a noun or a pair of an adjective and a noun, respectively. It should be noted that it is a good practice for the name of the parameter to hint at its type.

Some examples of **appropriate** naming of parameters:

- **first_name**
- **report**
- **speed_kmh**
- **users_list**
- **font_size_in_pixels**
- **font**

Some examples of **inappropriate** naming of parameters:

- **p**
- **p1**
- **p2**
- **populate**
- **LastName**
- **lastName**

More Good Practices in Working with Functions

Let's again recall how a function should execute **only one** particular **task** in the program. If this cannot be achieved, the next thing to do is to look at how to **separate** the function into a few other functions. As it was said, a name of a function should clearly and precisely state the purpose of the function. Another good practice will be to **avoid** making the function's name so long that it can no longer fit on the screen (relatively). If the code in the function gets too large nevertheless, it is recommended to **divide** the function's tasks into a few shorter functions within the function as demonstrated:

```
def print_receipt():
    print_receipt_header()
    print_receipt_body()
    print_receipt_footer()
```

```
print_receipt()
```

Structure and Formatting of The Code

In writing functions, caution must be applied to use the correct **indentation** (one Tab into the function). In **Python**, incompatible **indentation** leads to a direct error and does not allow the interpreter to execute the code.

An example of a **correctly** formatted Python code:

```
def read_full_name():
    first_name = input("First name:")
    last_name = input("Last name:")
    return first_name + " " + last_name
```

An example of an **incorrectly** formatted Python code (for this reason the last line is underlined in red):

```
def read_full_name():
    first_name = input("First name:")
last_name = input("Last name:")
    return first_name + " " + last_name
```

Another good rule of thumb is to leave **one empty line** between the loops and the conditional statements as well as **two empty lines** after the definition of a function. Also, try to **avoid long and complex expressions**. Practice shows that these practices improve the readability of the code and save the developer's time.

What Have We Learned from This Chapter?

In this chapter, we learned the fundamental concepts of working with functions, and:

- Learned that the **use** of functions comes very useful when larger programs must be **fragmented** into simpler tasks.
- Explored the **structure** of a function and how to **declare** and **call** it.
- Looked at examples of **complex** functions and their implementation in a program.
- Discovered what is a **signature** and **returned value** as well as the context of the **return** keyword.
- Get acquainted with the **good practices** in working with functions – how to name them and their parameters, how to format the code, etc.

Problems

To consolidate our knowledge about working with functions, a few problems will be solved as an exercise. Each exercise will ask for a function with specific functionality. Then the function must be called, with arguments being passed to the predefined parameters read as inputs from the console. Inputs and outputs beneath each exercise are given as an example.

Problem: Hello, Name!

Write a function that receives a name as a parameter and prints to the console "Hello, \!".

Sample Input and Output

Input	Output
Peter	Hello, Peter!

Hints and Guidelines

Define function `print_name(name)` and implement its functionality. Then take a string variable (name) as an input argument and call the function, passing the read name.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#7>.

Problem: Min Method

Create a function called `get_min(a, b)`, which returns the smaller number of the two. Write a program that reads three input values from the console and prints the smallest number. Use the function `get_min(...)` which is already created.

Sample Input and Output

Input	Output	Input	Output
1	1	-100	-102
2		-101	
3		-102	

Hints and Guidelines

Define a function `get_min(a, b)` and implement its functionality, after which call it as it's shown. To find the smallest of three numbers, first process one pair, and the result can be combined with the third number such as:

```
minimum = get_min(get_min(number1, number2), number3)
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#8>.

Problem: String Repeater

Write a function called `repeat_string(str, count)`, which receives one argument of type string, another argument `N` of type integer, and returns the string argument `N` times. The result of the function must be printed to the console.

Sample Input and Output

Input	Output	Input	Output
str 2	strstr	roki 6	rokirokirokirokirokiroki

Hints and Guidelines

Finish the function, as the input argument of type string, must be concatenated to the current output of the loop:

```
def repeat_string(str, count):
    repeated_string = ""
    for i in range(0, count):
        # TODO
    return repeated_string
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#9>.

Problem: Nth Digit

Write a function `find_nth_digit(number, index)`, which receives a number and index `N` as parameters and prints the `N`-th digit from right to left of the number (the first digit from right to left has index 1). The result of the function must be printed on the console.

Sample Input and Output

Input	Output	Input	Output	Input	Output
83746 2	4	93847837 6	8	2435 4	2

Hints and Guidelines

To execute the algorithm, we'll use a `while` loop which is running until the number becomes zero. At each iteration of the `while` loop, we'll check if the current index of the digit matches with the index passed to the function. If it matches, the digit of the number (`number % 10`)

with this index will be returned from the function. If not, then another digit will be removed from the current number (**number = number / 10**). Caution must be taken when checking, as the index starts from 1 from right to left. When the current index matches with the given one, the returned value of the function will be the digit from the number with the matched index.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#10>.

Problem: Integer to Base

Write a function called **integer_to_base(number, to_base)**, which receives as parameters an integer, and a number that indicates the base of the system. The function returns the input number converted to the required numeral system. Next, the result of the function must be printed to the console. The integer will always be of base 10 while the required base will be between 2 and 10.

Sample Input and Output

Input	Output	Input	Output	Input	Output
3 2	11	4 4	10	9 7	12

Hints and Guidelines

To solve the problem, a variable will be declared in which the result of the current state of the conversion will be stored as a string. The next steps required for the conversion are as follows:

- Calculate the **remainder** of the argument's integer division by the base argument.
- **Insert the remainder** of the divided argument at the beginning of the predeclared **result** variable.
- **Divide** the new argument by the base argument.
- **Repeat** the algorithm until the current argument becomes 0.

Fill up the missing part of the following function:

```
def integer_to_base(number, to_base):  
    result = ""  
    while number != 0:  
        # implement the missing conversion logic  
  
    return result
```

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#11>.

Problem: Notifications

Write a program that reads an integer **N** and enters **N** messages, each of which can be of multiple lines. For each of the messages' type will be used a different number of parameters. Each message begins with **message_type**, **success**, **warning**, or **error**:

- When **message_type** is a **success**, read **operation** + **message** (each on a new line).
- When **message_type** is a **warning**, read only the **message**.
- When **message_type** is an **error**, read **operation** + **message** + **errorCode** (each on a new line).

Each **read message** must be printed to the console, formatted depending on its **message_type**. After the first line from each output, the next line must be filled with as many equal signs as characters (including spaces) are used on the **first line** of each printed message. After the last line of each message, an **empty line** must be included before the beginning of the next message (look at the given output).

In the exercise, you have to use four functions: **show_success_message()**, **show_warning_message()**, **show_error_message()** and **read_and_process_message()**:

```
def show_success_message(operation, message):
    # TODO

def show_warning_message(operation, message):
    # TODO

def show_error_message(operation, message, error_code):
    # TODO

def read_and_process_message():
    # TODO
```

Sample Input and Output

Input	Output
4	Error: Failed to execute credit card purchase.
error	=====
credit card purchase	Reason: Invalid customer address.
Invalid customer	Error code: 500.
address	
500	Warning: Email not confirmed.
warning	=====
Email not confirmed	
success	Successfully executed user registration.
user registration	=====
User registered	User registered successfully.
successfully	
warning	Warning: Customer has not email assigned.
Customer has not email	=====
assigned	

Hints and Guidelines

Define and implement the four given functions.

In the function `read_and_process_message()` the type of the message is read from the console and depending on the type of the message, the remaining elements are also read, which can appear on either one, two, or three consecutive lines, depending on the type of the message. Next, the required function is called to execute the printing of the message.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#12>.

Problem: * Numbers to Words

Write a function `letterize(number)`, which reads an integer and translates it into words using the following requirements:

- Print each word on the same line (including minus sign if applicable) as shown in the output.
- If the number is bigger than 999, the output must be "too large".
- If the number is smaller than -999, the output must be "too small".
- If the number is negative, minus sign must be printed before the expression.
- If the number does not constitute of three digits, it must not be printed.

Sample Input and Output

Input	Output
3	nine-hundred and ninety nine
999	minus four-hundred and twenty
-420	too large
1020	

Input	Output
2	fifteen
15	three-hundred and fifty
350	

Input	Output
4	three-hundred and eleven
311	four-hundred and eighteen
418	five-hundred and nine
509	too small
-9945	

Input	Output
3	five-hundred
500	one-hundred and twenty three
123	nine
9	

Hints and Guidelines

We can first print **the hundreds** as a text – **(the number / 100) % 10**, after that **the tens** – **(the number / 10) % 10** and at the end **the ones** – **(the number % 10)**.

The first special case is when the number is exactly **rounded to 100** (e.g., 100, 200, 300, etc...). In this case, we print "one-hundred", "two-hundred", "three-hundred" etc.

The second special case is when the number formed by the last two digits of the input number is **less than 10** (e.g., 101, 305, 609, etc). In this case, we print "one-hundred and one", "three-hundred and five", "six-hundred and nine", etc.

The third special case is when the number formed by the last two digits of the input number is **larger than 10 and smaller than 20** (e.g., 111, 814, 919, etc.). In this case, we print "one-hundred and eleven", "eight-hundred and fourteen", "nine-hundred and nineteen", etc.

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#13>.

Problem: * String Encryption

Write a function **encrypt(letter)**, which encrypts a given letter in the following way:

- It takes the first and the last digit from the ASCII code of the letter and concatenates them into a string, which will represent the result.
- At the beginning of the string, which represents the result, we will insert the symbol which matches the following condition:
 - ASCII code of the letter + the last digit of the ASCII code of the letter.

- After that, at the end of the string, which represents the result, you concatenate the character which matches the following condition:
 - ASCII code of the letter - the first digit of the ASCII code of the letter.
- The function should return the encrypted string.

Example:

- $j \rightarrow p16i$
 - ASCII code of j is **106** $\rightarrow$ First digit - **1**, last digit - **6**.
 - Concatenate the first and the last digit $\rightarrow$ **16**.
 - At the beginning of the string, which represents the result, concatenate the symbol, which you get from the sum of the ASCII code + the last digit $\rightarrow 106 + 6 \rightarrow 112 \rightarrow p$.
 - **At the end** of the string, which represents the result, concatenate the symbol, which you get from subtracting the ASCII code - the first digit $\rightarrow 106 - 1 \rightarrow 105 \rightarrow i$.

Using the function shown above, write a function that takes **a sequence of characters**, **encrypts** them, and prints the result on one line.

The input data will always be valid. The Main function must read the data given by the user – an integer **N**, followed by a character for each of the following **N** lines.

Encrypt the symbols and add them to the encrypted string. In the end, as a result, you must print an **encrypted string** as in the following example:

- S, o, f, t, U, n, i $\rightarrow$ V83Kp11nh12ez16sZ85Mn10mn15h

Sample Input and Output

Input	Output
7 S o f t U n i	V83Kp11nh12ez16sZ85Mn10mn15h
Input	Output
7 B i	H66<n15hv14qh97XJ72Ah97xx10w

Input	Output
r a H a x	

Hints and Guidelines

Our variable **result**, in which we will save the value of the result, we will give the initial value "" (empty string). We must run a loop **n** times so that in each iteration we can add the encrypted symbol to the result string.

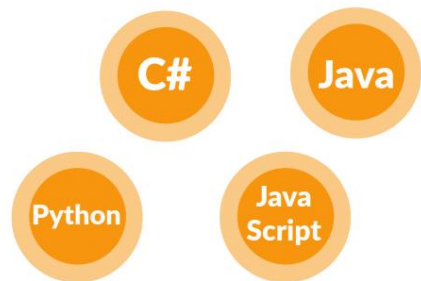
To find the first and the last digit of the ASCII code, we will use the same algorithm that we used to solve "N-th number" problem, while to create the string, we will proceed as in "Number to a numeral system"

Testing in The Judge System

Test your solution here: <https://judge.softuni.org/Contests/Practice/Index/1063#14>.

High-quality education profession and job for software engineers

- ✓ Free start for **absolute beginners**
- ✓ Choose among the **most in-demand** professions in the software industry
- ✓ Assistance for **career start**
- ✓ **Top trainers**, mentors and active learning **community**

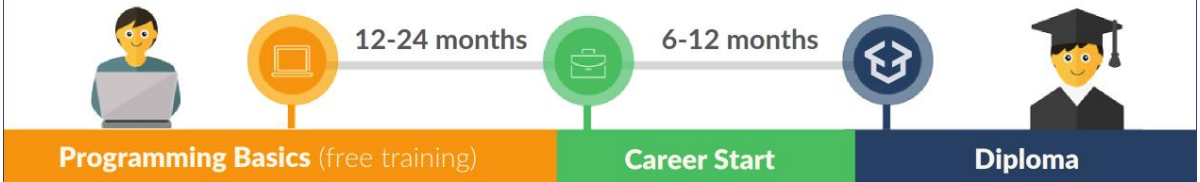


SoftUni provides practical, modern and **innovative education** for programming, IT, design, marketing and digital skills for thousands of young people. The "**Software University**" program builds true **software engineering professionals**.

Join the end-to-end **software engineering learning program** at SoftUni to master the most in-demand programming concepts, languages, software technologies and skills through a modern teaching methodology, **interactive learning platform** and tons of practical exercises and projects. Our **curriculum** is developed together with the **IT companies** to closely match the industry demands.

SoftUni works directly with the **companies** from the software industry to **find a job** for its students and to make them successful software engineers.

The path of the student at SoftUni



Apply now

softuni.org/apply

Chapter 11. Tricks and Hacks

In the current chapter, we are going to see some tricks, hacks, and techniques, which will make our work with **Python** easier in the **PyCharm** IDE. In particular, we will see:

- How to properly **format our code**.
- Conventions for **naming elements in the code**.
- Some **keyboard shortcuts**.
- Some **code snippets**.
- Techniques to **debug our code**.

Code Formatting

The right formatting of our code will make it **easier to read and understand** in case someone else needs to work with it. This is important because in practice we will need to work in a team with other people and it is highly important to write our code in a way that our colleagues can **quickly understand** it.

There are some defined rules for correct formatting of code, which are collected in one place and are called **conventions**. The conventions are a group of rules, generally accepted by the programmers using a given language, which are massively used. These conventions help to build norms in given languages - what the best way to write is and what **good practices** are. It is accepted that if a programmer follows them then their code will be easy to read and understand.

The **Python** language is created by the organization **Python Software Foundation** (PSF). Good writing practices can be different in each project, but most of them are done by the authors of the language (mostly Guido van Rossum). You should know that, even if you do not follow the imposed conventions recommended by **PSF**, the code will **work** (as long as it is written correctly), but simply **will not be easy to understand**. Of course, this is not fatal on a base level, but the faster you get used to writing quality code, the better.

The official **Python code convention** by PSF is located on the page **PEP 8 – Style Guide for Python Code**: <https://www.python.org/dev/peps/pep-0008/>. It is important to note that the examples which we gave until now and the ones that we will give in this book are guided by it.

For formatting the code, PSF recommends each inside level to be **4 spaces inside** (or one tab), as it is in the example below. **The language does not accept mixing tabs with spaces**:

```
if some_condition:
    print("Inside the if statement")
```

It is possible to instead of using 4, use 2 spaces, but it is not recommended.

Here is how two nested constructions look. Each block starts at 4 intervals inside. The number of spaces at the beginning of each row indicates what the indentation level is:

```
if some_condition:
```

```
print("Start of outer if statement")
if another_condition:
    print("Inside the inner if statement")
print("End of outer if statement")
```

The formatting of the code in **Python** is extremely important for it to work. Code that is not formatted throws an **error**.

This is an example of **badly formatted code** according to the accepted conventions for writing code in Python:

```
if some_condition:
print("Inside the if statement")
```

Once you run the code the following error is thrown and the program stops working:

- **IndentationError: expected an indented block**

The command inside **if** construction should be **4 spaces inside (or one tab)**. Right after the keyword **if** and before the condition of the check you should write a **space**.

The same rule is applied for the **for** loops and the other nested constructions. This is another example:

Correct:

```
for i in range(5):
    print(i)
```

Wrong:

```
for i in    range(5):
print( i )
```

For our convenience, there are **keyboard shortcuts in PyCharm** which will be explained later in this chapter, but for now, we can check one combination. It is used for **formatting the code** in the whole document: **[CTRL + ALT + L]**.

Let's use the wrongly formatted example from earlier:

```
for i in    range(5):
print( i )
```

If we press **[Shift + Alt + F]**, which is our combination to format **the entire document**, we will get code formatted according to **generally accepted Python conventions** and it will look the following way:

```
for i in range(5):
    print(i)
```

This key combination can help us if we work with badly formatted code. However, automatic formatting does not affect the naming of our variables (as well as other elements in the code), which we have to take care of ourselves.

Naming Code Elements

In this section, we will focus on the accepted conventions for naming projects, files, and variables, defined by PSF.

Naming Projects and Files

It is recommended to use a descriptive name for naming projects and files, which suggests the role of the respective file / project, and at the same time, the **lowercase_with_underscores** convention is also recommended. This is a convention for naming elements, in which each word, including the first one, starts with a lowercase character, and each word after is connected with an underscore (`_`), for example, **expression_calculator**. It's acceptable to name projects using the **PascalCase** convention - each word, including the first one, starts with a capital letter, and the next are appended to each other - **ExpressionCalculator**.

Example: The book starts with the **First steps in coding** chapter, therefore an exemplary name for the solution for this lecture can be **first_steps_in_coding** or **FirstStepsInCoding**. The files in a project should follow the **lowercase_with_underscores** convention. If we take for example the first problem in the **First steps in coding** lecture, it is called **Hello World**, therefore our file in the project will be called **hello_world.py**.

Naming Variables

In programming, variables keep data, and for the code to be more understandable, the name of a variable should suggest its purpose. Here are some recommendations for naming variables:

- The name should be **short and descriptive** and explain what the variable serves for.
- The name should only contain the letters **a-z, A-Z**, the numbers **0-9**, and the symbol `'_'`.
- It is accepted in Python for the variables to follow the **lowercase_with_underscores** convention.
- You should be careful with uppercase and lowercase letters because Python distinguishes them. For example, **age** and **Age** are different variables.
- The names of the variables **cannot coincide with keywords** in the Python language, for example **for** is an invalid name for a variable.



Although using capital letters in the names of variables is allowed, in Python it is not recommended and is considered a bad style of naming.

Here are some examples for **well-named** variables:

- `first_name`
- `age`
- `start_index`
- `last_negative_number_index`

Here are some examples for **badly named variables**, even though the names are correct according to the Python compiler:

- `firstName` (named after another convention, which is not accepted in Python).
- `AGE` (written in capital letters).
- `Start_Index` (contains capital letters).
- `lastNegativeNumber_Index` (missing '_' before each word).

At a first look all these rules can seem meaningless and unnecessary, but with time passed and experience gained you will see the need for conventions for writing quality code to be able to work easier and faster in a team. You will understand working with code, which is written without complying with any rules for code quality, is annoying.

Shortcuts in PyCharm

In the previous section, we mentioned some of the combinations, that are used for formatting code: `[Ctrl + Alt + L]` is used for **formatting the whole code in a file**. These combinations are called **shortcuts** and now we will give more thorough information about them.

Shortcuts are **combinations**, that give us the ability to perform some actions **easier and faster**, and each software development environment has its shortcuts, although most are repetitive. We will now look at some of the **shortcuts** in **PyCharm**.

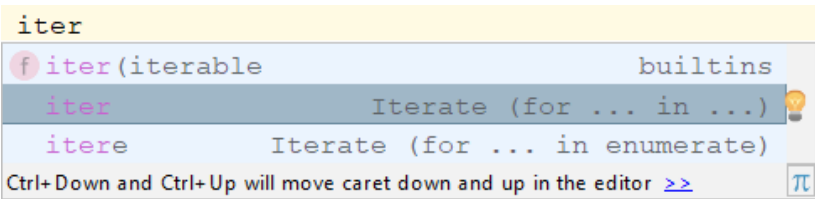
Combination	Action
[CTRL + F]	The combination opens the search window , by which we can search in our code .
[CTRL + /]	Comments part of our code and accordingly removes the comment which is already commented.
[CTRL + Z]	Brings back one change (so-called Undo).
[CTRL + SHIFT + Z]	The combination is opposite of [CTRL + Z] (the so-called Redo).
[CTRL + ALT + L]	Formats the code according the default conventions.
[CTRL + Backspace]	Deletes the word to the left of the cursor.

Combination	Action
[CTRL + Del]	Deletes the word to the right of the cursor.
[CTRL + S]	Saves all files in the project.

More about the **shortcuts in PyCharm** can be found on the website of JetBrains: <https://www.jetbrains.com/help/pycharm/mastering-keyboard-shortcuts.html>.

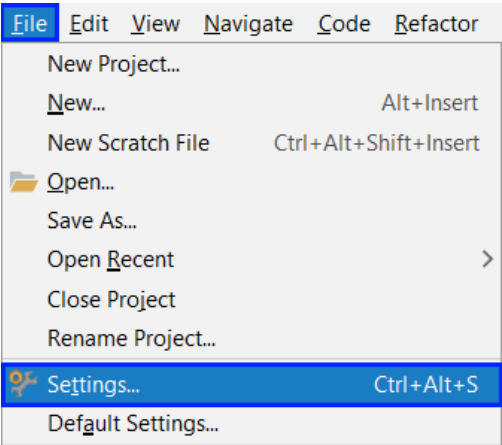
Code Snippets

In **PyCharm** there are the so-called **code snippets** (code snippets), which write a block of code by using a code template. For example, by writing the shortcode **"iter"** and then pressing [Tab] code **for ... in ...:** is generated in the body of our program, in the place of the shortcode. This is called "unfolding a code snippet". On the figure below you can see the **"iter"** snippet in action:

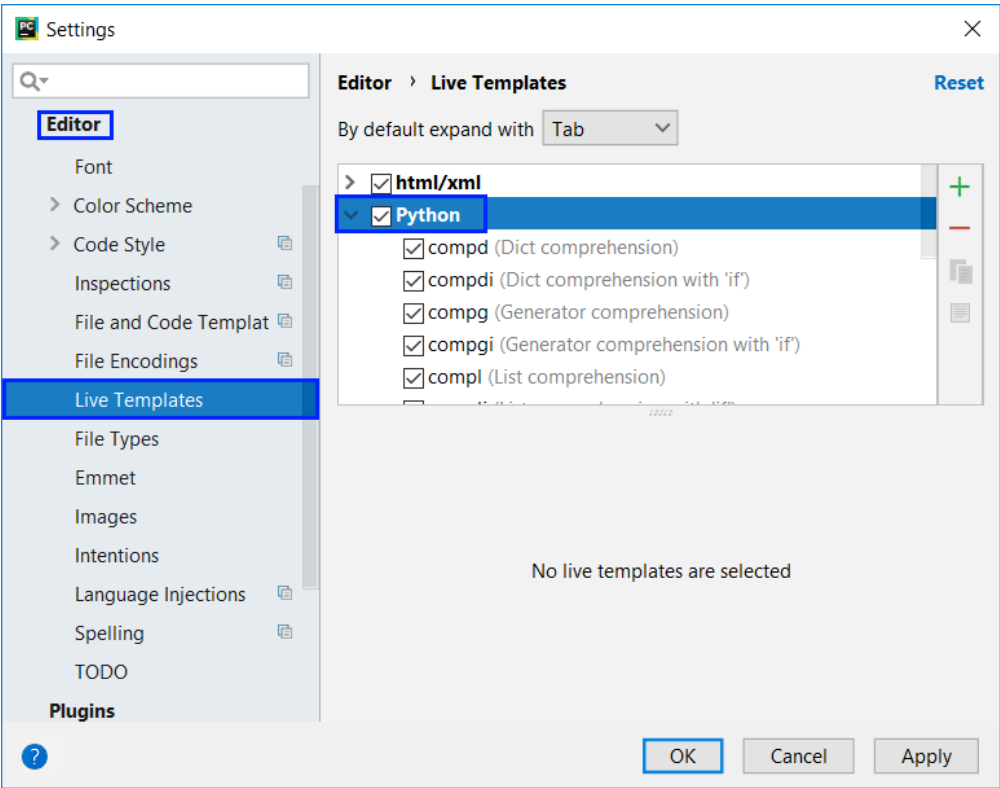


Creating Your Code Snippet

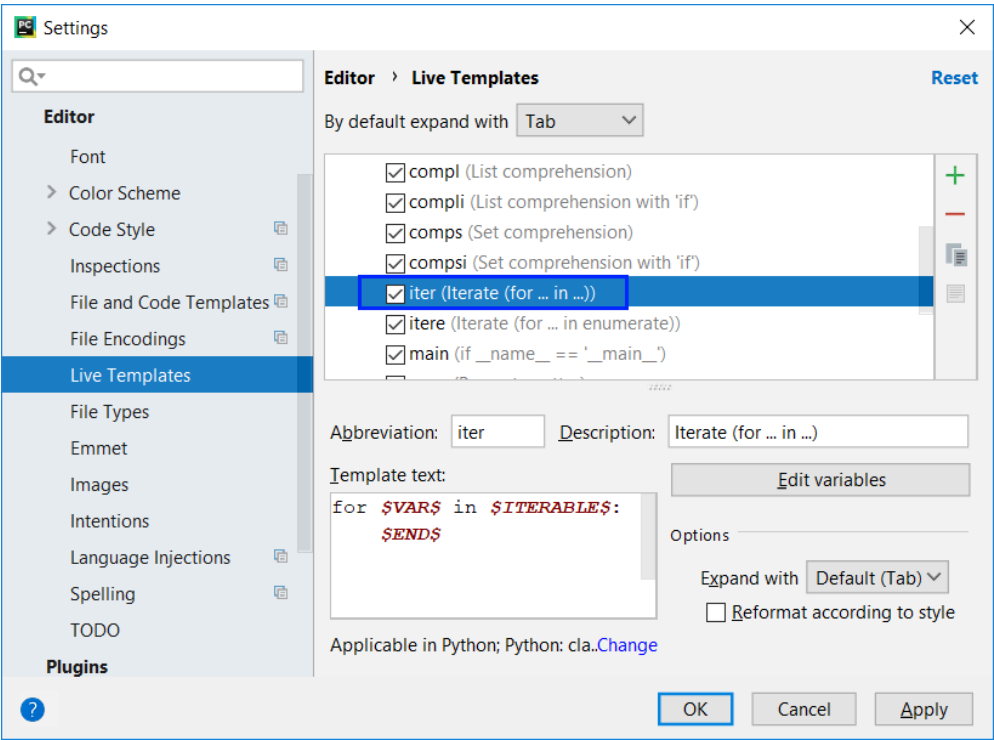
In this section, we are going to show you how to **make your code snippet**. We will see **how to make a live template for two nested loops**. To begin we must create a new empty project and go to [File] -> [Settings], as shown in the picture:



In the window that we opened, we must choose [Editor] -> [Live Templates], and from the section, we should choose the arrow **before** the checkbox for **Python**. This is where all the existing snippets for **Python** are located:

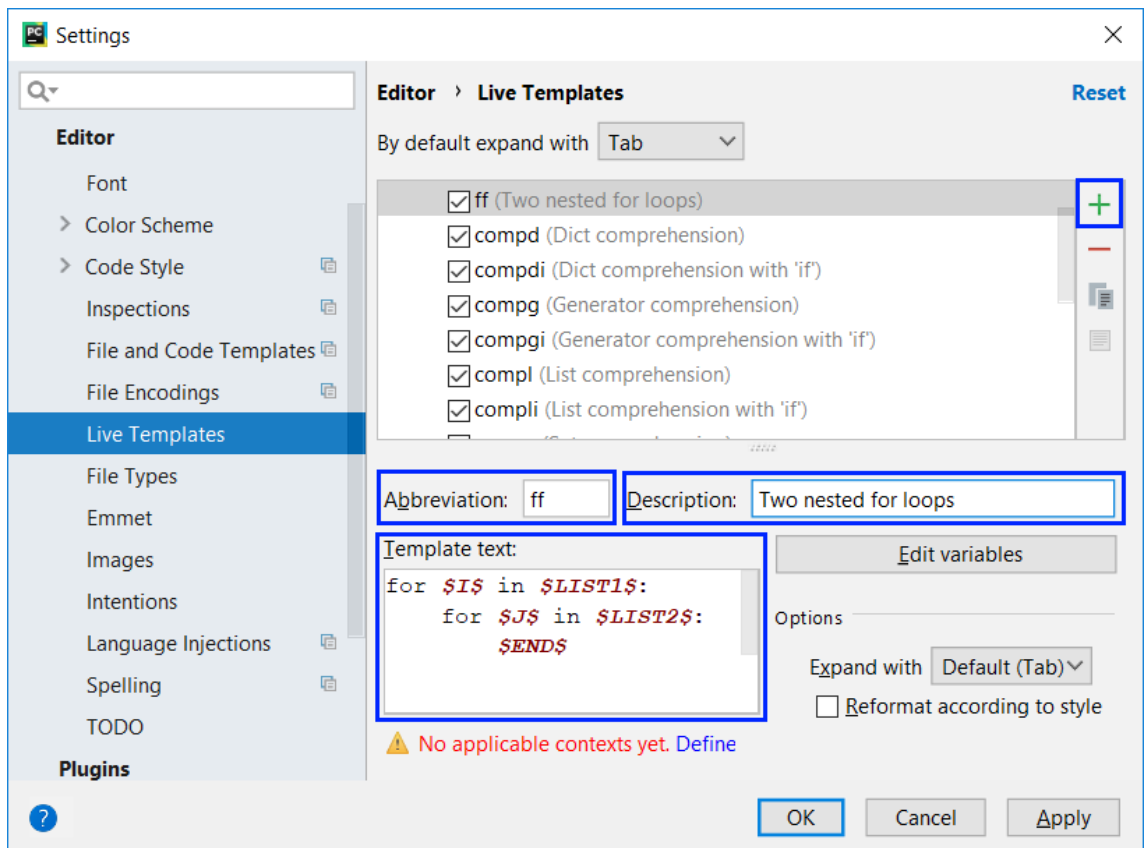


We choose a snippet, for example, `iter`, we explore it:

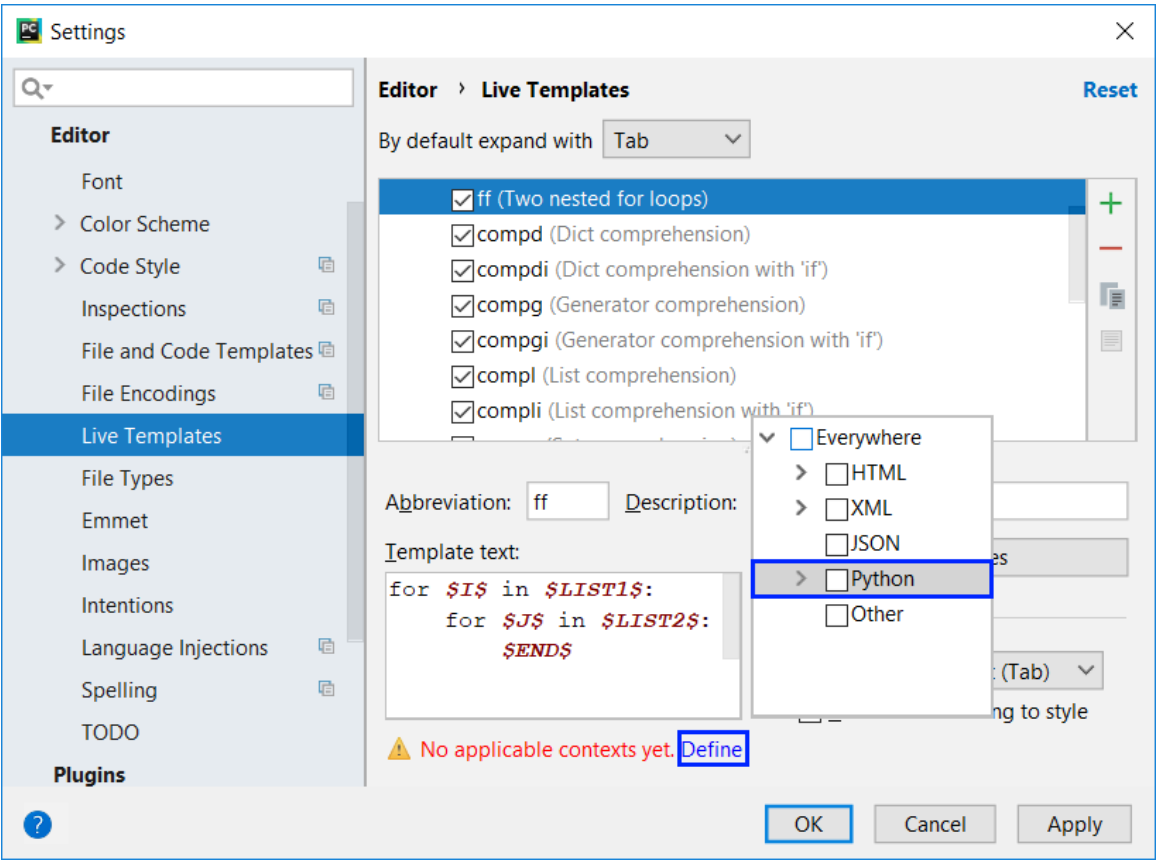


We see many things we haven't seen before yet, but don't worry, we will become acquainted with them later. For now, we want to create our snippet. To do so we choose the button Add (the green + on the right). From the list, we choose [1. Live Template]. In the bottom part of the window, on the place of **Abbreviation**, we should write a short name, which will be used to call the snippet (for example **ff**, for **for**, **for**), for **Description** we give a useful description, and for **Template Text**, we input the following:

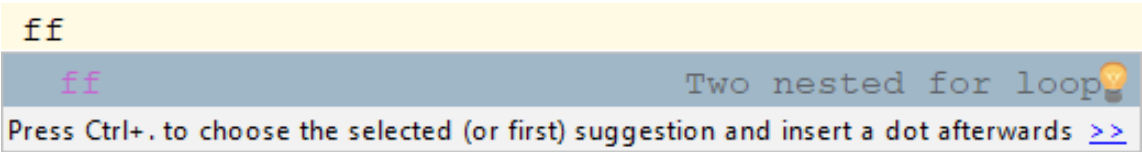
```
for $I$ in $LIST1$:
    for $J$ in $LIST2$:
        $END$
```



We receive a warning to choose context or the cases our snippet will show (the red text under Template text). We choose **Define** and from the appeared menu we check **Python**.



Now, if we write **ff** in PyCharm, our new live template appears:



Code Debugging Techniques

Debugging plays an important role in the process of creating software, which is to allows us to follow the implementation of our program step by step. With this technique, we can follow the values of the local variables because they are changing during the execution of the program and remove possible errors (bugs). The process of debugging includes:

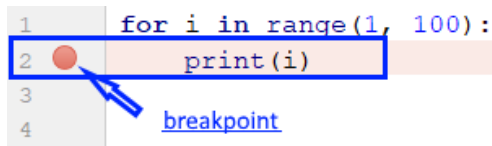
- Finding the problems (bugs).
- Locating the code, which causes the problems.
- Correcting the code, which causes the problems so that the program works correctly.
- Testing, to make sure that the program works correctly after the corrections we have made.

PyCharm gives us a **built-in debugger**, thanks to which we can place **breakpoints**, at places we have chosen. When it reaches a **breakpoint**, the program **stops running** and allows **step-by-step running** of the remaining lines. Debugging allows us to **get into the details of the program** and see where exactly the errors occur and what is the reason for this.

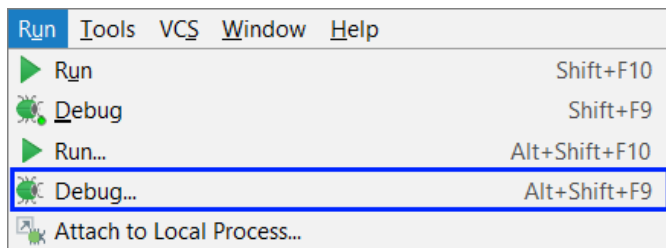
To demonstrate how to use the debugger, we will use the following program:

```
for i in range(1, 100):
    print(i)
```

We will place a **breakpoint** on the function **print(...)**. For this, we will need to move our cursor to the line, which prints on the console, and press [CTRL + F8]. A **breakpoint**, where the program will **stop** its execution:



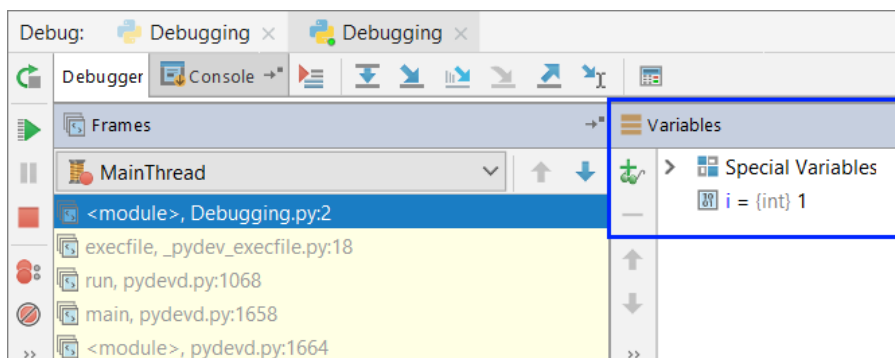
To start the program in debug mode, we choose [Run] -> [Debug...] or press [Alt + Shift + F9]:



After starting the program, we can see that it **stops executing** at line 2, where we placed our breakpoint. The code in the current line is **colored in yellow** and we can **run it step by step**. To go to the next line we use the key [F8]. We can see that the code on the current line hasn't been executed yet. It will execute when we go-ahead to the next line:

```
1 for i in range(1, 100): i: 1
2 print(i)
```

From the **Debugger** window, we can observe **the changes in the local variables**. The window opens when we start debugging. To open it manually, we should choose [View] -> [Tool Windows] -> [Debug]:



A guide to tricks

In this section, we will briefly show **tricks and techniques** of programming with the **Python** language, some of which are already mentioned in this book, which will be very useful, if you were to take a programming exam for beginners:

Inserting Variable Values in Strings

```
text = "some text"
print(f"{text}")
# This will print "some text" on the console
```

In this case, we are using a **placeholder** – **{x}**, where **x** is the **name of variable** that we want to show. It's possible to use more than one variable, for example:

```
text = "some text"
number = 5
print(f"{text} {number} {text}")
# This will print "some text 5 some text"
```

In this example, we can see that we can insert **not only text variables**. We can also use a given variable **several times**.

Rounding Numbers

In case we need to round numbers we can use one of the following methods:

- **round(num, digits)** – accepts two arguments – the first one is **the number to be rounded**, and the second – the **number of decimals to use when rounding the number** (this number should always be an integer). Rounding is performed according to the basic rule of rounding - if the decimal part is less than 5, rounding is down and vice versa, if it is greater than 5 - up:

```
first_number = 5.431
print(round(first_number, 2))
# This will print "5.43"

second_number = 5.539
print(round(second_number, 2))
# This will print "5.54" on the console
```

in case we want the rounding always to be **down to the previous integer**.

- **math.floor(...)** – in case we want the rounding always to be **down**. It is important to pay attention that this function is rounding the number to an integer. For example, if we have the number 5.99 and use **math.floor(5.99)**, we get the number **5**:

```
number_to_floor = 5.99
print(math.floor(number_to_floor))
# This will print "5" on the console
```

- **math.ceil(...)** – in case we want the rounding always to be **up**. This function also is rounding the number to an integer. For example, if we have the number 5.11 and use **math.ceil(5.11)**, we get the number 6:

```
number_to_ceil = 5.11
print(math.ceil(number_to_ceil))
# This will print "6" on the console
```

- **math.trunc(...)** – in case we want to **remove the fractional part**. For example, if we have the number 2.63 and use **math.trunc(2.63)**, we will get the number 2:

```
number_to_truncate = 2.63
print(math.trunc(number_to_truncate))
# This will print "2" on the console
```

Rounding with a Placeholder

```
number = 5.432424
print(f"{number:.2f}")
```

In this case, after the number, we add **:.2f**, which will limit the number to 2 digits after the decimal point and will work like **round(...)**. You should keep in mind that the number after the letter **f** means how many digits after the decimal point the number is rounded (i.e. it can be **3f** or **5f**). Don't forget the **dot before the number** - it is necessary.

How to Write a Conditional Statement?

The conditional **if** statement contains the following elements:

- Keyword **if**
- A Boolean expression (condition).
- Body of the conditional construction.
- Optional: **else** clause.

```
if condition:
    # Body
else:
    # Body
```

To make it easier we can use a live template for an **if** construction:

- **if** + [Tab]

How to Write a `for` Loop?

For a `for` loop we need a couple of things:

- Initializing block, in which the counter variable is declared (`let i`).
- Condition for repetition: `range(5)` are the numbers from `0` to `4` included.
- Body of the loop.

```
for i in range(5):  
    # Body
```

To be easier to write it, we can use a live template for the `for` loop:

- `for` + [Tab]
- or `iter` + [Tab]

What Have We Learned from This Chapter?

In the current chapter, we learned how to **correctly format** and name the elements of our code, some **shortcuts** for work in PyCharm, some **code snippets** (Live Templates), and how to **debug code**.

Conclusion

If you have **read the entire** book and you've solved all the problems from the exercises and reached the present conclusion, **congratulations!** You've already made the **first step** in learning the **profession of a programmer**, but there is a **long way** to go until you become **really good** and make **software writing** your **profession**.

Remember the [four main groups of skills](#), that each programmer must have to work in the industry:

- Skill #1 – **writing the program code** (20% of programmer's skills) – covered to a large degree by this book, but you must learn additional basic data structures, classes, functions, strings, and other elements of code writing.
- Skill #2 – **algorithmic thinking** (30% of programmer's skills) – covered partially by this book and developed mostly by solving a large amount of diverse algorithmic problems.
- Skill #3 – a **fundamental understanding of the profession** (25% of programmer's skills) – acquired for a few years in combination with learning and practice (reading books, watching video lessons, attending courses, and mostly by writing diverse projects in various technological areas).
- Skill #4 – **programming languages and software technologies** (25% of programmer's skills) – acquired in a long period, by a lot of practice, consistent reading, and writing projects. Such knowledge and skills quickly get outdated and need to be updated frequently. Good programmers are involved in studying new technologies every day.

This Book is Only the First Step!

The **present** book on programming basics is just the **first step** in building the skills of a programmer. If you were able to solve **all problems**, this means you have **obtained valuable knowledge** in the programming principles with **Python** language on a **basic level**. You are about to start **in-depth** studying of programming, develop **your algorithmic thinking**, and then add **technological knowledge** regarding the Python language, Django ecosystem, front-end technologies (HTML, CSS, Angular, React, AJAX, HTML5), and many other concepts, technologies and instruments for software development.

If you were **not able** to solve all problems or a large part of them, go back and solve them! Remember that **becoming a programmer** requires **a lot of work and effort**. This profession is not for lazy people. There is no way to learn it unless **you seriously practice** programming for years!

As we already explained, the first and basic skill of a programmer is **to learn to write code** with ease and pleasure. This is namely the mission of this book: to teach you how to code.

What to Do After Reading This Book?

This book **gives you solid foundations**, thanks to which it will be easy for you to continue developing as a programmer. If you wonder how to continue your development, you have the following possibilities:

- to study for a [software developer](#) at SoftUni and make programming your profession.
- to continue developing as a programmer **on your own**, for example through self-training or via free online lessons.
- to **stay at a coder level**, without going more seriously into programming.

Study Software Engineering at SoftUni

The first option, which we recommend, is to enroll in a [Software Developer program](#) to master programming on a professional level. The SoftUni curriculum is carefully developed by **Dr. Svetlin Nakov and his team**, to provide you consequently and with gradually increasing complexity all the skills that a software engineer must have, **to start a career as a software developer** in an IT company.

SoftUni's self-paced programs are designed as a **sequence of modules** giving you fundamental knowledge in software development and practical skills to prepare you to work as a programmer with the latest software technologies. The classes are divided into **learning theory (30%)** and doing exercises and **projects (70%)**.

Training Duration at SoftUni

The training in SoftUni has a duration of **2-3 years** (depending on the profession and the selected specializations) and during that period it is normal to reach a good starting level (junior developer), but this is **only if you study seriously** and write code intensely every day. Upon having good grades, a typical student **starts a job around the middle of the training (after around 1.5 years)**. Thanks to the well-developed partners' network, **the career center of SoftUni offers work** in a software or IT company to all SoftUni students who have very good or excellent grades. **Starting a job** by having good grades at SoftUni, combined with a willingness to work and reasonable expectations towards the employers, is almost guaranteed.

It Takes at Least a Year of Intense Code Writing to Become a Programmer

Keep in mind that **to become a programmer takes a lot of effort**, writing tens of thousands of lines of code, and solving hundreds, even thousands of practical problems, and this takes years! If someone offers "**an easier program**" with the promise of you becoming a programmer and landing a job within 3-4 months of starting the course, they are either **lying** to you, or providing a significantly lower-quality education that will not prepare you enough for even an intern position. There are exceptions, of course - for example, if you are not starting from scratch, or if you have extremely well-developed engineering thinking, or if you apply for a very low position (for example technical support), but in general, **you cannot**

become a programmer if you haven't spent at least 1 year of intense learning and code-writing!

SoftUni for People Who Work and Study

We recommend to everyone who gets **an excellent score at Judge Problems** and is passionate about making programming their profession, to consider leaving the rest of their commitments aside to **spend more time** on learning the profession of a software engineer and start making a living through it.

We recommend to all who **cannot get an excellent score at the Judge problems** to spend more time on better learning, understanding, and most of all, practicing the material studied in the present book. If you cannot easily solve the problems in this book, you will not be able to cope with programming and software development in the future.

Study Software Engineering on Your Own

Another possibility to develop after this book is to **continue studying programming outside of SoftUni**. You can enroll or subscribe to **video training**, that goes into more details in programming with **Python** or other languages and development platforms. You can **read books** on programming and software technologies, follow **online tutorials** and other online resources – there are plenty of free materials on the Internet. However, keep in mind that the most important thing about the profession of a programmer is **to do practical projects!**

You cannot become a programmer without a lot of code writing and intense practice. Allocate **sufficient time** to it. You cannot become a programmer for a month or two. On the Internet, you will find a wide variety of **free resources** such as books, manuals, and video lessons. However, you need to invest **at least a year or two**, to acquire a foundation level, needed for starting a job.

After you gain some experience, find a way to start **an internship in a company** (which will be almost impossible unless you'd spent at least a year of intense code writing before that) or come up with **your practical project**, on which you need to spend a few months, even a year, to learn based on the trial-and-error principle.



Keep in mind that there are many ways to become a programmer, but they all have something in common: **code writing and years of practice!**

Online Communities for Beginners in Programming

Regardless of the path you have chosen, if you are seriously involved in programming, we recommend subscribing to specialized **online forums, discussion groups, and communities**, from which you can get assistance from your colleagues and track the novelties in the software industry.

If you aim to study programming seriously, **surround yourselves with people who are involved in programming** seriously. Join **communities of software developers**, attend software conferences, go to events for programmers, find friends with whom you can talk about programming and discuss problems and bugs, find an environment that can help you.

In most large towns there are free events for programmers, a few times a week. In smaller localities, you have the Internet and access to the entire online community.

Here are some recommended **resources**, that will be useful for your development as a programmer:

- <https://learn.softuni.org> – the official page for SoftUni's **self-paced Software Developer programs and open courses**.
- <https://www.facebook.com/softuni.org> – official **Facebook page of SoftUni**. By it, you will learn about new courses, seminars, and events related to programming and software development.
- <https://introprogramming.info> - official website of the books "**Programming Basics**" with **C#, Java, JavaScript**, and **Python** by Dr. Svetlin Nakov and his team. The books examine in-depth programming basics, basic data structures and algorithms, object-oriented programming, and other basic skills, and are an excellent continuation for reading after the current book. However, **besides reading, you need to do intense code writing**, do not forget that!
- <https://stackoverflow.com> - **Stack Overflow** is one of the **widest** discussion forums for programmers worldwide, where you will find assistance for any possible question in the world of programming. If you are fluent in English, look up at StackOverflow and ask your questions there.
- <https://www.meetup.com> - search **tech meetups** around your town and enter the community that you like. More of the tech meetups are free and the newcomers are welcome.

Good Luck to All of You!

On behalf of the entire authors' team, we **wish you endless success in the profession and your life!** We will be really happy if we have helped you get **passionate about programming** and we have inspired you to go bravely towards becoming a "software engineer", which will bring you a good job that you will work with pleasure, give you a high-quality life, and prosperity, as well as amazing perspectives for development and possibilities for making impressive projects with inspiration and passion.

Sofia, October 19, 2018



In your hands you hold something more than a **programming book, textbook, or tutorial**. This modern teaching resource guides you through the **first steps in programming** using a little text and many code examples with explanations, combined with carefully selected **practical problems** and a **judge system** for immediate **automatic code evaluation**.

The teaching content was designed personally by **Svetlin Nakov**, who has helped more than **150 000 beginners** in the last 20 years to **start coding** and learn the **software engineering profession**.

Svetlin Nakov is known for his **inspiration** and ability to teach programming skills gradually, using **small learning units**, full of real-world examples and **practical problem solving**.

Remember that **programming skills** can only be learned through a lot of **code writing** and a lot of **practical problem solving**, not through reading books and watching videos, so be sure that you solve the exercises in each book section. Good luck!

Web site: **python-book.softuni.org**



AUTHORS TEAM:

Boncho Vulkov
Hristo Minkov
Iliya Iliev
Martin Tsarev
Miglen Evlogiev
Milena Angelova
Mirela Damyanova
Nikolay Kostov
Petar Ivanov
Petya Gospodinova
Svetlin Nakov
Tanya Evtimova
Tanya Staneva
Teodor Kurtev
Ventsislav Petrov
Vladimir Damyanovski
Yordan Darakhchiev

ISBN 978-619-00-1403-4



9 786190 014034 >